

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

LEONARDO FAIX PORDEUS

**PROPOSIÇÃO DE UMA ARQUITETURA PARA ÁRVORES DE DECISÃO
ORIENTADA A NOTIFICAÇÕES - ADON**

CURITIBA

2024

LEONARDO FAIX PORDEUS

**PROPOSIÇÃO DE UMA ARQUITETURA PARA ÁRVORES DE DECISÃO
ORIENTADA A NOTIFICAÇÕES – ADON**

**PROPOSAL OF A NOTIFICATION-ORIENTED DECISION TREE ARCHITECTURE
- NODTA**

Tese de Doutorado apresentada ao Programa de Pós-Graduação em Engenharia Elétrica e Informática Industrial da Universidade Tecnológica Federal do Paraná como requisito parcial para obtenção do título de “Doutor em Ciências” – Área de Concentração: Engenharia de Computação.

Candidato: Leonardo Faix Pordeus

Orientador(a): Prof. Dr. Jean Marcelo Simão.

Coorientador(a): Prof. Dr. Robson Ribeiro Linhares

CURITIBA

2024



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.



LEONARDO FAIX PORDEUS

**PROPOSIÇÃO DE UMA ARQUITETURA PARA ÁRVORES DE DECISÃO ORIENTADA A NOTIFICAÇÕES
ADON**

Trabalho de pesquisa de doutorado apresentado como requisito para obtenção do título de Doutor Em Ciências da Universidade Tecnológica Federal do Paraná (UTFPR). Área de concentração: Engenharia De Computação.

Data de aprovação: 19 de Dezembro de 2024

Dr. Andre Eugenio Lazzaretti, Doutorado - Universidade Tecnológica Federal do Paraná

Dr. Andre Schneider De Oliveira, Doutorado - Universidade Tecnológica Federal do Paraná

Dr. Antonio Augusto Medeiros Frohlich, Doutorado - Universidade Federal de Santa Catarina (Ufsc)

Dr. Herve Panetto, Doutorado - Universite de Lorraine

Dr. Paulo Cezar Stadzisz, Doutorado - Universidade Tecnológica Federal do Paraná

Documento gerado pelo Sistema Acadêmico da UTFPR a partir dos dados da Ata de Defesa em 31/01/2025.

AGRADECIMENTO

Agradecimentos de ordem pessoal

Dedico este trabalho primeiramente a Deus, por ter me concedido todas as condições e oportunidades para realizar este trabalho.

Em especial reforço meu agradecimento a todo amor, carinho, apoio, incentivos e cobranças recebidos de meu pai José Elber Vedam Pordeus, que infelizmente faleceu no ano de 2019. Todos os seus ensinamentos e esforços me permitiram chegar até o presente momento e deram forças para continuar este trabalho de doutorado.

Agradeço a todos meus familiares, em especial a minha mãe Rosicler De Fátima Faix e minha irmã Tathyane Faix Pordeus pelo apoio durante esse período, além de todo amor, carinho, compreensão e incentivo em toda a minha vida.

Agradeço a minha esposa Karina Tie Hirayama por todo amor, carinho, apoio e paciência comigo nesse período de doutoramento.

Agradecimentos de ordem profissional.

Primeiramente agradeço ao Grupo Volvo e ao CITS pela oportunidade de conciliar a realização deste doutorado em conjunto com as atividades de trabalho. Agradeço também aos professores Omero Francisco Bertol e Danillo Leal Belmonte pela oportunidade de lecionar duas disciplinas no Curso de Especialização de Internet das Coisas, na própria UTFPR.

Principalmente, agradeço aos professores Jean Marcelo Simão e Robson Ribeiro Linhares pela amizade, pelos conselhos e orientação e coorientação durante este período de doutorado. Em especial, agradeço ao professor André Eugênio Lazzaretti, juntamente com o professor Simão, ao sugerir a possibilidade de utilizar o PON em Árvores de Decisão e *Random Forest*, enquanto hipótese, além de toda a ajuda, coautoria e suporte que recebi ao longo deste percurso.

Igualmente, agradeço aos professores André Schneider de Oliveira, Antônio Augusto Medeiros Fröhlich, Hervé Panetto e Paulo César Stadysz, membros da banca, pela disponibilidade em avaliar este trabalho, trazendo todas suas sempre muito pertinentes sugestões e correções.

Agradeço ainda aos professores, colegas e amigos do grupo de pesquisa e da instituição e geral, bem como ao CPGEI especificamente que já me havia

acolhido no período no mestrado e à UTFPR que já havia também me acolhido na época da graduação em Engenharia Eletrônica.

Por fim, agradeço a todas as outras tantas pessoas, que embora não citadas, contribuíram para a realização deste trabalho, seja de forma direta, seja indireta.

RESUMO

O aumento da complexidade dos modelos de *machine learning* tem exigido uma evolução dos sistemas computacionais associados, no que diz respeito a processamento mais eficiente, em termos de uso de recursos, consumo de energia e desempenho. Essa necessidade tem motivado pesquisas e o desenvolvimento de soluções especializadas para suportar tais modelos diretamente em *hardware*. Neste contexto, a execução de modelos baseados em algoritmos de árvores de decisão e derivados do tipo *ensemble*, como *Random Forest* e *Gradient Boosting*, em dispositivos de lógica reconfigurável (FPGAs) é alvo destas pesquisas dado o seu potencial de aceleração. No entanto, o tamanho limitado dessas plataformas restringe a quantidade de árvores de decisão que podem ser alocadas, comprometendo a eficiência e o desempenho destes algoritmos em *hardware*. Isto se agrava com o fato de que as avaliações presentes nestes algoritmos frequentemente apresentam importantes redundâncias estruturais e temporais, que resultam em ineficiência em relação ao tempo de execução e ao tamanho do *hardware* a ser sintetizado nos dispositivos FPGAs. Esta característica ocorre principalmente devido às estruturas das árvores de decisão compostas essencialmente por avaliações lógicas do tipo ‘se-então’, que são repetidamente reavaliadas em diferentes pontos da árvore. Entretanto, esses algoritmos seriam especialmente propícios para se beneficiar dos conceitos e propriedades do Paradigma Orientado a Notificações (PON) que evitam tais redundâncias e, ademais, permitem paralelismo fino em FPGA. Esta tese propõe a Arquitetura para Árvores de Decisão Orientada a Notificações (ADON), uma arquitetura distinta que integra os princípios do PON para a composição e execução de árvores de decisão, com destaque para *Random Forest* e *Gradient Boosting*. A ADON, a partir da conformação e ampliação de conceitos oriundos do PON, tem um encadeamento próprio de notificações, permitindo a sua implementação e validação em *hardware* digital, em forma denominada de ADON-HD, alcançando a aceleração do processo de inferência e melhor uso do *hardware*. Experimentos realizados comparam o desempenho da implementação da ADON-HD em relação ao estado da arte de execução de árvores de decisão em *hardware*, precisamente utilizando a biblioteca *open-source hls4ml*. Neste cenário, foi verificado que inclusive devido à sua capacidade de eliminação das redundâncias presentes nos algoritmos de árvores de decisão, o uso da ADON-HD permitiu a redução da quantidade de recursos de *hardware* de até 70,5% menos LUTs e 94,8% menos FFs, além da melhora no desempenho de até 13,02 vezes na fase de inferência pela diminuição do tempo de processamento e aumento na taxa de classificação. A partir dos resultados obtidos, a ADON-HD é confirmada como uma alternativa promissora para a execução dos algoritmos derivados de árvores de decisão em *hardware*, proporcionando um melhor desempenho e uso dos recursos disponíveis.

Palavras-chave: Paradigma Orientado a Notificações, Arquiteturas Paralelas, Aceleração em Hardware, Field Programmable Gate Arrays, Árvores de Decisão, Random Forest, Gradient Boosting.

ABSTRACT

The increasing complexity of machine learning models requires evolving computing systems towards more efficient processing capabilities in terms of resource utilisation, energy consumption, and performance. This need has motivated research and the development of specialised hardware solutions to support those models. Due to their acceleration potential, a set of literature research focuses on implementing decision tree algorithms, namely ensemble methods, such as Random Forest and Gradient Boosting, on reconfigurable logic devices (FPGAs). However, the limited size of these platforms restricts the number of decision trees that can be allocated, compromising the algorithm's efficiency and hardware performance. In addition, the structural and temporal redundancies inherent in these algorithm's evaluations lead to inefficiencies in execution time and FPGA hardware synthesis size. These inefficiencies occur mainly due to decision tree structures composed essentially of 'if-then' logical evaluations that require repeated assessment at various tree nodes. However, these algorithms would be especially suitable for benefiting from the concepts and properties of the Notification Oriented Paradigm (NOP), which avoids such redundancies and, moreover, allows fine-grained parallelism in FPGA. This thesis proposes the Notification-Oriented Decision Tree Architecture (NODTA), a novel architecture that integrates NOP principles for decision tree composition and execution, particularly emphasising Random Forest and Gradient Boosting. NODTA implements a unique notification chaining mechanism by adapting and extending NOP concepts. Its digital hardware implementation, NODTA-DH, presents enhanced inference acceleration and hardware efficiency. Experiments comparing the NODTA-DH performance against current state-of-the-art hardware implementation of decision trees, specifically using the open-source hls4ml library, present significant improvements. In this scenario, NODTA-DH's logical model eliminates the redundancies in decision tree algorithms and achieves up to 70.5% reduction in LUT usage and 94.8% in FF requirements. Moreover, the inference phase showed a 13.02 times performance improvement through reduced processing time and enhanced classification rates. These results establish NODTA-DH as a viable solution for hardware implementation of decision tree-based algorithms, offering better performance and resource utilisation.

Keywords: Notification Oriented Paradigm, Parallel Architectures, Hardware Acceleration, Field Programmable Gate Arrays, Decision Trees, Random Forest, Gradient Boosting.

LISTA DE FIGURAS

Figura 1 Exemplo de uma <i>Rule</i> .	33
Figura 2 Colaborações pontuais entre entidades do metamodelo do PON.	34
Figura 3 Taxonomia de paradigmas de programação incluindo o PON.	36
Figura 4 Diagrama de blocos internos representando os componentes do PON-HD.	46
Figura 5 Componente <i>Attribute</i> .	47
Figura 6 Componente <i>Method</i> .	47
Figura 7 Componente <i>Premise</i> .	48
Figura 8 Exemplo de árvore de decisão.	53
Figura 9 Exemplo de divisão das amostras do <i>dataset Iris</i> em um modelo de árvore de decisão.	55
Figura 10 Etapa de teste do <i>Random Forest</i> .	61
Figura 11 Exemplo contendo a primeira árvore treinada com o algoritmo <i>Gradient Boosting</i> .	63
Figura 12 Exemplo contendo a segunda árvore treinada com o algoritmo <i>Gradient Boosting</i> .	63
Figura 13 Exemplo contendo a terceira árvore treinada com o algoritmo <i>Gradient Boosting</i> .	64
Figura 14 Diagramas de blocos <i>SysML</i> referente a solução ADON.	83
Figura 15 Diagrama de blocos internos <i>SysML</i> referente a exemplificação de duas árvores de decisão <i>Gradient Boosting</i> .	85
Figura 16 Conformação das Avaliações Lógico-Causais ao PON.	86
Figura 17 Exemplos de Regras mapeadas.	88
Figura 18 Diagrama de blocos internos <i>SysML</i> para um exemplo de regra mapeada ao PON.	89
Figura 19 Diagrama de blocos internos <i>SysML</i> exemplificando uma árvore de decisão conformada ao PON.	90
Figura 20 Diagrama de blocos internos <i>SysML</i> exemplificando <i>Premise Dual</i> .	92
Figura 21 Diagrama de blocos internos <i>SysML</i> exemplificando conformação após aplicar <i>Premise Dual</i> .	93
Figura 22 Exemplo de dependência de <i>SubCondition</i> .	95
Figura 23 Diagrama de blocos internos <i>SysML</i> exemplificando o Encadeamento de <i>SubConditions</i> .	97
Figura 24 Diagramas de Blocos <i>SysML</i> de análise da ADON.	98
Figura 25 Diagrama de blocos <i>SysML</i> da ferramenta para mapeamentos de modelos à ADON.	102
Figura 26 Diagrama de atividades para o treinamento.	103

Figura 27 Diagrama de atividades do processo de leitura e mapeamento do modelo.	105
Figura 28 Diagrama de atividades para geração de código PON-HD.	107
Figura 29 Componente <i>Attribute</i> com ponto fixo.	110
Figura 30 Componente <i>Method</i> com ponto fixo.	110
Figura 31 Componente <i>Premise</i> com ponto fixo.	111
Figura 32 Resultados dos testes obtidos via simulação.	118
Figura 33 Resultados dos testes obtidos em FPGA.	119
Figura 34 Uso de LUTs com o algoritmo <i>Random Forest</i> para o <i>dataset Iris</i> na comparação ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) vs Vivado HLS (<i>hls4ml</i>).	132
Figura 35 Uso de FFs com o algoritmo <i>Random Forest</i> para o <i>dataset Iris</i> na comparação ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) vs Vivado HLS (<i>hls4ml</i>).	132
Figura 36 Uso de LUTs com o algoritmo <i>Random Forest</i> para o <i>dataset Iris</i> .	133
Figura 37 Uso de FFs com o algoritmo <i>Random Forest</i> para o <i>dataset Iris</i> .	134
Figura 38 Uso de LUTs com o algoritmo <i>Random Forest</i> para o <i>dataset Wine</i> .	134
Figura 39 Uso de FFs com o algoritmo <i>Random Forest</i> para o <i>dataset Wine</i> .	135
Figura 40 Uso de LUTs com o algoritmo <i>Random Forest</i> para o <i>dataset Digit</i> .	135
Figura 41 Uso de FFs com o algoritmo <i>Random Forest</i> para o <i>dataset Digit</i> .	136
Figura 42 Frequência Máxima de operação (MHz) com o algoritmo <i>Random Forest</i> para o <i>dataset Iris</i> .	138
Figura 43 Predições/s com o algoritmo <i>Random Forest</i> para o <i>dataset Iris</i> .	138
Figura 44 Frequência Máxima de operação (MHz) com o algoritmo <i>Random Forest</i> para o <i>dataset Wine</i> .	139
Figura 45 Predições/s com o algoritmo <i>Random Forest</i> para o <i>dataset Wine</i> .	139
Figura 46 Frequência Máxima de operação (MHz) com o algoritmo <i>Random Forest</i> para o <i>dataset Digit</i> .	140
Figura 47 Predições/s com o algoritmo <i>Random Forest</i> para o <i>dataset Digit</i> .	140
Figura 48 Uso de LUTs com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Iris</i> .	142
Figura 49 Uso de FFs com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Iris</i> .	142
Figura 50 Uso de LUTs com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Wine</i> .	143
Figura 51 Uso de FFs com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Wine</i> .	143
Figura 52 Uso de LUTs com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Digit</i> .	144
Figura 53 Uso de FFs com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Digit</i> .	144
Figura 54 Frequência Máxima de operação (MHz) com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Iris</i> .	146
Figura 55 Predições/s com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Iris</i> .	147

Figura 56 Frequência Máxima de operação (MHz) com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Wine</i> .	147
Figura 57 Predições/s com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Wine</i>	148
Figura 58 Frequência Máxima de operação (MHz) com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Digit</i> .	148
Figura 59 Predições/s com o algoritmo <i>Gradient Boosting</i> para o <i>dataset Digit</i>	149
Figura 60 Experimentos adicionais referente ao consumo de energia para o algoritmo <i>Random Forest</i> com <i>dataset Iris</i>	152
Figura 61 Experimentos adicionais referente ao consumo de energia para o algoritmo <i>Gradient Boosting</i> com <i>dataset Iris</i>	152
Figura 62 Uso de LUTs para <i>dataset Iris</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	187
Figura 63 Uso de FFs para <i>dataset Iris</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	188
Figura 64 Frequência Máxima de Operação para <i>dataset Iris</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	188
Figura 65 Previsões/s para <i>dataset Iris</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	189
Figura 66 Uso de LUTs para <i>dataset Wine</i> com ADON-HD preliminar vs Vivado LHS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	189
Figura 67 Uso de FFs para <i>dataset Wine</i> com ADON-HD preliminar vs Vivado LHS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	190
Figura 68 Frequência Máxima de Operação para <i>dataset Wine</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	190
Figura 69 Previsões/s para <i>dataset Wine</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	191
Figura 70 Uso de LUTs para <i>dataset Digit</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	191
Figura 71 Uso de FFs para <i>dataset Digit</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	192
Figura 72 Frequência Máxima de Operação para <i>dataset Digit</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	192
Figura 73 Previsões/s para <i>dataset Digit</i> com ADON-HD preliminar vs Vivado HLS (<i>ad hoc</i>) e Vivado HLS (<i>hls4ml</i>).....	193

LISTA DE TABELAS

Tabela 1 Propriedades elementares do PON contempladas nas suas materializações.	44
Tabela 2 Conceitos do PON contemplados nas suas materializações.	44
Tabela 3 Revisão pragmática da literatura.	68
Tabela 4 Ferramental referente ao estado da arte e técnica acessível.	74
Tabela 5 Requisitos da ADON.	80
Tabela 6 Requisitos da ADON HD.	81
Tabela 7 Números de elementos após treinamento e mapeamento do <i>dataset Iris</i>	127
Tabela 8 Números de elementos após treinamento e mapeamento do <i>dataset Wine</i>	128
Tabela 9 Números de elementos após treinamento e mapeamento do <i>dataset Digit</i>	129

LISTA DE CÓDIGOS

Código 1 Código VHDL com tipos de dados do <i>Attribute</i>	109
Código 2 Votação Majoritária em VHDL para classificação em <i>Random Forest</i>	112
Código 3 Predição para probabilidade.	112
Código 4 Aproximação numérica para cálculo da função <i>ex</i> em VHDL.	113
Código 5 Aproximação numérica para cálculo da função <i>ln(x)</i> em VHDL.	114
Código 6 Código VHDL usado como interface para a realização de testes da ADON-HD.	116
Código 7 Configuração da ferramenta <i>hls4ml</i>	125

LISTA DE SIGLAS, ACRÔNIMOS E ABREVIATURAS

ADON	Árvores de Decisão Orientada a Notificações (<i>Notification-Oriented Decision Tree Architecture</i>)
ANN	Rede Neural Artificial (<i>Artificial Neural Network</i>)
ASIC	<i>Application-Specific Integrated Circuit</i> (Circuitos Integrado de aplicação específica)
ARQPON	Arquitetura de Computador para o Paradigma Orientado a Notificações (<i>Notification-Oriented Computer Architecture</i>)
ARQPONSIM	Simulador da Arquitetura de Computador para o Paradigma Orientado a Notificações (<i>Notification-Oriented Computer Architecture Simulator</i>)
ARQTOTALPON	Arquitetura de Computador Totalmente Voltada para o Paradigma Orientado a Notificações (<i>Total Notification-Oriented Computer Architecture</i>)
ARQTOTALPONSIM	Simulador da Arquitetura de Computador Totalmente Voltada para o Paradigma Orientado a Notificações (<i>Total Notification-Oriented Computer Architecture Simulator</i>)
CART	<i>Classification And Regression Tree</i> (Árvore de Classificação e Regressão)
CHAID	<i>Chi-squared Automatic Interaction Detection</i> (Detecção Automática de Interação por Qui-Quadrado)
CRF	<i>Compact Random Forests</i> (Florestas Aleatórias Compactas)
CTA	Controle de Tráfego de Automóveis (<i>Car Traffic Control</i>)
CON	Controle Orientado a Notificações (<i>Notification Oriented Control</i>)
COPON	Coprocessador PON (<i>NOP Coprocessor</i>)
DON	Desenvolvimento Orientado a Notificações (<i>Notification-Driven Development</i>)
FBE	<i>Fact Base Element</i> (Elemento da Base de Fatos)
FF	<i>Flip-Flops</i> (Registradores)
FPGA	<i>Field Programmable Gate Array</i> (Matriz de Portas

	Programáveis em Campo)
GB	<i>Gradient Boosting</i> (Incremento Gradiente)
GPU	<i>Graphics Processing Unit</i> (Unidade de Processamento Gráfico)
HLS	<i>High-Level Synthesis</i> (Síntese em alto nível)
ID3	<i>Iterative Dichotomiser 3</i> (Iterador Dicotomizador 3)
IDE	<i>Integrated Development Environment</i> (Ambiente de Desenvolvimento Integrado)
ION	Inferência Orientada a Notificações (<i>Notification Oriented Inference</i>)
IoT	<i>Internet of Things</i> (Internet das Coisas)
IP	<i>Internet Protocol</i> (Protocolo de Internet)
LightGBM	<i>Light Gradient Boosting Machine</i> (Máquina de Incremento Gradiente Leve)
LingPON	Linguagem do Paradigma Orientado a Notificações (<i>Notification-Oriented Paradigm Language</i>)
LingPON-HD	Linguagem do Paradigma Orientado a Notificações – Hardware Digital (<i>Notification-Oriented Paradigm Language – Digital Hardware</i>)
LUT	<i>Look-Up Tables</i> (Tabelas de Consulta)
MCPON	Método de Compilação para o PON (<i>NOP Compilation Method</i>)
NOCA	<i>Notification-Oriented Computer Architecture</i> (Arquitetura de Processador para o Paradigma Orientado a Notificações)
NOCASIM	<i>Notification-Oriented Computer Architecture Simulator</i> (Simulador da Arquitetura de Processador para o Paradigma Orientado a Notificações)
NOM	<i>Notification Oriented Software Design Methodology</i> (Metodologia de Projeto para o Paradigma Orientado a Notificações)
NOP	<i>Notification-Oriented Paradigm</i> (Paradigma Orientado a Notificações)

NOP-DH	<i>Notification-Oriented Paradigm – Digital Hardware</i> (Paradigma Orientado a Notificações – Hardware Digital)
NOPL	<i>Notification-Oriented Paradigm Language</i> (Linguagem do Paradigma Orientado a Notificações)
PD	Paradigma Declarativo (<i>Declarative Paradigm</i>)
PF	Paradigma Funcional (<i>Functional Paradigm</i>)
PI	Paradigma Imperativo (<i>Imperative Paradigm</i>)
PL	Paradigma Lógico (<i>Logical Paradigm</i>)
POD	Paradigma Orientado a Dados (<i>Data Oriented Paradigm</i>)
POE	Paradigma Orientado a Eventos (<i>Event-Driven Paradigm</i>)
PON	Paradigma Orientado a Notificações (<i>Notification Oriented Paradigm</i>)
POO	Programação Orientada a Objetos (<i>Object Oriented Programming</i>)
POR	Paradigma Orientado a Regras (<i>Rule Oriented Paradigm</i>)
PP	Paradigma Procedimental (<i>Procedural Paradigm</i>)
RAM	<i>Random Access Memory</i> (Memória de acesso aleatório)
RF	<i>Random Forest</i> (Floresta Aleatória)
ROM	<i>Read Only Memory</i> (Memória de Acesso Aleatório)
SBR	Sistema Baseado em Regras (<i>Rule Based Systems</i>)
SVM	<i>Support Vector Machine</i> (Máquina de Suporte Vetorial)
SYSML	<i>Systems Modeling Language</i> (Linguagem de modelagem de sistemas)
TCP	<i>Transmission Control Protocol</i> (Protocolo de Controle de Transmissão)
TOTALNOCA	<i>Total Notification-Oriented Computer Architecture</i> (Arquitetura de Computador Totalmente Voltada para o Paradigma Orientado a Notificações)
TOTALNOCASIM	<i>Simulator for Total Notification-Oriented Computer Architecture</i> (Simulador da Arquitetura de Computador Totalmente Voltada para o Paradigma Orientado a Notificações)
ULA	Unidade Lógica e Aritmética (<i>Arithmetic Logic Unit</i>)

UML	<i>Unified Modeling Language</i> (Linguagem de Modelagem Unificada)
VHDL	<i>VHSIC Hardware Description Language</i> (Linguagem de Descrição de Hardware VHSIC)
VHSIC	<i>Very-High-Speed Integrated Circuit</i> (Circuito Integrado de Velocidade Muito Alta)
XGBoost	<i>Extreme Gradient Boosting</i> (Incremento de Gradiente Extremo)

SUMÁRIO

1. INTRODUÇÃO	17
1.1. Contextualização	19
1.2. Motivação	22
1.3. Justificativa	26
1.4. Objetivos	28
1.4.1. Objetivo geral.....	28
1.4.2. Objetivos específicos	28
1.5. Organização do documento	29
2. FUNDAMENTAÇÃO TEÓRICA	31
2.1. Paradigma Orientado a Notificações - PON.....	31
2.1.1. Materializações do PON	36
2.1.2. Propriedades elementares e conceitos do PON	41
2.1.3. PON em hardware	44
2.2. Árvores de decisão	52
2.2.1. Introdução às árvores de decisão	52
2.2.2. Modelos de árvores de decisão	57
2.2.3. Implementação de árvores de decisão em <i>hardware</i>	66
2.3. Conclusão do capítulo	76
3. ARQUITETURA PARA ÁRVORES DE DECISÃO ORIENTADA A NOTIFICAÇÕES (ADON).....	79
3.1. Requisitos	80
3.2. Estruturação da arquitetura ADON	81
3.2.1. Conformação das avaliações lógico-causais ao PON	83
3.2.2. Modelagem da ADON.....	97
3.3. Ferramentas para treinamento e mapeamento	101
3.4. Mapeamento da ADON ao PON-HD.....	108
3.4.1. Melhorias no PON-HD	108
3.4.2. Implementação dos métodos de votação	111
3.4.3. Testes da ADON-HD	115
3.5. Considerações finais.....	119
4. EXPERIMENTOS E RESULTADOS - ADON em HD	122
4.1. Materiais e métodos.....	122
4.1.1. Recursos utilizados.....	122
4.1.2. Método aplicado	123

4.1.3. Configuração do modelo.....	124
4.1.4. Processo de síntese com a ferramenta estado da arte	124
4.2. Casos de estudo	126
4.3. Resultados e discussões	130
4.3.1. Resultados ADON-HD preliminar	130
4.3.2. Resultados com <i>Random Forest</i>	133
4.3.3. Resultados com Gradient Boosting	141
4.3.4. Resultados suplementares sobre eficiência energética	150
4.4. Considerações finais.....	153
5. CONCLUSÃO E TRABALHOS FUTUROS	156
5.1. Principais contribuições	157
5.2. Trabalhos futuros	163
6. REFERÊNCIAS	168
APÊNDICE A.....	187

1. INTRODUÇÃO

As inovações em sistemas computacionais, em relação a *hardware* e *software*, permitiram uma evolução significativa no campo da assim chamada inteligência artificial, especialmente no dito aprendizado de máquina ou *machine learning* em inglês (REUTHER *et al.*, 2020). Em tal contexto, o aumento da complexidade dos modelos de *machine learning*, associados a uma demanda por processamento mais eficiente em termos de menor consumo de energia e alto desempenho, tem motivado diversas pesquisas e o desenvolvimento de soluções especializadas para suportar tais algoritmos diretamente em *hardware* (CAPOGROSSO *et al.*, 2024).

Neste cenário acima dado, técnicas de aceleração baseadas em Unidade de Processamento Gráfico (GPUs - *Graphics Processing Unit*), Matriz de Portas Programáveis em Campo (FPGAs - *Field Programmable Gate Arrays*) e Circuitos Integrados de Aplicação Específica (ASICs - *Application-Specific Integrated Circuit*) vêm sendo empregadas para aprimorar o processo de treinamento e o processo de inferência/predição de diferentes modelos de *machine learning*. Exemplos de tais modelos são Máquina de Suporte Vetorial (SVM - *Support Vector Machine*), Redes Neurais Profundas (*Deep Neural Networks*), Redes Neurais Convolucionais (*Convolutional Neural Networks*), Redes Neurais Recorrentes (*Recurrent Neural Networks*), Transformadores ou modelos baseados em atenção (*Transformers*) e modelos baseados em Árvores de Decisão, incluindo Floresta Aleatória (RF - *Random Forest*) e Incremento de Gradiente (GB - *Gradient Boosting*; REUTHER *et al.*, 2020; SAIDI *et al.*, 2021; NECHI *et al.*, 2023).

No contexto de árvores de decisão, mais precisamente, pesquisas exploraram a implementação de seus processos de treinamento e execução em *hardware*, podendo ser destacados GPUs e FPGAs (VAN ESSEN *et al.*, 2012; BARBARESCHI; BARONE; MAZZOCCA, 2021). Embora o uso de árvores de decisão tradicionais não exija tantos recursos computacionais e, portanto, não sejam tipicamente consideradas boas candidatas para aceleração em *hardware*, algoritmos de grupo ou conjunto (*ensemble*), como *Random Forest* e *Gradient Boosting*, podem se beneficiar de um ambiente altamente paralelizável, como GPUs e particularmente, FPGAs (VAN ESSEN *et al.*, 2012). No entanto, ao implementar esses modelos em FPGAs, o aumento no número de árvores requer mais recursos

lógicos, sendo que rapidamente o limite de capacidade do dispositivo pode ser atingido. Assim, neste contexto dado, economizar o uso de tais recursos seria imperativo, tal qual o uso do paralelismo possível e disponível (BARBARESCHI; BARONE; MAZZOCCA, 2021).

Justamente, o Paradigma Orientado a Notificações (PON), desenvolvido por pesquisadores da UTFPR, surge como uma abordagem alternativa para a concepção enxuta de sistemas computacionais que permite o desacoplamento do processamento lógico-causal, tanto em *software* quanto em *hardware* (SIMÃO; STADZISZ, 2002; SIMÃO; STADZISZ, 2008; SIMÃO; STADZISZ, 2009; SIMÃO; TACLA; STADZISZ, 2009; SIMÃO *et al.*, 2012b; KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; LINHARES *et al.*, 2020; PORDEUS *et al.*, 2021; PORDEUS *et al.*, 2023). Em resumo, o PON organiza o sistema computacional em entidades facto-execucionais e entidades lógico-causais, as quais são desacopladas entre si graças a colaborações precisas por notificações. Com isso, quando a plataforma computacional permite, é possível alcançar paralelismo e/ou distribuição de forma fina e granular (SIMÃO; STADZISZ, 2008; BANASZEWSKI, 2009; SIMÃO *et al.*, 2016; BELMONTE *et al.*, 2016; PORDEUS, 2017; NEGRINI, 2019). Neste sentido, o PON é particularmente pertinente quando aplicado na composição de sistemas em dispositivos FPGAs (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; KERSCHBAUMER, 2018; SCHÜTZ, 2019).

Com base em todo o contexto anteriormente apresentado, este presente capítulo apresenta as considerações introdutórias a respeito deste trabalho de doutorado, que está naturalmente inserido em um projeto de pesquisa sobre PON, particularmente no tocante ao PON em *Hardware* Digital (PON-HD) via FPGA. Mais especificamente ainda, este trabalho diz respeito à pesquisa chamada de Proposição de uma Arquitetura para Árvores de Decisão Orientada a Notificações (ADON) implementada em conjunto com os conceitos oriundos do PON-HD, que é a essência da pesquisa desta presente tese de doutorado.

Isto tudo dito, esses assuntos acima aventados são mais efetivamente introduzidos nas seções que seguem deste corrente capítulo. Na Seção 1.1 é apresentada a contextualização do tema de pesquisa, incluindo e mesmo destacando o PON. Além disso, a Seção 1.2 apresenta as motivações deste trabalho, enquanto a Seção 1.3 apresenta a sua respectiva justificativa. Por sua vez,

a Seção 1.4 apresenta os objetivos geral e específicos da pesquisa em questão. Por fim, a Seção 1.5 apresenta a organização dos capítulos subsequentes deste documento de tese.

1.1. Contextualização

A aplicação de algoritmos de aprendizado de máquina (*machine learning*), uma subárea da inteligência artificial, tem se tornado cada vez mais relevante em diversas áreas do conhecimento. Estes algoritmos permitem a tomada de decisões por meio da identificação de padrões ‘aprendidos’ a partir da análise de conjuntos de dados relacionados a um tema específico. Exemplos mais usuais de tais algoritmos são modelos de regressão linear (*i.e.*, previsão de valores contínuos) e logística (*i.e.*, classificação binária), algoritmos de agrupamento como *k-means*, algoritmos como SVM, redes neurais e árvores de decisão (RUSSELL; NORVIG, 2021; SARKER, 2021; BERTOLINI *et al.*, 2021). Entre os algoritmos que se destacam nesse contexto, por oferecerem soluções eficazes para tarefas de classificação (*i.e.*, identificação de categorias) e regressão (*i.e.*, previsão de valores contínuos), estão as Árvores de Decisão e seus derivados, como os modelos de conjunto ou *ensemble*. Exemplos destacados de *ensemble* são *Random Forest* e o *Gradient Boosting* que, em suma, servem para aumentar a precisão preditiva e reduzir o risco de sobreajuste (*overfitting*), combinando múltiplos modelos simples para produzir uma solução mais robusta (BENTÉJAC; CSÖRGŐ; MARTÍNEZ-MUÑOZ, 2021; SARKER, 2021; MIENYE; SUN, 2022; COSTA, PEDREIRA, 2023; MIENYE; JERE, 2024). Em tempo, o *overfitting* ou sobreajuste (em tradução direta), ocorre quando há uma adaptação excessiva do modelo aos dados de treinamento, que reduz sua capacidade de funcionar bem com novos dados, prejudicando sua generalização (MIENYE; SUN, 2022)

Entretanto, via de regra, todos esses algoritmos, de *ensemble* inclusive, exigem cada vez mais recursos computacionais, especialmente à medida que os modelos e problemas a serem resolvidos se tornam mais complexos, bem como o volume de dados necessários para o aprendizado aumenta (MOHAMMADI *et al.*, 2018; REUTHER *et al.*, 2020; ZAMAN *et al.*, 2022; WANG *et al.*, 2022). Como alternativa, tecnologias de *hardware* como GPUs e FPGAs têm sido aplicadas para acelerar tanto os processos de treinamento ou aprendizado, quanto os processos de

tomada de decisões desses algoritmos (REUTHER *et al.*, 2020; SAIDI *et al.*, 2021; ZAMAN *et al.*, 2022; WANG *et al.*, 2022; CAPOGROSSO *et al.*, 2024). Entre os algoritmos que estão nesse contexto de aceleração por *hardware*, há naturalmente também as Árvores de Decisão e seus derivados, salientando aqui novamente os modelos de *ensemble*, particularmente as já citadas Árvores de Decisão do tipo *Random Forest* e o *Gradient Boosting* (VAN ESSEN *et al.*, 2012; SUMMERS *et al.*, 2020; MOLINA *et al.*, 2021; BARBARESCHI; BARONE; MAZZOCCA, 2021; SAIDI *et al.*, 2021; SHAH *et al.*, 2022; DINH *et al.*, 2023).

As Árvores de Decisão, em suma, são algoritmos de *machine learning* que podem ser utilizados para uma série de aplicações com os objetivos de classificação ou regressão, como aplicações para domínio financeiro (NARARETH; REDDY, 2023), segurança cibernética (KAUR; GABRIJELČIČ; KLOBUČAR, 2023), diagnósticos médicos (KUMAR *et al.*, 2023), previsão de demanda em sistemas de logística (JAHANI; JAIN; IVANOV, 2023) e análise em sistemas de IoT (SOUZA *et al.*, 2022). Como referência, o algoritmo *Random Forest* consiste em uma série de árvores de decisão treinadas individualmente e de forma aleatória, que por sua vez, são combinadas para se obter um resultado com menor erro. Em seguida, cada árvore neste conjunto é avaliada separadamente, permitindo a execução paralela caso a plataforma suporte tal paralelismo de processamento. Ao final, os resultados são combinados para retornar o valor da previsão dado pela combinação de todo o conjunto de árvores (CRIMINISI; SHOTTON; KONUKOGLU, 2011).

Como as árvores de decisão que compõem estes algoritmos são essencialmente compostas por expressões condicionais na clássica forma 'se-então' (*i.e.*, avaliações lógico-causais) em cada uma de suas árvores, estes algoritmos apresentam simplicidade no treinamento e na classificação em relação a outros métodos de *machine learning*, como redes neurais, que são classicamente baseadas em cálculos matriciais (HAYKIN, 2009; RUSSELL; NORVIG, 2021). Além disso, o desacoplamento das árvores nos algoritmos do tipo *ensemble*, aqui destacando o *Random Forest*, permite o paralelismo nas árvores em si, além do paralelismo entre elas, pois cada árvore contém muitas avaliações lógico-causais independentes que também podem ser executadas em paralelo, desde que com as soluções apropriadas em ambientes propícios (VAN ESSEN *et al.*, 2012).

Devido a tal simplicidade de execução, bom desempenho em si e considerável possibilidade de paralelismo, o desempenho de algoritmos como

Random Forest e *Gradient Boosting* pode ser particularmente aprimorado por abordagens de execução em *hardware* (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; KERSCHBAUMER, 2018; SCHÜTZ, 2019; PORDEUS *et al.*, 2021; SAIDI *et al.*, 2021), (SOZZO *et al.*, 2022). Entretanto, geralmente soluções em *hardware* têm uma tendência a demandar capacidade técnica de programação e desenvolvimento em geral de baixo nível, como acontece no caso das soluções para desenvolvimento em *FPGAs*, o que se agrava quando se almeja execução paralela granular em escala. Uma alternativa seria usar ferramentas de programação em alto nível para *FPGA* que, entretanto, usam paradigmas de programação oriundos do processamento sequencial e, portanto, não focam naturalmente no uso do paralelismo (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; KERSCHBAUMER, 2018; SCHÜTZ, 2019; PORDEUS *et al.*, 2021; MOLINA *et al.*, 2021; SOZZO *et al.*, 2022).

Neste contexto, o assim chamado Paradigma Orientado a Notificações (PON) oferece novas possibilidades, pois sua estrutura permite o processamento lógico-causal desacoplado e otimiza a execução paralela de maneira mais eficaz do que as abordagens tradicionais (SIMÃO; STADZISZ, 2008; SIMÃO; STADZISZ, 2009; LINHARES, 2015; KERSCHBAUMER, 2018; SCHÜTZ, 2019; PORDEUS, 2017; RONSZCKA, 2019). Desde sua origem, como uma solução de controle discreto para sistemas de manufatura inteligente, o PON foi expandido para um paradigma geral de desenvolvimento e execução de sistemas computacionais (SIMÃO, 2005; LINHARES, 2015; KERSCHBAUMER, 2018; SCHÜTZ, 2019; RONSZCKA, 2019; MENDONÇA, 2020).

Inicialmente aplicado ao *software* e depois ao *hardware*, o PON acabou se tornando um tema de pesquisa em várias áreas. Há pesquisas e protótipos pertinentes ao PON envolvendo *softwares* como CRUD, jogos (SIMÃO *et al.*, 2012c; RONSZCKA *et al.*, 2015), futebol de robôs (SANTOS; SIMÃO; FABRO, 2017), simulação de controle de semáforos de trânsito (NEGRINI *et al.*, 2020; NEVES, 2021; OSHIRO, 2021), sistemas inteligentes de manufatura (SIMÃO; STADZISZ, 2009; LUCCA *et al.*, 2009) (SIMÃO; TACLA; STADZISZ, 2009), controle nebuloso ou *fuzzy* (MELO, 2016), redes neurais em *software* e *hardware* (SCHÜTZ *et al.*, 2018; SCHÜTZ, 2019), Internet das Coisas (IoT) em *software* distribuído (OLIVEIRA *et al.*, 2018; FIGUEIREDO; SIMÃO; VENDRAMIN, 2022), controle de robôs em *hardware* (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b) e

ordenação via *Bitonic Sort* em *hardware* (PORDEUS *et al.*, 2021), dentre outros. Tais pesquisa têm como cerne um ou mais itens como arquétipos ou *frameworks* em *software* e *hardware* (RONSZCKA, 2012; KERSCHBAUMER, 2018), arquitetura de computadores (LINHARES, 2015; PORDEUS, 2017; LINHARES *et al.*, 2020), sistema para compor linguagens e compiladores em PON chamado de Tecnologia LingPON (RONSZCKA, 2019; OSHIRO, 2021) e soluções próprias de engenharia de *software/sistema* (WIECHETECK, 2011; NOVAES, 2019; MENDONÇA, 2020).

1.2. Motivação

A aceleração de algoritmos de *machine learning* pode ser realizada por diferentes plataformas de *hardware*, tais como as plataformas acima contextualizadas GPUs e FPGAs. Entre as opções citadas, as GPUs apresentam uma arquitetura focada na realização de cálculos matriciais de forma paralela, característica que viabiliza a sua adoção em tarefas de *machine learning*, com destaque a redes neurais (REUTHER *et al.*, 2020). Por outro lado, as FPGAs são dispositivos compostos por blocos lógicos programáveis que podem ser reconfigurados com o objetivo de implementar circuitos digitais especializados conforme a necessidade de cada aplicação (SAIDI *et al.*, 2021; ZAMAN *et al.*, 2022; CAPOGROSSO *et al.*, 2024).

Neste contexto, em sistemas embarcados, sistemas de tempo real ou sistemas de alto desempenho, o consumo de energia e a baixa latência são fatores críticos. Nestes sistemas, as FPGAs apresentam vantagens em comparação com GPUs devido a implementações de *hardware* customizadas e otimizadas para cada aplicação (SAIDI *et al.*, 2021; ZAMAN *et al.*, 2022; CAPOGROSSO *et al.*, 2024). Essas vantagens tornam-se relevantes em aplicações que envolvem a aceleração de árvores de decisão e seus derivados do tipo *ensemble*, pois a flexibilidade ao conceber arquiteturas personalizadas possibilitaria um melhor aproveitamento dos recursos de *hardware*, contribuindo para um desempenho mais eficiente destas arquiteturas (NARAYANAN *et al.*, 2007; VAN ESSEN *et al.*, 2012). Justamente, no âmbito de correto aproveitamento de *hardware*, uma solução que se faz pertinente é o chamado Paradigma Orientado a Notificações (PON).

O PON apresenta uma nova abordagem para o processamento lógico-causal e facto-execucional, baseada em entidades notificadoras colaborativas

mínimas e precisas. Em geral, essa abordagem é mais eficiente e desacoplada em comparação com os paradigmas tradicionais de programação. Tais paradigmas são acoplados e sequenciais por definição e origem, cujas linguagens têm sido usadas para o desenvolvimento de *software* e mesmo para o desenvolvimento de *hardware* quando são utilizadas ferramentas de Síntese de Alto Nível (HLS – *High Level Synthesis*; AWAD, 2009; COMPTON; HAUCK, 2002; LIU *et al.*, 2019; NANE *et al.*, 2016; WINDH *et al.*, 2015; CANIS *et al.*, 2013; MEEUS *et al.*, 2012; HUANG *et al.*, 2020; LAHTI *et al.*, 2019; MOLINA *et al.*, 2021; SOZZO *et al.*, 2022).

No caso particular em *Hardware Digital*, ao contrário das abordagens tradicionais de HLS, o PON permite explorar implicitamente ambientes de execução paralelos ou distribuídos, o que já foi testado em FPGAs, de maneira mais simples e eficiente. Isso ocorre devido à sua capacidade de evitar redundâncias e, principalmente, ao desacoplamento natural das diferentes partes mínimas do programa (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021). Além disso, o PON oferece um desenvolvimento mais fácil do que as abordagens tradicionais baseadas em linguagens de descrição de *hardware* e ferramentas HLS, por permitir abordagens declarativas de alto nível orientadas por regras. Ainda assim, mesmo em mais alto nível, ele alcança melhor desempenho por meio de menor número de ciclo de *clocks*, menor frequência máxima de operação e tamanho de circuito similar às abordagens tradicionais (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

O PON alcança esses resultados em *hardware* ao combinar suas características de alto nível de abstração, ausência de redundância na avaliação lógica-causal e desacoplamento intrínseco. Em termos experimentais, já é sabido que efetivamente essas características são úteis para melhorar o desempenho e são vantajosas para o processamento paralelo em aplicações utilizando dispositivos FPGA segundo um conjunto de trabalhos prévios (LINHARES, 2015; PORDEUS *et al.*, 2016; KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; LINHARES *et al.*, 2020; PORDEUS *et al.*, 2021; PORDEUS *et al.*, 2023). Essa nova abordagem de desenvolvimento de *hardware* baseada no PON é chamada de PON para *Hardware Digital* (PON-HD) (PORDEUS *et al.*, 2016; KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a;

KERSCHBAUMER *et al.*, 2018b; LINHARES *et al.*, 2020; PORDEUS *et al.*, 2021; PORDEUS *et al.*, 2023).

O PON-HD explora de forma notável a propriedade de paralelismo inerente ao PON, devido às características específicas da plataforma de execução em *hardware*. Isso ocorre porque, no contexto de *hardware*, o PON, através da abordagem PON-HD, permite mapear uma aplicação PON na forma de circuitos especializados, que se beneficiam do conceito de entidades mínimas e especializadas. Estes circuitos são gerados diretamente a partir do mapeamento dos elementos do modelo PON para componentes específicos baseados em linguagens de descrição de *hardware*, como VHDL (*VHSIC “Very-High-Speed Integrated Circuit” Hardware Description Language*) (LINHARES, 2015; PORDEUS *et al.*, 2016; KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; LINHARES *et al.*, 2020; PORDEUS *et al.*, 2021; PORDEUS *et al.*, 2023).

Quando aplicado o PON-HD em conjunto com a brevemente supracitada Tecnologia LingPON, há uma maior facilidade de desenvolvimento em relação a abordagens tradicionais baseadas em linguagens de descrição de *hardware*, como VHDL e Verilog, e mesmo ferramentas de síntese em alto nível (p. ex. *Bluespec*, *LegUp*, *OpenCL*, *VivadoHLS*, entre outros (INGGS *et al.*, 2014; NANE *et al.*, 2016)). Ainda, o PON-HD proporciona, por si só, outras facilidades, como desempenho, frequência máxima de operação e tamanho de circuito semelhantes às abordagens tradicionais (PORDEUS *et al.*, 2016; KERSCHBAUMER *et al.*, 2015; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; KERSCHBAUMER, 2018; PORDEUS *et al.*, 2021; PORDEUS *et al.*, 2023).

Uma vez que o PON-HD é uma alternativa para a execução performática e/ou paralela de sistemas baseados em cálculo lógico-causal, conforme aventado em trabalhos como (SIMÃO; STADZISZ, 2008; SIMÃO; STADZISZ, 2009) e indicado em resultados de outras pesquisas do grupo do PON já acima citados, como (LINHARES, 2015; KERSCHBAUMER, 2018; SCHÜTZ, 2019; PORDEUS, 2017; RONSZCKA, 2019), a sua conformação aos algoritmos de árvores de decisão poderia trazer benefícios em termos de desempenho e eficiência. Eis que a organização de avaliações lógico-causais presente nos algoritmos de árvores de decisão, os tornaria especialmente aptos para se beneficiarem do PON, em particular do PON-HD. Além disso, as avaliações lógico-causais independentes nos algoritmos *ensemble* não só são adequadas para aproveitar a abordagem do PON-

HD, mas também particularmente para testá-lo enquanto promessa de uma solução computacional performante e altamente paralelizável, conforme explicado a seguir.

Justamente, neste contexto, o *Random Forest* e o *Gradient Boosting*, devido às suas estruturas de múltiplas árvores de decisão que operam de maneira independente, fornecem um cenário ideal para examinar como o PON poderia melhorar o paralelismo e reduzir a latência de execução, particularmente em termos de paralelismo em nível granular, conforme já argumentava Simão e Stadzisz para sistemas em geral em (SIMÃO; STADZISZ, 2009) e começavam a demonstrar e confirmar pesquisadores como (PETERS, 2012; LINHARES, 2015; PORDEUS *et al.*, 2016; KERSCHBAUMER, 2018; PORDEUS *et al.*, 2021; PORDEUS *et al.*, 2023) em aplicações de prova de conceito. Através da implementação e testes dos algoritmos *Random Forest* e *Gradient Boosting* em FPGA, por meio de uma solução própria, seria possível, de forma sinérgica e efetiva, validar em escala se os princípios do PON realmente oferecem vantagens significativas em termos de redução de redundâncias e, portanto, paralelismo granular, com o decorrente aumento da eficiência computacional quando comparado aos métodos tradicionais de execução em FPGA.

Considerando todos esses pontos, é pertinente explorar o modelo de execução de árvores de decisão, particularmente os algoritmos *Random Forest* e *Gradient Boosting*, utilizando os conceitos do PON em *hardware*, ainda que naturalmente eles apenas não bastariam, necessitando, portanto, de invenção e engenho próprios para tal. Naturalmente, o domínio de Árvores de Decisão tem suas particularidades e idiossincrasias, como alto encadeamento de e inter-relacionamento entre avaliações lógico-causais (cf. já apontava Rokach (2016)), o que faria necessário mapear seus princípios para os conceitos da orientação a notificações, bem como expandi-los conforme tais idiossincrasias demandem. Neste âmbito, este trabalho diz respeito à contribuição para o desenvolvimento de uma nova arquitetura para a execução de árvores de decisão conforme as propriedades do PON.

De prima, como requisito básico, esta arquitetura deveria incorporar as propriedades elementares do PON, eliminando redundâncias estruturais presentes nas implementações convencionais de árvores de decisão e permitindo a execução paralela conforme a natureza do algoritmo. Na verdade, à luz de algum método e mapeamento vislumbrados, tal arquitetura deveria conformar os princípios do PON

ao domínio de Árvores de Decisão, expandindo-os conforme necessário, de forma a atender cada peculiaridade do domínio visado.

1.3. Justificativa

A execução do algoritmo de árvores de decisão e seus derivados, como os *ensemble Random Forest* e *Gradient Boosting*, em dispositivos de lógica reconfigurável, nomeadamente FPGAs, tem sido objeto de diversas pesquisas recentes devido ao seu potencial de aceleração em *hardware* (SUMMERS *et al.*, 2020; BARBARESCHI; BARONE; MAZZOCCA, 2021; MOLINA *et al.*, 2021; SAIDI *et al.*, 2021; SHAH *et al.*, 2022; DINH *et al.*, 2023; PORDEUS *et al.*, 2023). No entanto, o tamanho limitado dessas plataformas restringe a quantidade de árvores de decisão que podem ser alocadas, comprometendo a eficiência e o desempenho do algoritmo (DAMIANI *et al.*, 2022).

Nesse contexto, o uso dos algoritmos *Random Forest* e *Gradient Boosting* se destacam por sua composição específica, sendo um conjunto de árvores de decisão que contém uma grande quantidade de expressões condicionais 'se-então' em cada uma de suas árvores. O fato dessas expressões supostamente poderem ser totalmente desacopladas e executadas em paralelo favoreceriam a paralelização e a distribuição destes algoritmos, tanto quanto a solução de desenvolvimento utilizada e a plataforma de *hardware* visada permitisse. Entretanto, esta organização de avaliações lógico-causais, presente nos algoritmos de árvores de decisão, frequentemente apresenta redundâncias estruturais e temporais, que resultam em ineficiência em relação ao tempo de execução e ao tamanho do *hardware* a ser sintetizado. Assim, este conjunto de características, torna este conjunto de algoritmos ímpar para se beneficiar dos conceitos e propriedades do PON, caso seja elaborado uma solução própria e harmônica para tal.

Nos algoritmos de árvores de decisão, particularmente, as redundâncias estruturais podem ser observadas pela repetição desnecessária de comparações e decisões idênticas em diferentes pontos da árvore, gerando uma duplicidade de cálculos. Por sua vez, as redundâncias temporais ocorrem quando o mesmo conjunto de avaliações é repetido ao longo do tempo, sem que haja uma variação significativa no estado das variáveis do sistema, resultando em processamento de condições de forma repetida e sem ganhos efetivos no resultado. Estas

redundâncias todas e os problemas delas decorrentes poderiam ser evitados com a ajuda de uma solução que seja oriunda dos princípios notificante-desacoplantes que norteiam o PON, o que seria potencializado na execução altamente paralela em *hardware* viabilizadas pelos mesmíssimos princípios.

Ao ensejo, eis que a combinação de princípios do PON em sinergia para com as características de FPGAs, no assim chamado PON-HD, surge como uma alternativa promissora de caminho para se alcançar uma solução arquitetural distinta de execução dos algoritmos derivados de árvores de decisão em *hardware*, visando proporcionar um melhor uso, dos recursos disponíveis. Em tese, essa abordagem permitiria propor uma arquitetura distinta e única para a execução de árvores de decisão, explorando, conformando e também expandindo as propriedades e os princípio presentes no PON em geral e no PON-HD em particular. Efetivamente, faz-se necessário criar uma solução distinta, mais precisamente uma arquitetura inédita, que permita conformar aqueles princípios citados de forma harmônica e transparente para o domínio de Árvores de Decisão, em particular para algoritmo *ensemble* e derivados, salientando *Random Forest* e *Gradient Boosting*.

Portanto, a pesquisa apresentada nesse trabalho de doutorado visa propor e avaliar uma solução arquitetural, de antemão batizada como Arquitetura para Árvores de Decisão Orientada a Notificações (ADON), a qual é justificada pela necessidade de superar as limitações atuais na execução de árvores de decisão em FPGAs e, particularmente, pela saliente sinergia observável entre princípios do PON-HD e as necessidades de execução de árvores de decisão. Assim, por meio da ADON, esta pesquisa busca aprimorar, ou mesmo otimizar, o uso de recursos e acentuadamente melhorar a eficiência e o desempenho dos algoritmos derivados de árvores de decisão em *hardware*, levando em consideração as limitações de capacidade das FPGAs.

Dito de outra forma, em uma visão mais generalista, vislumbra-se que a solução ADON possa trazer avanços significativos na eficiência do uso de recursos computacionais e no desempenho geral de aplicações que demandam alta performance no tocante às Árvores de Decisão, mas que possuem limitações quanto à quantidade de tais recursos disponíveis. Com esta presente justificativa apresentada, o objetivo geral e os objetivos específicos encontram-se descritos na próxima seção de forma cartesiana e precisa.

1.4. Objetivos

Esta seção apresenta os objetivos gerais e específicos deste trabalho de pesquisa, que constitui esta tese de doutorado.

1.4.1. Objetivo geral

O objetivo geral deste trabalho de doutorado constitui-se em:

Propor e avaliar uma arquitetura orientada a notificações distinta, de acrônimo ADON (Árvores de Decisão Orientada a Notificações), para a composição e execução de árvores de decisão agrupadas em conjuntos correlatos (*i.e.*, tipo *ensemble*), com destaque para o *Random Forest* e *Gradient Boosting*, a partir do mapeamento, conformação e ampliação de conceitos oriundos do PON (Paradigma Orientado a Notificações) em um encadeamento próprio, que permita a sua implementação em diversas materializações, em particular a sua aplicação direta em *hardware* digital, de acrônimo ADON-HD, na qual dever ser propriamente validada.

1.4.2. Objetivos específicos

Para atingir o objetivo geral nesta pesquisa de doutoramento, visa-se os seguintes objetivos específicos:

- Revisar os aspectos técnicos e teóricos a respeito do estado da arte no tocante às arquiteturas para execução de árvores de decisão agrupadas em conjuntos correlatos, com destaque para os algoritmos *Random Forest* e *Gradient Boosting* em *hardware*, para compreender e destacar a lacuna existente.
- Revisar os aspectos técnicos e teóricos fundamentais a respeito do PON e do PON-HD, avaliando suas propriedades e materializações, com o objetivo de guiar as decisões sobre quais de seus conceitos e demais elementos são essenciais para alcançar a ADON e a ADON-HD, bem como quais deles devem ser reinventados, rearticulados, simplificados e expandidos.

- Estabelecer a ADON via um processo de mapeamento entre conceitos de árvores de decisão e conceitos oriundos do PON, os rearticulando, conformando e expandindo, para a execução orientada a notificações em um encadeamento próprio de árvores do tipo *ensemble*, particularmente *Random Forest* e *Gradient Boosting*.
- Instanciar a ADON-HD via um processo de mapeamento entre conceitos da ADON e conceitos oriundos do PON-HD, os rearticulando, conformando e expandindo, para a execução orientada a notificações em FPGA via um encadeamento próprio de árvores do tipo *ensemble*, particularmente *Random Forest* e *Gradient Boosting*.
- Estabelecer para a ADON um módulo para tratamento de árvores de decisão treinadas (*Random Forest* e *Gradient Boosting*) a serem conformadas às entidades notificantes, o qual é implementado por meio de uma ferramenta que possa receber modelos treinados e gerar os códigos específicos para a ADON-HD.
- Aplicar conjuntos de *datasets* clássicos, variando suas configurações, para validar o modelo proposto com base nas métricas: números de elementos lógicos, frequência operacional máxima e a quantidade de ciclos de *clock* necessários para a execução de uma previsão, fazendo comparações da ADON-HD em relação a arquitetura do estado da arte em árvores de decisão do tipo *ensemble* (*Random Forest* e *Gradient Boosting*), nomeadamente *hls4ml*.

1.5. Organização do documento

Este documento de tese está organizado na forma de 5 capítulos. O presente capítulo 1 contextualizou o problema que levou à motivação e justificativa para a elaboração deste trabalho, bem como seus objetivos geral e específicos. O capítulo 2 aborda os principais conceitos que fundamentam teoricamente o desenvolvimento da arquitetura ADON

No Capítulo 3, é apresentada a proposta da arquitetura denominada ADON para a execução de árvores de decisão, em particular dos algoritmos *ensemble* utilizando o PON-HD. Além disso, são destacadas as melhorias nos componentes do PON-HD que permitem a implementação desse modelo. Este capítulo também explica em detalhes a construção das ferramentas necessárias para a avaliação da arquitetura proposta, abordando as especificações de *hardware* e a implementação do *Random Forest* e *Gradient Boosting* na ADON.

O capítulo 4 apresenta os casos de estudos utilizados nas análises comparativas, seguido dos resultados dos experimentos realizados e pelas discussões que confirmam a relevância da ADON como uma arquitetura eficaz no contexto de execução de árvores de decisão em *hardware* para dispositivos FPGA. Por fim, no capítulo 5, são apresentadas as conclusões deste trabalho de doutorado, além dos trabalhos futuros a serem realizados no âmbito do grupo de pesquisa do PON.

2. FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, são abordados os principais conceitos que fundamentam o desenvolvimento da arquitetura ADON.

Inicialmente, na Seção 2.1, apresenta-se o estado da arte do PON¹, destacando suas propriedades e os benefícios em termos de eficiência e desacoplamento em comparação aos paradigmas de programação tradicionais. Em seguida, ainda na mesma seção, são detalhadas as materializações do PON tanto em *software* quanto em *hardware*, demonstrando como essas implementações têm servido de base para a evolução e a aplicação do PON em diferentes contextos computacionais. Ainda na mesma Seção 2.1, a revisão detalha a materialização do PON em *hardware*, especificamente o PON-HD, *i.e.*, PON em *Hardware Digital*.

Na Seção 2.2, são explorados os conceitos de árvores de decisão pertinentes a este trabalho de pesquisa. É iniciada a seção com a introdução às árvores de decisão, onde são exemplificados os conceitos fundamentais deste modelo de *machine learning*. Em seguida é dado foco aos principais modelos e algoritmos utilizados, com ênfase nos modelos *Random Forest* e *Gradient Boosting*. Ao fim da Seção 2.2, é apresentada uma revisão dos principais trabalhos relacionados à implementação de árvores de decisão e seus derivados em *hardware*, com foco nos avanços relevantes para os objetivos desta tese. Por fim, a Seção 2.3 apresenta as conclusões deste capítulo.

2.1. Paradigma Orientado a Notificações - PON

A base do PON foi inicialmente proposta por Simão (2001, 2005) como uma solução de controle discreto para sistemas inteligentes de manufatura. Esta solução definiu uma forma de orquestrar o controle de manufatura por meio de colaboração por notificações precisas e pertinentes entre entidades relacionadas. Posteriormente, essa solução foi estendida e aplicada como uma solução genérica de inferência de *software*, evoluindo para um paradigma de programação e desenvolvimento e, finalmente, alcançando a forma de um paradigma de desenvolvimento de sistemas computacionais (BANASZEWSKI *et al.*, 2007; SIMÃO;

¹ A leitura desta seção, seja na íntegra ou em partes, poderia ser dispensável para o leitor conhecedor do assunto tratado. Isto salvo se o leitor conhecedor tiver interesse em revisar os assuntos abordados, seja em parte, seja na íntegra.

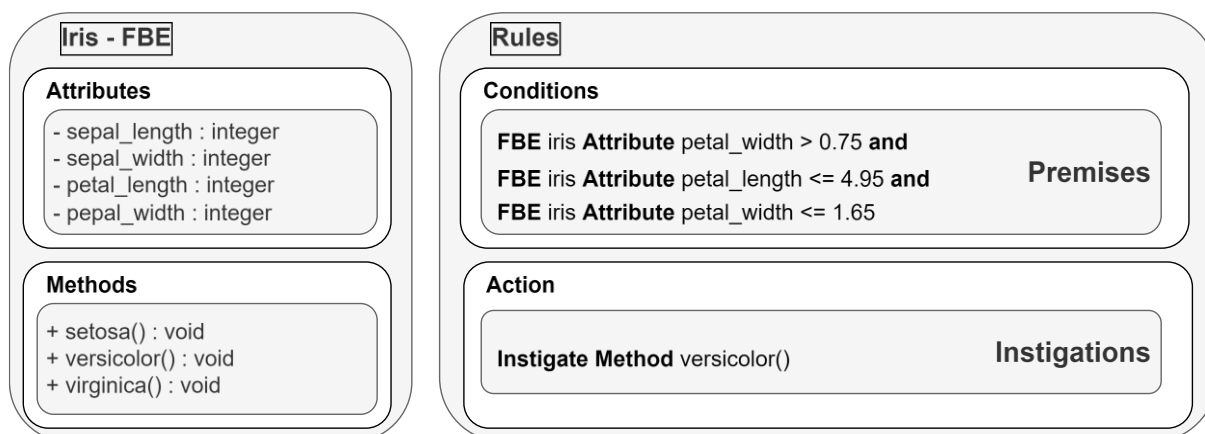
STADZISZ, 2002; SIMÃO; FABRO; STADZISZ, 2003; SIMÃO; STADZISZ, 2008; SIMÃO; STADZISZ, 2009; SIMÃO *et al.*, 2012a; PORDEUS, 2017).

O PON foi proposto como um novo paradigma de desenvolvimento, primeiramente de *software*. Este paradigma oferece um nível apropriado de abstração, proporcionando melhor desempenho e facilidades para o paralelismo e a distribuição, em comparação com sistemas baseados em paradigmas tradicionais, como a programação advinda do Paradigma Orientado a Objetos - Imperativo (POO/PI) e os Sistemas Baseados em Regras (SBR) do Paradigma Lógico-Declarativo (PL/PD) (SIMÃO; STADZISZ, 2008; SIMÃO; STADZISZ, 2009; SIMÃO *et al.*, 2012a; PORDEUS, 2017).

Efetivamente, o PON traz solução para alguns dos problemas destes paradigmas, apresentando vantagens quando comparado a eles. Uma das principais vantagens é a maior facilidade na concepção de sistemas que apresentem paralelismo ou distribuição, por evitar implicitamente o acoplamento excessivo entre entidades computacionais. Outra vantagem é a efetiva redução de alguns dos problemas clássicos de *software* PI e PD, como redundâncias de execução, que também está relacionado ao evitar de acoplamento excessivo (SIMÃO; STADZISZ, 2008; SIMÃO; STADZISZ, 2009; SIMÃO *et al.*, 2012a; PORDEUS, 2017).

Estruturalmente, uma aplicação segundo o modelo PON é representada na forma de *Fact Base Element (FBE – Elemento de Base de Fatos)* e de *Rules (Regras)*. As *FBEs* (*i.e.*, instâncias de *FBE*) são entidades facto-execucionais utilizadas para representar objetos do mundo (entidades reais ou abstratas) em um sistema computacional, por meio de conjuntos exclusivos de estados tratados por entidades chamadas de *Attributes (Atributos)* e por meio de serviços tratados por entidades chamadas de *Methods (Métodos)* (PORDEUS, 2017).

Cada *Rule* (*i.e.*, cada instância de *Rule*), por sua vez, é uma entidade lógico-causal que se compõe de uma entidade *Condition (Condição)* associada a entidades do tipo *Premise (Premissa)* e de uma entidade *Action (Ação)* que se associa a entidades do tipo *Instigation (Instigação)*. Estas entidades podem ser conjuntamente tratadas de forma orientada a regras, tal qual exemplifica a Figura 1 (PORDEUS, 2017), bem como por outros formalismos (WIECHETECK, 2011; MENDONÇA *et al.*, 2015; MENDONÇA; SIMÃO; STADZISZ, 2020).

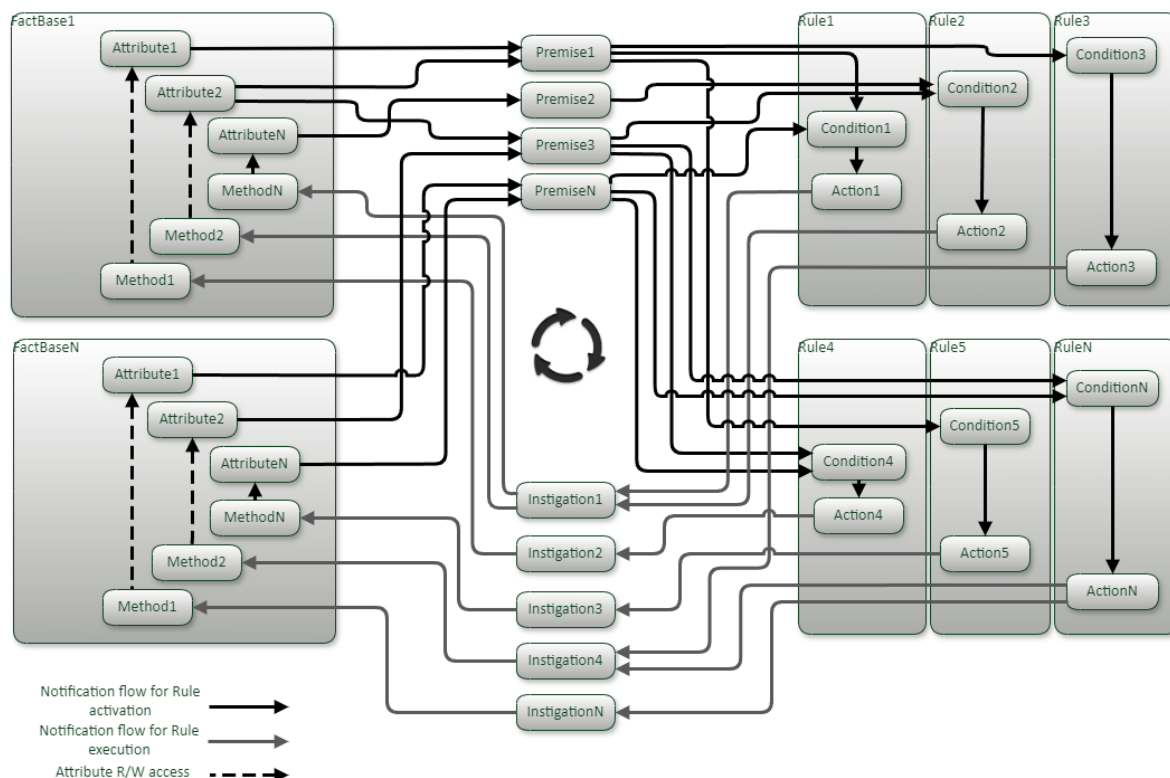
Figura 1 Exemplo de uma *Rule*.

Fonte: Autoria Própria

Em suma, as entidades *Rules* definem o cálculo lógico-causal a ser efetuado sobre os estados dos *FBEs*, controlando a execução dos seus serviços. A colaboração entre estes elementos ocorre por meio de notificações diretas e somente as estritamente necessárias, conforme deixa a entender a Figura 1. Isto é atualmente chamado no grupo de pesquisa de PON, como Inferência Orientada a Notificações (ION) (BANASZEWSKI, 2009; VALENÇA, 2012; RONSZCKA, 2012; FERREIRA, 2015; PORDEUS, 2017).

Conforme esboça a Figura 2, o processo de ION funciona da seguinte forma: quando cada *Attribute-FBE* tem seu valor alterado, ele notifica apenas as *Premises* efetivamente pertinentes; quando cada *Premise* tem seu valor booleano alterado, ela notifica apenas as *Conditions-Rules* efetivamente pertinentes; quando cada *Condition-Rule* é aprovada, ela notifica apenas a *Action-Rule* efetivamente pertinente; quando cada *Action-Rule* é notificada a executar, ela notifica apenas as *Instigations* efetivamente pertinentes; e quando cada *Instigation* é notificada para executar, ela notifica apenas os *Methods-FBEs* efetivamente pertinentes, os quais podem alterar os *Attributes-FBEs* pertinentes realimentando este ciclo de ION (PORDEUS, 2017).

Figura 2 Colaborações pontuais entre entidades do metamodelo do PON.



Fonte: Linhares, 2015

Todo o processo de colaborações pontuais entre as entidades na ION aporta um conjunto de vantagens. Dado que a ION se constitui de notificações diretas entre as entidades que compõem o PON, reativamente disparadas pelas mudanças de estados destas entidades, cria-se um desacoplamento (ou acoplamento mínimo) entre as entidades. O emprego de notificações diretas desacoplantes entre as entidades que constituem os *FBEs* e as *Rules* (por meio de sub-entidades) elimina a necessidade de mecanismos de inferência monolíticos para realizar o *matching* (ou casamento) entre elas. Isto torna o ION do PON um processo de inferência essencialmente distinto dos processos utilizados em *software* PI (PP e POO) e em PD (SBR-PL, PL e PF) (SIMÃO; STADZISZ, 2002, 2008, 2009; RONSZCKA *et al.*, 2015; PORDEUS, 2017).

Os conceitos lógicos e estruturais inerentes à ION permitem que o PON apresente uma forma inovadora de conceber soluções para problemas, de forma que estas soluções sejam conceitualmente descentralizadas devido ao desacoplamento implícito, permitindo alcançar paralelismo e distribuição tão finos quanto a arquitetura computacional o permitir. Este modelo de inferência peculiar do

PON tende a reduzir alguns dos problemas clássicos das abordagens tradicionais em PI e PD, como as redundâncias estruturais (repetições de código) e temporais (repetições ao longo do tempo) (SIMÃO; STADZISZ, 2002, 2008, 2009; RONSZCKA *et al.*, 2015).

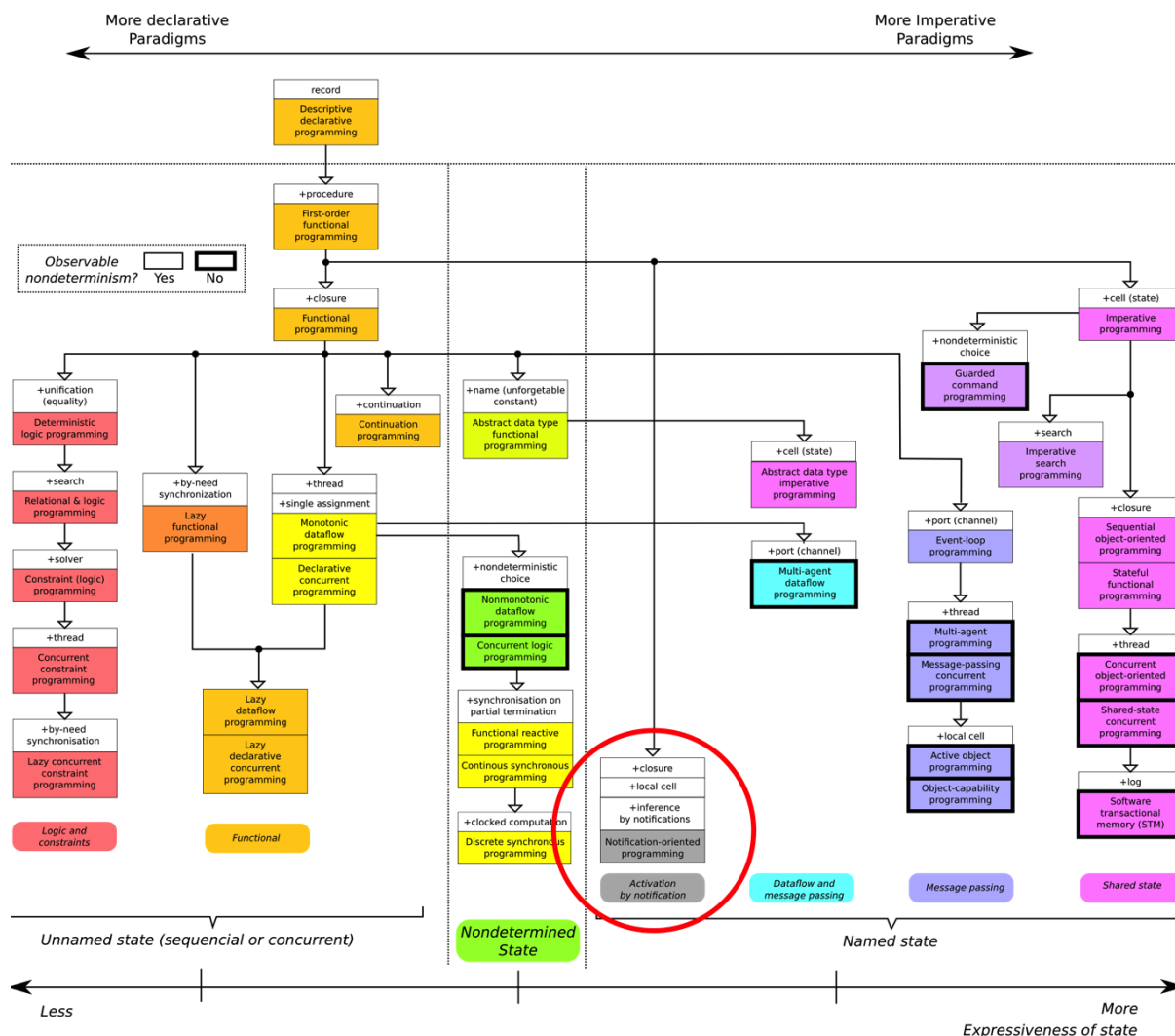
Além disso, os conceitos do PON permitem a concepção de soluções para esses problemas com um nível de abstração mais intuitivo à forma cognitiva humana (*i.e.*, em alto nível e orientado a regras e entidades factuais), inspirando-se na orientação a regras (SIMÃO; STADZISZ, 2002, 2008, 2009; XAVIER, 2014; PORDEUS, 2017). Neste âmbito, ainda que o PON aproveite alguns conceitos de paradigmas anteriores como Sistemas Baseados em Regras, Orientação a Objetos e Orientação a Eventos, ele é distinto em função de suas propriedades ímpares e orquestração *suis-generis* do cálculo lógico-causal segundo a ION. Oportunamente, a Figura 3 apresenta uma adaptação da taxonomia proposta por Van Roy (2009), incluindo o PON em relação aos principais paradigmas de programação (XAVIER, 2014).

Em tal taxonomia definida por Van Roy (2009), cada elemento representa um paradigma e seu respectivo conjunto de conceitos gerais. As setas entre os quadros indicam a inclusão de novos conceitos, mostrando que os quadros derivados contemplam os conceitos dos paradigmas anteriores, acrescidos de um ou mais conceitos novos que, conjuntamente, os definem como um paradigma distinto dos demais. Os conceitos gerais abordados na taxonomia em questão são basicamente elementos primitivos básicos que, em conjunto, dão origem aos paradigmas. Os principais conceitos gerais abordados são: *record* (elementos como estruturas de dados), *closures* (recipientes com escopo léxico como funções, classes, métodos etc), *concurrency* (concorrência), *named state* (estado nomeado) e *determinism* (determinismo) (VAN ROY, 2009; XAVIER, 2014; RONSZCKA, 2019).

Adicionalmente, o posicionamento do PON apresentado na Figura 3 se deve aos fatores descritos nas sentenças a seguir. O posicionamento de um paradigma (graficamente e semanticamente) ocorre pela estrutura intrínseca da própria taxonomia: expressividade de estado (ativação por notificações tem expressividade menor que passagem de mensagem e maior que estado não nomeado), forma de programação (mais declarativa ou mais imperativa – eixo horizontal - equilibrado entre *Rules*, *FBEs* e seus constituintes) e a soma de características únicas (segundo o eixo vertical). De forma geral, os paradigmas adjacentes ao PON apresentam

proximidades ou distâncias relativas de acordo com suas características. Por fim, o PON se posiciona em um ponto equidistante, tanto graficamente quanto conceitualmente, entre seus paradigmas predecessores: Orientação a Objetos e Programação Declarativa (XAVIER, 2014).

Figura 3 Taxonomia de paradigmas de programação incluindo o PON.



Fonte: Xavier, 2014.

2.1.1. Materializações do PON

O PON foi alvo de pesquisas em diversas áreas nos últimos anos, tais como engenharia de *software*, engenharia de sistemas, redes neurais, lógica *fuzzy*, sistemas embarcados e *hardware* em lógica reconfigurável, podendo ser expandido para novos domínios do conhecimento. Atualmente, existem tanto materializações em *software* quanto em *hardware* do PON. Entretanto, as primeiras foram

essencialmente em *software* e, apenas em um segundo momento, surgiram as em *hardware* (PORDEUS, 2017; RONSZCKA, 2019).

Para o desenvolvimento de *softwares* fazendo uso do PON, foram inicialmente realizadas pesquisas por meio de um *Framework* PON Prototipal (versão 0.5) (SIMÃO; STADZISZ, 2008), este derivado dos *Frameworks* (versão 1.0 e 2.0) do Controle Orientado a Notificações (CON), todos em C++, que surgiram a partir dos esforços de pesquisa de Simão (2001, 2005). Subsequentemente foi desenvolvido e utilizado o *Framework* PON C++ 1.0, também chamado de “Original” em alguns momentos (SIMÃO; STADZISZ, 2008; BANASZEWSKI, 2009; SIMÃO *et al.*, 2012c).

Depois, evolutivamente, foi desenvolvida e utilizada uma segunda versão melhorada de *framework* PON em C++, sendo este chamado de *Framework* PON C++ 2.0, também por vezes chamado de “Otimizado” (RONSZCKA, 2012; VALENÇA, 2012). Essas materializações do PON em forma de *frameworks* ou arquétipos permitiram a criação de *softwares* PON sob a linguagem C++ do POO/PI, mas aplicando nela a abordagem por notificações, portanto, permitindo-a operar de forma orientada a notificações (PORDEUS, 2017). Ainda, no mesmo contexto, foram desenvolvidas versões de *framework* PON em linguagens de programação C# e Java (BARRETO *et al.*, 2018; HENZEN, 2015; OLIVEIRA, 2019).

Do mesmo modo, houve uma pesquisa a respeito do balanceamento de carga de *software* PON em ambientes *multicore* utilizando uma versão modificada do *Framework* PON C++ 2.0, chamada de 3.0 (BELMONTE, 2012; BELMONTE *et al.*, 2012; BELMONTE *et al.*, 2016) e depois um *Framework* PON Erlang/Elixir e mesmo um *Framework* PON Akka alcançando sinergia para com estas tecnologias (MARTINI, 2018; NEGRINI, 2019). Por fim, outras implementações prototipais, baseadas em *Framework* PON em Java, C# e C++, tiveram experimentos envolvendo distribuição via TCP/IP (*Transmission Control Protocol/Internet Protocol*) (TALAU, 2016; PORDEUS, 2017; BARRETO *et al.*, 2018; OLIVEIRA, 2019) e, mais recentemente, MQTT no quadro de IoT com o *Framework* PON C++ IoT (FIGUEIREDO, 2022).

Os *frameworks* em PON permitiram a demonstração de viabilidade do PON, como a possibilidade efetiva de paralelismo e/ou distribuição. No entanto, o peso das estruturas de dados utilizadas na sua concepção, mesmo nas versões mais recentes e mais ‘otimizadas’ em C++ (e.g., *Framework* PON C++ 4.0 e 4.5) (NEVES,

2021; BABU, 2022; LEDESMA; SIMÃO, 2022), apresentaram imperfeições no tocante à performance. Particularmente, o peso das estruturas de dados e afins não permitiram alcançar todo o potencial do PON em termos de tempo de processamento. Isso é relevante à luz de suas propriedades de evitar redundâncias e de seu cálculo assintótico que é $O(n)$ no caso médio e $O(n^3)$ no inverossímil pior caso, conforme cálculos apresentados em Simão (2005) (FERREIRA, 2015; RONSZCKA *et al.*, 2015; RONSZCKA, 2019). Dado este fato, fez-se necessário vislumbrar outros tipos de materializações.

Neste sentido, houve a concepção de um conjunto de linguagens e compiladores específicos para o PON denominado Tecnologia LingPON, em uma versão 1.0, os quais permitiram alcançar tempos de processamento mais apropriados (FERREIRA, 2015). Não obstante, a Tecnologia LingPON continuou em evolução para se alcançar uma segunda versão, chamada Tecnologia LingPON 2.0, ou apenas Tecnologia NOPL (SANTOS, 2017; PORDEUS, 2017; SANTOS; SIMÃO; FABRO, 2017; RONSZCKA *et al.*, 2017). Tanto a Tecnologia LingPON 1.0 quanto a Tecnologia LingPON 2.0 são subordinadas a um método próprio intitulado de MCPON (RONSZCKA, 2019; OSHIRO, 2021), bem como o é a assim chamada Tecnologia LingPON 3.0 (LEDESMA; SIMÃO, 2022; SKORA; CHIERICI; SIMÃO, 2022).

Ronszcka (2019) propôs o método MCPON para a criação de materializações para o PON de forma padronizada. Esse método trata principalmente da concepção e definição de linguagens de programação específicas e da implementação de compiladores próprios para o PON, de forma tal que se permita coerência em plataformas distintas. A construção destas linguagens e particularmente destes compiladores é orientada a partir de um grafo diferenciado, denominado Grafo PON, que é materializado como um arquétipo ou *framework*. Esse grafo permite mapear os elementos de uma aplicação PON, visando organização apropriada deles, de forma desacoplada à luz do próprio paradigma. Ainda, é justamente o Grafo PON que permite a integração e compatibilidade entre as diferentes materializações construídas para plataformas distintas (RONSZCKA, 2019). Por fim, o GrafoPON e MCPON foram refinados e expandidos, em esforços subsequentes de pesquisa, cf. (OSHIRO, 2021; SKORA; CHIERICI; SIMÃO, 2022).

No contexto de engenharia de *software*, por sua vez, Wiecheteck (2011) propôs um método de modelagem denominado Desenvolvimento Orientado a

Notificações (DON). Esse método consiste na concepção de *software* segundo o PON baseando-se em práticas de engenharia de *software*, com adaptações oriundas da UML e também das Redes de Petri (WIECHETECK, 2011; BATISTA, 2013; MENDONÇA *et al.*, 2015; MENDONÇA; SIMÃO; STADZISZ, 2020). No entanto, o DON aplica uma adaptação da abordagem convencional de modelagem orientada a objetos, não necessariamente favorecendo tanto quanto esperado a modelagem de *software* PON. Assim, Mendonça (2020) propôs uma nova metodologia de projeto de *software* para o PON, chamada de NOM (*Notification Oriented Software Design Methodology*), que visa facilitar a concepção de aplicações para esse paradigma (MENDONÇA; SIMÃO; STADZISZ, 2020). Além disso, houve pesquisas no tocante a concepção, requisitos e testes relativos ao PON (BATISTA, 2013; KOSSOSKI, 2015; NOVAES, 2019). Certamente, todas estas pesquisas seriam integráveis à Tecnologia LingPON mencionada anteriormente, inclusive à luz do Grafo PON já considerado.

Por outro lado, pesquisas também exploraram a implementação do PON em *hardware*. Uma primeira pesquisa e projeto, chamado de PON em *Hardware Digital* (PON-HD) prototipal, utilizou lógica reconfigurável (FPGA - *Field Programmable Gate Array*) visando explorar o seu potencial de paralelização (WITT *et al.*, 2011; SIMÃO *et al.*, 2012b; KERSCHBAUMER *et al.*, 2015; PORDEUS *et al.*, 2016; KERSCHBAUMER *et al.*, 2018a). Outra pesquisa correlata e subsequente em PON para *Hardware Digital* permitiu alcançar uma solução mais efetiva, chamada PON-HD 1.0, integrando-se em boa hora à Tecnologia LingPON. Esta solução permite a geração de uma aplicação PON em circuitos específicos a partir de um código em alto nível, por meio da linguagem e compilador próprios da Tecnologia LingPON. Naturalmente, essa solução foi eventualmente denominada de Tecnologia LingPON-HD (KERSCHBAUMER *et al.*, 2015; KERSCHBAUMER *et al.*, 2018b; KERSCHBAUMER, 2018).

Em outra pesquisa, Peters (2012) propôs a implementação de um coprocessador PON (CoPON) em lógica reconfigurável. Isto se constitui em uma solução híbrida, em que a parte da aplicação responsável pelo processamento factual é executada em um núcleo von Neumann, usando uma adaptação do *Framework* PON C++ 1.0. Enquanto a parte da aplicação responsável pelo cálculo lógico-causal e, portanto, pela propagação de notificações é executada por meio de um coprocessador, utilizando FPGA, baseado nos princípios do PON. O resultado foi

a paralelização no cálculo lógico causal e efetivo ganho de tempo de processamento (PETERS, 2012; PETERS *et al.*, 2012). Entretanto, apesar dos excelentes resultados, tanto em CoPON quanto em PON-HD têm limitação no número de entidades PON a serem executadas em *hardware* próprio para tal (PETERS, 2012; LINHARES, 2015; PORDEUS, 2017; PORDEUS *et al.*, 2023).

No contexto acima dado, uma arquitetura de computação foi desenvolvida de acordo com o modelo do PON para resolver aquela deficiência, baseado no dueto de memória e processadores, sendo denominada em inglês de *Notification-Oriented Computer Architecture* (NOCA) e em português de ArqPON. A ArqPON é uma arquitetura de computador alternativa às arquiteturas de computadores tradicionais, tais como von Neumann e fluxo de dados. A ArqPON tem como objetivo implementar um ambiente de execução mais propício à dinâmica de notificações do PON, permitindo a execução de aplicações desenvolvidas segundo este paradigma, de forma que seja possível aproveitar, até certo ponto, as suas propriedades intrínsecas de paralelização (LINHARES, 2015; LINHARES; SIMÃO; STADZISZ, 2015; PORDEUS, 2017).

Esta arquitetura foi implementada na forma de um protótipo por meio do uso de linguagem de descrição de *hardware* em conjunto com um dispositivo FPGA (LINHARES, 2015; LINHARES; SIMÃO; STADZISZ, 2015). Esta arquitetura também foi avaliada por meio de um simulador em *software*, o qual permitiu a realização de experimentos com uma alta escalabilidade simulada. Por meio deste simulador, denominado ArqPONSIm ou NOCASim, foi possível (dentre outros) obter um número de unidades de processamento na mesma proporção do que o número de elementos PON que compõem uma aplicação, reduzindo assim as limitações de escala impostas pela plataforma em *hardware* para estudo desta arquitetura (PORDEUS, 2017).

Ainda, uma evolução da ArqPON, batizada de Total ArqPON, foi vislumbrada durante os esforços de pesquisa desta presente tese, cf. (PORDEUS, 2020). Entretanto, essa evolução foi deixada para o porvir do grupo de pesquisa dado o interesse da pesquisa doutoral no tema de árvores de decisão *vis-à-vis* PON-HD. Muito embora o ArqPON e/ou suas evoluções sejam pertinentes para árvores de decisão, o PON-HD continua sendo a melhor opção em termos de performance e paralelismo justamente por não necessitar trabalhar com o dueto memória processamento, tal qual a família ArqPON o necessita. Ademais, a simplicidade

inerente da natureza das árvores de decisão, particularmente quando vislumbradas via PON, potencialmente atenuariam a questão das limitações de espaço em *hardware*, o que faz parte do objeto de pesquisa desta tese.

Outrossim, outras pesquisas foram e vêm sendo realizadas explorando o uso do PON em novas frentes, como: lógica *fuzzy* em PON (MELO; SIMÃO; FABRO, 2015; MELO, 2016) e redes neurais em PON tanto em *software* (via conformações de *Framework* PON 3.0 e também da LingPON 1.0) quanto *hardware* (via conformações da Tecnologia LingPON-HD) alcançando a chama NeuroPON (SCHÜTZ *et al.*, 2015; SCHÜTZ, 2019), sistemas em tempo real em teoria e protótipos em algumas aplicações mais recentes (LINHARES *et al.*, 2014; ATHAYDE, 2016; MARTINI, 2020; MAMANN *et al.*, 2021), computação senciente em teoria e prática (OLIVEIRA, 2019; SIMÃO *et al.*, 2014; FIGUEIREDO, 2022; FIGUEIREDO; SIMÃO; VENDRAMIN, 2022) e sistemas assaz complexos exemplificado na forma de futebol de robôs e simulação de Controle de Tráfego de Automóveis (CTA) (SANTOS, 2017; NEGRINI, 2019; NEGRINI *et al.*, 2020).

2.1.2. Propriedades elementares e conceitos do PON

Embora alguns trabalhos tenham implementado materializações para o PON, tanto em *software* quanto em *hardware*, elas não contemplaram por completo e conjuntamente as propriedades elementares do paradigma (nomeadamente desenvolvimento orientado a regras em alto-nível, não redundância visando desempenho e desacoplamento visando inclusive paralelismo/distribuição) e, muitas vezes, apresentaram-se de maneira incompleta e mesmo inconsistente entre si. Isto dito, mais precisamente pode-se definir as propriedades elementares como segue:

- **Programação em alto nível visando facilidade de desenvolvimento:** Facilidade de desenvolvimento pela orientação a regras e alto nível sendo cada programa organizado em um conjunto de entidades facto-execucionais e outro conjunto correlato de entidades lógico-causais.
- **Não-Redundância visando desempenho apropriado:** Baixo tempo de processamento devido à ausência de redundâncias temporais e estruturais à luz da estratégia utilizada para implementação e da plataforma utilizada para tal.

- **Desacoplamento visando Paralelismo/Distribuição:** Modelo preparado para execução paralela/distribuída fina em função do desacoplamento implícito entre as entidades que compõem o PON, tanto quanto a plataforma de execução permitir.

Por mais que as materializações do PON tenham evoluído ao longo dos anos de modo a contemplar as propriedades elementares (definidas no parágrafo anterior), bem como boa parte dos chamados conceitos de programação pertinentes do paradigma (como *Formation Rules*, *Master Rule* etc)², restaram alguns conceitos sem implementação em algumas materializações e mesmo sem contemplar conjuntamente todas as propriedades elementares (RONSZCKA, 2019). Em tempo, cf. Ronszcka (2019), pode-se resumir estes conceitos, derivados das propriedades elementares, em:

- **Reatividade das entidades:** O PON é naturalmente composto por pequenas entidades reativas, que colaboram entre si via notificações precisas e pontuais.
- **Escalonamento de *Rules*:** Mecanismo de decisão das ordens das execuções das *Rules* aprovadas.
- **Estratégias de resolução de conflito:** Mecanismos implementados para resolver impasses quando um dado *Attribute* representa um recurso exclusivo e é acessado por dois elementos ao mesmo tempo.
- **Compartilhamento de entidades:** Compartilhamento de entidades do PON com o objetivo de eliminar a criação de entidades redundantes.
- **Regras de formação (*Formation Rules*):** Conceito que permite a criação de *Rules* específicas, a partir da representação genérica de uma *Rule*. Este conceito é aplicado quando o conhecimento causal de uma *Rule* é comum para diferentes conjuntos de instâncias de *FBEs*, ou seja, um conjunto de

² Estes conceitos são amplamente discutidos na literatura sobre PON, com detalhamento em Pordeus (2017) e Ronszcka (2019). Em suma, Regras de Formação são “classes” de *Rules* que permitem instanciar automaticamente *Rules* similares, como as que variam apenas as combinações de FBE observados. Ainda, *Regra Mestra* faz com que suas *Rules* subordinadas também precisam que ela esteja aprovada para que ela execute, permitindo alguma hierarquização.

Rules específicas se diferencia apenas nas combinações das instâncias referenciadas.

- **Propriedades reativas dos *Attributes*:** Propriedades que determinam o comportamento de um *Attribute* ao ter seu valor alterado: padrão (*i.e.*, gera notificação somente quando seu estado mudar de valor), renotificante (*i.e.*, gera uma notificação mesmo que seu estado não sofra alterações) e sem notificação (*i.e.*, não gera notificações).
- ***Master Rule*:** Conceito aplicado quando uma *Rule* é responsável pela ativação de outra *Rule*, ou seja, quando uma *Rule* depende de outra.
- **Entidades impertinentes:** Quando uma entidade PON permite desativar temporariamente a geração de notificações para evitar notificações desnecessárias e redundantes, as quais podem ser reativadas quando o contexto ditar a sua pertinência para aquele momento.
- ***FBE Rules*:** Permite agregar uma *Rule* dentro do escopo de uma dada *FBE*.
- ***FBE Aggregator*:** Permite agregar uma *FBE* dentro do escopo de outra *FBE*.

Neste âmbito, de conceitos e propriedades do PON, as materializações do PON variam em quais deles são efetivamente implementados. Justamente a Tabela 1 apresenta uma síntese das principais materializações em relação às propriedades elementares do PON, enquanto a Tabela 2 apresenta uma síntese das principais materializações em relação aos conceitos de programação implementados. A partir destas tabelas, percebe-se que há propriedades elementares e conceitos de programação que são tratados em uma materialização, mas não em outra e vice-versa. Certamente, um fator agravante para o não reaproveitamento do que foi feito em uma materialização na outra é a falta de padronização no processo de proposição e desenvolvimento dessas implementações, o que tende a dificultar a concepção de materializações mais efetivas (RONSZCKA, 2019).

Tabela 1 Propriedades elementares do PON contempladas nas suas materializações.

Materialização Propriedades	Software						Hardware			
	FW Prot.	FW 1.0	FW 2.0	FW 3.0	FWs Java e C#	FW 3.0 + PON IP	PONHD Prot.	CoPON	NOCA	PONHD 1.0
	2005	2009	2012	2014	2016	2016	2011	2012	2015	2018
Prog. Alto nível										
Desacoplamento/Paralelismo				✓	✓	✓	✓	✓	✓	✓
Desacoplamento/Distribuição						✓				
Não-Redundância/Desempenho							✓	✓	✓	✓

Fonte: Adaptado de Ronszcka, 2019

Tabela 2 Conceitos do PON contemplados nas suas materializações.

Materialização Conceitos de programação	Software						Hardware			
	FW Prot.	FW 1.0	FW 2.0	FW 3.0	FWs Java e C#	FW 3.0 + PON IP	PONHD Prot.	CoPON	NOCA	PONHD 1.0
	2005	2009	2012	2014	2016	2016	2011	2012	2015	2018
Escalonamento de Rules	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Estratégias de resolução de conflito		✓	✓	✓	✓	✓			✓	
Compartilhamento de entidades		✓	✓	✓	✓	✓			✓	
Regras de formação			✓	✓	✓	✓			✓	
Propriedades reativas dos Attributes	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Master Rule			✓	✓		✓			✓	✓
Entidades impertinentes			✓	✓		✓			✓	
FBE Rules			✓	✓		✓			✓	
FBE Agregator			✓	✓		✓				

Fonte: Adaptado de Ronszcka, 2019

2.1.3. PON em hardware

Esta seção explora os conceitos desenvolvidos para a materialização do PON em *hardware* digital, denominado PON-HD 1.0. Essa materialização utiliza lógica reconfigurável para maximizar o paralelismo implícito e possibilita a execução direta em *hardware* de aplicações desenvolvidas no PON. Como resultado, o PON-HD apresenta vantagens significativas em relação às arquiteturas tradicionais de execução sequencial e também ao próprio ArqPON, especialmente em termos de desempenho e eficiência (LINHARES, 2015; LINHARES; SIMÃO; STADZISZ, 2015; PORDEUS, 2017; LINHARES *et al.*, 2020).

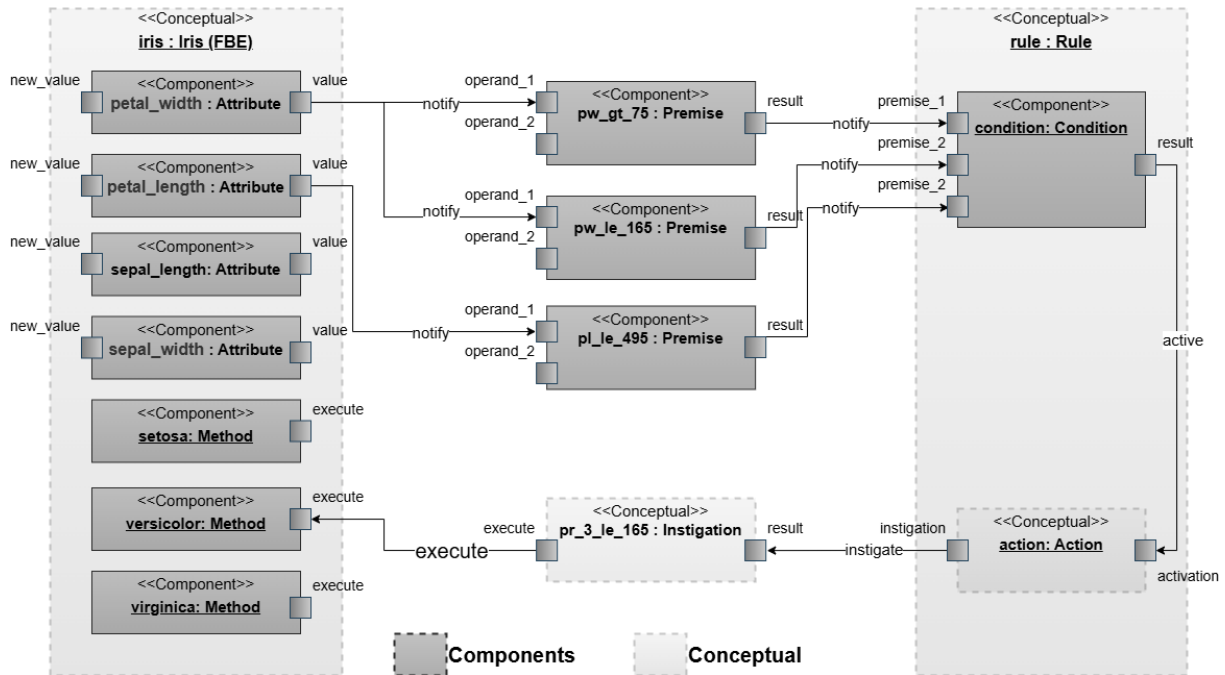
2.1.3.1. Visão geral do PON-HD 1.0

O PON-HD 1.0 ou simplesmente PON-HD, também referido como NOP-DH (*Notification-Oriented Paradigm - Digital Hardware*) em inglês, é uma materialização do Paradigma Orientado a Notificações (PON) implementada em *hardware* reconfigurável, como FPGAs. Essa abordagem permite que aplicações desenvolvidas no contexto do PON sejam executadas diretamente no *hardware* de maneira paralela e eficiente, até então utilizando lógica reconfigurável (FPGA) para replicar a dinâmica de notificações intrínseca ao paradigma (KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

Essa materialização explora o paralelismo inerente ao PON, baseando-se na propagação de sinais digitais entre componentes de circuitos que representam os elementos NOP. Esses componentes são mapeados diretamente em VHDL para alguns elementos do modelo do PON, como *Attributes*, *Methods*, *Premise* e *Condition*, enquanto outros elementos, como *FBE*, *Rule*, *Action* e *Instigation*, são apenas organizacionais neste âmbito. Justamente, a Figura 4 ilustra os componentes do PON com dois estereótipos: "*Component*" (Cinza) que denota com implementação física e "*Conceptual*" (Cinza Claro) que denota sem implementação física (KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

Os componentes do PON-HD são projetados para gerar o circuito mais otimizado para cada função, permitindo que cada ciclo de notificação ocorra em apenas um ciclo de *clock*. Além disso, os componentes possuem mecanismos de prioridade para resolver conflitos de acesso aos *Attributes* e *Methods*. Isto dito, a seguir, são descritos brevemente os componentes do PON-HD e suas respectivas implementações em VHDL (KERSCHBAUMER, 2018; KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

Figura 4 Diagrama de blocos internos representando os componentes do PON-HD.

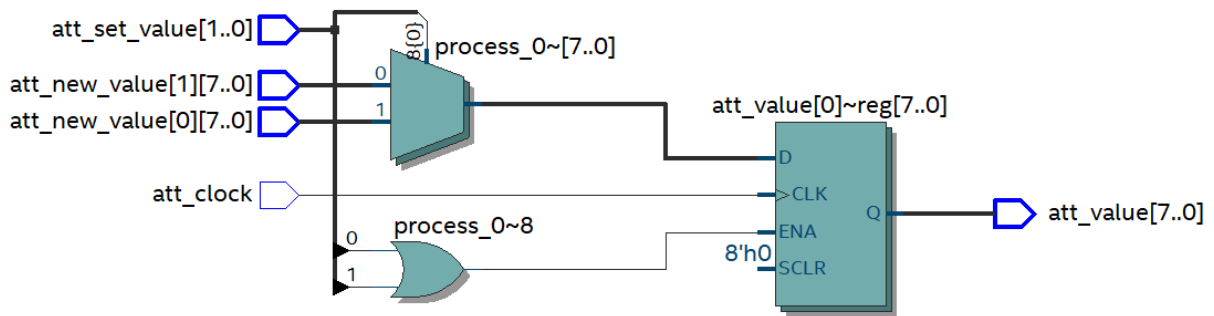


Fonte: Autoria Própria

2.1.3.2. Componentes PON-HD

O componente *Attribute* é responsável por armazenar o estado de uma variável no sistema e por notificar eventuais mudanças em seu valor. Conforme ilustrado na Figura 5, o circuito gerado para um componente *Attribute* do tipo inteiro é composto por um multiplexador, que é utilizado para atribuição de valores e por um conjunto de registradores que armazenam o valor do *Attribute*. Para resolver conflitos, o componente *Attribute* utiliza a prioridade das entradas do multiplexador, atribuindo diferentes níveis de prioridade a cada entrada (KERSCHBAUMER *et al.*, 2018a). Além disso, vale destacar que esse componente é o único que depende do sinal de *clock* devido à sua lógica sequencial, enquanto os demais componentes implementados são puramente combinacionais.

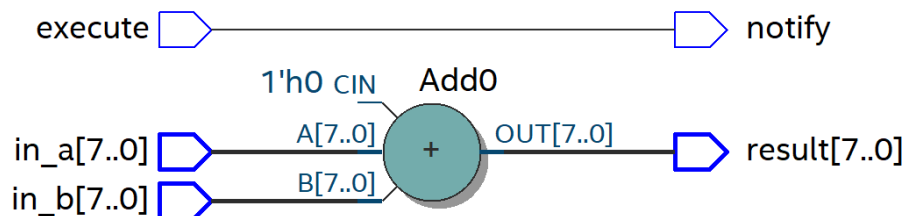
Figura 5 Componente *Attribute*.



Fonte: Kerschbaumer, 2018

Por sua vez, há o componente *Method* que é responsável por realizar cálculos factuais e atualizar os valores dos *Attributes*, reiniciando assim o ciclo de notificações. Na implementação fundamental, ele executa apenas atribuições e cálculos aritméticos simples. No entanto, pode ser usada nele qualquer função que possa ser implementada em uma FPGA (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021). A Figura 6 ilustra um componente *Method* que realiza a adição de dois *Attributes*. Como dois ou mais *Methods* podem tentar acessar o mesmo *Attribute* simultaneamente, foi implementado um mecanismo de prioridade. Nesse mecanismo, a saída "*notify*" do componente *Method* é utilizada como um dos elementos binários do array `attr_set_value`. Este array, por sua vez, é utilizado como entrada pelo sistema de prioridade do componente *Attribute*, determinando qual valor deve ser atribuído ao *Attribute* (conforme apresentado na Figura 5) (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

Figura 6 Componente *Method*.

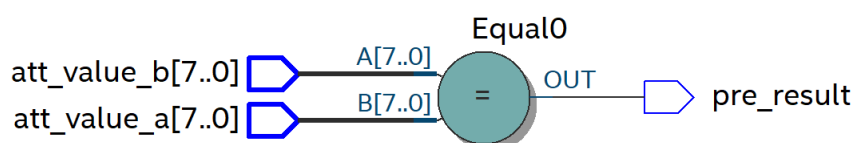


Fonte: Kerschbaumer, 2018

Cada componente *Attribute* no PON-HD é responsável por notificar um conjunto de componentes *Premise* conectados a ele. O componente *Premise*, por sua vez, realiza comparações lógicas entre dois valores notificados de *Attributes* ou entre o valor de um *Attribute* e um valor constante. Por exemplo, a Figura 7 ilustra

um componente *Premise* implementado como um comparador lógico, que gera um *bit* indicando o resultado *true* ou *false* para uma operação de igualdade. As operações lógicas que podem ser realizadas por este componente incluem igualdade, desigualdade, maior que, maior ou igual, menor que, e menor ou igual (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

Figura 7 Componente *Premise*.



Fonte: Kerschbaumer, 2018

Após a execução de suas funções, cada componente *Premise* notifica um conjunto de componentes *Condition* aos quais está conectado. O componente *Condition* é responsável por combinar os valores de várias *Premises* usando operações lógicas. Para isso, a implementação deste componente PON em *hardware* utiliza portas lógicas, como *AND* e *OR*, ou combinações dessas portas para realizar as operações necessárias (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

As portas lógicas empregadas na implementação dos componentes *Condition*, bem como os códigos dos módulos VHDL utilizados para os componentes *Attribute*, *Method* e *Premise*, fazem parte de um *framework* VHDL para o PON, que é a essência da tecnologia PON-HD (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021). Ao ensejo, os elementos *FBE* e *Rules* não possuem implementação em *hardware*. No PON-HD, cada *Action* é apenas um conjunto de *Instigations* ativadas pela aprovação de uma *Condition* pertinente, e cada *Instigation* é um caminho que transmite sinais de ativação dos componentes *Conditions/Actions* para os componentes *Methods* (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

A implementação de componentes PON em *hardware* no PON-HD 1.0 permitiu que cada ciclo de notificação ocorra em apenas um único ciclo de *clock*. Esse resultado é alcançado devido à natureza do PON e ao fato de que cada implementação de componente foi projetada para gerar o circuito mais otimizado

para a função de cada elemento PON (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b; PORDEUS *et al.*, 2021).

2.1.3.3. Trabalhos Anteriores sobre PON-HD 1.0

Pesquisas anteriores destacaram a utilidade do PON-HD 1.0 em *benchmarks* específicos, como o controle de robôs hexápodes, ou em *benchmarks* menos adequados, como o algoritmo de ordenação *Bubble Sort*. Embora esses estudos tenham empregado cenários não universais ou menos apropriados, eles demonstraram com sucesso a adequação do PON-HD como uma plataforma de desenvolvimento de alto nível para dispositivos de lógica reconfigurável (KERSCHBAUMER *et al.*, 2018a; KERSCHBAUMER *et al.*, 2018b). A seguir, é apresentado um resumo dos experimentos já realizados principalmente com o PON-HD 1.0 e seus respectivos resultados, sendo alguns já no âmbito desta presente tese.

Em sua tese, Kerschbaumer (2018) apresentou oito experimentos distintos com o objetivo de demonstrar o PON-HD como uma ferramenta apropriada para o desenvolvimento de *hardware* em lógica reconfigurável, além de comparar o desempenho do PON-HD 1.0 e LingPON-HD 1.0 em relação a implementações diretas em VHDL e à ferramenta de síntese em alto nível Vivado HLS. Entre os experimentos realizados são: contador, driver para display de 7 segmentos, contador decimal, calculador de média, ordenação de dados (*Bubble Sort*), controlador de esteira e controlador de robô. Por sua vez, os resultados demonstraram a viabilidade do PON-HD como ferramenta de desenvolvimento, permitindo o desenvolvimento de *hardware* com um maior nível de abstração, sem gerar perdas em relação ao desempenho dos circuitos gerados (KERSCHBAUMER, 2018).

Pordeus *et al.* (2016) e Kerschbaumer *et al.* (2018a) realizaram experimentos implementando aplicações com o PON-HD, como um simulador de telefone e o algoritmo *Bubble Sort*, respectivamente. Os resultados mostraram que as implementações com PON-HD apresentaram desempenho equivalente às versões equivalentes desenvolvidas em VHDL, mas com a vantagem de oferecer uma maior expressividade de paralelismo em um nível mais elevado de abstração (KERSCHBAUMER, 2018).

Em outro estudo, Kerschbaumer *et al.* (2018b) investigaram o controle de um robô hexápode em um ambiente simulado, utilizando a plataforma V-REP. Este experimento teve como objetivo avaliar o PON-HD como uma ferramenta de Síntese em Alto Nível (HLS) em contextos que requerem decisões complexas de controle. Os resultados indicaram que o robô controlado pelo PON-HD seguiu o mesmo trajeto e tomou as mesmas decisões que um robô controlado por uma abordagem tradicional em VHDL, porém com a vantagem adicional de possibilitar programação em um nível mais alto de abstração (KERSCHBAUMER, 2018).

Schütz *et al.* (2018) e Schütz (2019) compararam a implementação de uma Rede Neural Artificial (ANN) usando tanto a abordagem tradicional em VHDL quanto a tecnologia PON-HD, denominada NeuroNOP. Utilizando redes do tipo *Perceptron* Multicamadas para a função XOR e o conjunto de dados *Iris*, com treinamento por retropropagação, o estudo mostrou que, embora a implementação em VHDL tenha gerado um circuito com menos elementos lógicos e uma frequência de operação mais alta, o PON-HD permite uma abordagem declarativa, facilitando a personalização de cada neurônio e a configuração da cadeia de notificações sinápticas (SCHÜTZ, 2019).

Pordeus *et al.* (2021) conduziram um experimento para avaliar o uso do PON-HD no desenvolvimento do algoritmo de ordenação *Bitonic Sort* como *benchmark*. Este algoritmo possui características que favorecem a execução paralela, especialmente em FPGAs. Os resultados dos experimentos compararam o desempenho, a quantidade de elementos lógicos e a frequência máxima de operação do PON-HD em relação à abordagem tradicional em VHDL, mostrando que os circuitos PON-HD alcançam resultados semelhantes aos obtidos com o desenvolvimento tradicional.

Esses experimentos demonstraram a viabilidade do PON-HD como um tipo de protótipo de ferramenta HLS e validaram o design desta tecnologia também. No entanto, o PON-HD ainda apresenta algumas limitações a serem exploradas na próxima subseção.

2.1.3.4. Limitações do PON-HD

Atualmente, o PON-HD representa uma abordagem única para a execução de aplicações baseadas no PON, utilizando processamento de fato em paralelo, via componentes simples de *hardware*. No entanto, essa abordagem não incorpora relações de memória típicas de sistemas de memória convencionais, como os presentes nas máquinas de arquitetura *von Neumann*. Isto pode ser uma vantagem (conforme já sublinhada), mas pode tal qual ser uma desvantagem em certos contextos.

No contexto do PON-HD, o uso exclusivo de registradores para armazenar elementos do tipo *Attribute* apresenta algumas limitações. Embora o uso de registradores permita a execução paralela das aplicações desenvolvidas para essa abordagem, ele restringe a quantidade de dados que pode ser armazenada ao número de registradores disponíveis na FPGA. Além disso, como os elementos *Premise*, *Condition* e *Method* são essencialmente combinacionais, a execução pode consumir muitos recursos da FPGA, dependendo da complexidade da aplicação (PORDEUS *et al.*, 2016).

Uma possível solução para essas limitações seria o uso de blocos de memória RAM (internos ou externos) para o armazenamento dos *Attributes*. Contudo, devido ao fato de que um *Attribute* precisa notificar outros elementos do circuito e pode ser acessado simultaneamente por vários outros *Attributes*, o PON-HD atual não é capaz de utilizar eficientemente a estrutura de memória RAM convencional. Portanto, são necessárias outras abordagens, que embora possam resultar em uma perda de desempenho em comparação com o uso de registradores, essas alternativas possivelmente permitiriam um aproveitamento mais eficiente dos recursos internos das FPGAs.

Por fim, apesar dos bons resultados alcançados, a abordagem atual do PON-HD ainda carece de generalidade e escalabilidade. Pois, qualquer modificação na lógica da aplicação requer a reconfiguração completa do circuito de *hardware*, além de que o seu tamanho e complexidade estão limitados à capacidade do dispositivo de lógica reconfigurável (LINHARES; SIMÃO; STADZISZ, 2015; LINHARES, 2015). Entretanto, certos domínios, como o de árvores de decisão podem se beneficiar, tanto do PON quanto do PON-HD em particular, dado a seu desacoplamento entre elementos decisoriais, sinérgicos a natureza do PON. Nestes

cenários, as aplicações com o PON-HD se beneficiam do paralelismo natural que a sua plataforma de execução permite, além da simplicidade de seus componentes que não necessitam gerenciamento de acessos a memória.

2.2. Árvores de decisão

Esta seção apresenta uma revisão de apoio sobre árvores de decisão, contemplando seus fundamentos teóricos, principais algoritmos, além de uma revisão de trabalhos pertinentes relacionados à execução em *hardware*. As seções 2.2.1 e 2.2.2 são baseadas em um conjunto *surveys* relativos a árvores de decisão, que abordam seus conceitos fundamentais e os principais algoritmos utilizados (CRIMINISI; SHOTTON; KONUKOGLU, 2012; ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022; MIENYE; JERE, 2024). Em seguida, a Seção 2.2.3 foca na revisão das implementações de árvores de decisão em *hardware*, identificando oportunidades de otimização em relação às limitações das implementações em *hardware* existentes.

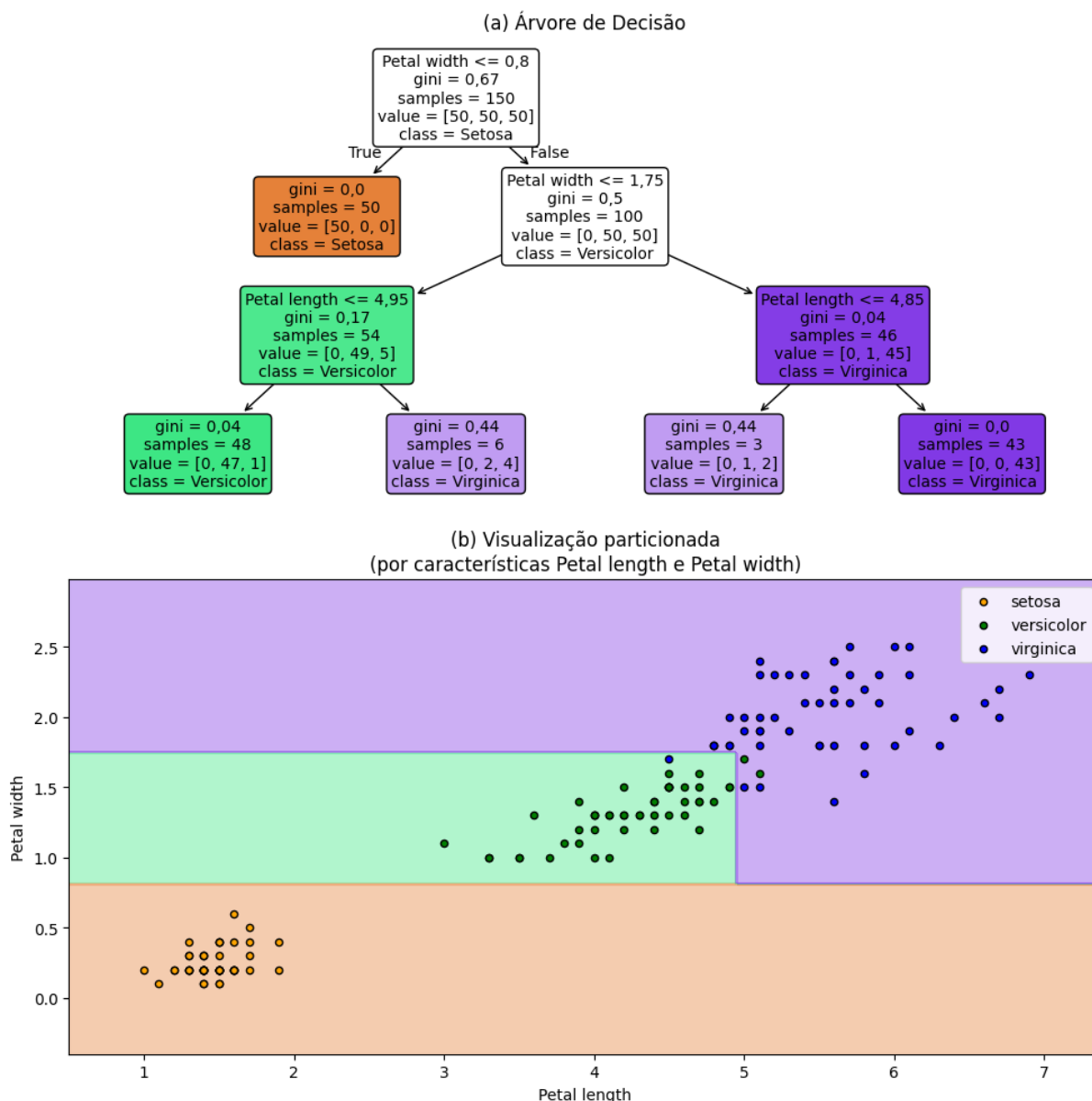
2.2.1. Introdução às árvores de decisão

As árvores de decisão são um modelo de previsão amplamente utilizado tanto para regressão (*i.e.*, previsão de valores contínuos) quanto para classificação (*i.e.*, identificação de categorias). Esse modelo é composto por um conjunto hierárquico de nós de teste organizados em uma estrutura de árvore. Em cada nó dessa estrutura, são realizadas avaliações de teste sobre um atributo específico, relacionado a uma característica particular do conjunto de dados em análise. Com base nos resultados dessas avaliações, o conjunto de dados é dividido em subconjuntos menores, o que facilita a sua análise e interpretação (ROKACH, 2016).

A Figura 8 (a) apresenta um exemplo de árvore de decisão para o conjunto de dados (*dataset*) *Iris* (*i.e.*, lírio) (FISHER, 1936; UNWIN; KLEINMAN, 2021). Este conjunto de dados é um *benchmark* bem conhecido para algoritmos e soluções de classificação, contendo 150 elementos divididos em três classes da planta lírio, representando três espécies distintas chamadas *setosa* (laranja), *versicolor* (verde) e *virginica* (roxo). Para treinar e prever modelos com este *dataset*, são avaliadas quatro características: o comprimento e largura das sépalas e pétalas (UNWIN; KLEINMAN, 2021). Entretanto, para fins de exemplificação, são utilizados nos

exemplos abaixo apenas as características referentes a largura (*Petal Width*) e comprimento (*Petal Length*) das pétalas.

Figura 8 Exemplo de árvore de decisão.



Fonte: Autoria Própria

Conforme a Figura 8 (a), a árvore de decisão inicia com um teste na característica *Petal Width* ≤ 0.8 , que separa as amostras da classe *setosa* (50 amostras) das demais. Para as amostras restantes, é realizado um novo teste com *Petal Width* ≤ 1.75 , que separa novamente as amostras nas classes *versicolor* (54 amostras) e *virginica* (46 amostras). Em seguida, cada subconjunto resultante é reavaliado usando um novo teste, que continua dividindo os dados até atingir um

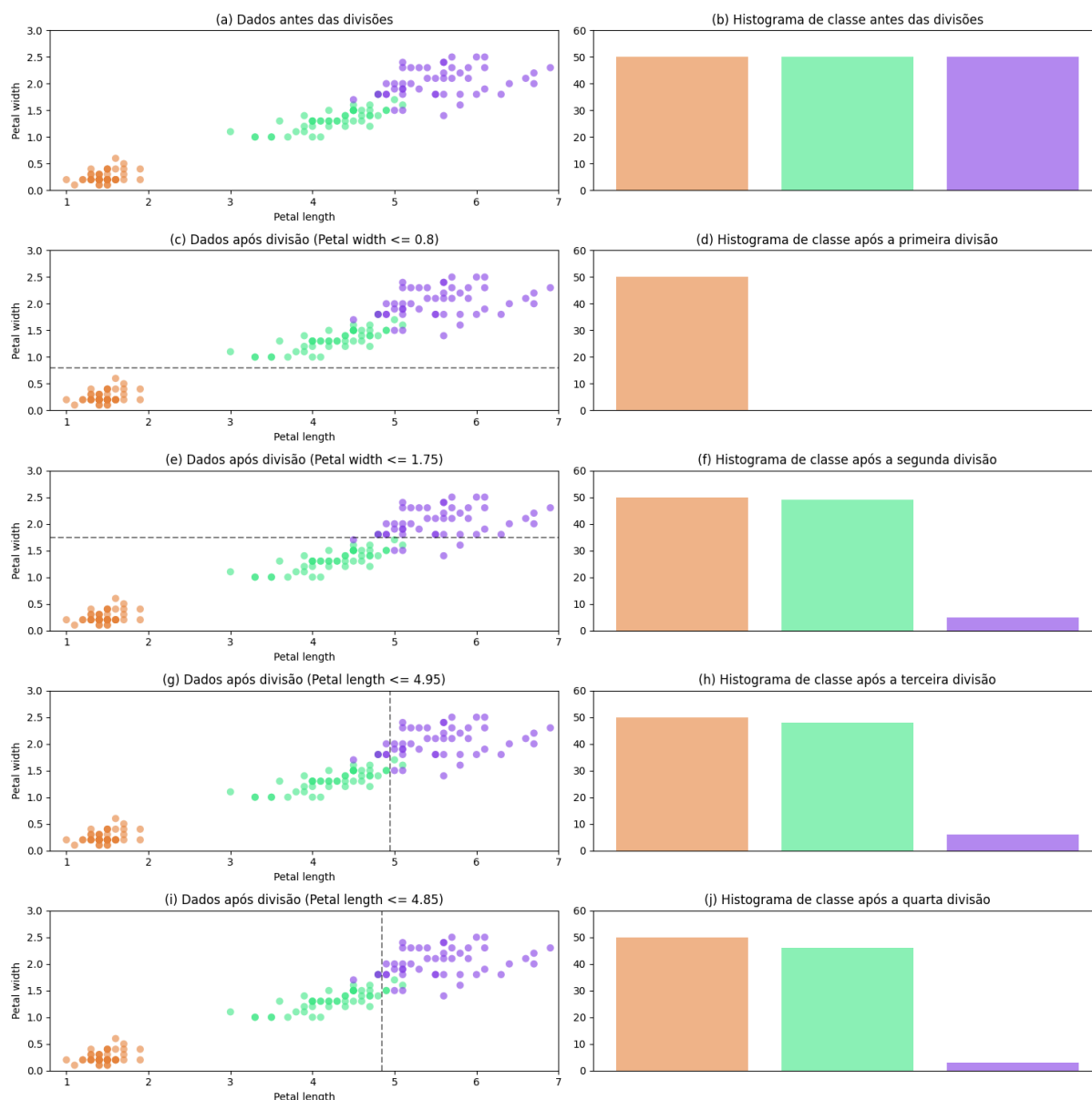
ponto limite. Este ponto limite é alcançado quando não são possíveis novas divisões ou quando são atingidas restrições definidas pelo usuário, que determinam o resultado da decisão. Cada um desses pontos de término é chamado de folha (*leaf*), em que são atribuídas as classes *setosa*, *versicolor* e *virginica* (ROKACH, 2016).

Além disso, cada teste realizado depende do resultado do teste anterior, criando uma relação hierárquica de decisões que determina a classificação final a partir das características de entrada. Essa relação de teste é representada na forma de um caminho que determina a decisão de classificação ou regressão (CRIMINISI; SHOTTON; KONUKOGLU, 2012). Por sua vez, Figura 8 (b) apresenta visualização dos resultados da árvore de exemplo por meio da separação das classes em três regiões distintas conforme as características de entrada, demonstrando graficamente as divisões realizadas pela árvore.

O processo de treinamento de árvores de decisão é guiado pelas estatísticas do conjunto de dados de treinamento, sendo aplicados os conceitos de entropia e ganho de informação com o objetivo de aprender os parâmetros que melhor dividam os dados de treinamento (CRIMINISI; SHOTTON; KONUKOGLU, 2012). Em árvores de decisão, a entropia é uma métrica utilizada para quantificar o grau de incerteza ou impureza em um conjunto de dados. Quanto menor a entropia, mais homogêneo é o conjunto de dados analisado. O ganho de informação, por sua vez, quantifica a redução da entropia quando os dados são divididos, auxiliando na identificação das divisões que melhor separam as classes.

Entretanto, é possível observar na Figura 8 (a) que há mais de um nó folha compartilhando o mesmo resultado (e.g., classe *virginica* em roxo). Esse comportamento ocorre porque o algoritmo de treinamento fraciona os dados de treinamento sempre que encontra uma subdivisão capaz de reduzir ainda mais a impureza dos dados, mesmo que os ramos resultem na mesma classe. Consequentemente, cada nó folha pode refletir diferentes intervalos de valores para as características, levando duas ou mais regiões a terem o mesmo resultado de predição. Além disso, no exemplo apresentado, as subdivisões aparecem com tonalidades diferentes (e.g., roxo claro e roxo escuro) para indicar nós distintos que resultam na mesma classe, porém representando variações nos testes de decisão ou nos níveis de probabilidade associados à predição. Neste contexto, a Figura 9 apresenta um exemplo de classificação em que o objetivo é separar as diferentes classes da melhor maneira possível.

Figura 9 Exemplo de divisão das amostras do *dataset Iris* em um modelo de árvore de decisão.



Fonte: Autoria Própria

No início, todos os dados são agrupados, contendo uma distribuição mista de classes *setosa*, *versicolor* e *virginica*. Após a primeira divisão baseada no teste $\text{Petal Width} \leq 0.8$, observamos uma separação clara da classe em laranja (*setosa*) em relação as demais classes *versicolor* (verde) e *virginica* (roxo), evidenciada pelo histograma correspondente, em que a classe *setosa* se torna predominantemente pura no subconjunto resultante.

A segunda divisão considera o teste $\text{Petal Width} \leq 1.75$ para separar a classe *virginica* das demais classes. Ao encadear as duas avaliações, há uma redução adicional na mistura das classes, particularmente ao separar a classe

virginica das demais. Em cada etapa subsequente, são utilizados novos critérios de divisão melhorar a separação dos dados, como *Petal Length* ≤ 4.95 e *Petal Length* ≤ 4.85, refinando ainda mais a organização dos dados.

Os histogramas a cada exemplo de divisão mostram visualmente como a entropia diminui em cada nó da árvore de decisão, com um aumento gradual da pureza das classes em cada grupo resultante. Esse processo demonstra como a árvore de decisão divide as características dos dados de treinamento de forma hierárquica, buscando sempre otimizar as divisões para reduzir a incerteza e melhorar a precisão da classificação final.

Como exemplificado anteriormente, o processo de treinamento de árvores de decisão, para o caso de classificação, baseia-se na maximização do ganho de informação ou na redução da entropia (I_j) em um nó (j), que é dado por:

$$I_j = H(S_j) - \sum_{i \in L,R} \frac{|S_j^i|}{|S_j|} H(S_j^i), \quad (1)$$

onde S_j é o conjunto de exemplos do nó j , $|S_j^i|$ é a cardinalidade da divisão i , esquerda (L) e direita (R), do nó j , e $H(S_j)$ é a entropia em S_j , dada por:

$$H(S) = - \sum_{c \in \Omega} P(c) \log(P(c)), \quad (2)$$

onde $P(c)$ é a porcentagem/probabilidade de exemplos da classe c em relação ao número total de exemplos de todas as classes Ω (CRIMINISI; SHOTTON; KONUKOGLU, 2012).

Em resumo, cada nó busca uma divisão que maximize o ganho de informação, de modo que os nós folha apresentem predominantemente exemplos de uma dada classe c (CRIMINISI; SHOTTON; KONUKOGLU, 2012). Nesse contexto, a Figura 8 (a) mostra um exemplo de uma árvore gerada pela maximização apresentada na Equação 1. Em cada nó da árvore são exibidos os seguintes parâmetros: o índice de Gini, que mede a impureza do nó; *samples*, que indica o número de amostras que chegaram até esse nó; *value*, que representa a distribuição das amostras entre as diferentes classes (e.g., [0, 47, 5] significa 0 amostras da primeira classe, 47 amostras da segunda classe e 5 amostras da terceira); e *class*, que indica a classe prevista para aquele nó, com base na maioria das amostras nele presentes.

O uso de árvores de decisão oferece como principal vantagem a simplicidade na avaliação dos dados e o bom desempenho na previsão. No entanto, quando esse modelo é utilizado isoladamente, com apenas uma única árvore de decisão, ele pode apresentar desvantagens significativas, resultando em baixo desempenho preditivo devido ao *overfitting* (CRIMINISI; SHOTTON; KONUKOGLU, 2012). Essa limitação é ainda mais evidente em casos em que há um grande número de atributos, pois a análise isolada pode levar à replicação de testes e à perda de relevância dos atributos (ROKACH, 2016).

Para superar essas limitações, métodos de *ensemble*, como o *Random Forest* e o *Gradient Boosting*, foram desenvolvidos. Esses métodos combinam múltiplas árvores de decisão para melhorar a precisão e a robustez das previsões. O *Random Forest*, em particular, consiste em um conjunto de T árvores de decisão, cada uma treinada de forma independente e aleatória, utilizando diferentes subconjuntos de dados e atributos. Essa abordagem reduz as correlações (*i.e.*, grau de associação entre duas variáveis) entre as previsões das diferentes árvores, resultando em um modelo de previsão mais robusto e confiável. Ao combinar os resultados de múltiplas árvores, o *Random Forest* consegue capturar melhor a variabilidade nos dados e reduzir o risco de *overfitting*, proporcionando uma melhoria significativa em relação ao uso de uma única árvore de decisão (CRIMINISI; SHOTTON; KONUKOGLU, 2012; ROKACH, 2016).

2.2.2. Modelos de árvores de decisão

As árvores de decisão são reconhecidas na área de *machine learning* por sua simplicidade e eficácia em diversas aplicações. Nesse contexto, diversos algoritmos de árvores de decisão foram desenvolvidos, cada um com características, vantagens e aplicações específicas. Esta seção apresenta uma revisão dos principais modelos de árvores de decisão encontrados na literatura, como *Classification And Regression Tree* (CART), *Iterative Dichotomiser 3* (ID3), *C4.5*, *Chi-squared Automatic Interaction Detection* (CHAID), *Random Forests*, e *Gradient Boosting*, destacando que cada um desses algoritmos adota abordagens distintas para a divisão de nós, seleção de atributos e aplicação de técnicas de *pruning* ou poda, que elimina ramos que contribuem pouco para a precisão do modelo, com o

objetivo de minimizar a complexidade do modelo e evitar o *overfitting* (CRIMINISI; SHOTTON; KONUKOGLU, 2012; ROKACH, 2016).

O algoritmo CART foi introduzido por Breiman *et al.* (1984) para tarefas de classificação e regressão. Este modelo apresentou novos conceitos às árvores de decisão como o critério de impureza de *Gini* e a divisão binária, que também são adotados posteriormente por outros algoritmos para a seleção de atributos a serem utilizados nas divisões dos nós. Ainda, o algoritmo CART se destaca pela sua flexibilidade, pois é capaz de lidar tanto com variáveis categóricas quanto numéricas, e sua estrutura binária permite divisões simples e de fácil interpretação. Além disso, o processo de *pruning* utilizado pelo CART é um processo crucial para evitar o *overfitting* do modelo (MIENYE; JERE, 2024).

O Índice de Gini, também chamado de medida de impureza de Gini, é utilizado para avaliar a qualidade de uma divisão em uma árvore de decisão. Essa métrica calcula a probabilidade de um elemento ser classificado incorretamente ao ser atribuído a uma classe de forma aleatória. Este índice varia de 0 a 1, sendo que valores próximos a 0 indicam uma maior pureza, ou seja, a maioria dos elementos pertencem à mesma classe. Em contrapartida, valores próximos de 1 indicam maior impureza, apresentando classes distintas nos grupos resultantes. A Equação 3 apresenta o cálculo do Índice de Gini:

$$Gini(S) = 1 - \sum_{i=1}^n p_i^2 \quad (3)$$

Onde S representa o conjunto de amostras, n é o número de classes, e p_i é a proporção das amostras que pertencem à classe i .

O algoritmo ID3, introduzido por Quinlan (1986), utiliza o ganho de informação para selecionar atributos e determinar a melhor divisão em cada nó. Esse ganho de informação mede a redução da entropia após uma divisão, criando uma estrutura de árvore que maximiza a informação obtida a cada decisão. Entretanto, apesar de ser eficaz para conjuntos de dados pequenos e bem definidos, o algoritmo ID3 também pode apresentar problemas de *overfitting*, bem como não lida bem com atributos contínuos sem uma discretização prévia (MIENYE; JERE, 2024).

Já o algoritmo C4.5 é uma extensão do algoritmo ID3 introduzida por Quinlan (1993), que corrige limitações de seu predecessor. Ele inclui a capacidade

de lidar com atributos contínuos e a introdução da razão de ganho (*gain ratio*) para seleção dos atributos, evitando o viés de seleção para atributos que possuem grande número de valores distintos. Além disso, o algoritmo C4.5 também implementa técnicas de *pruning* para reduzir a complexidade da árvore e reduzir o *overfitting*. Essas melhorias permitem que o algoritmo C4.5 seja mais flexível e robusto para uma variedade maior de conjuntos de dados, incluindo atributos contínuos ou conjuntos de dados com valores ausentes. Já o algoritmo C5.0, é uma melhoria em relação ao C4.5, sendo mais rápido e eficiente em relação ao uso de memória. Além disso, ele traz aprimoramentos em relação ao *pruning* e a capacidade de lidar com novos tipos de dados (MIENYE; JERE, 2024).

O CHAID é um algoritmo de árvore de decisão proposto por Kass (1980) que utiliza o teste Qui-quadrado (X^2) para selecionar as melhores divisões entre os nós. Esta medida estatística é usada para avaliar a associação entre variáveis categóricas, comparando as frequências observadas com as frequências esperadas em uma distribuição. Um valor de X^2 mais alto indica uma associação mais forte entre o atributo e a variável-alvo, sugerindo que o atributo é um bom preditor para dividir o conjunto de dados, ou seja, reduz incertezas e aumenta a capacidade do modelo de fazer previsões corretas (MIENYE; JERE, 2024).

Os algoritmos tradicionais de árvores de decisão, como CART, ID3, C4.5, C5.0 e CHAID trazem como sua principal vantagem a sua simplicidade e capacidade de interpretar facilmente os resultados *overfitting* (CRIMINISI; SHOTTON; KONUKOGLU, 2012; ROKACH, 2016; MIENYE; JERE, 2024). No entanto, esses algoritmos apresentam algumas limitações significativas que afetam sua eficácia em certos contextos. Um dos principais problemas é o *overfitting*, que ocorre quando as árvores de decisão tendem a se ajustar muito bem aos dados de treinamento, mas não se ajustam bem para novos dados, ocasionando erros em maior quantidade na classificação ou regressão. Outro problema comum é a sensibilidade a pequenas variações nos dados de entrada, de forma que mudanças mínimas nos dados podem resultar em árvores significativamente diferentes, o que implica em instabilidade nos modelos criados por algoritmos de árvores de decisão tradicionais (ROKACH, 2016).

Os métodos de *ensemble* surgem como alternativas para superar estas limitações dos modelos de árvores de decisão tradicionais. Os métodos de *ensemble* são técnicas de *machine learning* que combinam múltiplos modelos para

melhorar a precisão e a robustez das previsões. A ideia central destes modelos é que, ao combinar as previsões de várias árvores de decisão, os erros individuais de cada árvore podem ser compensados, resultando em um modelo mais estável e preciso (ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022).

Entre as técnicas de *ensemble*, destacam-se as técnicas como *Bagging* (*Bootstrap Aggregating* ou agregação por bootstrap) e *Boosting* (ou Impulsionamento) que utilizam abordagens específicas para construir e combinar diferentes árvores de decisão. Na técnica *Bagging*, são construídas múltiplas versões de um modelo de treinamento usando diferentes amostras de dados e, em seguida, combina suas previsões para produzir um resultado final mais robusto. Enquanto nos modelos *Boosting*, há uma série de modelos simples que são ajustados de maneira iterativa, com cada novo modelo corrigindo os erros dos anteriores para criar um modelo final mais preciso e robusto (BREIMAN, 2001; ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022).

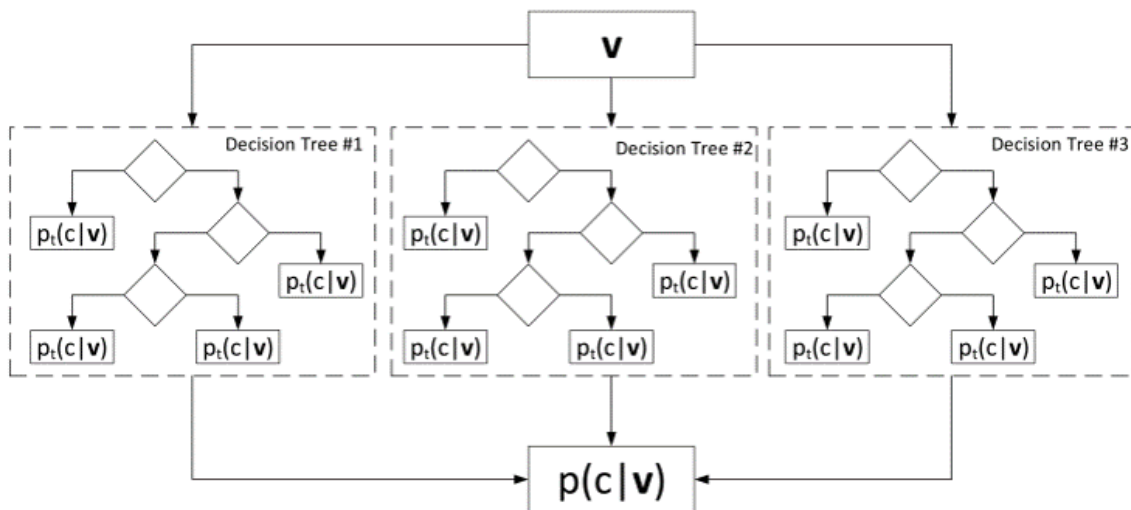
Entre os algoritmos que utilizam os conceitos de *Bagging*, destaca-se o *Random Forest*. Esse algoritmo cria múltiplas árvores de decisão de forma aleatória e independente para melhorar a precisão e robustez das previsões. Cada árvore é treinada em um subconjunto aleatório dos dados e dos atributos, sendo que as previsões são agregadas por votação majoritária (para classificação) ou cálculo da média (para regressão). A principal vantagem deste algoritmo é a sua capacidade de resolver problemas relacionados ao *overfitting*, bem como a sua capacidade de lidar com dados ausentes. No entanto, uma limitação é que, para alcançar melhores resultados de previsão, é necessário utilizar um grande número de árvores, o que aumenta o tempo necessário para o treinamento e a realização das previsões (ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022; MIENYE; JERE, 2024).

A Figura 10 mostra um exemplo da etapa de teste do algoritmo *Random Forest* combinando a decisão de três diferentes árvores de decisão. O funcionamento do algoritmo *Random Forest* pode ser resumido nos seguintes passos:

1. **Etapa de Treinamento:** Para a árvore $t = 1$ até $t = T$:
 - a. Seleciona aleatoriamente um subconjunto $\mathcal{F}$ dos dados de treinamento.

- b. Para cada subconjunto $\mathcal{F}$, gera-se uma árvore de tamanho predeterminado, repetindo os seguintes passos recursivamente:
 - i. Seleciona-se aleatoriamente um subconjunto de m atributos dos p disponíveis;
 - ii. Treina-se a árvore neste subconjunto $\mathcal{F}$ com m atributos, utilizando critérios com base no ganho de informação.
 2. **Etapas de Teste:** Para um novo exemplo caracterizado pelo vetor de características v , o resultado (ou seja, o processo de votação) é determinado pela média das probabilidades $p_t(c | v)$ calculadas pelas T árvores.

Figura 10 Etapas de teste do *Random Forest*.



Fonte: Pordeus et al., 2023

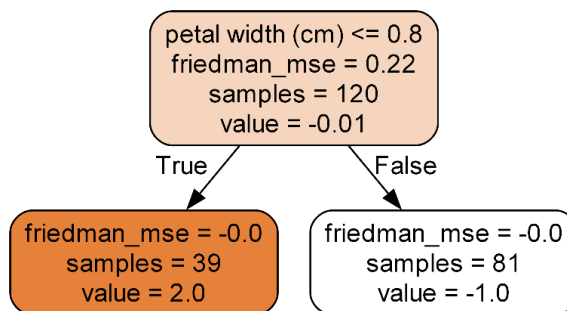
Outra abordagem para a construção de modelos *ensemble* com maior acurácia, é o uso do método *Boosting*. Neste método, os algoritmos de treinamento aprendem sequencialmente, ajustando-se para corrigir os erros cometidos por modelos anteriores. Em cada iteração, as instâncias de treinamento que foram previstas incorretamente recebem um peso maior, aumentando sua importância no treinamento subsequente. Isso permite que o modelo final se concentre em corrigir os erros dos modelos anteriores, resultando em um desempenho geral aprimorado.

Um exemplo de algoritmo baseado em *Boosting* é o algoritmo *Gradient Boosting Trees* que também pode ser utilizado tanto para previsões de classificação,

quanto previsões de regressão. Diferente do modelo *Random Forest* que faz uso de múltiplas árvores treinadas de forma independente, o *Gradient Boosting* adota uma abordagem sequencial para o treinamento das árvores em que realiza a otimização do modelo baseada em gradiente descendente para minimizar os erros. Desta forma, este algoritmo se destaca por sua capacidade de aprender padrões complexos nos dados de entrada, corrigindo iterativamente os erros das árvores treinadas anteriormente. Apesar de sua alta eficácia, o algoritmo *Gradient Boosting* pode ser sensível ao *overfitting*, especialmente quando o número de árvores é elevado ou quando o ajuste dos hiperparâmetros não é realizado de maneira adequada (ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022).

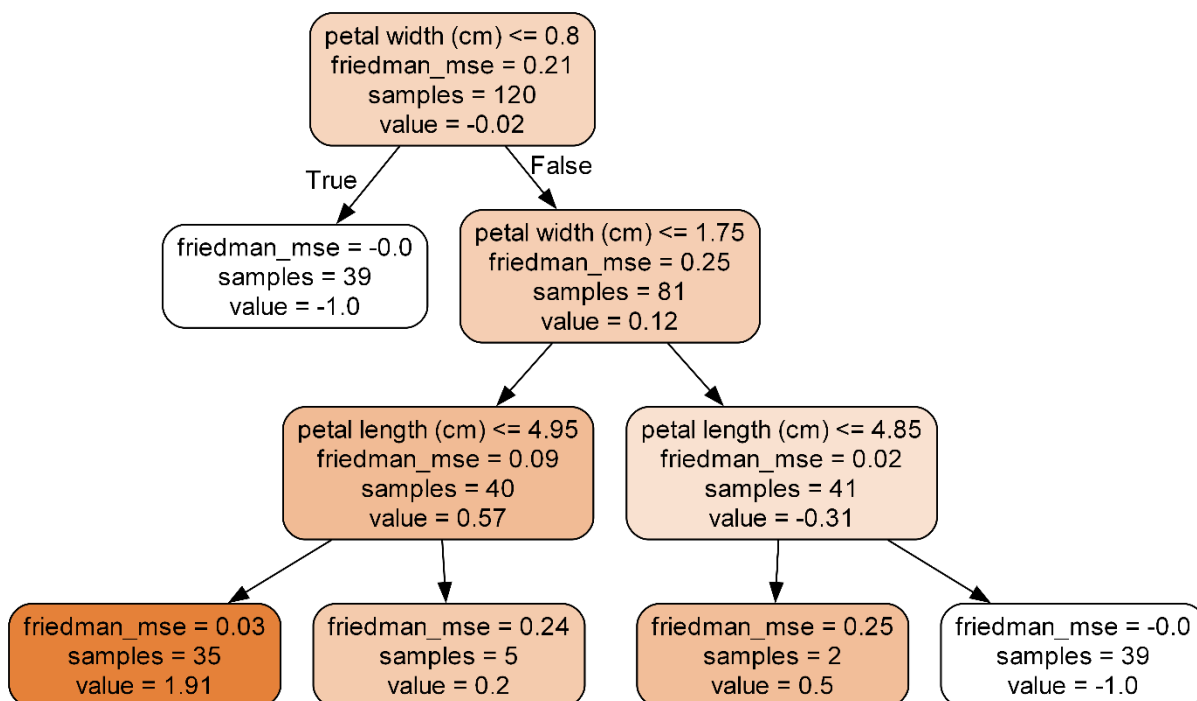
Durante este processo iterativo de refinamento das previsões, observa-se que várias árvores acabam realizando avaliações sobre as mesmas condições em diferentes momentos do treinamento e execução deste modelo, conforme apresentado na Figura 11, Figura 12 e Figura 13. Por exemplo, na Figura 11, a árvore inicialmente utiliza a avaliação *Petal Width* ≤ 0.8 como critério de divisão inicial. Já na Figura 12, observa-se que o modelo utiliza a mesma avaliação da Figura 11 e adiciona novos critérios como *Petal Width* ≤ 1.75 , *Petal Length* ≤ 4.95 e *Petal Length* ≤ 4.85 . Por fim, a Figura 13 reforça este refinamento ao repetir parte das avaliações da Figura 12 e novamente adicionar novas divisões, que tornam o modelo cada vez mais especializado em corrigir os erros residuais deixados pelas árvores anteriores. Embora apresente redundâncias, este comportamento é essencial para o modelo, pois permite que cada nova árvore refine progressivamente as previsões anteriores, resultando em um modelo final mais robusto e preciso.

Figura 11 Exemplo contendo a primeira árvore treinada com o algoritmo *Gradient Boosting*.



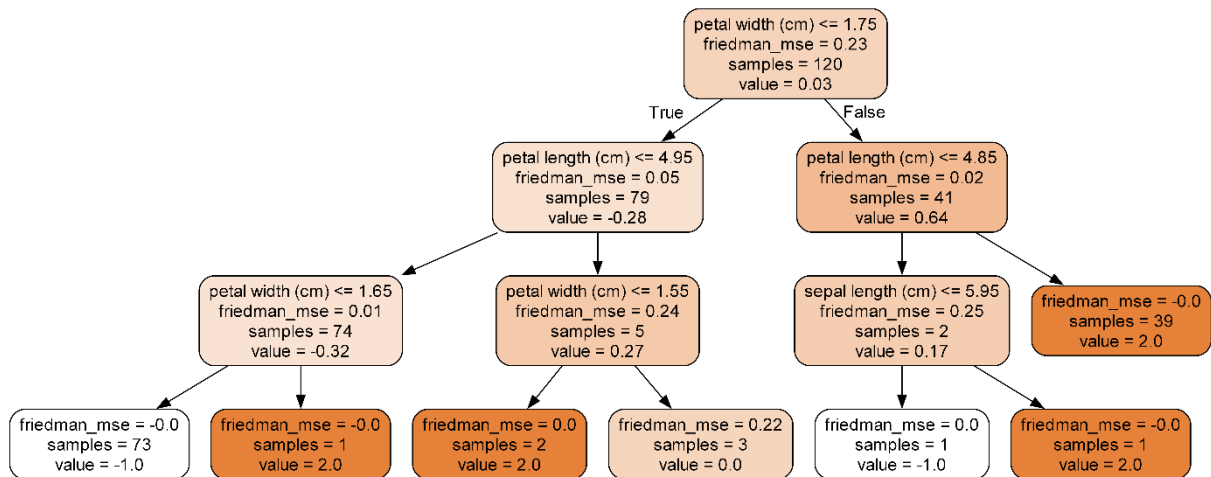
Fonte: Autoria Própria

Figura 12 Exemplo contendo a segunda árvore treinada com o algoritmo *Gradient Boosting*.



Fonte: Autoria Própria

Figura 13 Exemplo contendo a terceira árvore treinada com o algoritmo *Gradient Boosting*.



Fonte: Autoria Própria

O funcionamento do algoritmo *Gradient Boosting* pode ser resumido nos seguintes passos:

1. **Inicialização do modelo:** Inicializa o modelo com um valor constante.
2. **Etapa de Treinamento:** Para a árvore $t = 1$ até $t = T$:
 - a. Para cada instância nos dados de treinamento, calcular o gradiente negativo da função de perda em relação às previsões atuais do modelo.
 - b. Ajustar a árvore de decisão aos valores de gradiente negativo, usando os dados de entrada como características e os valores de gradiente negativo como variáveis alvo.
 - c. Atualizar o modelo adicionando a nova árvore, ponderada por uma taxa de aprendizado η , ao modelo atual.
3. **Etapa de Teste:** Para um novo exemplo, é feita a previsão para uma nova entrada de dados somando as previsões das várias árvores.
 - a. Para regressão, a predição final é a soma das previsões de todas as árvores, dada por:

$$f(x) = \sum_{t=1}^T \eta h_t(x) \quad (4)$$

onde η é a taxa de aprendizado e $h_t(x)$ é a previsão da árvore t .

- b. No caso binário (duas classes), a predição final é a probabilidade da classe positiva, calculada aplicando uma função sigmoide à soma das predições de todas as árvores.

$$f(x) = \frac{1}{1 + e^{-\sum_{t=1}^T \eta h_t(x)}} \quad (5)$$

onde η é a taxa de aprendizado e $h_t(x)$ é a previsão da árvore t . Por sua vez, a taxa de aprendizado (*learning rate*) corresponde a um fator multiplicativo que permite reduzir a contribuição de cada árvore para o modelo em cada iteração.

- c. Para classificação com múltiplas classes:

$$f(x) = \operatorname{argmax} \left(e^{\left(\left(\sum_{t=1}^T \eta h_t(x) \right) - \left(\log \sum_{c=1}^C e^{\sum_{t=1}^T \eta h_t(x)} \right) \right)} \right) \quad (6)$$

onde η é a taxa de aprendizado e $h_t(x)$ é a previsão da árvore t , e C é o número de classes.

Há outros modelos que também estendem os conceitos do algoritmo *Gradient Boost* em busca de melhorias em relação à acurácia dos resultados ou melhorias no desempenho. Por exemplo, o modelo *Extreme Gradient Boosting* (XGBoost) busca reduzir o *overfitting* e diminuir a necessidade de pré-preparação dos dados antes do treinamento, enquanto o algoritmo *Light Gradient Boosting Machine* (LightGBM) busca tornar o processo de treinamento mais rápido, diminuindo a complexidade do modelo (MIENYE; SUN, 2022).

Os métodos de *ensemble* ajudam a mitigar problemas comuns nos métodos tradicionais de árvores de decisão, como *overfitting* e variância elevada. A capacidade de generalizar melhor para novos dados e de lidar de maneira mais eficaz com variações nos conjuntos de dados faz com que os métodos de *ensemble* sejam amplamente utilizados em aplicações modernas de aprendizado de máquina (ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022). Além disso, a possibilidade dos métodos de *ensemble*, como *Random Forest* e *Gradient Boost*, serem executados de forma paralela, os torna adequados para execução em plataformas de *hardware*, como *GPUs* e *FPGAs*, aumentando significativamente sua eficiência computacional.

2.2.3. Implementação de árvores de decisão em *hardware*

O crescente aumento do volume de dados e a complexidade dos problemas a serem resolvidos por modelos de *machine learning* levaram à aplicação de tecnologias de *hardware* como GPUs e FPGAs para acelerar tanto os processos de treinamento ou aprendizado, quanto os processos de tomada de decisões desses algoritmos (REUTHER *et al.*, 2020; SAIDI *et al.*, 2021; ZAMAN *et al.*, 2022; WANG *et al.*, 2022; CAPOGROSSO *et al.*, 2024). Neste contexto, Narayanan *et al.* (2007) apresentaram uma das primeiras implementações de uma arquitetura de *hardware* para acelerar o processo de treinamento, especificamente, através da aceleração do cálculo do índice Gini.

Posteriormente, Van Essen *et al.* (2012) propuseram o conceito de *Compact Random Forests* (CRFs) e realizou uma análise comparativa deste conceito entre FPGAs, GPUs e CPUs *multicore*. Este trabalho explorou o paralelismo inerente dos algoritmos de árvore de decisão, em que cada árvore classifica as amostras de forma independente e só necessita sincronização ao combinar os resultados finais, permitindo a execução de uma árvore treinada com tamanho limitado que se adequasse aos dispositivos avaliados sem comprometer a precisão (VAN ESSEN *et al.*, 2012).

É pertinente destacar que a implementação dos algoritmos *Random Forest* e *Gradient Boosting* em dispositivos FPGA continuou sendo objeto de pesquisa em diferentes trabalhos na literatura. Portanto, esta seção visa apresentar uma revisão simples dos principais trabalhos relacionados à implementação de árvores de decisão e seus derivados em *hardware*, com foco nos avanços relevantes para os objetivos desta tese.

Esta revisão pragmática da literatura foi conduzida utilizando cinco bases de dados principais: Google Scholar, IEEE Xplore, Science Direct, Springer Link e ACM Digital Library, com período de busca entre 2014 e 2024. Como primeiro passo para esta revisão, foi elaborada a estratégia de busca combinando palavras-chave relacionadas aos algoritmos de árvore de decisão, plataformas de execução em *hardware* e tipos de revisão, utilizando os operadores booleanos AND (e) e OR (ou). Em seguida, foram aplicados critérios de inclusão e exclusão para refinar os trabalhos encontrados, tendo foco em trabalhos que tiveram como objetivo a aceleração da execução ou treinamento de árvores de decisão em *hardware*.

Foram aplicadas as seguintes expressões para busca:

- ("decision tree" OR "ensemble learning" OR "random forest" OR "gradient boosting") AND ("GPU" OR "FPGA")
- ("decision tree" OR "ensemble learning" OR "random forest" OR "gradient boosting") AND ("GPU" OR "FPGA") AND ("systematic review" OR "literature review" OR "survey")
- ("random forest" OR "gradient boosting") AND ("FPGA") AND ("systematic review" OR "literature review" OR "survey")
- ("random forest" OR "gradient boosting" OR "XGBoost") AND ("fpga" OR "hls" or "high level synthesis") AND ("systematic review" OR "literature review" OR "survey")

Foram aplicados os seguintes critérios de inclusão:

- Artigos publicados em periódicos e conferências revisados por pares.
- Estudos que abordam algoritmos baseados em árvores de decisão, especialmente *Random Forest* e *Gradient Boosting*, e suas implementações com aceleração de *hardware*.
- Trabalhos que incluam uma revisão da literatura ou revisão sistemática.
- Estudos focados em implementações com FPGA e GPU.

Também foram aplicados critérios de exclusão de artigos:

- Estudos que se concentraram apenas em árvores de decisão, sem discutir métodos de aceleração de *hardware*.
- Estudos que se concentraram apenas em métodos de aceleração em *hardware* de algoritmos de *machine learning* em geral.
- Artigos que não estejam disponíveis em inglês ou sem texto completo acessível

No final, foram analisados de forma completa 27 artigos, conforme os critérios apresentados, que compõem a revisão deste trabalho. Nesse sentido, a Tabela 3 apresenta um resumo de cada trabalho, apresentando seu objetivo e os resultados alcançados.

Tabela 3 Revisão pragmática da literatura.

Autor	Categoria	Objetivo	Resultado
(OWAIDA et al., 2014)	Execução	Propor um mecanismo de classificação que utiliza a combinação de CPU e FPGA para processar conjuntos de árvores de decisão, como o <i>XGBoost</i> , de maneira eficiente e escalável.	A solução proposta apresentou uma aceleração de até 20 vezes em comparação com a versão em CPU para processar o conjunto de árvores em FPGA. Em cenários em que o conjunto de árvores não coube inteiramente na memória da FPGA, a combinação CPU-FPGA resultou em um ganho de desempenho de até uma ordem de magnitude em relação ao uso exclusivo de CPU.
(BARBA et al., 2015)	Execução	Propor um componente de <i>hardware</i> dinâmico e recursivo para acelerar operações de árvores semânticas e compará-las com uma implementação de <i>software</i> equivalente.	A implementação em FPGA mostrou diminuição significativa no tempo de execução em comparação a versão equivalente em <i>software</i> .
(BARBARESCHI et al., 2015)	Execução	Propor uma abordagem em <i>hardware</i> para a regra de votação majoritária em <i>Random Forest</i> .	A abordagem de <i>hardware</i> teve sucesso na votação majoritária com aceleração significativa em relação ao <i>software</i> .
(NAKAHARA et al., 2016)	Execução	Acelerar a classificação do algoritmo <i>Random Forest</i> usando o <i>Altera SDK</i> para OpenCL.	A implementação do acelerador em FPGA foi 10.7 vezes mais rápida que em GPU e 14.0 vezes mais rápida que CPU, com eficiência de consumo energético 61.3 vezes melhor que em GPU 12.1 vezes melhor do que CPU.
(LIN; BLANTON; THOMAS, 2017)	Execução	Este artigo explora três arquiteturas diferentes para implementação de <i>Random Forest</i> em FPGAs, visando otimizar o uso de recursos e o tempo de troca de contexto entre aplicações. As arquiteturas propostas são: centrada em memória, centrada em comparadores e centrada em síntese.	A arquitetura centrada em memória apresentou o menor tempo de troca de contexto, mas usou mais recursos do FPGA. A arquitetura centrada em comparadores alcançou um equilíbrio entre uso de recursos e tempo de troca. A arquitetura centrada em síntese usou menos recursos, mas teve o maior tempo de troca de contexto.

(STRUHARIK; VUKOBRATOVIĆ, 2018)	Treinamento	O artigo propõe o DTEEP, um coprocessador que acelera a indução de conjuntos de árvores de decisão por meio de uma abordagem evolutiva, de forma que "indução" se refere ao processo de treinar modelo de árvores de decisão a partir dos dados.	Os resultados mostraram que o DTEEP conseguiu acelerar significativamente o tempo de treinamento de árvores de decisão em comparação com soluções de <i>software</i> , melhorando o desempenho em todos os testes realizados.
(BUSCHJÄGER <i>et al.</i> , 2018)	Execução	Comparar as arquiteturas de CPUs e FPGAs para a filtragem de dados de sensores usando <i>Random Forest</i> e identificar qual delas oferece melhor desempenho em termos de tempo de execução e consumo de energia em sistemas embarcados.	As CPUs ofereceram maior, maior <i>throughput</i> com uma implementação <i>if-else</i> , enquanto as FPGAs se destacaram em termos de baixo consumo de energia.
(OWAIDA; ALONSO, 2018)	Execução	Explorar técnicas para mapear a inferência de <i>ensembles</i> de árvores de decisão em clusters de FPGAs em <i>datacenters</i> .	As técnicas propostas permitem distribuir eficientemente as computações em múltiplas FPGAs, superando limitações de recursos e balanceando a largura de banda de rede e computação.
(WANG, LI, HE, 2020)	Execução	Propor um <i>framework</i> chamado HERALD, um novo método para a redução e execução em <i>hardware</i> de conjuntos de árvores de decisão a partir do algoritmo <i>Random Forest</i> , visando otimizar recursos computacionais em dispositivos restritos, como FPGAs.	O <i>framework</i> proposto permitiu reduzir tanto o consumo de energia, quanto a quantidade de recursos 21.5% e 62% respectivamente. Além disso, a abordagem proposta conseguiu reduzir recursos computacionais sem comprometer (e às vezes até melhorando) a performance de classificação.
(SAYED <i>et al.</i> , 2020)	Execução	Apresentar um <i>framework</i> que automatiza a criação de circuitos integrados (IC) a partir de modelos <i>Random Forest</i> .	Implementação em ASIC apresentou menor consumo de energia com alta precisão em classificadores gerados automaticamente.
(SUMMER <i>et al.</i> , 2020)	Execução	Descrever a implementação de árvores de decisão (BDTs) na biblioteca <i>hls4ml</i> , que traduz modelos treinados para <i>firmware</i> baseados VHDL e ferramentas HLS para acelerar a inferência em tempo real.	A <i>hls4ml</i> mostrou-se eficiente para converter modelos de árvores de decisão, com economia de recursos e baixo tempo de inferência em FPGAs. A implementação de BDTs mostrou desempenho comparável a redes neurais, mas com diferentes padrões de uso de recursos.

(ALCOLEA; RESANO, 2021)	Execução	Apresentar um acelerador baseado em FPGA para executar árvores de decisão baseadas em <i>Gradient Boosting</i> , visando a execução em sistemas embarcados. Particularmente, o processamento de imagens hiperespectrais.	Os resultados obtidos demonstram o potencial dos aceleradores FPGA para GBDT em sistemas embarcados. O acelerador desenvolvido mostrou-se significativamente mais eficiente que soluções baseadas em CPU.
(BARBARESCHI; BARONE; MAZZOCCA, 2021)	Execução	Otimizar o uso de recursos em sistemas de classificação múltipla baseados em árvores de decisão. O foco é reduzir a sobrecarga de área de <i>hardware</i> em dispositivos FPGA, mantendo uma precisão aceitável nas classificações.	A metodologia proposta mostrou-se eficaz, alcançando reduções substanciais nos requisitos de área de <i>hardware</i> (até 50% em alguns casos) enquanto mantém perdas mínimas de precisão (aproximadamente 0,2%).
(KANANI; VAIDYA; AGARWAL, 2021)		Desenvolver uma biblioteca chamada <i>LightFPGA</i> , que automatiza a extração de detalhes de um modelo <i>LightGBM</i> pré-treinado e gera o código Verilog correspondente para implementação em FPGA.	A implementação em FPGA alcançou uma melhoria de 100 a 400 vezes na latência em comparação com execuções em CPU, sem redução na precisão de inferência. Foi observada uma redução de 7 a 8 vezes no consumo de energia na implementação em FPGA em comparação à CPU. Além disso, a solução demonstrou ser eficiente e escalável, otimizando automaticamente o consumo de recursos e alcançando o máximo de paralelismo possível.
(MOLINA <i>et al.</i> , 2021)	Execução	Apresentar e avaliar arquitetura baseadas em <i>System-on-Chip</i> (SoC) baseadas em FPGAs para acelerar inferência com modelos de <i>machine learning</i> baseados em <i>ensembles</i> de árvores de decisão. Especificamente, o estudo foca na implementação eficiente do algoritmo <i>QuickScorer</i> (QS) em dispositivos SoC, buscando melhorar o desempenho em termos de tempo de inferência e escalabilidade, mesmo em cenários com grandes quantidades de árvores.	Os resultados mostram que a solução proposta em FPGA apresente tempos de execução quase constantes, independentemente do número de árvores no modelo.
(BAUMGARTNER; HUEMER; LUNGLMAYR, 2022)	Execução	Apresentar uma nova arquitetura para votação majoritária em <i>hardware</i> digital e demonstrar sua aplicação em um acelerador de <i>hardware</i> para <i>Random Forest</i> . A eficácia da arquitetura é avaliada através do desempenho na classificação de dígitos manuscritos.	A arquitetura proposta permite obter os resultados da votação majoritária em que o número de ciclos de <i>clock</i> necessários tem relação logarítmica com a quantidade de entradas. Para o exemplo de reconhecimento de dígitos manuscritos, a arquitetura

			implementada em FPGA permitiu a classificação de mais de 7 milhões de imagens por segundo, demonstrando o potencial de execução de árvores de decisão em <i>hardware</i> .
(DAMIANI et al., 2022)	Execução	Desenvolver o <i>Entree</i> , um fluxo de design automático que integra e estende significativamente os projetos <i>hls4ml</i> e Conifer (SUMMER et al., 2020), permitindo a implementação de conjuntos de árvores de decisão arbitrariamente grandes em FPGAs, superando as limitações de recursos enfrentadas por implementações estáticas.	A arquitetura proposta permitiu a geração de um acelerador de <i>hardware</i> sem limitações em relação a quantidade de árvores de decisão exploradas. Além disso, a latência média é comparável aos obtidos pela implementação estática do <i>Conifer</i> (SUMMER et al., 2020), porém apresentar erros em relação aos limites de recursos.
(GAJJAR et al., 2022)	Execução	Apresentar um acelerador FPGA para inferência binária com o algoritmo <i>XGBoost</i> chamado FAXID, usando ferramentas de síntese de alto nível (HLS). Em seguida, comparar o desempenho do acelerador proposto com implementações em CPU e GPU para demonstrar a eficiência da solução, utilizando diferentes <i>datasets</i> e avaliando aspectos como latência, eficiência energética e custo.	O acelerador proposto alcançou melhorias de desempenho de até 65,8x em relação à CPU e 5,3x em relação à GPU em termos de latência. Este acelerador permite processar múltiplos problemas em paralelo, oferecendo maior flexibilidade e utilização de recursos.
(HE; THOTTETHODI ; VIJAYKUMAR, 2022)	Treinamento e Execução	Propor um acelerador de <i>hardware</i> chamado <i>Booster</i> para otimizar o treinamento e a inferência de modelos de árvores de decisão baseadas em <i>gradient boosting</i> . O <i>Booster</i> explora as características exclusivas deste algoritmo, para maximizar o paralelismo de acesso à memória e melhorar o desempenho em comparação com CPUs <i>multicores</i> e GPUs tradicionais.	O <i>Booster</i> alcançou melhorias de desempenho de 11,4x e 6,4x para o treinamento, e de 45x e 22x para inferência, em comparação com uma CPU <i>multicore</i> ideal de 32 núcleos e uma GPU ideal, respectivamente.
(POORHERAVI ; GAUDET, 2022)	Treinamento	Propor uma arquitetura de <i>hardware</i> para acelerar o treinamento de <i>Random Forest</i> em FPGAs, com o objetivo principal de reduzir o tempo de treinamento sem comprometer a precisão. Além disso, a proposta considera fatores como tempo de execução e utilização de memória na implementação da arquitetura.	A arquitetura proposta permitiu reduzir o tempo de treinamento de modelos usando o algoritmo <i>Random Forest</i> em média 50x comparado à implementação em <i>software</i> , mantendo uma precisão comparável entre as abordagens em <i>hardware</i> e <i>software</i> .

(SHAH et al., 2022)	Execução	Acelerar a classificação do algoritmo <i>Random Forest</i> em GPUs e FPGAs para grandes conjuntos de dados. Modelos atuais enfrentam dificuldades em relação ao tamanho das árvores de decisão geradas e aos acessos irregulares à memória, o que limita o desempenho, especialmente em GPUs. Para resolver esse problema, os autores propõem uma estrutura hierárquica de dados para otimizar o <i>layout</i> de memória das árvores de decisão e desenvolver três variantes de código que exploram paralelismo e eficiência de memória, visando superar as limitações do formato (<i>Compressed Sparse Row</i>) em GPUs e FPGAs. O formato CSR é usado para armazenar estruturas esparsas de forma compacta, mas causa acessos à memória irregulares, o que prejudica a performance em arquiteturas que dependem de acessos de memória sequenciais e eficientes, como as GPUs.	O trabalho apresentou melhorias na classificação de modelos <i>Random Forest</i> em GPUs e FPGAs. Na implementação usando GPU, a implementação foi até 9 vezes mais rápida que a implementação baseada em CSR e superou em até 2x a biblioteca cuML da Nvidia. Em FPGA, a variante em que cada árvore de decisão é processada separadamente foi a mais eficaz, alcançando uma aceleração de até 109 vezes em relação ao CSR. Esses resultados demonstram que as otimizações específicas para cada arquitetura melhoraram substancialmente o desempenho da classificação, especialmente quando a profundidade das subárvores foi ajustada para explorar melhor os recursos das GPUs e FPGAs.
(ZHU et al., 2022)	Execução	Propor um acelerador de <i>hardware</i> eficiente para modelos de <i>Deep Forest</i> implementado em FPGA, visando melhorar a velocidade de processamento, precisão de classificação e consumo de energia. Este acelerador aborda desafios específicos do <i>Deep Forest</i>, como o armazenamento de um grande conjunto de árvores de decisão em espaço limitado e o desequilíbrio no processamento dos nós devido a diferentes comprimentos nos caminhos das árvores.	Os experimentos realizados mostraram que o <i>hardware</i> proposto atinge uma aceleração de cerca de 40 vezes em relação a uma CPU de 40 núcleos. A arquitetura implementada no FPGA (<i>Intel Stratix V</i>) também resultou em uma significativa melhoria de eficiência energética em comparação com trabalhos anteriores, utilizando menos recursos de <i>hardware</i>.
(DINH et al., 2023)	Execução	Propor uma arquitetura em FPGA para acelerar a inferência do algoritmo <i>Random Forest</i> em aplicações de computação em borda para <i>IoT</i>. A solução envolve a criação de processadores dedicados à execução de Árvores de Decisão em FPGAs, permitindo ajustar o número de processadores conforme os recursos de <i>hardware</i> disponíveis na plataforma.	Experimentos com as plataformas Kria KV260 e PYNQ-Z2 mostraram melhorias de desempenho de até 28,37x e 13,11x em comparação com um Intel Core i7. Além disso, a arquitetura proposta é energeticamente eficiente, oferecendo uma solução viável para implementar o algoritmo <i>Random Forest</i> em plataformas de computação de borda com recursos limitados, combinando alto desempenho e eficiência energética.

(WANG <i>et al.</i> , 2023)	Execução	Propor o <i>framework HardGBM</i> para otimizar o uso de <i>hardware</i> na implementação do <i>Gradient Boosting</i> , reduzindo o tamanho do <i>ensemble</i> de árvores de regressão sem comprometer a precisão preditiva.	Os experimentos com o <i>framework HardGBM</i> mostram uma redução de 81,60% na utilização de área e de 21,15% no consumo de energia, mantendo ou superando a precisão dos ensembles originais (<i>XGBoost</i> e <i>LightGBM</i>). A solução de <i>hardware</i> proporciona uma aceleração superior a 12 vezes em relação ao uso de CPU. Além disso, o uso de métodos de quantização economiza até 34,37% de recursos de <i>hardware</i> sem comprometer a precisão.
(ALSHARARI <i>et al.</i> , 2024)	Execução	Desenvolver técnicas de quantização para modelos baseados em <i>Gradient Boosting</i> visando implementações eficientes em <i>hardware</i> . Mais especificamente, o trabalho propõe o treinamento com consciência de quantização (QAT) para equilibrar eficiência e desempenho, através do uso apenas de números inteiros e aritmética binária.	Os modelos quantizados e binários propostos superaram trabalhos anteriores em eficiência de recursos e velocidade. O QAT superou a quantização pós-treinamento, mantendo a precisão mesmo com baixas larguras de bits. No melhor caso, a implementação em FPGA usou apenas 170 LUTs e 233 flip-flops alcançou frequência de operação de 724 MHz.

Fonte: Autoria Própria

Em síntese, os trabalhos relacionados apresentados na Tabela 3 discutem diferentes abordagens para treinar, executar e/ou votar em paralelo nos algoritmos de árvores de decisão e seus derivados *ensemble* como *Random Forest* e *Gradient Boosting*. No âmbito de execução em FPGAs, as arquiteturas para execução podem ser classificadas como estáticas ou dinâmicas, cada uma com suas características e vantagens específicas. (LIN; BLANTON; THOMAS, 2017; DAMIANI *et al.*, 2022).

Nas arquiteturas dinâmicas, como as descritas por Lin *et al.* (2017) e Damiani *et al.* (2022), o uso de blocos de memória permite a reconfiguração parcial dos dispositivos FPGA a cada troca de contexto. Isso permite a implementação de novos parâmetros de modelo na memória sem a necessidade de reprogramar completamente o *hardware*. Um exemplo deste modelo de arquitetura é a abordagem *Entree* proposta por Damiani *et al.* (2022), que deriva do estado da técnica, nomeadamente *hls4ml* (aventado na tabela acima e detalhado nos parágrafos que seguem), e utiliza partições reconfiguráveis do FPGA para carregar diferentes árvores de decisão conforme necessário. Esta abordagem é útil para

executar grandes conjuntos de árvores de decisão em FPGAs com recursos limitados, pois permite alternar entre diferentes árvores ou subconjuntos de árvores.

Por outro lado, as arquiteturas estáticas, como também discutidas por Lin *et al.* (2017) utilizam uma abordagem fixa, em que os parâmetros do modelo e a topologia das árvores de decisão são definidos no momento da síntese e carregados permanentemente no *hardware*. Estas arquiteturas são focadas na otimização para um único modelo ou um conjunto específico de modelos que não mudarão ao longo do tempo. Isso resulta em uma execução mais rápida e previsível, já que não há a possibilidade de reconfiguração durante a execução. Contudo, essa abordagem carece de flexibilidade, pois qualquer alteração nos modelos requer uma nova síntese e reprogramação do dispositivo FPGA. Além disso, a quantidade de árvores implementadas depende dos recursos disponíveis no dispositivo FPGA utilizado. Um exemplo de modelo estático seria a implementação de Árvores de Decisão em FPGAs usando a biblioteca *hls4ml*, do estado da técnica e da arte, conforme descrito no artigo de Summers *et al.* (2020). Nessa abordagem, o modelo de árvore de decisão, incluindo todos os parâmetros e limiares, é sintetizado diretamente no *hardware* do FPGA como lógica estática.

A partir dos trabalhos apresentados na Tabela 3 analisou-se quais ferramentas referentes ao estado da arte e da técnica acessível poderiam ser alvos de comparações a uma arquitetura distinta orientada a notificações, à luz dos objetivos deste trabalho de pesquisa. Para fazer esta análise, que é apresentada pela Tabela 4, foram avaliadas as seguintes propriedades pertinentes a esta pesquisa: bibliotecas para treinamento, os algoritmos suportados, ferramental associado e disponibilidade da ferramenta ser acessível via repositório público.

Tabela 4 Ferramental referente ao estado da arte e técnica acessível.

Ferramenta	Algoritmos	Bibliotecas	Acessível	Saída	Referências
<i>hls4ml</i> (Conifer)	• <i>Random Forest</i> • <i>Gradient Boosting</i> • <i>XGBoost</i> • <i>LightGBM</i>	• <i>scikit-learn</i> • <i>XGBoost</i> • <i>TMVA</i> • <i>Tensorflow</i> • <i>ONNX</i>	Sim ³	• Xilinx HLS (Vivado/ Vitis) • VHDL • Forest Processing Unit (FPU)	(Summers <i>et al.</i> , 2020) (DAMIANI <i>et al.</i> , 2022) (ALSHARARI <i>et al.</i> , 2023) (ALSHARARI <i>et al.</i> , 2024)

³ Repositório da ferramenta *hls4ml* (Conifer): <https://github.com/thesps/conifer>

					(GONSKI <i>et al.</i> , 2024) (KARN; KNECHTEL; SINANOGLU, 2024)
<i>LightFPGA</i>	• <i>LightGBM</i>	• <i>LightGBM</i>	Sim ⁴	• Verilog	(KANANI; VAIDYA; AGARWAL, 2021)
<i>FAXID</i>	• <i>XGBoost</i>	• <i>XGBoost</i>	Não	• Xilinx HLS	(GAJJAR <i>et al.</i> , 2022)
<i>HardGBM</i>	• <i>XGBoost</i> • <i>LightGBM</i>	• <i>XGBoost</i> • <i>LightGBM</i>	Não	• HDL	(WANG <i>et al.</i> , 2023)

Fonte: Autoria Própria

A partir da análise feita na Tabela 4, muito embora inexista (ao melhor saber obtido neste trabalho) comparações de performance entre eles, foi escolhida a ferramenta *hls4ml* como estado da arte e da técnica acessível para a realização das comparações dos experimentos deste trabalho de pesquisa. Muito embora não exista comparações em termos de árvores de decisão, o *hls4ml* gera código em Vivado HLS, para o qual sim há uma série de comparações de HLS, conforme Cong *et al.* (2022) e Sozzo *et al.* (2022), nas quais ele se destaca em relação ao estado da técnica e da arte. Além disso, a ferramenta *hls4ml* é adotada e validada em trabalhos recentes da literatura como Summers *et al.* (2020), Damiani *et al.* (2022), Alsharari *et al.* (2023), Alsharari *et al.* (2024), Gonski (2024) e Karn, Knechtel e Sinanoglu (2024).

Por outro lado, as demais ferramentas implementam apenas os algoritmos *XGBoost* e *LightGBM*. Embora esses algoritmos sejam implementações mais recentes e otimizadas do *Gradient Boosting*, optou-se por utilizar as implementações de base dos algoritmos do tipo *ensemble*, como *Random Forest* e *Gradient Boosting*. Esta escolha se deve principalmente pela simplicidade das implementações da biblioteca *scikit-learn*, que mantêm a estrutura fundamental dos algoritmos derivados de árvores de decisão, sem otimizações específicas em relação à acurácia dos modelos. Isto permite uma avaliação mais clara da eficácia da arquitetura a ser proposta neste trabalho de pesquisa.

Outrossim, essa escolha do *hls4ml* também ocorre por esta ferramenta oferecer maior opções no suporte a algoritmos e bibliotecas de treinamento, como a

⁴ Repositório da ferramenta *LightFPGA*: <https://github.com/AlishKanani/LightFPGA>

biblioteca *scikit-learn* (PEDREGOSA *et al.*, 2011), o que não ocorre com as outras abordagens citadas acima. Esta biblioteca é amplamente utilizada como referência para o treinamento de múltiplos modelos de *machine learning* em Python (PEDREGOSA *et al.*, 2011). Mais especificamente no contexto de árvores de decisão, a biblioteca *scikit-learn* oferece suporte a diferentes implementações de algoritmos derivados de árvores de decisão como o *Random Forest* e *Gradient Boosting*, com uma interface de programação consistente, bem estruturada e documentada. Esta interface facilita a extração das estruturas das árvores de decisão após o treinamento, o que é essencial para o processo de mapeamento à arquitetura proposta neste trabalho de pesquisa.

2.3. Conclusão do capítulo

A execução dos algoritmos baseados em árvores de decisão em dispositivos de lógica reconfigurável, como FPGAs, tem sido objeto de diversas pesquisas recentes devido ao seu potencial de aceleração em *hardware*. No entanto, o tamanho limitado dessas plataformas restringe a quantidade de árvores de decisão que podem ser alocadas, o que compromete o grau de paralelismo e, portanto, a eficiência e o desempenho da execução destes algoritmos nas plataformas de *hardware*.

Nos trabalhos apresentados na revisão da literatura em relação à execução de árvores de decisão em *hardware* foram propostas arquiteturas que buscaram “otimização” da execução das árvores de decisão principalmente por meio de técnicas que melhoram o acesso a memória (SHAH *et al.*, 2022), redução das árvores de decisão (WANG *et al.*, 2023), ou por meio de técnicas de quantização⁵ (WANG *et al.*, 2023; ALSHARARI *et al.*, 2024). No entanto, esses trabalhos utilizaram ferramentas principalmente inspiradas em técnicas dos paradigmas tradicionais de computadores, herdando ao menos parte das redundâncias temporais e estruturais já explicadas nesse presente capítulo. Assim sendo, existe a tendência de não explorar adequadamente o paralelismo intrínseco fornecido pela

⁵ Modelos quantizados são aqueles em que a precisão é reduzida para utilizar menos recursos de *hardware*, e.g. uso de valores inteiros de 8 bits ao invés de valores *float* de 32 bits (WANG *et al.*, 2022; WEI *et al.*, 2024).

plataforma de execução. Justamente como uma arquitetura baseada nos princípios do PON supostamente permitiria.

Além das questões acima apresentadas de inércia de paradigmas, as próprias árvores de decisão exacerbariam aquelas redundâncias. Em suma, os métodos tradicionais de árvores de decisão, frequentemente, apresentam problemas como *overfitting* e alta variância, implicando em problemas na qualidade do resultado. Para superar essas limitações, há os métodos de *ensemble*, como o *Random Forest* e o *Gradient Boosting*, que aumentam consideravelmente as redundâncias. Justamente, esses métodos combinam múltiplas árvores de decisão para melhorar a precisão e a robustez das previsões. O *Random Forest*, especificamente, consiste em um conjunto de múltiplas árvores de decisão, cada uma treinada de forma independente e aleatória, utilizando diferentes subconjuntos de dados e atributos. Ao combinar os resultados de múltiplas árvores, o *Random Forest* captura melhor a variabilidade dos dados e reduz o risco de *overfitting*, melhorando significativamente o desempenho em relação ao uso de uma única árvore de decisão (CRIMINISI; SHOTTON; KONUKOGLU, 2012; ROKACH, 2016). Entretanto, nas técnicas de computacionais tradicionais isto naturalmente extrapola as redundâncias já citadas, conforme demonstrado subsequentemente na Seção 4.2.

Por sua vez, o método *Gradient Boosting* apresenta uma abordagem em que o algoritmo de treinamento aprende sequencialmente, ajustando-se para corrigir os erros cometidos por modelos anteriores. Especificamente, este algoritmo adota uma abordagem sequencial para o treinamento das árvores em que realiza a “otimização” do modelo baseada em gradiente descendente para minimizar os erros. Em cada iteração, as instâncias de treinamento que foram previstas incorretamente recebem um peso maior, o que aumenta sua importância no treinamento subsequente. Isso permite que o modelo final foque na correção dos erros dos modelos anteriores, resultando em um melhor desempenho geral (ROKACH, 2016; GONZÁLEZ *et al.*, 2020; MIENYE; SUN, 2022). Isso acarreta uma maior quantidade de redundâncias estruturais e temporais durante a execução para a realização das previsões, principalmente quando há um grande número de árvores utilizadas.

Em resumo, o uso de algoritmos como *Random Forest* e o *Gradient Boosting* se destaca em relação às árvores de decisão tradicionais por combinarem múltiplas árvores em um único modelo. Entretanto cada uma dessas árvores contém uma

grande quantidade de expressões condicionais '*se-então*', que aumentam à medida que novas árvores são acrescentadas ao modelo. Essas expressões, que podem ser totalmente desacopladas e executadas em paralelo, até se beneficiam da paralelização e da distribuição nestes algoritmos, mas ainda sim continuam sendo redundantes em si. Além disso, diferente do modelo *Random Forest* que faz uso de múltiplas árvores treinadas de forma independente, o *Gradient Boosting* adota uma abordagem sequencial para o treinamento das árvores, que acarreta a repetição de diversas partes das árvores de decisão, o que aumenta particularmente a redundância estrutural e mesmo a temporal.

Desta forma, a proposta de uma arquitetura orientada a notificações, oriunda dos conceitos e propriedades de base do PON e do PON-HD surge como uma alternativa promissora para a composição e execução de árvores de decisão agrupadas em conjuntos correlatos (e.g. *ensemble*). A partir da conformação e ampliação de conceitos oriundos do PON em um encadeamento próprio a esta arquitetura vislumbrada, ela poderia proporcionar um melhor uso dos recursos disponíveis através da diminuição das redundâncias estruturais e explorando paralelismo intrínseco da plataforma de execução. Tal arquitetura poderia ser aplicada tanto em soluções dinâmicas, quanto em estáticas. Na verdade, inicialmente pensou-se em evoluir a ArqPON para aplicações como árvores de decisão (conforme Pordeus (2020)), mas subsequentemente percebeu-se que o mais natural seria uma arquitetura específica para árvores de decisão e estática visando validações mais acuradas dos benefícios almejados.

3. ARQUITETURA PARA ÁRVORES DE DECISÃO ORIENTADA A NOTIFICAÇÕES (ADON)

Neste capítulo são apresentados os detalhes da proposta Arquitetura para Árvores de Decisão Orientada a Notificações (ADON), uma arquitetura distinta que conforma os princípios do Paradigma Orientado a Notificações (PON) para a composição e execução de árvores de decisão agrupadas em conjuntos correlatos, com destaque para o tipo *ensemble*, como *Random Forest* e *Gradient Boosting*. A ADON é concebida com a intenção de ‘otimizar’ (*i.e.*, aprimorar veementemente) tanto o desempenho quanto o uso de recursos em implementações de árvores de decisão em *hardware*, especialmente em FPGAs, *vis-à-vis* o estado da arte.

A arquitetura proposta é fundamentada nas propriedades elementares e conceitos do PON, aproveitando sua capacidade de processamento lógico-causal desacoplado-descentralizado e a eliminação de redundâncias, características essas que permitem o paralelismo granular necessário para o processamento eficiente de modelos baseados em árvores de decisão em *hardware*. Tal arquitetura faz-se relevante para superar as limitações atuais das implementações tradicionais de árvores de decisão em *hardware*, que são baseadas em técnicas dos paradigmas tradicionais de computadores no tocante a desenvolvimento em alto nível.

Isto tudo dito e lembrado, este capítulo detalha a estrutura da ADON, com seus componentes principais, novos conceitos introduzidos e como ele se integra ao fluxo de desenvolvimento de modelos de árvores de decisão, oferecendo uma visão completa desta arquitetura. Além disso, é detalhada a implementação da ADON por meio da conformação do PON-HD, cuja implementação foi a que melhor se ajustou às propriedades de paralelismo do PON em relação às demais materializações. Neste contexto, em suma, são apresentadas as contribuições desta pesquisa no que concerne o desenvolvimento da ADON.

Finalmente, apresenta-se a organização deste presente capítulo fundamental desta tese de doutorado. Inicialmente, a Seção 3.1 apresenta os requisitos necessários para o projeto da arquitetura ADON. Em seguida, a Seção 3.2 apresenta a conformação das árvores de decisão de acordo com as entidades do modelo PON e vice-versa a fim de se alcançar a ADON. A Seção 3.3 descreve os processos e as ferramentas desenvolvidas para o treinamento e mapeamento de um modelo de árvores de decisão, visando a sua execução conforme a arquitetura

ADON. A Seção 3.4 apresenta a implementação da ADON por meio da conformação dos componentes do PON-HD que permite viabilizar a implementação da arquitetura proposta. Por fim, a Seção 3.5 apresenta as considerações finais deste capítulo.

3.1. Requisitos

Com base nos objetivos e motivações descritos para a elaboração deste trabalho, são detalhados os requisitos essenciais para o desenvolvimento da arquitetura ADON, abrangendo aspectos funcionais e não funcionais desta arquitetura. A Tabela 5 apresenta os requisitos levantados para a ADON.

Tabela 5 Requisitos da ADON.

Identificador	Descrição
Req-01	A ADON deve permitir o mapeamento e conformação das estruturas de árvores de decisão colaborativas ao PON por meio de um encadeamento próprio para comportar e executar árvores de decisão.
Req-02	A ADON deve ser capaz de executar o processo de inferência de árvores de decisão segundo o PON, com as peculiaridades trazidas no mapeamento e conformação supracitados.
Req-02.1	A ADON deve suportar a execução desacoplada de constituintes elementares (<i>i.e.</i> , cada pedaço de decisão) de árvores de decisão, aproveitando o paradigma de notificações para distribuir e paralelizar a avaliação das árvores conforme a plataforma visada permitir.
Req-02.2	A ADON deve eliminar por definição redundâncias estruturais e temporais nas árvores de decisão, inclusive as não previstas inicialmente pelo PON, evitando avaliações desnecessárias e melhorando a eficiência do modelo.
Req-02.3	A ADON deve permitir a integração com módulos executáveis externamente ou mesmo incorporá-los, como transformações de pesos em probabilidades e funções exponenciais e logarítmicas, necessários para a correta inferência dos modelos.
Req-03	A ADON deve instanciar e estruturar automaticamente os componentes das árvores de decisão, conforme os elementos

	definidos e conformados para si, a partir do PON enquanto modelo de base.
Req-03.1	A ADON deve integrar os modelos de árvores de decisão treinados pela biblioteca <i>scikit-learn</i> como referência, permitindo o treinamento e validação de modelos por métodos tradicionais e a execução por meio da implementação da arquitetura proposta.
Req-03.2	A ADON deve ser compatível com diferentes tipos de algoritmos de árvores de decisão, mas particularmente com <i>Gradient Boost</i> e <i>Random Forest</i> , permitindo a integração e execução destes modelos
Req-04	A ADON deve permitir gerar código específico para diferentes alvos, incluindo LingPON, Frameworks e arquiteturas específicas do PON, como PON-HD e ArqPON.

Fonte: Autoria Própria

Por sua vez, a Tabela 6 apresenta os requisitos específicos referentes a implementação em *hardware* da ADON, chamada ADON-HD.

Tabela 6 Requisitos da ADON HD.

Identificador	Descrição
Req-HD-01	A ADON-HD em <i>hardware</i> deve estender e validar os requisitos propostos para a implementação da arquitetura ADON
Req-HD-02	A ADON-HD permitir a geração de códigos para plataforma FPGA como alvo, utilizando os conceitos provenientes do PON-HD.
Req-HD-03	A ADON-HD deve suportar aritmética de ponto fixo necessária na execução de árvores de decisão.

Fonte: Autoria Própria

3.2. Estruturação da arquitetura ADON

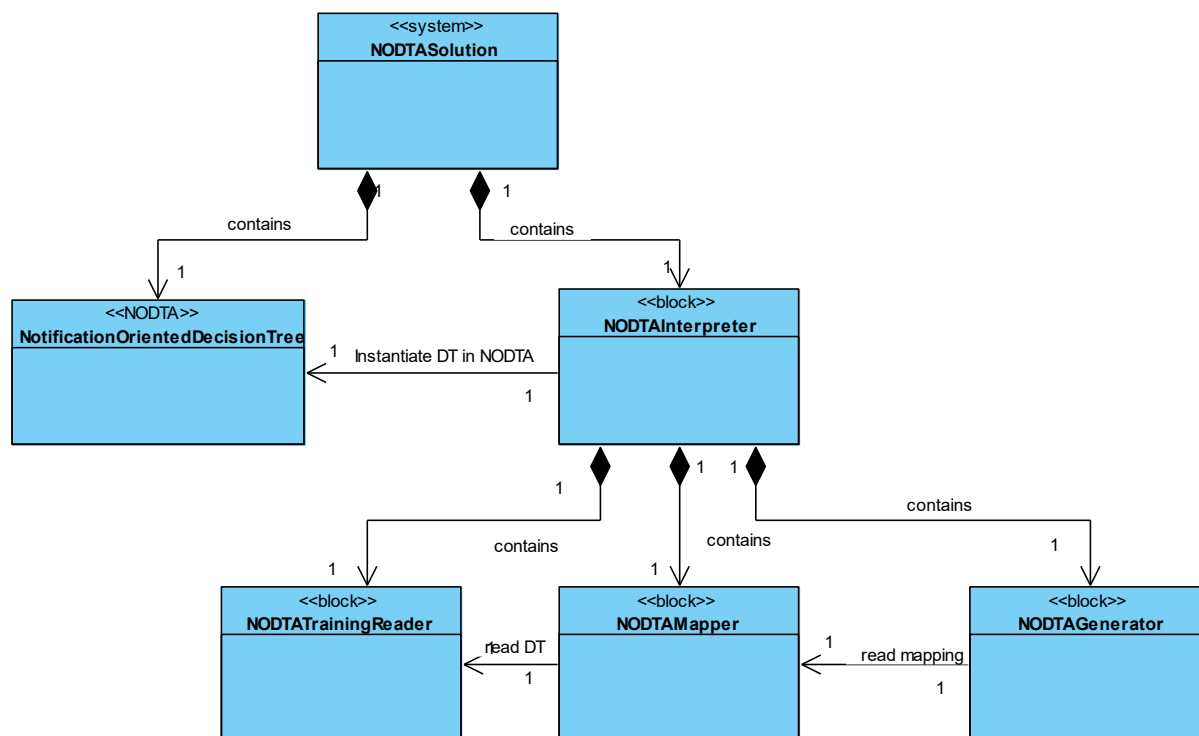
O projeto da arquitetura ADON apresenta uma forma distinta de execução do processo de inferência de árvores de decisão, com ênfase no tipo *ensemble*, por meio da conformação e ampliação dos conceitos oriundos do PON. Este novo processo de inferência de árvores de decisão parte da análise das estruturas convencionais de árvores de decisão apresentadas na Seção 2.2 e desenvolve-se através da utilização e expansão dos conceitos do PON, atendendo aos requisitos

apresentados na Seção 3.1. Além disso, a subsequente materialização desta arquitetura em *hardware*, por meio da conformação do PON-HD para tal, permite o uso da ADON-HD resultante para a composição e execução de árvores de decisão no contexto de aceleração do processo de inferência em *hardware*.

Considerando todas as análises e discussões já elaboradas ao longo deste presente documento, as subseções a seguir descrevem a arquitetura ADON⁶ do ponto de vista da análise, abordando seu projeto por meio de diagramas em Linguagem de Modelagem de Sistemas (*SysML –System Modelling Language*). Em tempo, a *SysML* é um padrão que permite a modelagem de sistemas de forma detalhada e padronizada, possibilitando a representação de estruturas, comportamentos e requisitos do sistema em diagramas, cf. Wolny *et al.* (2020). Isto dito, por sua vez, a modelagem em *SysML* da ADON servirá como base para o desenvolvimento da implementação da ADON em conformidade com o PON-HD, alcançando a ADON-HD no contexto de *hardware*. Por sua vez a Figura 14 apresenta o diagrama de blocos *SysML* referente a solução ADON, composta pela arquitetura em si representada pelo bloco *NotificationOrientedDecisionTree*, além do ferramental para interpretação das árvores de decisão treinadas externamente e instanciação dos elementos da ADON.

⁶ Como padrão de desenvolvimento da ADON, os nomes das entidades, atributos, métodos e relações seguirão em inglês nos diagramas e na implementação. A sigla ADON, traduzida para o inglês, refere-se à *Notification-Oriented Decision Tree Architecture* (NODTA)

Figura 14 Diagramas de blocos SysML referente a solução ADON.



Fonte: Autoria própria

3.2.1. Conformação das avaliações lógico-causais ao PON

Com base na estrutura e elementos das árvores de decisão, é possível abordar o processo de conformação destes elementos ao PON, particularmente no que diz respeito às avaliações lógico-causais presentes nas árvores de decisão. Esta conformação visa traduzir os elementos estruturais e funcionais das árvores de decisão tradicionais para as entidades definidas pelo metamodelo do PON, derivando e inovando tal meta-modelo à medida que se faça necessário.

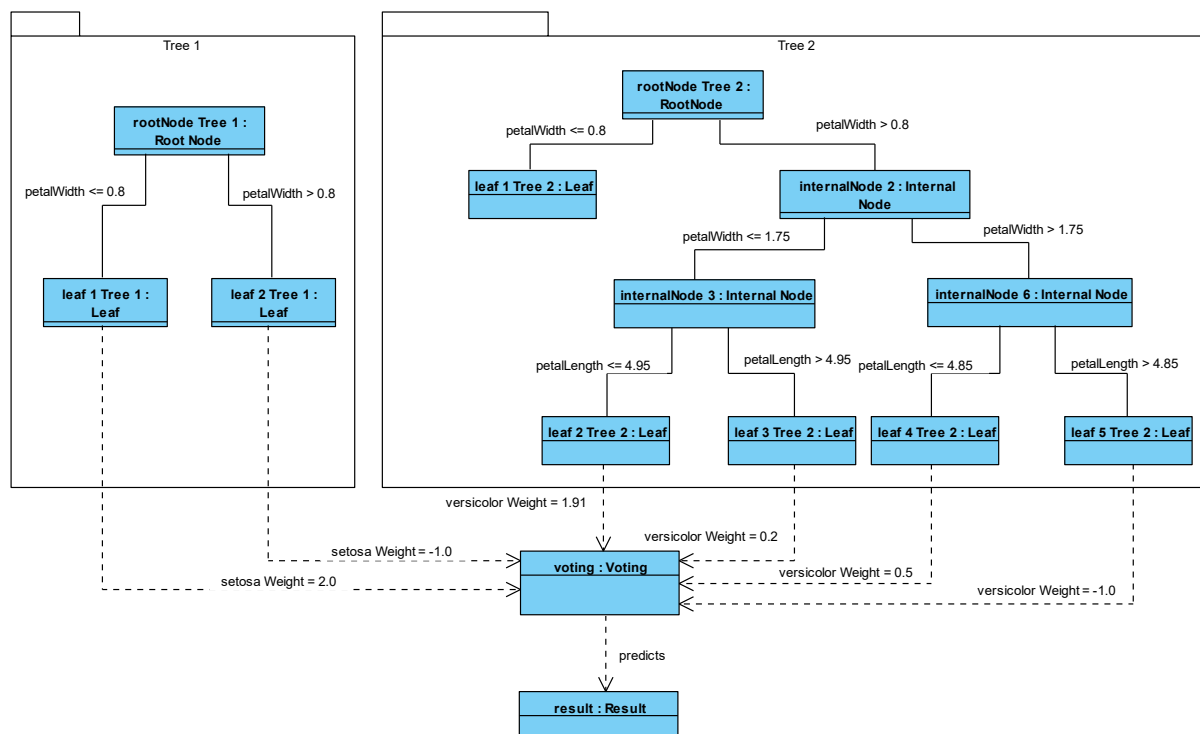
Para início do processo de conformação, são listados os principais elementos que compõem a estrutura de uma árvore de decisão, tais como:

- **Features:** características dos dados de entrada que serão analisados pelo modelo de *machine learning* para tomar decisões em relação às previsões.
- **Root:** é o ponto de partida de uma árvore de decisão. Ou seja, neste elemento que ocorre a primeira avaliação da árvore de decisão sobre as *Features* do sistema.

- **Internal Nodes:** corresponde às avaliações intermediárias de uma árvore de decisão sobre as *Features* do sistema.
- **Leaf:** representam as decisões finais do conjunto de avaliações sobre as *Features* realizadas ao longo do caminho de aprovação. Cada *Leaf* contém uma previsão ou classe que é atribuída aos dados que chegam a essa folha.
- **Voting:** é o processo de combinar previsões de várias árvores de decisão para tomar uma decisão final. Cada árvore “vota” individualmente em uma classe com maior probabilidade, e a partir deste processo de votação, a classe com mais votos é escolhida como a previsão final.
- **Result:** é a saída final de um modelo de árvore de decisão após a avaliação das *Features*. Este resultado pode ser uma previsão, uma classificação ou um valor contínuo.

A Figura 15 apresenta um exemplo da estrutura de duas árvores de decisão baseadas no *dataset Iris*, em que são utilizadas duas características de entrada (*Feature*) deste *dataset*: *petal width* e *petal length*. O elemento de partida (*Root*) de ambas as árvores corresponde à avaliação “*petalWidth 1 <= 0.8* ou *petalWidth 1 > 0.8*”. Na primeira árvore, o nó principal (*RootNode*) é diretamente subdividido nos elementos finais (*Leaf*). A segunda árvore é subdividida em nós intermediários (*Internal Nodes*), que avaliam as condições “*petalWidth <= 1.75* ou *petalWidth > 1.75*”, “*petalLength <= 4.95* ou *petalLength > 4.95*” e “*petalLength <= 4.85* ou *petalLength > 4.85*”. Ao final de cada caminho completo até o elemento final (*Leaf*), as decisões são tomadas considerando as probabilidades do resultado da árvore ser cada uma das possíveis classes (*setosa*, *versicolor* ou *virginica*). Por fim, o resultado desta árvore é avaliado pelo método de votação (*Voting*), que combina os resultados das demais árvores de decisão e seleciona a classe com mais votos como a previsão final.

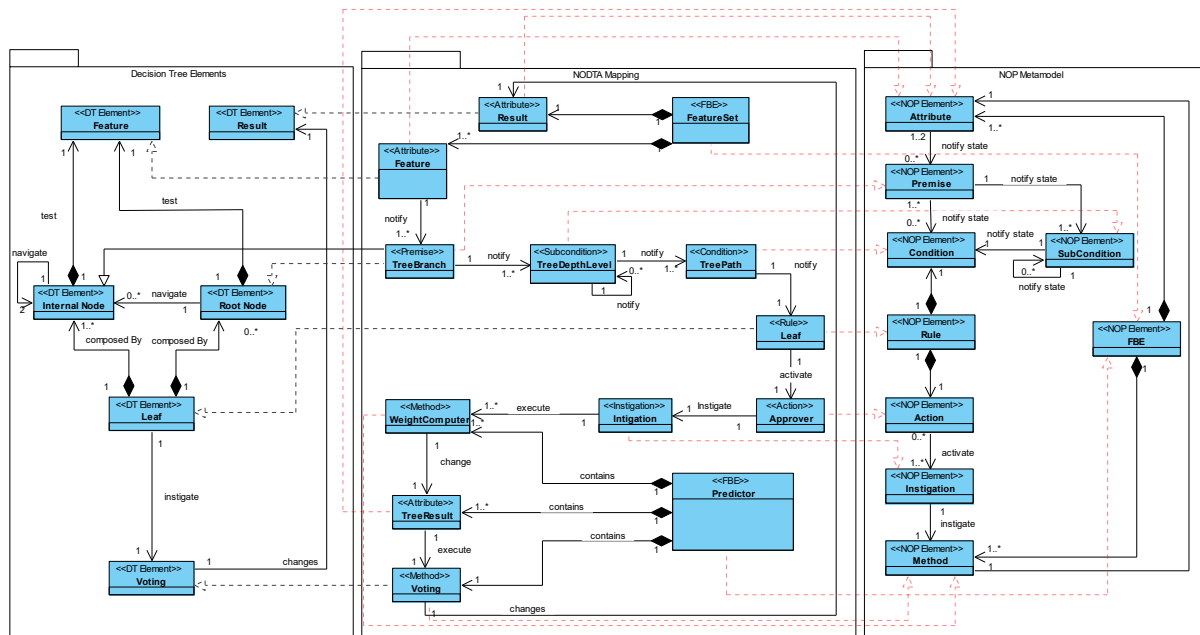
Figura 15 Diagrama de blocos internos SysML referente a exemplificação de duas árvores de decisão *Gradient Boosting*.



Fonte: Autoria própria

O processo de conformação de uma árvore de decisão à ADON inicia-se pela identificação dos principais elementos do PON e a sua correspondência conceitual aos elementos que compõem as árvores de decisão, conforme apresentado na Figura 16. Os blocos referentes ao pacote *Decision Tree Elements* representam uma modelagem dos principais elementos que compõem a estrutura de uma árvore de decisão e seus relacionamentos. Já os blocos do pacote *NOP Metamodel* representam o metamodelo do PON. Enquanto o pacote *NODTA Mapping*, demonstra a extrapolação dos elementos que irão compor a ADON em relação às árvores de decisão e ao metamodelo do PON. É importante ressaltar que as ligações entre os blocos em diferentes os pacotes apresentados na Figura 16 não representam relações de herança, mas sim realizações conceituais que estabelecem as correspondências e mapeamentos entre os elementos dos diferentes domínios modelados.

Figura 16 Conformação das Avaliações Lógico-Causais ao PON.



Fonte: Autoria própria

Este processo de conformação inicia-se pela identificação e mapeamento das *Features* do sistema na forma de elementos *Attributes* do PON. Em seguida, cada avaliação referente ao nó de partida (*Root Node*) e cada avaliação intermediária (*Internal Node*) é mapeada para um elemento *Premise* (*TreeBranch*). A conjunção entre duas *Premises* é mapeada como uma *SubCondition* (*TreeDepthLevel*), do mesmo modo em que a conjunção entre uma *Premise* e uma *SubCondition* também pode ser igualmente mapeada como uma nova *SubCondition*. Assim, cada caminho de avaliações que se estende desde o nó inicial (*Root*) até uma folha (*Leaf*), que representa uma *Rule* (*Leaf*), é composto pela combinação de elementos *Premises* e *SubConditions*, formando uma *Condition* (*TreePath*) para cada folha.

As *FBEs* e as *Rules* são definidas por meio da composição dos elementos conformados ao metamodelo do PON. A primeira *FBE*, denominada *FeatureSet*, é composta pelos *Attributes* *Feature* e *Result*, responsáveis por representar, respectivamente, as características de entrada do sistema e o resultado consolidado da classificação ou regressão. A segunda *FBE*, denominada *Predictor*, é composta pelo *Attribute* *TreeResults*, que armazena os resultados individuais de cada árvore de decisão, e pelos *Methods* *WeightComputer* e *Voting*, que são responsáveis, respectivamente, pelo cálculo dos pesos associados a esses resultados e pela consolidação dos resultados de inferência por meio de um processo de votação.

A Figura 17 mostra o mapeamento de quatro caminhos de decisão dos elementos das árvores mostradas na Figura 15 para os elementos do metamodelo do PON no formato de regras. As *Rules* são definidas a partir das *Conditions* que representam os caminhos lógicos das árvores de decisão, desde a raiz até as folhas. Cada *Rule* é composta por uma *Condition*, formada pelo compartilhamento e interdependência de *Premises* e *SubConditions* mapeadas para as avaliações lógicas dos nós das árvores de decisão. Quando uma *Condition* associada a uma *Rule* é satisfeita, o sistema aciona uma *Action* (*Approver*), que instiga a execução dos métodos associados essa regra como *WeightComputer* e *Voting*. Desta forma, as *Rules* não apenas determinam os caminhos lógicos seguidos na árvore de decisão, mas também definem as operações a serem realizadas com base nos resultados dessas decisões, assegurando que o sistema responda de forma coerente e eficaz às condições estabelecidas. Essa conformação permite uma avaliação lógica eficiente e paralela, alinhada com os princípios do PON, garantindo que as decisões sejam tomadas de maneira otimizada e precisa com base nos dados de entrada fornecidos pelas *Features*.

Figura 17 Exemplos de Regras mapeadas.

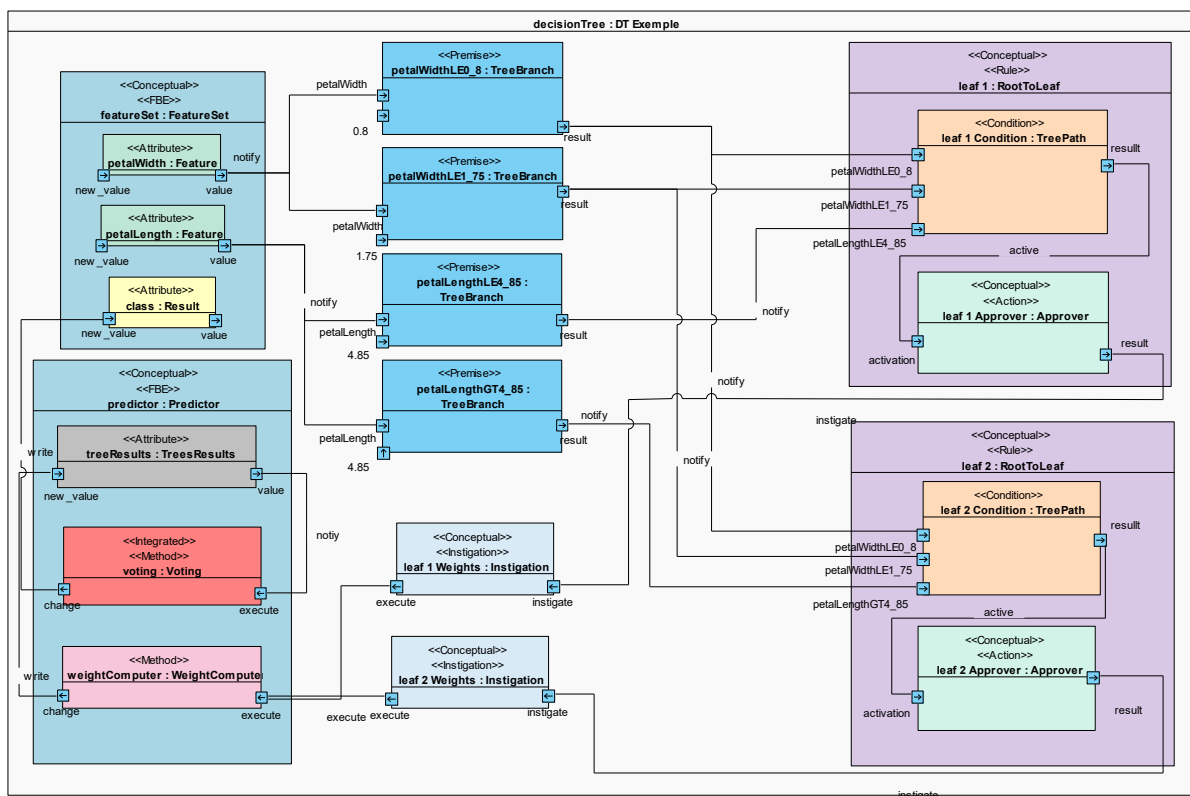


Fonte: Autoria própria

Conforme demonstrado no diagrama interno de blocos da Figura 18, o modelo da ADON permite que cada caminho de decisão na árvore, que consiste em múltiplas avaliações lógicas, seja representado dentro da estrutura do PON. Cada *Condition* (Laranja), por sua vez, determina se um caminho específico foi satisfeito, acionando *Methods* (Rosa) associados para a atualização dos pesos mapeados por meio de novos elementos do tipo *Attributes* chamados *treeResults* (Cinza). Por fim, os resultados de todas as árvores de decisão são consolidados através de um único

Method responsável pelo processo de *Voting* (Vermelho), que por sua vez escreve os resultados de predição em um elemento *Attribute Result* (Amarelo).

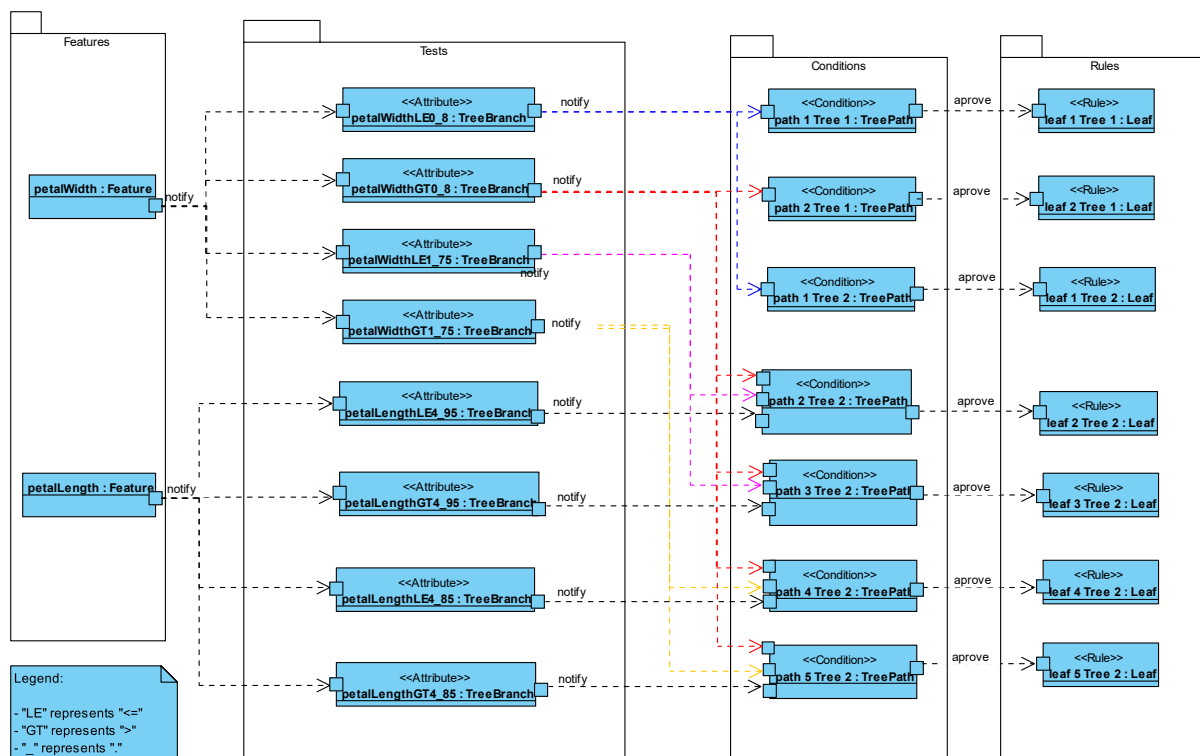
Figura 18 Diagrama de blocos internos SysML para um exemplo de regra mapeada ao PON.



Fonte: Autoria própria

Por fim, a Figura 19 apresenta um diagrama de objetos após o mapeamento dos elementos da árvore de decisão da Figura 15 para os elementos do modelo PON. Entretanto, essa modelagem ainda apresenta algumas avaliações em duplicidade em relação aos elementos *Premises* e *Conditions*. No caso das *Premises*, para cada nó de decisão é avaliado o mesmo atributo em condições opostas, gerando duas entidades *Premises* que nunca podem ser aprovadas simultaneamente. Enquanto para *Conditions*, observam-se redundâncias quando diferentes *Rules* compartilham partes comuns do caminho de decisão, mas não há um mecanismo sistemático para compartilhar essas subestruturas comuns.

Figura 19 Diagrama de blocos internos SysML exemplificando uma árvore de decisão conformada ao PON.



Fonte: Autoria própria

Ao bem da verdade, há dois trabalhos do grupo do PON que utilizaram a conformação do PON às árvores de decisão proposta no âmbito deste projeto de pesquisa (a luz de Pordeus (2020)), mas no âmbito de *software*, nominalmente “*Framework PON C++ 4.0: contribuição para a concepção de aplicações no paradigma orientado a notificações por meio de programação genérica*” (NEVES, 2021) e “*Contribuição em paradigma orientado a notificações: evolução da tecnologia LingPON 2.0 via aprimoramento da linguagem e compilador para código notificante modular em C++*” (OSHIRO, 2021). Entretanto os resultados não foram tão bons quanto esperados justamente em função das duplicidades não tratadas pelo PON no âmbito de árvores de decisão.

Segundo Oshiro (2021), apesar do algoritmo ser composto essencialmente por expressões do tipo *se-então*, a eliminação de redundâncias não foi o suficiente para que a implementação da Tecnologia LingPON 2.0 apresentasse um desempenho melhor do que a implementação feita usando a linguagem de programação C. Enquanto a implementação de Neves (2021) fez uso de sinais de controle que não permitiram que o *Framework PON C++ 4.0* explorasse os benefícios de paralelização nos experimentos em relação ao algoritmo *Random*

Forest. Além disso, em ambos os experimentos realizados, os valores de entrada do sistema eram alterados de maneira aleatória, fazendo com que os experimentos não explorassem um dos benefícios do PON que consiste na execução apenas das entidades que tiveram seus valores alterados.

Muito embora a ADON possa ser aplicada em *software*, a lacuna de árvore de decisão em *hardware* impele a sua utilização nesse contexto. Justamente no contexto de *hardware*, as redundâncias observadas nas implementações prototipais em *software* são agravantes em função do espaço em dispositivos de FPGA serem um elemento restritivo.

Desta forma, para explorar as propriedades elementares do PON durante o processo de conformação à ADON, foi necessário o desenvolvimento de novos conceitos que expandem o PON e permitem explorar essas propriedades de forma adequada. Em Pordeus (2020) as primeiras implementações de árvores de decisão já consideravam os novos conceitos implicitamente, portanto não tendo sido ainda explicitados, tanto que as implementações em *software* não os implementaram e exploraram, o que permitiu a observar o impacto de seu não uso. Ainda, em Pordeus (2020) considerava-se expandir o ArqPON para uso em aplicações da envergadura de árvores de decisão, sendo que subsequentemente percebeu-se a pertinência de árvores de decisão diretamente em PON-HD e as insuficiências neste âmbito, levando à ADON.

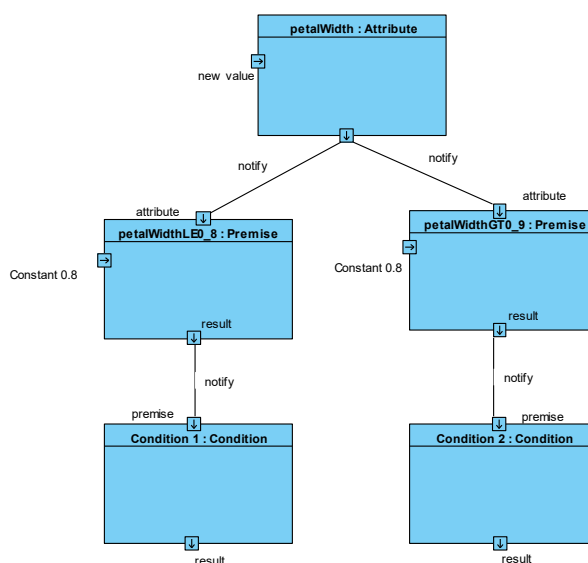
3.2.1.1. Novos Conceitos no PON para ADON

Ao realizar esta conformação, busca-se assegurar o aproveitamento eficiente dos elementos do metamodelo do PON com base nas suas propriedades elementares e conceitos. Entretanto, os conceitos atuais do PON por si só não são suficientes para a concepção de árvores de decisão de forma a aproveitar as propriedades elementares do PON, eliminando todas as redundâncias visando desempenho apropriado e desacoplamento visando performance e paralelismo. Desta forma, durante o processo de conformação foi necessário a concepção de novos conceitos que expandem o PON a favor da ADON e então permita explorar adequadamente suas propriedades elementares neste contexto.

3.2.1.2. Premise Dual

Ao conformar os elementos *Root* e *Internal Nodes*, que representam as avaliações lógico-causais de uma árvore de decisão, observa-se que cada elemento gera duas *Premises* opostas. Um exemplo são as avaliações “*petalWidth 1* ≤ 0.8 ou *petalWidth 1* > 0.8 ” conforme a Figura 20, que avaliam o mesmo atributo, mas nunca podem ser aprovadas simultaneamente.

Figura 20 Diagrama de blocos internos SysML exemplificando *Premise Dual*.



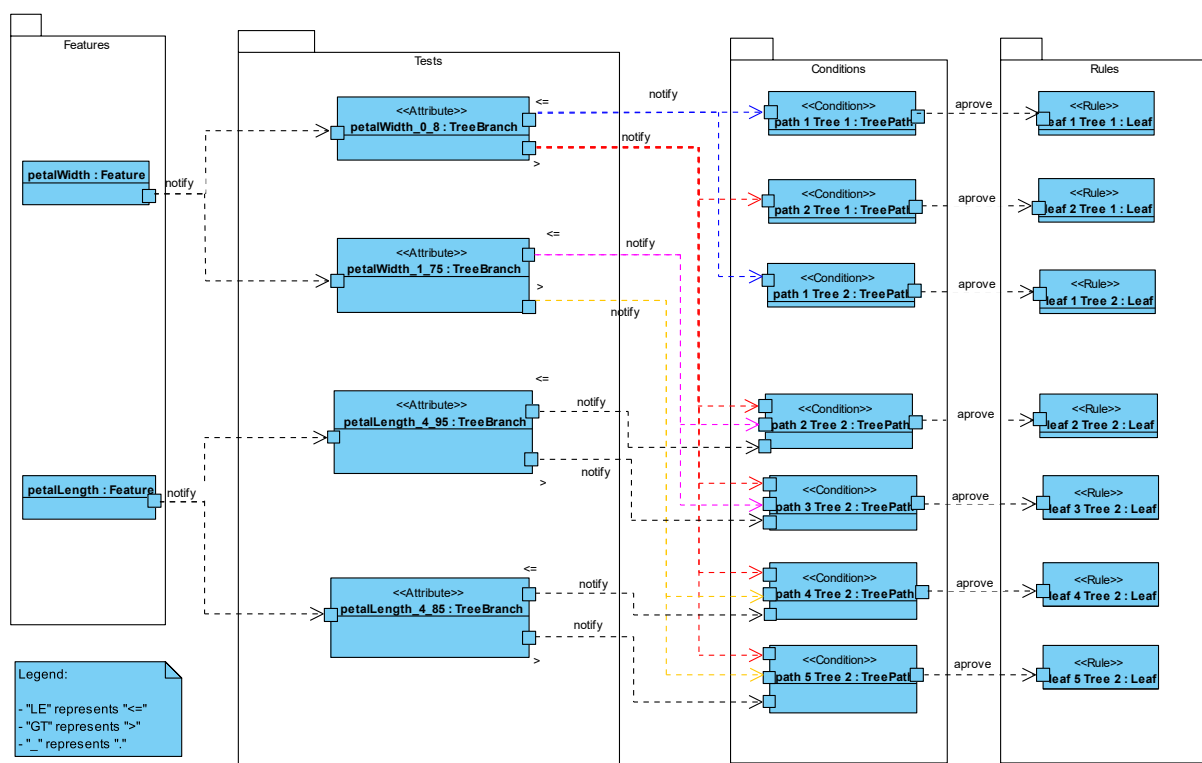
Fonte: Autoria própria

Neste cenário, utilizando apenas os conceitos atuais do PON, são mapeadas duas *Premises* distintas que são notificadas assim que o *Attribute* *petalWidth* tem seu valor alterado. Em seguida, ambas as *Premises* são avaliadas e caso tenham seus valores alterados, notificam as *Conditions* e *SubConditions* pertinentes. Entretanto, essas duas *Premises* opostas acabam por gerar uma redundância estrutural avaliando em duplicidade o mesmo *Attribute*. Uma vez que essas *Premises* são complementares, é proposto um novo tipo de *Premise* chamada *Premise Dual*.

A *Premise Dual* permite que essas avaliações opostas sejam tratadas como uma única entidade lógica, eliminando a necessidade de múltiplas avaliações do mesmo atributo. Quando um *Attribute* tem seu valor alterado, a *Premise Dual* é notificada e realiza a avaliação das condições opostas de maneira integrada. Quando a alteração do *Attribute* satisfizer uma das condições da *Premise Dual*, ela notifica as *Conditions* correspondentes, evitando a necessidade de processamento

duplicado. Isso otimiza o modelo de inferência ao eliminar processamentos redundantes. Conforme mostrado na Figura 21, que é uma extensão da Figura 20, a *Premise Dual* integra essas avaliações opostas em uma única entidade lógica, evitando redundâncias no processo de notificação. Neste cenário, é observada a redução de elementos *Premises* mapeados no diagrama de objetos, eliminando a duplicidade na avaliação deste elemento.

Figura 21 Diagrama de blocos internos SysML exemplificando conformação após aplicar *Premise Dual*.



Fonte: Autoria própria

3.2.1.3. Encadeamento de *SubConditions*

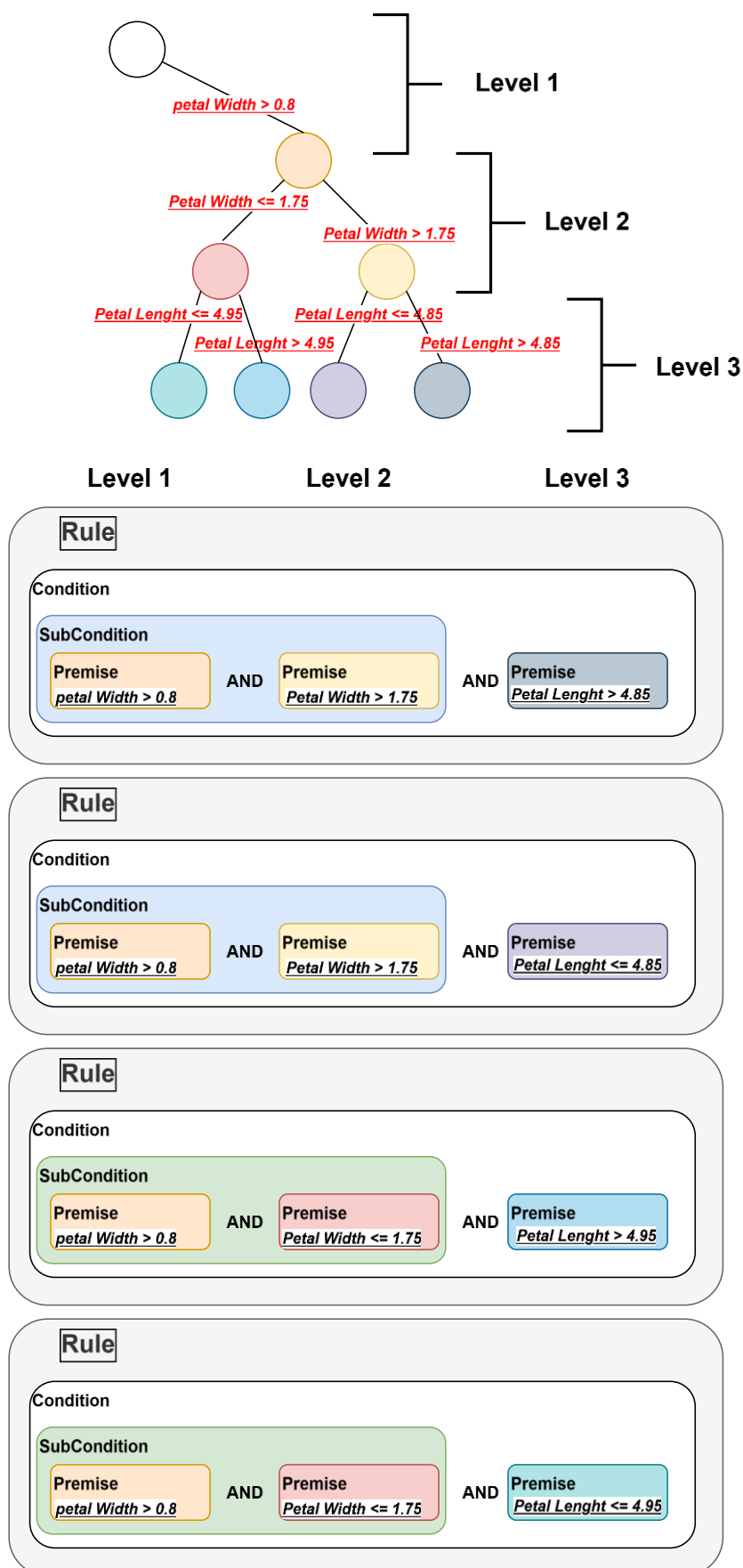
Na ADON, as árvores de decisão são formadas com base nas avaliações lógicas 'se-então', juntamente com o caminho de aprovação seguido até o elemento final da árvore (*Leaf*). Cada avaliação 'se-então' é representada pelo modelo PON por meio de elementos do tipo *Premises*. Cada caminho lógico da avaliação inicial (*Root Node*) até o elemento final (*Leaf*) determina as ações a serem executadas no sistema. Cada elemento de decisão corresponde a uma *Rule* distinta, composta por uma *Condition* e uma *Action*. Por sua vez, as *Conditions* são compostas por meio de conjunções de *Premises* ou outras *SubConditions*. Desta forma, um encadeamento

interdependente de *SubConditions* ocorre quando uma ou mais *SubConditions* compartilham as mesmas *Premises* ou outras *SubConditions*, influenciando diretamente sua validação.

A Figura 22 apresenta um exemplo, baseado na Figura 15, que demonstra o compartilhamento de *SubConditions* na composição de quatro caminhos de decisão da árvore, em que elementos de mesma cor representam os testes redundantes na estrutura, os quais ainda não são resolvíveis naturalmente pelo estado da arte do PON. Mais precisamente, cada caminho pode ser definido como uma *Rule* que contém uma *Condition* composta por outras três *SubConditions* dependentes, que correspondem a cada nível da árvore, sendo que a *Subcondition* do último nível depende da *SubCondition* do nível anterior, que por sua vez depende da *SubCondition* do nível anterior ainda. Ou seja, no âmbito de árvore de decisão há uma dependência de *SubConditions* que é algo não tratado pelo estado da arte do PON e, portanto, tem que ser resolvido no âmbito da ADON o que é chamado de encadeamento de *Subconditions*.

A luz do encadeamento de *SubConditions*, passa a existir a dependência de *SubConditions*, sendo que então cada *SubCondition* pode depender de outras *SubConditions* além de suas *Premises*. Assim no exemplo dado, no primeiro nível, a *SubCondition* inicial é constituída por uma *Premise* que é compartilhada entre todas as quatro *Rules* representadas na figura. No segundo nível, cada *SubCondition* é formada pela *SubCondition* do primeiro nível, em conjunto com *Premises* adicionais que refinam o caminho de decisão. No terceiro e último nível, as *SubConditions* tornam-se ainda mais específicas, sendo formadas a partir das *SubConditions* do segundo nível, combinadas com novas *Premises* que introduzem maior especialização no caminho de decisão. Justamente, a Figura 22 exemplifica como as *SubConditions* se encadeiam ao longo dos diferentes níveis, resultando na composição final de cada *Rule* estendida, representando um caminho da árvore de decisão.

Figura 22 Exemplo de dependência de *SubCondition*.

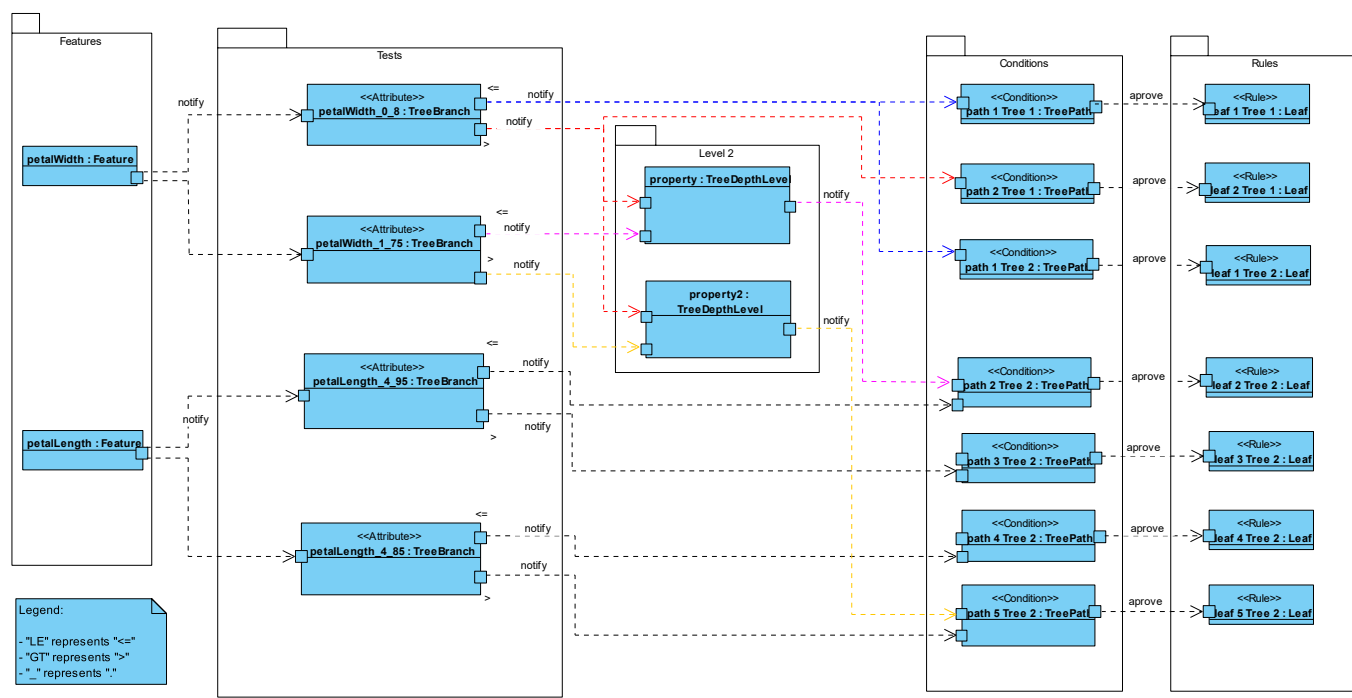


Fonte: Autoria própria

Ao compartilhar *SubConditions* entre diferentes níveis da árvore de decisão, no âmbito deste novo dispositivo, nomeadamente dependência de *SubConditions*, a ADON extrapola o princípio basilar do PON de não redundância. Eis que Ronszcka (2012, 2019) até discute que as *Rules* no PON não operam de forma isolada, ou seja, elas podem ter dependências que exigem sincronização e uma ordem de execução específica. Essa interdependência entre *Rules* foi definida por Ronszcka, mas que é diferente do encadeamento de *SubConditions* aqui apresentado, o qual se dá no âmago de cada *Rule*. No âmbito de árvores de decisão, o proposto encadeamento de *SubConditions* que realmente permite tratar cada caminho da árvore orientada a notificações sem redundâncias estruturais e temporais aí associadas.

Na prática, ao conformar uma árvore de decisão à ADON, deve-se considerar que o encadeamento de *SubConditions* efetivamente ameniza diretamente a quantidade de redundâncias presentes na estrutura das árvores de decisão. Por exemplo, quando múltiplas *SubConditions* dependem de uma mesma *Premise*, é crucial que a ADON seja capaz de identificar essas dependências e otimizar a ordem de avaliação. A Figura 23 apresenta um exemplo de encadeamento entre *SubConditions* em uma árvore de decisão, em que múltiplas *SubConditions* compartilham as mesmas *Premises*. Além disso, a integração a outra solução proposta no âmbito do PON, a *Premise Dual* mencionada anteriormente, pode ajudar ainda mais a mitigar as redundâncias causadas por essas dependências.

Figura 23 Diagrama de blocos internos SysML exemplificando o Encadeamento de SubConditions.

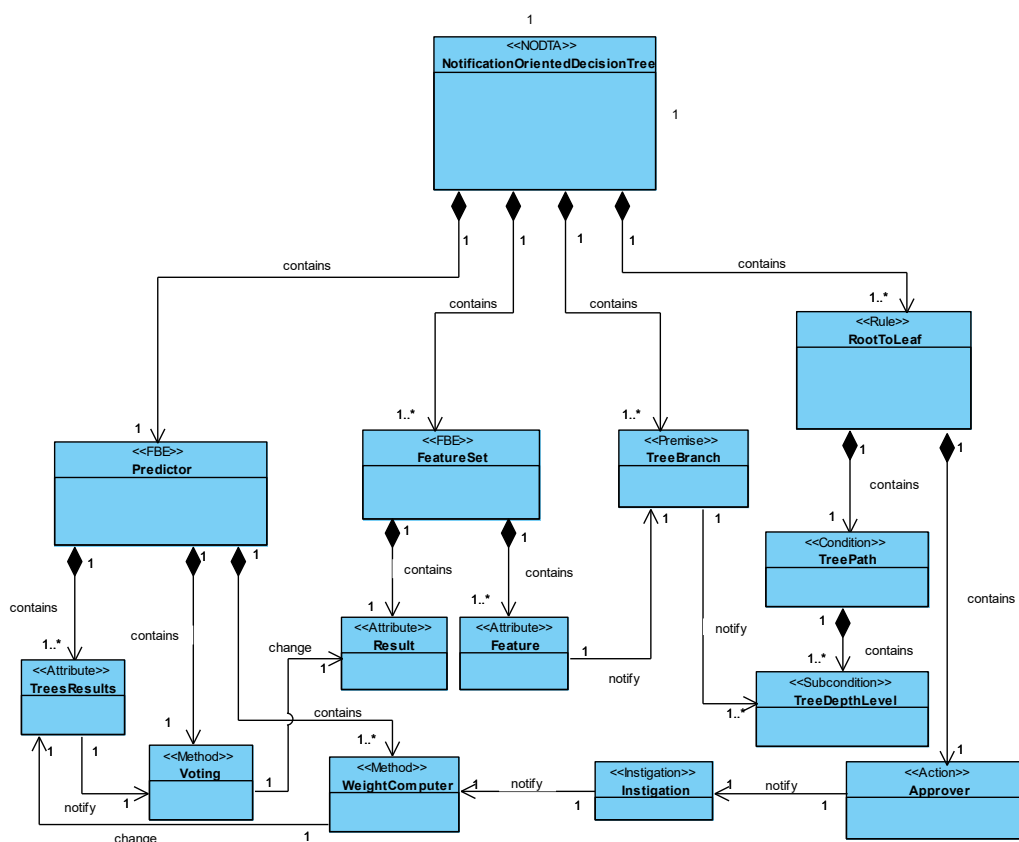


Fonte: Autoria própria

3.2.2. Modelagem da ADON

A partir da conformação das avaliações lógico-causais das árvores de decisão, foi concebido o diagrama de blocos em SysML da ADON, detalhando os subsistemas que fazem parte dele e suas respectivas interconexões. A Figura 24 apresenta esse diagrama de blocos.

Figura 24 Diagramas de Blocos SysML de análise da ADON.



Fonte: Autoria própria

A seguir são detalhados os elementos presentes no diagrama de blocos apresentado na Figura 24:

- **FeatureSet:** este bloco corresponde à representação de um conjunto de características de entrada e o resultado de uma classificação ou regressão. Este bloco é mapeado para um *FBE*, composto pelos atributos *Feature* e *Result*.
- **Feature:** este bloco é responsável por representar uma característica individual do *dataset*. Cada *Feature* é mapeada como uma entidade *Attribute*, permitindo a representação do valor de uma característica de entrada. Caso o seu valor seja alterado, este componente notifica os demais componentes pertinentes, que podem então desencadear ações subsequentes no sistema.

- **Result:** este bloco é responsável por representar o resultado consolidado da classificação ou regressão de um modelo de árvore de decisão. Cada *Result* também é representado como uma entidade *Attribute*, permitindo a representação do valor final da previsão.
- **TreeBranch:** este bloco é responsável por representar uma ramificação ou derivação dentro de uma árvore de decisão. Cada *TreeBranch* é mapeada como uma *Premise*. Quando o valor associado a uma *TreeBranch* é alterado, ele notifica as *Conditions* e *SubConditions* pertinentes, desencadeando ações subsequentes no sistema.
- **RootToLeaf:** bloco responsável por representar *Rule* da ADON por meio de uma condição (*TreePath*) que cobre o caminho completo de uma árvore de decisão, desde a raiz até uma folha, e as ações a serem realizadas a partir da aprovação (*Approver*). Cada *Condition* determina a aprovação de uma regra específica, estruturando um caminho de decisão. Quando a *Condition* de um caminho *RootToLeaf* é satisfeita, a regra é aprovada, e desencadeia a execução de ações específicas, como a computação de pesos e a atualização de resultados.
- **TreePath:** este bloco é responsável por representar um caminho completo de uma árvore de decisão, desde a raiz até uma folha. Cada *TreePath* é composto por múltiplas *SubConditions* que, juntas, formam uma *Condition*. A *Condition* determina a aprovação de uma regra específica, estruturando o caminho de decisão ao avaliar todas as *Premises* e *SubConditions* associadas. Quando todas as *Premises* e *SubConditions* de um *TreePath* são satisfeitas, a *Condition* associada a *Rule* (*RootToLeaf*) é aprovada.
- **TreeDepthLevel:** este bloco é responsável por representar um nível específico de profundidade dentro de uma árvore de decisão. Ele

organiza e agrupa os elementos que precisam ser avaliados a cada nível de uma árvore de decisão na forma de um conjunto de *SubConditions*. Cada *TreeDepthLevel* facilita a execução das avaliações lógicas na ADON, começando da raiz e notificando os níveis mais profundos da árvore. Quando todas as *SubConditions* em um nível são satisfeitas, o sistema avança para o próximo nível, até atingir uma folha sem necessitar reavaliar as *SubConditions* previamente avaliadas (conforme previsto no *Encadeamento de SubConditions* na Seção 3.2.1.3). Essa estrutura ajuda a reduzir redundâncias estruturais e temporais, otimizando o uso dos recursos de *hardware*, trabalhando em conjunto com outros componentes do PON para uma execução eficiente das avaliações lógicas na ADON.

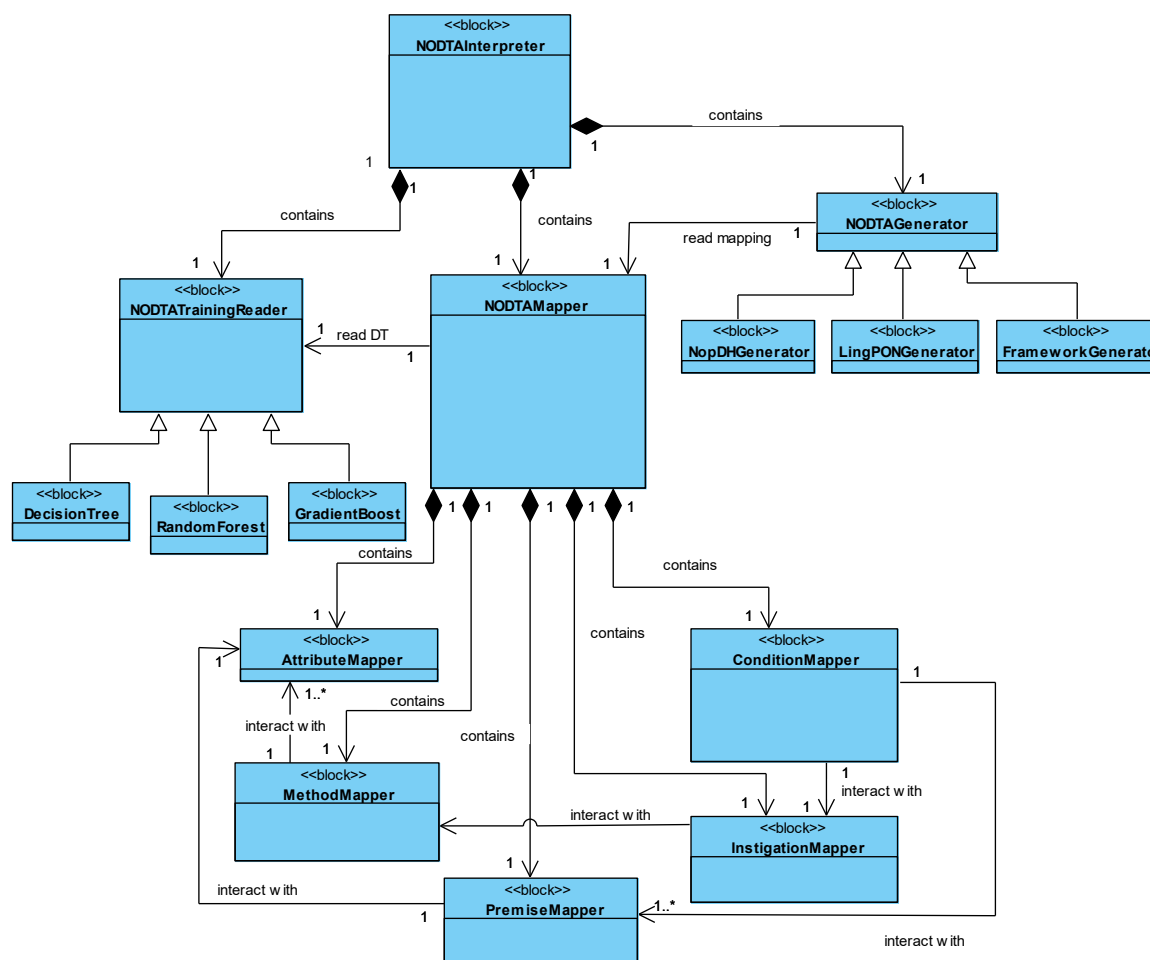
- **Approver:** este bloco é responsável por representar um elemento *Action* do metamodelo do PON. Quando a *Condition* associada a uma regra é verificada como verdadeira, o bloco *Approver* aprova a regra, instigando a execução das ações específicas definidas por essa regra por meio do bloco *Instigation*.
- **Predictor:** este bloco é responsável por consolidar os resultados individuais das árvores de decisão e o processamento para gerar uma previsão final. O bloco *Predictor* avalia as saídas de múltiplas árvores, utilizando técnicas de votação para determinar a classe ou valor previsto. Este bloco é mapeado para uma *FBE*, composta pelos atributos *TreeResults* e pelos métodos *WeightComputer* e *Voting*.
- **TreeResults:** este bloco é responsável por armazenar os resultados individuais de cada árvore de decisão. Cada *TreeResults* também é representado como uma entidade *Attribute*, permitindo a representação e armazenamento dos resultados de cada árvore. Isso facilita a consolidação dos resultados e a posterior utilização desses dados no cálculo das previsões finais.

- **WeightComputer**: este bloco corresponde a um elemento *Method* que é responsável por calcular os pesos associados aos resultados de cada árvore de decisão e persistir estes pesos no *Attribute TreeResults*.
- **Voting**: este bloco é responsável por consolidar os resultados das árvores de decisão e aplicar um mecanismo de votação para determinar a previsão final. O bloco *Voting* avalia os resultados de múltiplas árvores armazenados em *TreeResults* e utiliza estratégias específicas de cada algoritmo para selecionar a classe ou valor final previsto. Após a conclusão do processo de votação, o bloco *Voting* atualiza o valor do *Attribute Result* com o resultado de predição.

3.3. Ferramentas para treinamento e mapeamento

Além do processo de conformação de árvores de decisão para a arquitetura ADON, conforme descrito na Seção 3.2.1, foi essencial desenvolver um método para o mapeamento das entidades do PON a partir de um modelo de árvore de decisão treinado. Em conformidade com os requisitos apresentados na Seção 3.1, o processo de mapeamento deve permitir a leitura dos componentes que compõem o conjunto de árvores de decisão treinadas, conformando estes componentes aos elementos estabelecidos pela ADON e posteriormente instanciar as entidades conformadas para a plataforma de execução desejada. Desta forma, a Figura 25 apresenta o diagrama de blocos referente aos componentes responsáveis pelo processo de treinamento e mapeamento das entidades das árvores de decisão à ADON.

Figura 25 Diagrama de blocos SysML da ferramenta para mapeamentos de modelos à ADON.



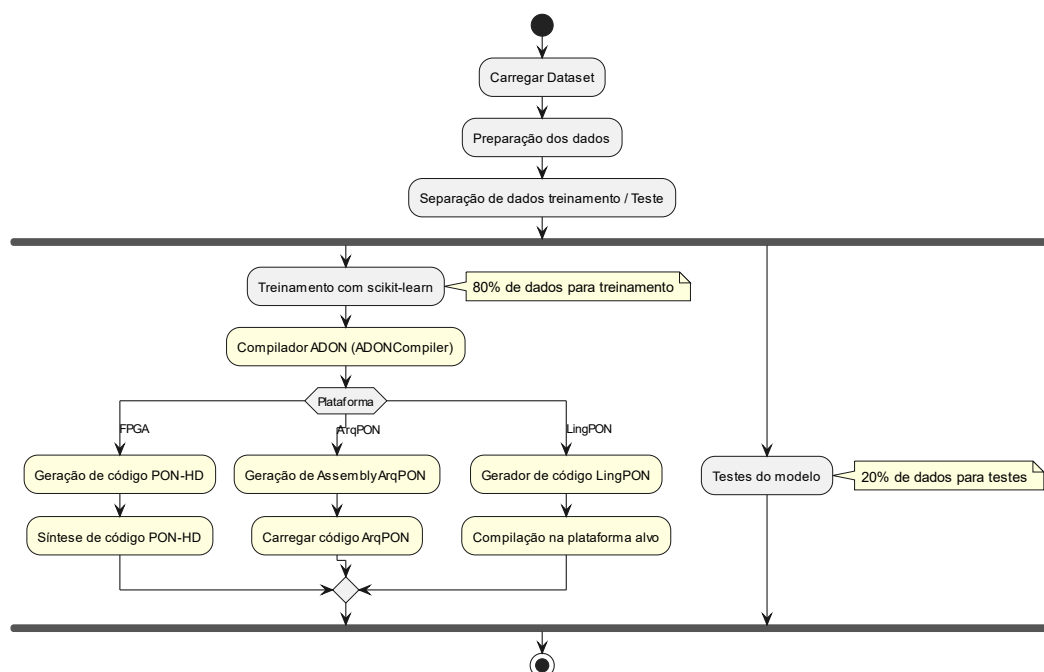
Fonte: Autoria própria

O início deste processo de treinamento envolve a integração dos modelos tradicionais de árvores de decisão, como *Gradient Boosting* e *Random Forest*, treinados por bibliotecas externas, como a biblioteca *scikit-learn* como referência. O bloco denominado *NODTATrainingReader* através de suas derivações, é responsável por realizar a integração da ADON com os métodos tradicionais de árvores de decisão. Suas derivações especializam a integração de cada tipo de modelo de árvore de decisão com a ADON. No diagrama, o bloco *NODTATrainingReader* possui derivações como *DecisionTreeReader*, *RandomForestReader* e *GradientBoostReader*, que representam métodos específicos de treinamento de árvores de decisão. Cada uma dessas derivações é responsável por ler os métodos de treinamento específicos desses algoritmos para que possam ser interpretados posteriormente pelo bloco *NODTAMapper*. Desta forma, este processo possibilita o treinamento e a validação dos modelos por

métodos tradicionais, além de sua execução por meio da implementação na arquitetura proposta.

Para a realização do processo de treinamento e conformação dos modelos de árvore de decisão à ADON, foi desenvolvida uma aplicação suplementar em *Python* que implementa o bloco *NODTAInterpreter*, conforme o diagrama apresentado na Figura 26, com o objetivo de realizar o treinamento das árvores de decisão com base em um conjunto de dados utilizando a biblioteca *scikit-learn*. Os blocos em cinza demonstram os passos tradicionais para o treinamento de um modelo de *machine learning*, em que primeiramente são carregados os dados de um *dataset*, realiza-se eventual tratamento nos dados previamente carregados, separa-se os dados entre dados de treinamento e testes para validação cruzada, e então realiza-se o treinamento do modelo por meio de uma biblioteca (e.g., *scikit-learn*). Em seguida, representados pelos blocos em amarelo, os resultados desse treinamento são então utilizados para mapear o modelo à ADON e, posteriormente, gerar o código necessário para as abordagens avaliadas.

Figura 26 Diagrama de atividades para o treinamento.

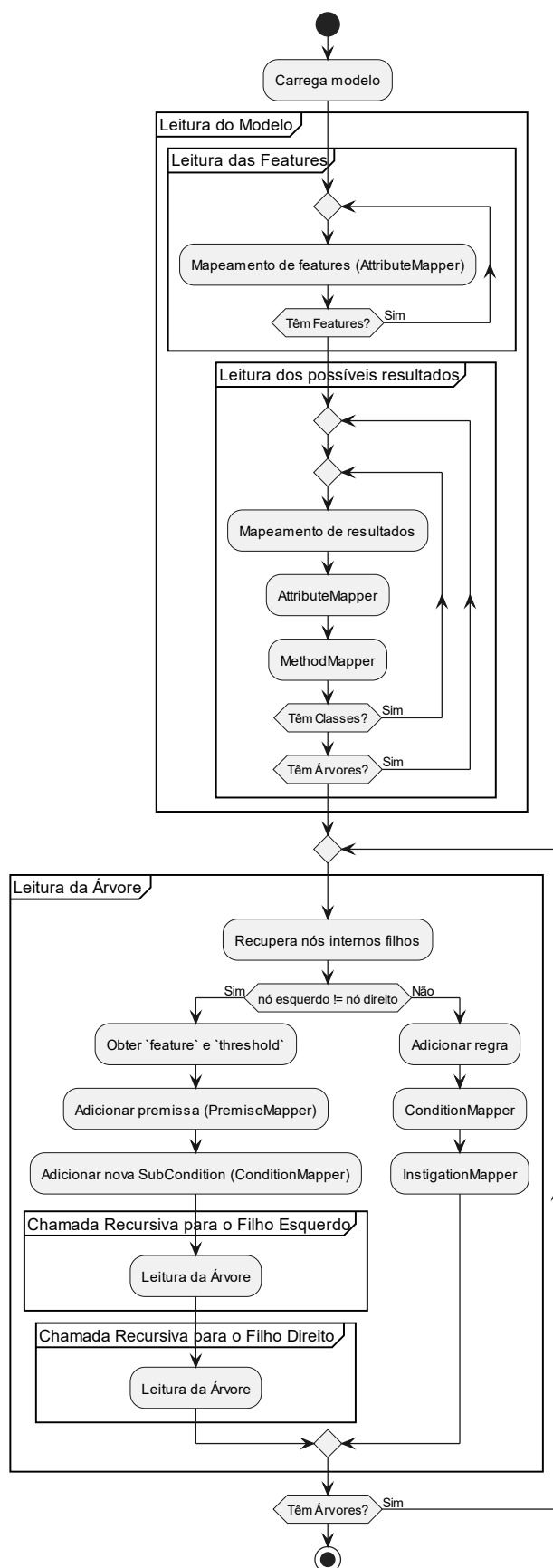


Fonte: Autoria própria

Para a integração com a etapa de treinamento, cada árvore do modelo é percorrida completamente e os dados de cada nó são armazenados em uma tabela para cada componente do PON que é instanciado efetivamente na ADON, como

Attributes, *Methods*, *Premises*, *Conditions* e *Instigations*. Para isso, o bloco *NODTAMapper* atua como um elemento intermediário que traduz os modelos de árvores de decisão treinados em componentes do PON. Este bloco utiliza subcomponentes especializados, tais como *AttributeMapper*, *PremiseMapper*, *ConditionMapper*, *MethodMapper* e *InstigationMapper*, para realizar o mapeamento detalhado de cada parte do modelo de árvore de decisão para seu equivalente no PON, conforme detalhado na Seção 3.2.1. A Figura 27 apresenta um diagrama de atividades exemplificando o processo de leitura de um modelo de árvore de decisão, seguido do mapeamento das entidades da árvore de decisão aos elementos do PON contemplados na ADON.

Figura 27 Diagrama de atividades do processo de leitura e mapeamento do modelo.

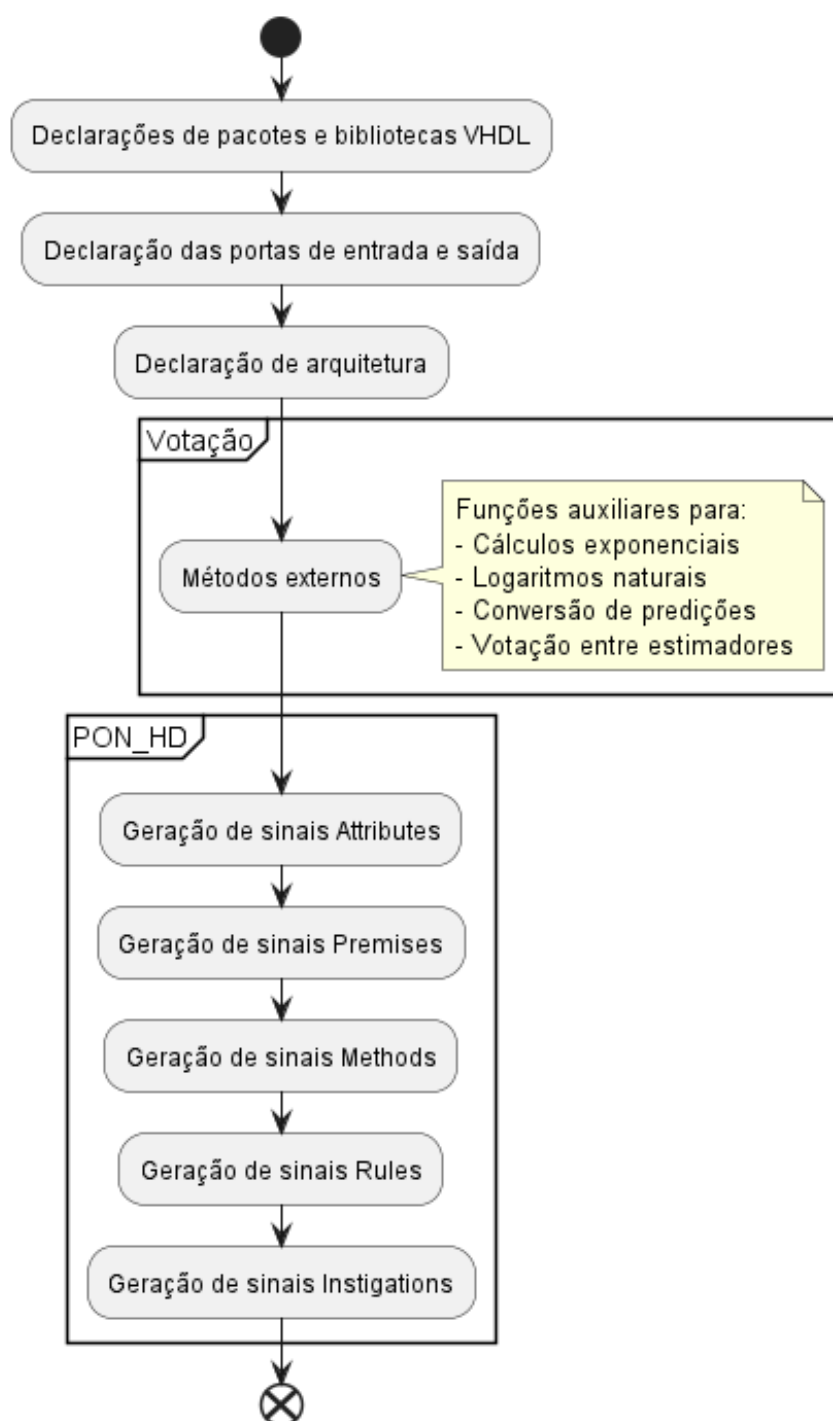


Fonte: Autoria própria

Após o mapeamento das entidades ADON a partir do modelo de árvore de decisão treinado, o próximo passo no processo envolve a geração de códigos específicos para diferentes plataformas de execução. Este processo de geração de códigos é realizado pelo bloco *NODTAGenerator*, que deve ser implementado por blocos especializados, como o *NopDHGenerator*, *NOPLGenerator* e *FrameworkGenerator*. Cada bloco é responsável por traduzir os elementos ADON mapeados para uma forma adequada ao ambiente de execução selecionado.

Na Figura 28 é demonstrado um diagrama de atividades contendo o processo de geração de código VHDL realizado pelo bloco *NopDHGenerator* que é responsável por instanciar os elementos da ADON em um formato compatível com *hardware* digital específico, como FPGAs, utilizando a abordagem ADON-HD. Os subcomponentes responsáveis pelo mapeamento, *AttributeMapper*, *PremiseMapper*, *ConditionMapper*, *MethodMapper*, e *InstigationMapper*, são percorridos para a geração de cada elemento da ADON e suas interdependências, conforme os componentes da ADON-HD são definidos. Esta conversão é fundamental para explorar o paralelismo e a eficiência energética oferecidos por plataformas de *hardware* reconfiguráveis, maximizando o desempenho de execução dos modelos de árvores de decisão implementados.

Figura 28 Diagrama de atividades para geração de código PON-HD.



Fonte: Autoria própria

Cada gerador dentro da *NODTAGenerator* é projetado para garantir que o modelo ADON resultante esteja otimizado para a plataforma de execução específica, atendendo às propriedades elementares do PON como não redundância, visando desempenho apropriado, e desacoplamento, visando paralelismo. A arquitetura modular do processo de geração permite que novos geradores sejam adicionados

conforme surgem demandas por novas plataformas ou tecnologias. Embora o *NODTAGenerator* seja extensível, este trabalho de doutorado concentra-se especificamente na implementação da ADON-HD no contexto de *hardware* digital com o uso de FPGAs. A seção a seguir detalha como é realizada a implementação da ADON ao PON-HD, alcançando a ADON-HD, apresentando também as expansões necessárias ao PON-HD para atender os requisitos específicos apresentados na Seção 3.1.

3.4. Mapeamento da ADON ao PON-HD

Uma vez que o mapeamento dos componentes da estrutura de árvores de decisão foi mapeado para as entidades PON, o passo seguinte é a geração de código para a plataforma de execução alvo. Este processo é realizado utilizando o bloco especializado *NopDHGenerator*, que consiste em percorrer os elementos mapeados anteriormente e convertê-los para uma representação que faça uso do PON-HD, alcançando a implementação específica da ADON em *hardware* ADON-HD. O código VHDL gerado segue a estrutura das entidades PON-HD, mapeando cada elemento do modelo de árvore de decisão para componentes específicos de *hardware*.

3.4.1. Melhorias no PON-HD

Para a implementação de alguns modelos, como *Gradient Boost*, é fundamental o uso de valores decimais para a execução dos cálculos de votação e combinação das previsões. No entanto, a versão original do PON-HD, denominada PON-HD 1.0, proposta por Kerschbaumer (2018), não suporta diretamente operações com números reais, limitando-se ao uso de números inteiros. Para superar essa limitação, é necessário expandir o PON-HD através do desenvolvimento de novos componentes que utilizam aritmética de ponto fixo. Esses novos componentes foram projetados com base nas abordagens sugeridas por Kerschbaumer (2018), adaptando a infraestrutura do PON-HD para suportar operações aritméticas mais complexas sem comprometer a eficiência do *hardware*.

Na versão do PON-HD 1.0, a estrutura proposta para o *Attribute* exigiu a definição de um tipo de dados específico devido à característica que o número de entradas pode variar dependendo do número de *Methods* que acessam o *Attribute*

(KERSCHBAUMER, 2018). Desta forma, o PON-HD 1.0 define um tipo de dado em que há duas dimensões variáveis, a primeira dimensão consiste em variar o número de entradas, enquanto uma segunda dimensão permite variar o tamanho em número de bits conforme as características dos dados da aplicação a ser executada.

A criação desses novos componentes exigiu a definição de novos tipos de dados para serem utilizados pelos elementos do PON-HD, conforme apresentado no Código 1. Utilizou-se a biblioteca *ieee.fixed_pkg* para a definição desses dados. No entanto, diferente do tipo genérico *data* utilizado nos componentes do PON-HD 1.0, que permite a escolha do número de *bits* ao declarar os componentes, foi necessário especificar um número fixo de 24 *bits*. Nesse cenário, 16 *bits* são destinados a valores inteiros e 8 *bits* para a representação de valores decimais. Em seguida, o tipo de dado denominado *SFIXED_ARRAY* é aplicado na construção dos novos componentes *Attribute*, *Premise* e *Method*, replicando a mesma estrutura interna dos componentes do PON-HD 1.0 propostos por Kerschbaumer (2018).

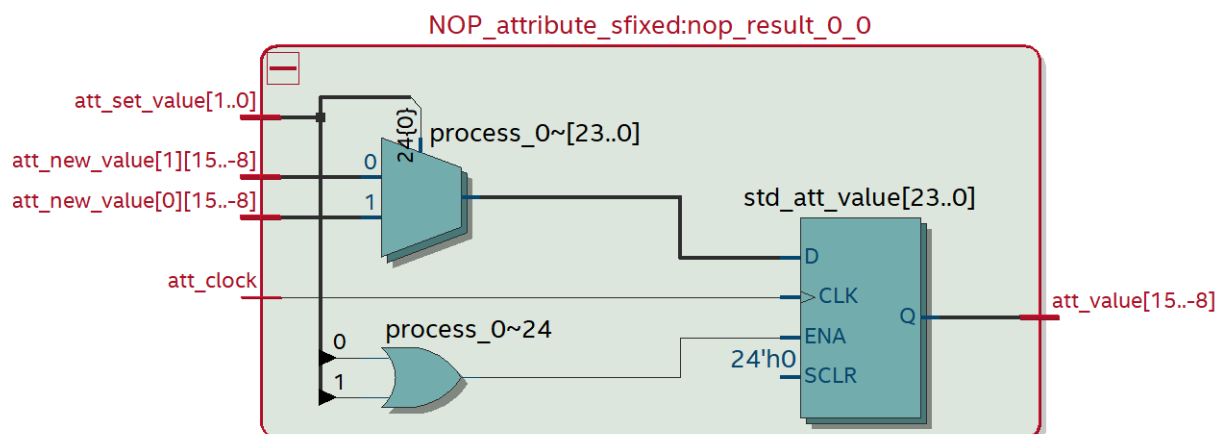
Código 1 Código VHDL com tipos de dados do *Attribute*.

1	LIBRARY ieee;
2	USE ieee.std_logic_1164.all;
3	USE ieee.fixed_pkg.all;
4	
5	PACKAGE data_type_pkg IS
6	TYPE data IS ARRAY(NATURAL RANGE <>) OF STD_LOGIC_VECTOR;
7	TYPE SFIXED_ARRAY IS ARRAY(NATURAL RANGE <>) OF SFIXED(15 downto -8);
8	END PACKAGE data_type_pkg;

Fonte: Autoria própria

Após a adaptação dos componentes originais para o uso de ponto fixo, foi realizada a síntese dos componentes, resultando em figuras que ilustram as estruturas dos *Attributes*, *Methods* e *Premises*. A Figura 29 mostra a estrutura do componente *Attribute*, destacando os componentes internos como registradores e mecanismo para priorização das atribuições de valor.

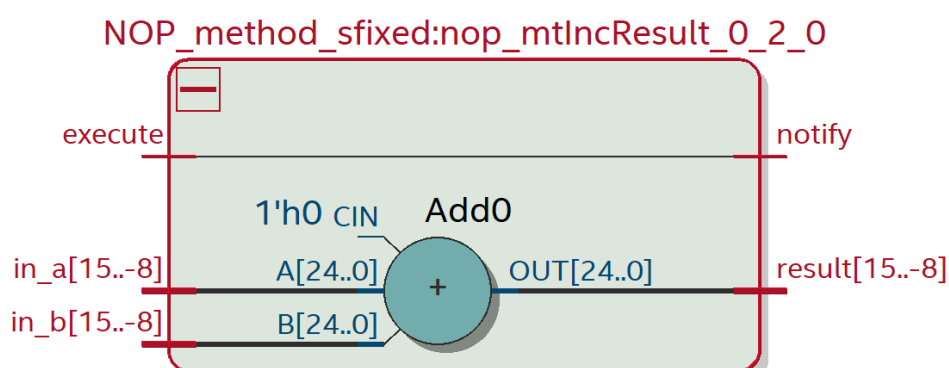
Figura 29 Componente *Attribute* com ponto fixo.



Fonte: Autoria própria

Por sua vez, a Figura 30 apresenta a estrutura do componente *Method*, que é responsável por realizar operações e cálculos factuais.

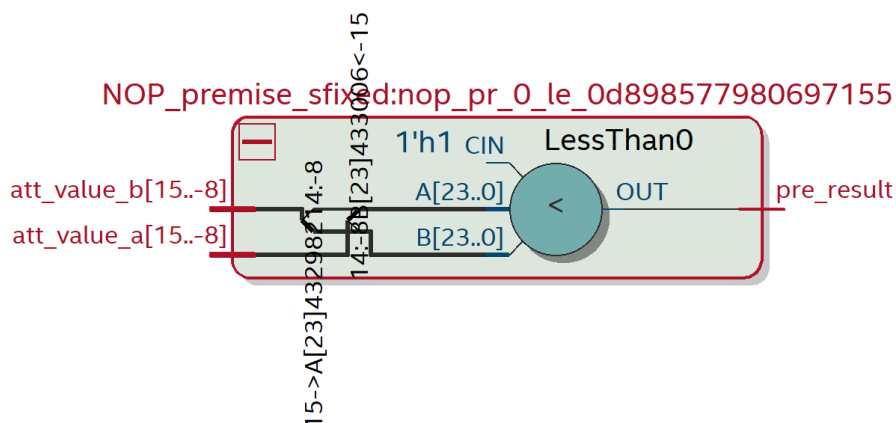
Figura 30 Componente *Method* com ponto fixo.



Fonte: Autoria própria

Por fim, a Figura 31 exibe a estrutura do componente *Premise*, que realiza comparações lógicas entre valores de *Attributes*.

Figura 31 Componente *Premise* com ponto fixo.



Fonte: Autoria própria

A partir da implementação dos componentes, foi possível habilitar na ADON a execução de modelos como *Gradient Boost*, em que os métodos de votação requerem o uso de valores decimais para a sua execução para a conversão dos resultados de cada folha em probabilidade. Enquanto no método de votação do modelo *Random Forest* é necessária apenas a soma dos resultados e seleção do resultado com maior valor.

3.4.2. Implementação dos métodos de votação

Para a realização do processo de votação, devido à sua complexidade, foi necessário realizar a implementação dos métodos diretamente em VHDL.

Para problemas de classificação utilizando o algoritmo *Random Forest*, o processo de decisão é baseado em votação majoritária. Cada árvore do modelo fornece um voto, que é ponderado de acordo com a distribuição das amostras entre as classes. Esses votos são acumulados em um *array*, cujo tamanho corresponde ao número de classes do modelo. Após a avaliação de todas as árvores, o *array* resultante é utilizado como parâmetro na função *voting* (*votingArray*), que identifica o índice com o maior valor acumulado, retornando a classe correspondente. O Código 2 implementa a lógica do processo de votação em VHDL. A função *voting* recebe um *array* (*votingArray*) contendo os votos ponderados de cada árvore, processa-o, e retorna o índice associado ao maior valor no *array*, que representa a classe com maior valor relacionado aos votos recebidos.

Código 2 Votação Majoritária em VHDL para classificação em *Random Forest*.

```

1 FUNCTION voting(votingArray: SFIXED_ARRAY(0 TO 2)) RETURN INTEGER IS
2     VARIABLE maxIndex : INTEGER := 0;
3     VARIABLE maxValue: SFIXED(15 downto -8) := (others => '0');
4 BEGIN
5     FOR i IN 0 to 2 LOOP
6         IF(votingArray(i) > maxValue) THEN
7             maxValue := votingArray(i);
8             maxIndex := i;
9         END IF;
10    END LOOP;
11    RETURN maxIndex;
12 END FUNCTION;

```

Fonte: Autoria própria

Em relação a problemas de classificação usando o algoritmo *Gradient Boosting*, a previsão para uma nova entrada de dados é obtida a partir da soma das predições feitas por várias árvores. No entanto, após calcular essa soma, é necessário aplicar a Equação 7 para cada classe, seguido da aplicação deste resultado na Equação 6 (Seção 2.2.2), que por sua vez, utiliza a Equação 8 para realizar a conversão dos resultados em probabilidades. Essa equação envolve somar as predições associadas a cada classe possível e, em seguida, transformar esses valores em probabilidades, de forma que a soma total das probabilidades seja igual a 1. Isso garante que a predição final forneça a probabilidade de cada classe ser a correta, permitindo a escolha da classe com a maior probabilidade. Por fim, o Código 3 apresenta a implementação da Equação 8 em VHDL.

$$\text{Classe}_i = \text{Valor inicial} + \sum_{j=1}^n (\text{taxa de aprendizado} \times \text{resultado estimador}_j) \quad (7)$$

Onde n é o número de árvores, ou estimadores, para cada classe i

Código 3 Predição para probabilidade.

```

1 FUNCTION prediction_to_proba(input_array: SFIXED_ARRAY(0 TO 2)) RETURN SFIXED_ARRAY IS
2     VARIABLE exp_array : SFIXED_ARRAY(0 TO 2);
3     VARIABLE logsumexp : SFIXED(15 downto -8);
4     VARIABLE result_array : SFIXED_ARRAY(0 TO 2);
5 BEGIN
6     FOR i IN 0 TO 2 LOOP
7         exp_array(i) := exponential(input_array(i));
8     END LOOP;
9     logsumexp := ln(resize(exp_array(0) + exp_array(1) + exp_array(2), logsumexp'high,
10 logsumexp'low));

```

```

11     FOR i IN 0 TO 2 LOOP
12         result_array(i) := resize(exp_array(i) - logsumexp, result_array(i)'high,
13 result_array(i)'low);
14     END LOOP;
15     RETURN result_array;
16 END FUNCTION;

```

Fonte: Autoria própria

Para o cálculo da função e^x em VHDL, é necessário utilizar métodos de aproximação numérica por meio de séries de Taylor, conforme definida na Equação 8. Essa fórmula permite calcular o valor de e^x com maior precisão, à medida que são incluídos mais termos na soma. O Código 4 apresenta a implementação da função e^x em VHDL. Para evitar cálculos desnecessários e economizar recursos de *hardware* para os cálculos dos termos fatoriais, é possível calcular previamente e deixá-los como constantes no código

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, -\infty < x < \infty \quad (8)$$

Código 4 Aproximação numérica para cálculo da função e^x em VHDL.

```

1  FUNCTION exponential(exponent: sfixed) RETURN sfixed IS
2      VARIABLE result : sfixed(15 downto -8);
3      VARIABLE intermediate_multiply : sfixed(31 downto -16);
4      VARIABLE intermediate_division : sfixed(31 downto -16);
5      VARIABLE intermediate_sum : sfixed(20 downto -8);
6      VARIABLE exponents : SFIXED_ARRAY(0 to 8);
7      VARIABLE terms : SFIXED_ARRAY(0 to 8);
8      CONSTANT FAT: sfixed_array := (
9          "00000000000000000000000000000000", -- FAT(0)
10         "00000000000000000000000000000000", -- FAT(1)
11         "00000000000000000000000000000000", -- FAT(2)
12         "00000000000000000000000000000000", -- FAT(3)
13         "00000000000000000000000000000000", -- FAT(4)
14         "00000000000000000000000000000000", -- FAT(5)
15         "00000000000000000000000000000000", -- FAT(6)
16         "00000000000000000000000000000000", -- FAT(7)
17         "00000000000000000000000000000000");-- FAT(8)
18  BEGIN
19      exponents(0) := to_sfixed(1.0, exponent'high, exponent'low);
20      exponents(1) := exponent;
21      terms(0) := exponents(0);
22      terms(1) := exponents(1);
23      FOR i IN 2 TO 5 LOOP
24          intermediate_multiply := exponents(i-1) * exponent;
25          exponents(i) := resize(intermediate_multiply, exponents(i)'high,
26 exponents(i)'low);
27          intermediate_division := exponents(i) * FAT(i);

```

```

28     terms(i) := resize(intermediate_division, terms(i)'high, terms(i)'low);
29     END LOOP;
30     intermediate_sum := terms(0) + terms(1) + terms(2) + terms(3) + terms(4) + terms(5);
31     result := resize(intermediate_sum, result'high, result'low);
32     RETURN result;
33 END FUNCTION;

```

Fonte: Autoria própria

Por sua vez, para o cálculo de $\ln(x)$ também é necessário utilizar métodos de aproximação numérica. Porém para o caso de funções logarítmicas, pode ser utilizada a série de MacLaurin, conforme a Equação 9. Em seguida, o Código 5 apresenta a implementação desta aproximação usando séries de MacLaurin para fazer a aproximação dos valores de $\ln(x)$, e assim como nas séries de Taylor, os termos fatoriais também são previamente calculados para evitar usar recursos de *hardware* para cálculos dos mesmos.

$$\ln(1 + y) = y - \frac{y^2}{2} + \frac{y^3}{3} - \frac{y^4}{4} + \frac{y^5}{5} - \dots \quad (9)$$

Esta série converge para $|y| < 1$. Para calcular $\ln(x)$, onde $y = x - 1$.

Código 5 Aproximação numérica para cálculo da função $\ln(x)$ em VHDL.

```

1  FUNCTION ln(exponent: sfixed) RETURN sfixed IS
2      VARIABLE result : sfixed(15 downto -8);
3      VARIABLE intermediate_multiply : sfixed(31 downto -16);
4      VARIABLE intermediate_division : sfixed(31 downto -16);
5      VARIABLE intermediate_sum : sfixed(21 downto -8);
6      VARIABLE exponents : SFIXED_ARRAY(0 to 8);
7      VARIABLE terms : SFIXED_ARRAY(0 to 8);
8      CONSTANT FAT: sfixed_array := (
9          "000000000000000100000000",
10         "000000000000000100000000",
11         "0000000000000001010101",
12         "0000000000000001000000",
13         "000000000000000110011",
14         "000000000000000101010",
15         "000000000000000100100",
16         "000000000000000100000",
17         "00000000000000011100");
18  BEGIN
19      exponents(0) := resize(exponent - to_sfixed(1.0, exponent'high, exponent'low),
20      exponent'high, exponent'low);
21      terms(0) := exponents(0);
22      FOR i IN 1 TO 6 LOOP
23          intermediate_multiply := exponents(i-1) * resize(exponent - to_sfixed(1.0,
24      exponent'high, exponent'low), exponent'high, exponent'low);

```

25	exponents(i) := resize(intermediate_multiply, exponents(i)'high,
26	exponents(i)'low);
27	intermediate_division := exponents(i) * FAT(i);
28	terms(i) := resize(intermediate_division, terms(i)'high, terms(i)'low);
29	END LOOP;
30	intermediate_sum := terms(0) - terms(1) + terms(2) - terms(3) + terms(4) - terms(5) +
31	terms(6);
32	result := resize(intermediate_sum, result'high, result'low);
33	RETURN result;
34	END FUNCTION;

Fonte: Autoria própria

3.4.3. Testes da ADON-HD

Após a extensão dos componentes do PON-HD com ponto fixo e a implementação dos métodos para os cálculos de e^x e $\ln(x)$ em VHDL, foram realizados testes de unidade de cada componente para verificar o seu funcionamento. Os testes de unidade foram executados por meio de simulação, auxiliando na correção de cada método de uma forma mais ágil.

Em seguida, foi desenvolvida uma aplicação que serve como interface para a execução de testes da implementação da ADON-HD de modelos de árvores de decisão, comparando-os com o resultado de inferência esperado quando o mesmo modelo é executado com a biblioteca *scikit-learn*. Esta aplicação de interface, é apresentada no Código 6, em que é criada uma memória ROM (*Read Only Memory*) preenchida com os valores das características (*features*) de um modelo treinado. Essa memória é lida sequencialmente com o auxílio de um controlador desenvolvido por meio de máquinas de estados finitos, que aplica os valores lidos da memória as entradas do componente em *hardware* da ADON, que após um ciclo de *clock*, devolve o resultado em uma porta de saída.

Código 6 Código VHDL usado como interface para a realização de testes da ADON-HD.

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4  use IEEE.fixed_pkg.ALL;
5  USE work.data_type_pkg.all;
6  ENTITY RPInterfaceDigit IS
7  PORT(
8      CLK      : IN  STD_LOGIC;
9      RST      : IN  STD_LOGIC;
10     EN       : IN  STD_LOGIC;
11     INDEX    : OUT STD_LOGIC_VECTOR(31 DOWNTO 0);
12     Y        : OUT STD_LOGIC_VECTOR(23 DOWNTO 0)
13 );
14 END RPInterfaceDigit;
15 ARCHITECTURE RPInterfaceDigit OF RPInterfaceDigit IS
16     COMPONENT NOP_attribute_sfired IS
17         GENERIC (
18             N_bits: INTEGER;
19             fractional: INTEGER;
20             N_new_values: INTEGER;
21             initial_value: SFIXED
22         );
23         PORT(
24             -- inputs
25             att_clock :IN STD_LOGIC;
26             att_new_value :IN SFIXED_ARRAY; -- the new values for the attribute
27             att_set_value :IN STD_LOGIC_VECTOR; -- write the new values to the attribute
28             -- outputs
29             att_value :OUT SFIXED(N_bits - fractional - 1 downto 0 -
30 fractional):=initial_value -- the value of the attribute
31         );
32     END COMPONENT;
33     TYPE state IS (Idle, Preparing, SetNewData, Running, Done);
34     SIGNAL pr_state, nx_state: state;
35     SIGNAL in_attr0 : SFIXED_ARRAY(0 DOWNTO 0);
36     SIGNAL in_attr1 : SFIXED_ARRAY(0 DOWNTO 0);
37     SIGNAL in_attr2 : SFIXED_ARRAY(0 DOWNTO 0);
38     SIGNAL in_attr3 : SFIXED_ARRAY(0 DOWNTO 0);
39     SIGNAL in_attr0_set: STD_LOGIC_VECTOR(0 DOWNTO 0);
40     SIGNAL in_attr1_set: STD_LOGIC_VECTOR(0 DOWNTO 0);
41     SIGNAL in_attr2_set: STD_LOGIC_VECTOR(0 DOWNTO 0);
42     SIGNAL in_attr3_set: STD_LOGIC_VECTOR(0 DOWNTO 0);
43     SIGNAL NEXT_EVENT : STD_LOGIC;
44     SIGNAL rp_output : STD_LOGIC_VECTOR(23 DOWNTO 0);
45     SIGNAL COUNTER: INTEGER RANGE 0 TO 19;
46
47     type memory_type is array(0 to 19, 0 to 3) of SFIXED(15 downto -8);
48     constant ROM_CONTENT : memory_type := (
49         (to_sfired(-15.5, 15, -8), to_sfired(-8.0, 15, -8), to_sfired(4.0, 15, -8),
50         to_sfired(5.6, 15, -8)),
51         ...
52     );
53 BEGIN
54     Y <= rp_output;
55     PROCESS (CLK, RST)
56     BEGIN
57         IF (rst = '1') THEN
58             pr_state <= Idle;
59         ELSIF (CLK'EVENT AND CLK='1') THEN
60             pr_state <= nx_state;
61         END IF;
62     END PROCESS;
63
64     PROCESS (NEXT_EVENT, RST)
65     BEGIN
66         IF (RST = '1') THEN
67             COUNTER <= 0;
68         ELSIF (NEXT_EVENT'EVENT AND NEXT_EVENT='1') THEN
69             IF(COUNTER = 19) THEN
70                 COUNTER <= 0;
71             ELSE
72                 COUNTER <= COUNTER + 1;
73             END IF;
74         END IF;

```

```

75     INDEX <= STD_LOGIC_VECTOR(TO_UNSIGNED(COUNTER, INDEX'length));
76 END PROCESS;
77 PROCESS (pr_state, CLK, EN)
78 BEGIN
79     CASE pr_state IS
80     WHEN Idle=>
81         in_attr0(0) <= ROM_CONTENT(COUNTER, 0);
82         in_attr1(0) <= ROM_CONTENT(COUNTER, 1);
83         in_attr2(0) <= ROM_CONTENT(COUNTER, 2);
84         in_attr3(0) <= ROM_CONTENT(COUNTER, 3);
85         in_attr0_set <= "0";
86         in_attr1_set <= "0";
87         in_attr2_set <= "0";
88         in_attr3_set <= "0";
89         NEXT_EVENT <= '0';
90         IF(EN = '1') THEN
91             nx_state<=Preparing;
92         ELSE
93             nx_state<=Idle;
94         END IF;
95     WHEN Preparing=>
96         in_attr0(0) <= ROM_CONTENT(COUNTER, 0);
97         in_attr1(0) <= ROM_CONTENT(COUNTER, 1);
98         in_attr2(0) <= ROM_CONTENT(COUNTER, 2);
99         in_attr3(0) <= ROM_CONTENT(COUNTER, 3);
100        in_attr0_set <= "0";
101        in_attr1_set <= "0";
102        in_attr2_set <= "0";
103        in_attr3_set <= "0";
104        NEXT_EVENT <= '0';
105        nx_state<=SetNewData;
106    WHEN SetNewData=>
107        in_attr0(0) <= ROM_CONTENT(COUNTER, 0);
108        in_attr1(0) <= ROM_CONTENT(COUNTER, 1);
109        in_attr2(0) <= ROM_CONTENT(COUNTER, 2);
110        in_attr3(0) <= ROM_CONTENT(COUNTER, 3);
111        in_attr0_set <= "1";
112        in_attr1_set <= "1";
113        in_attr2_set <= "1";
114        in_attr3_set <= "1";
115        NEXT_EVENT <= '0';
116        nx_state<=Running;
117    WHEN Running=>
118        in_attr0(0) <= ROM_CONTENT(COUNTER, 0);
119        in_attr1(0) <= ROM_CONTENT(COUNTER, 1);
120        in_attr2(0) <= ROM_CONTENT(COUNTER, 2);
121        in_attr3(0) <= ROM_CONTENT(COUNTER, 3);
122        in_attr0_set <= "1";
123        in_attr1_set <= "1";
124        in_attr2_set <= "1";
125        in_attr3_set <= "1";
126        NEXT_EVENT <= '1';
127        nx_state<=Done;
128    WHEN Done=>
129        in_attr0(0) <= ROM_CONTENT(COUNTER, 0);
130        in_attr1(0) <= ROM_CONTENT(COUNTER, 1);
131        in_attr2(0) <= ROM_CONTENT(COUNTER, 2);
132        in_attr3(0) <= ROM_CONTENT(COUNTER, 3);
133        in_attr0_set <= "0";
134        in_attr1_set <= "0";
135        in_attr2_set <= "0";
136        in_attr3_set <= "0";
137        NEXT_EVENT <= '0';
138        nx_state<=Idle;
139    END CASE;
140 END PROCESS;
141 rp : entity work. AdonNOPDH
142 port map(
143     in_attr0 => in_attr0(0),
144     in_attr1 => in_attr1(0),
145     in_attr2 => in_attr2(0),
146     in_attr3 => in_attr3(0),
147     in_attr0_set => in_attr0_set(0),
148     in_attr1_set => in_attr1_set(0),
149     in_attr2_set => in_attr2_set(0),
150     in_attr3_set => in_attr3_set(0),
151     out_result => rp_output,

```

```

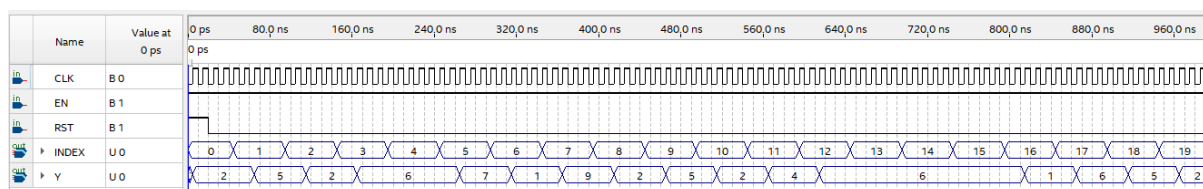
152     clock => CLK
153 );
154 END RPInterfaceDigit;

```

Fonte: Autoria própria

Após a execução dos testes, os resultados obtidos via simulação apresentados na Figura 32 foram comparados com aqueles gerados pela biblioteca *scikit-learn* utilizando o mesmo modelo e conjunto de dados. A implementação da ADON não altera o resultado do modelo, ou seja, a ADON atua em como o processamento é realizado, preservando a integridade das predições originais. Desta forma, a comparação confirmou que ambas as implementações produziram resultados idênticos para as mesmas entradas.

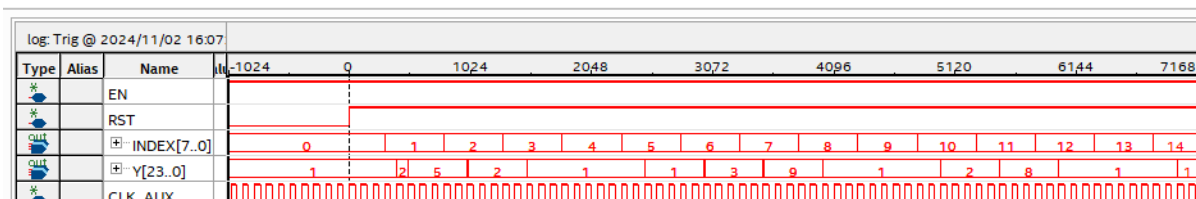
Figura 32 Resultados dos testes obtidos via simulação.



Fonte: Autoria própria

Posteriormente, a mesma interface para a execução de testes foi programada em um dispositivo FPGA, a fim de confirmar os resultados obtidos via simulação. Para essa validação, foi utilizado o kit de desenvolvimento Terasic DE10-Nano, que contém o dispositivo FPGA Cyclone V. O desenvolvimento e a programação foram realizados com o *software Quartus Prime*, que também foi usado para a análise dos sinais através da funcionalidade *SignalTap* presente na aplicação. Em tempo, é importante ressaltar que cada fabricante de dispositivos FPGA disponibiliza e licencia seu próprio ambiente de desenvolvimento integrado para síntese e programação, não sendo possível utilizar ferramentas de outros fabricantes para programar seus dispositivos. Por fim, os testes realizados na implementação da ADON-HD, através de simulação e verificação em *hardware*, confirmam a correta funcionalidade do sistema conforme os requisitos.

Figura 33 Resultados dos testes obtidos em FPGA.



Fonte: Autoria própria

3.5. Considerações finais

Este capítulo apresentou as contribuições deste trabalho, que são relacionadas à proposta da ADON, uma arquitetura distinta que integra os princípios do PON para a composição e execução de árvores de decisão agrupadas em conjuntos correlatos, com destaque para o tipo *ensemble*, como *Random Forest* e *Gradient Boosting*. Na primeira parte deste capítulo, são detalhados os requisitos para o desenvolvimento da arquitetura geral ADON, abrangendo seus aspectos funcionais. Além disso, também são apresentados os requisitos de uma implementação específica desta arquitetura *hardware* via PON-HD, chamada ADON-HD.

Para atender ao requisito Req-01, foram descritas como as árvores de decisão são conformadas de acordo com as entidades do modelo PON. Isso inclui a tradução dos elementos estruturais e funcionais das árvores de decisão tradicionais para as entidades definidas pelo metamodelo do PON. Elementos como *Root*, *Internal Nodes*, *Leaf*, e *Voting* são mapeados para componentes específicos do PON, como *FBEs*, *Attributes*, *Methods*, *Rules*, *Premises*, *Conditions*, *Actions* e *Instigations* permitindo uma conformidade às propriedades do PON e seu modelo de inferência, também atendendo ao requisito Req-02 e Req-02.1.

Além disso, a partir da conformação dos elementos presentes nas árvores de decisão, verificou-se que era necessário expandir o PON por meio de dois novos conceitos que visaram a eliminação de redundâncias que ainda persistiam mesmo após a conformação ao PON. Estes são a *Premise Dual* e o *Encadeamento de SubConditions*, que foram descritos respectivamente nas seções 3.2.2.1 e 3.2.2.2. Essas extensões, alinhadas com o requisito Req-02.2, enriqueceram o metamodelo PON e permitiram explorar suas propriedades elementares de forma adequada na ADON. Naturalmente, tais novos conceitos podem e normalmente se fariam pertinentes para o PON em si, em outros domínios e afins.

Por sua vez, também foram abordadas as ferramentas desenvolvidas para o treinamento e mapeamento de um modelo de árvores de decisão, visando a sua execução conforme a arquitetura ADON. Conforme o requisito Req-03, as ferramentas desenvolvidas neste trabalho, permitem a integração com bibliotecas externas (Req-03.1), como *scikit-learn*, permitindo o treinamento e validação de modelos por métodos tradicionais e a execução por meio da implementação da arquitetura proposta.

Para a implementação do requisito Req-03.2, as especializações do bloco *NODTATrainingReader* são responsáveis por adaptar e integrar os métodos de treinamento específicos para cada tipo de modelo de árvore de decisão com a ADON. Este processo não só facilita a integração de diferentes modelos de árvores de decisão, mas também permite a extensão e adaptação flexível de novos algoritmos ou métodos de treinamento no futuro, mantendo a arquitetura ADON escalável e modular.

Após o mapeamento das entidades PON a partir do modelo de árvore de decisão treinado, o requisito Req-04 é implementado através das especializações do bloco *NODTAGenerator* que são responsáveis pela geração de códigos específicos para diferentes plataformas de execução. De forma similar aos blocos especializados do *NODTATrainingReader*, as especializações do bloco *NODTAGenerator* permitem a extensão e adaptação flexível de novos alvos além do PON-HD, contribuindo para manter a arquitetura ADON escalável e modular.

A implementação específica em *hardware* digital da ADON, chamada de ADON-HD, validou os requisitos propostos para a arquitetura geral da ADON (Req-HD-01) e confirmou sua viabilidade ao utilizar os conceitos do PON-HD (Req-HD-02). Ainda, a implementação do requisito Req-HD-03 estendeu o PON-HD para suportar operações de ponto fixo, permitindo que a ADON-HD executasse os métodos de votação conforme a precisão numérica exigida pelos algoritmos, como no caso do modelo *Gradient Boosting*.

Em síntese, a concepção da arquitetura ADON e a implementação específica ADON-HD, conforme os requisitos previstos, atenuariam as limitações atuais das implementações tradicionais de árvores de decisão em *hardware*, que são baseadas em técnicas dos paradigmas tradicionais de computadores. A implementação da ADON-HD confirma a viabilidade da arquitetura proposta, cumprindo os requisitos estabelecidos, atenuando as limitações existentes e preenchendo uma lacuna na

literatura. A ADON-HD tende a proporcionar um melhor uso dos recursos disponíveis através da diminuição das redundâncias estruturais e explorando paralelismo intrínseco da plataforma de execução.

Por fim, o capítulo 4 a seguir apresenta os experimentos realizados com a implementação da ADON-HD com o objetivo de avaliar esta implementação em relação a outra abordagem para execução de árvores de decisão em *hardware*. Mais precisamente a outra abordagem é a *hls4ml* considerada o estado da arte e da técnica acessível no âmbito deste trabalho de pesquisa e nos termos dados no final do capítulo 2.

4. EXPERIMENTOS E RESULTADOS - ADON em HD

Este capítulo tem como objetivo descrever os casos de estudo para a avaliação da ADON-HD⁷ por meio da implementação desta arquitetura utilizando conformações do PON-HD, bem como, avaliar e discutir os resultados obtidos. A arquitetura proposta é avaliada utilizando os algoritmos *Random Forest* e *Gradient Boosting*. O objetivo destes experimentos é avaliar a implementação da arquitetura ADON-HD em relação à abordagem *hls4ml* para a execução destes algoritmos em *hardware*, especialmente no que se refere às arquiteturas de execução paralela de árvores de decisão em FPGAs. Nestes experimentos são avaliados o número de elementos lógicos, a frequência operacional máxima e a quantidade de ciclos de *clock* necessários para a execução de uma previsão. Desta forma, este capítulo apresenta os materiais e métodos utilizados, a descrição de cada caso de estudo e das diferentes configurações utilizadas, além dos resultados obtidos e a sua respectiva análise.

4.1. Materiais e métodos

Esta seção apresenta os detalhes em relação aos recursos utilizados para a realização dos experimentos, além do método empregado para obtenção e análise dos dados.

4.1.1. Recursos utilizados

Para a realização dos experimentos, foram utilizadas as seguintes ferramentas:

- **Hardware:** É utilizada como referência para a síntese, o dispositivo FPGA Xilinx Zynq-7000 (7z020clg484), que possui 53.200 elementos lógicos ou *Look-Up Tables* (LUTs) e 106.400 registradores ou *Flip-Flops* (FFs).
- **Software:**
 - **Vivado Design Suite:** *Software* utilizado para síntese e implementação dos circuitos no dispositivo FPGA utilizado como referência.

⁷ Os códigos referentes a implementação da ADON e ADON-HD utilizado nos experimentos apresentados nesta seção encontram-se no repositório institucional <https://nop.dainf.ct.utfpr.edu.br/nop-public/adon>.

- **Vivado High-Level Synthesis (HLS):** Ferramenta de síntese em alto nível da Xilinx, utilizada como parte do processo de implementação da biblioteca *hls4ml*.
- **Python 3.x:** Linguagem utilizada para o desenvolvimento da aplicação que realiza o treinamento dos modelos de árvores de decisão e o mapeamento para a arquitetura ADON.
- **Biblioteca *scikit-learn*:** Utilizada para o treinamento dos modelos de *Random Forest* e *Gradient Boosting*.
- ***hls4ml*:** Biblioteca *open-source* proposta por Summers *et al.* (2020) que permite a conversão de modelos de árvores de decisão para implementações em FPGA⁸. É pertinente recordar que esta abordagem é considerada o estado da arte e da técnica acessível no âmbito deste trabalho de pesquisa e nos termos dados no final do capítulo 2.

Dada a plataforma utilizada, o experimento foi realizado com dados de 24 bits, sendo 8 bits para representar valores decimais. Além disso, foi definido um sinal de *clock* com período de 10 ns, ou seja, 100 MHz para servir como ponto de referência de tempo para as análises relacionadas a tempo.

4.1.2. Método aplicado

O método para realizar os experimentos dos casos de estudo foi aplicado conforme as seguintes etapas:

1. **Treinamento dos Modelos:** Os modelos de *Random Forest* e *Gradient Boosting* foram treinados utilizando a biblioteca *scikit-learn*, com os conjuntos de dados selecionados.
2. **Mapeamento para ADON:** Os modelos treinados foram mapeados para a arquitetura ADON, conforme apresentado na Seção 3.3.
3. **Geração de Código para PON-HD:** A partir do mapeamento do conjunto de árvores para a ADON, foi gerado o código VHDL utilizando os componentes do PON-HD expandido, ou seja, incluindo as melhorias descritas na Seção 3.4.

⁸ Para este trabalho, a versão utilizada da ferramenta *hls4ml* (*Conifer*) foi a versão 1.5: <https://github.com/thesps/conifer/releases/tag/v1.5>

4. **Implementação em FPGA:** Os códigos gerados foram sintetizados e implementados no FPGA utilizando o Vivado Design Suite.
5. **Comparação com outra abordagem:** Para fins de comparação, os mesmos conjuntos de árvores de decisão foram implementados utilizando a biblioteca *hls4ml*.
6. **Avaliação de Desempenho:** Foram avaliados os seguintes parâmetros:
 - **Utilização de recursos lógicos:** Número de LUTs e FFs utilizados.
 - **Frequência de operação máxima:** Máxima frequência em que o circuito pode operar de forma estável.
 - **Número de previsões por segundo:** Número de previsões ou classificações que o circuito pode realizar por segundo.

4.1.3. Configuração do modelo

Para cada conjunto de dados (*dataset*), foram treinados modelos de *Random Forest* e *Gradient Boosting* com diferentes números de árvores (estimadores) para avaliar o impacto na utilização de recursos e desempenho. As configurações utilizadas foram:

- **Número de Árvores:** Variando de 10 a 200 estimadores.
- **Profundidade Máxima das Árvores:** Para fins de testes, as árvores foram limitadas a uma profundidade máxima de 10 níveis, conforme definido pelo modelo de treinamento.
- **Taxa de aprendizado:** Para o modelo *Gradient Boosting* utilizou-se uma taxa de aprendizado (*learning rate*) de 0,1.

4.1.4. Processo de síntese com a ferramenta estado da arte

Além da aplicação das ferramentas e processos apresentados na Seção 3 para o mapeamento de árvores de decisão ao ADON, mais especificamente o ADON-HD, este trabalho utilizou a ferramenta *hls4ml* para permitir a comparação dos casos de estudo com o ADON-HD e outra abordagem. Assim como as ferramentas desenvolvidas para o ADON, a ferramenta *hls4ml* também permite a integração de árvores de decisão treinadas por bibliotecas externas para a geração de código-alvo C/C++ compatível com a ferramenta Vivado HLS.

Este processo ocorre de forma similar ao exemplificado na Figura 26, em que primeiramente são carregados os dados de um *dataset*, realiza-se eventual tratamento nos dados previamente carregados, separa-se os dados entre dados de treinamento e testes para validação cruzada, e então realiza-se o treinamento do modelo por meio de uma biblioteca (e.g., *scikit-learn*). Em seguida, conforme o Código 7, os resultados desse treinamento são então utilizados para escrever a árvore de decisão em um modelo equivalente em Vivado HLS, gerando o código necessário para a abordagem avaliada.

Código 7 Configuração da ferramenta *hls4ml*.

```

1  # Cria um dicionário de configuração para o conifer/hls4ml
2  cfg = conifer.backends.xilinxhls.auto_config()
3
4  #Parametrização do dispositivo, ciclo de clock e quantidade de bits
5  cfg['XilinxPart']='xc7z020-clg400-3'
6  cfg['ClockPeriod']=10
7  cfg['Precision']='ap_fixed<24,8>'
8
9  # Define o diretório de saída como algo único
10 cfg['OutputDir'] = 'prj_iris_{_n_}'.format(tipo_modelo, qtde_estimadores)
11
12 # Criar o modelo do conifer/hls4ml
13 modelo_hls = conifer.converters.convert_from_sklearn(modelo_treinado, cfg)
14
15 # Escreve o projeto Vivado HLS
16 modelo_hls.write()
```

Fonte: Autoria própria

O processo apresentado no Código 7 gera um projeto em Vivado HLS que contém um *firmware* em C++, que por sua vez faz uso de diretivas de compilação específicas da ferramenta HLS com o objetivo de otimizar o desempenho e os recursos de *hardware*. Entre as diretivas usadas, destacam-se: *#pragma HLS array_partition*, que particiona os *arrays* para permitir o acesso paralelo aos dados; *#pragma HLS pipeline*, que instrui o compilador a implementar um *pipeline* a partir de um *loop*, iniciando uma nova iteração a cada novo ciclo de *clock*; e *#pragma HLS unroll*, que realiza o desenrolamento de loops para execução paralela.

Também são gerados os scripts em TCL (*Tool Command Language*) que permitem o processo de simulação e síntese do projeto gerado em Vivado HLS. Esses scripts podem ser integrados com a ferramenta *hls4ml* via *scripts* em *Python*, diretamente na IDE (*Integrated Development Environment* ou Ambiente de Desenvolvimento Integrado) do Vivado HLS ou através de interface de linha de comando que se integra diretamente com os *scripts* gerados. Desta forma, a execução do processo de síntese neste trabalho foi feita a partir da integração dos *scripts* gerados pela ferramenta *hls4ml* com as interfaces de linha de comando do

Vivado HLS, permitindo a automatização da execução da síntese para diferentes configurações. Já em relação às diretivas de compilação, foram mantidas as diretivas padrão geradas pela ferramenta *hls4ml*, pois estas representam um ponto de partida consistente e validado por outros pesquisadores, embora tanto a *hls4ml* quanto a ADON sejam passíveis de futuras otimizações.

4.2. Casos de estudo

Esta seção apresenta os casos de estudo a serem analisados neste trabalho. Para realizar os experimentos, são empregados três *datasets* amplamente conhecidos, nomeadamente *Iris*, *Wine* e *Digit*, cf. (FISHER, 1936; AEBERHARD; FORINA, 1992; ALPAYDIN; KAYNAK, 1998; UNWIN; KLEINMAN, 2021). Estes *datasets* foram selecionados para avaliar o desempenho da implementação da ADON-HD vis-à-vis *hls4ml*, utilizando os algoritmos *Random Forest* e *Gradient Boosting*.

Em cada caso, considerou-se a quantidade de avaliações lógicas (*i.e.*, avaliações “*se-então*”) geradas pelos modelos originalmente, e após a conformação à ADON. Neste sentido, foram consideradas as entidades mapeadas na arquitetura ADON, tais como *Attributes*, *Methods*, *Rules*, *Premises* e *Instigation*. A conformação à ADON dos modelos de árvore de decisão treinados tem como um dos objetivos eliminar grande parte das avaliações redundantes durante o processo. Essa eliminação de redundâncias tende a reduzir o uso de recursos em implementações de *hardware* em dispositivos FPGAs baseadas na implementação da ADON via PON-HD. Essas afirmativas serão corroboradas por meio dos experimentos apresentados nesse capítulo.

Para o primeiro caso de estudo, foi utilizado o conjunto clássico de dados *Iris* (*i.e.*, lírio) (UNWIN; KLEINMAN, 2021). Este *dataset* é utilizado em problemas de classificação com várias classes, contendo 150 elementos divididos em três classes da planta lírio, representando três espécies distintas chamadas *setosa*, *versicolor* e *virginica*. Para o treinamento e predição de um modelo com este *dataset*, são avaliadas quatro características: o comprimento e largura das sépalas e pétalas (UNWIN; KLEINMAN, 2021).

Ao realizar o treinamento deste *dataset* com os algoritmos *Random Forest* e *Gradient Boosting* utilizando as configurações apresentadas na Seção 4.1.3,

obtiveram-se as quantidades de avaliações lógicas. Também foram obtidas as entidades conformadas, tais como *Attributes*, *Methods*, *Rules*, *Premises* e *Instigations*, por meio da ferramenta de mapeamentos de modelos de árvores de decisão à ADON, conforme apresentado na Tabela 7.

Ao comparar o número de elementos *Premises* mapeados em relação à quantidade de avaliações *se-então* do modelo “original” sem conformação, observa-se a eliminação de avaliações redundantes após a conformação à ADON. Especificamente, no modelo *Gradient Boosting* com 200 árvores, o número de avaliações *se-então* diminui de 2.053 para 21 *Premises*. Esta quantidade de elementos *Premises* conformados à ADON representa 1% da quantidade inicial de avaliações, que por sua vez, corresponde a uma redução de aproximadamente 99% das avaliações redundantes.

Tabela 7 Números de elementos após treinamento e mapeamento do *dataset Iris*.

<i>Modelo</i>	<i>Árvores</i>	<i>Avaliações (se-então)</i>	<i>Attributes</i>	<i>Methods</i>	<i>Rules</i>	<i>Premises</i>	<i>Instigations</i>	<i>% Premises / Avaliações (se-então)</i>
<i>Random Forest</i>	10	80	34	60	90	42	60	53%
<i>Random Forest</i>	20	153	64	120	173	60	120	39%
<i>Random Forest</i>	50	387	154	300	437	86	300	22%
<i>Random Forest</i>	100	739	304	600	839	102	600	14%
<i>Random Forest</i>	200	1.490	604	1.200	1.690	128	1.200	9%
<i>Gradient Boosting</i>	10	164	34	60	194	15	60	9%
<i>Gradient Boosting</i>	20	324	64	120	384	15	120	5%
<i>Gradient Boosting</i>	50	804	154	300	954	15	300	2%
<i>Gradient Boosting</i>	100	1.604	304	600	1.904	15	600	1%
<i>Gradient Boosting</i>	200	2.053	604	999	2.653	21	999	1%

Fonte: Autoria própria

No segundo estudo de caso, foi utilizado o *dataset Wine* (ou seja, vinho). Este *dataset* inclui 178 amostras de vinhos, cada uma apresentando 13 características químicas, com a finalidade de determinar a origem entre três diferentes regiões (AEBERHARD; FORINA, 1992). As 13 características correspondem a diversos elementos químicos presentes no vinho, tais como teor alcoólico, acidez málica, cinzas, alcalinidade das cinzas, magnésio, fenóis totais, entre outros (AEBERHARD; FORINA, 1992).

Assim como no modelo treinado para o modelo *Iris*, realizou-se o treinamento e mapeamento à ADON do *dataset Wine* com os algoritmos *Random Forest* e *Gradient Boosting* com as configurações apresentadas na Seção 4.1.3. A partir

desse treinamento e mapeamento à ADON, obtiveram-se as quantidades de avaliações lógicas e as entidades conformadas à ADON, conforme apresentado na Tabela 8 que compara a quantidade de elementos da ADON em relação ao modelo “original” sem conformação. Cada avaliação presente em cada divisão das árvores de decisão representa uma avaliação lógica (*se-então*) para decidir qual o caminho irá seguir. Ao comparar o número de elementos *Premises* mapeados em relação a quantidade de avaliações *se-então*, observa-se a eliminação de avaliações redundantes após a conformação à ADON. Especificamente, no modelo *Gradient Boosting* com 200 árvores, o número de avaliações *se-então* diminui de 1.916 para 53 *Premises*. Esta quantidade de elementos *Premises* conformados à ADON representa 3% da quantidade inicial de avaliações, que por sua vez, corresponde a uma redução de aproximadamente 97% das avaliações redundantes.

Tabela 8 Números de elementos após treinamento e mapeamento do *dataset Wine*.

Modelo	Árvores	Avaliações (<i>se-então</i>)	Attributes	Methods	Rules	Premises	Instigations	% Premises / Avaliações (<i>se-então</i>)
<i>Random Forest</i>	10	96	43	60	106	89	60	93%
<i>Random Forest</i>	20	171	73	120	191	147	120	86%
<i>Random Forest</i>	50	458	163	300	508	353	300	77%
<i>Random Forest</i>	100	907	313	600	1.007	620	600	68%
<i>Random Forest</i>	200	1.787	613	1.200	1.987	988	1.200	55%
<i>Gradient Boosting</i>	10	156	43	60	186	32	60	21%
<i>Gradient Boosting</i>	20	306	73	120	366	32	120	10%
<i>Gradient Boosting</i>	50	756	163	300	906	32	300	4%
<i>Gradient Boosting</i>	100	1.506	313	600	1.806	32	600	2%
<i>Gradient Boosting</i>	200	1.916	613	1.002	2.516	53	1.002	3%

Fonte: Autoria própria

O terceiro caso de estudo avalia o dataset *Digit* (ALPAYDIN; KAYNAK, 1998). Este conjunto de dados consiste em imagens de dígitos manuscritos de 0 a 9, sendo que cada entrada é uma imagem 8x8 referente a um dígito, resultando em 64 atributos correspondentes a cada *pixel* da imagem (ALPAYDIN; KAYNAK, 1998). Assim como nos demais *datasets*, a partir do treinamento e mapeamento do dataset *Digit*, obtiveram-se as quantidades de avaliações lógicas, e as entidades conformadas à ADON conforme apresentado na Tabela 9. Ao comparar o número de elementos *Premises* mapeados em relação a quantidade de avaliações *se-então*, observa-se a eliminação de avaliações redundantes após a conformação à ADON. Especificamente, no modelo *Gradient Boosting* com 200 árvores, o número de

avaliações *se-então* diminui de 77.286 para 1.439 *Premises*. Esta quantidade de elementos *Premises* conformados à ADON representa 2% da quantidade inicial de avaliações, que por sua vez, corresponde a uma redução de aproximadamente 98% das avaliações redundantes.

Tabela 9 Números de elementos após treinamento e mapeamento do *dataset Digit*.

Modelo	Árvores	Avaliações (se-então)	Attributes	Methods	Rules	Premises	Instigations	% Premises / Avaliações (se-então)
<i>Random Forest</i>	10	990	164	200	1.000	599	200	61%
<i>Random Forest</i>	20	1.980	264	400	2.000	859	400	43%
<i>Random Forest</i>	50	4.950	564	1.000	5.000	1.173	1.000	24%
<i>Random Forest</i>	100	9.900	1.064	2.000	10.000	1.283	2.000	13%
<i>Random Forest</i>	200	19.800	2.064	4.000	20.000	1.381	4.000	7%
<i>Gradient Boosting</i>	10	12.935	164	200	13.035	1.371	200	11%
<i>Gradient Boosting</i>	20	28.371	264	400	28.571	1.423	400	5%
<i>Gradient Boosting</i>	50	59.123	564	1.000	59.623	1.437	1.000	2%
<i>Gradient Boosting</i>	100	77.257	1.064	1.993	78.257	1.439	1.993	2%
<i>Gradient Boosting</i>	200	77.286	2.064	2.998	79.286	1.439	2.998	2%

Fonte: Autoria própria

A partir das tabelas apresentadas, observa-se que à medida que o número de árvores nos modelos aumenta, a proporção de *Premises* em relação às avaliações lógicas (*i.e.*, avaliações *se-então*) diminui. Isso indica que a ADON, devido às propriedades e conceitos oriundos, expandidos e conformados a partir do PON, tende a se tornar mais eficiente na eliminação de redundâncias em modelos mais complexos.

Quando se analisa o modelo *Gradient Boosting*, observa-se uma redução mais acentuada das avaliações redundantes em comparação com o algoritmo *Random Forest*. Desta forma, é demonstrado que a ADON beneficia efetivamente os modelos baseados em *Gradient Boosting* devido à natureza sequencial destes algoritmos, que acarreta a repetição de diversas partes das árvores, conforme Seção 2.2.2. Além disso, o número de *Attributes* e *Methods* cresce mais lentamente em comparação com o aumento das avaliações lógicas, indicando que a ADON reutiliza esses componentes e evita duplicações desnecessárias.

Em resumo, a conformação dos modelos *Random Forest* e *Gradient Boosting* à ADON apresenta uma otimização da estrutura do modelo, levando à redução de recursos em implementações de *hardware*. A seção a seguir apresenta os resultados da síntese destes casos de estudo com o objetivo de demonstrar os

benefícios da ADON em relação ao estado da arte e da técnica acessível nos diferentes *datasets* analisados.

4.3. Resultados e discussões

Após a seleção e análise dos casos de uso apresentados na seção anterior, esta seção apresenta os resultados comparativos entre a implementação da arquitetura ADON proposta em *hardware* utilizando o PON-HD e uma segunda implementação feita com o uso de uma biblioteca *open-source* da literatura chamada *hls4ml* (conforme Seção 2.2.3). Neste contexto, aplica-se o conjunto de *datasets* definidos (conforme Seção 4.2), variando suas configurações, para validar o modelo proposto com base nas métricas definidas (conforme Seção 4.1.2). Faz-se então comparações da ADON em relação à implementação por meio da biblioteca *hls4ml* de árvores de decisão do tipo *ensemble*, em particular *Random Forest* e *Gradient Boosting*. Por fim são feitas as considerações finais sobre o capítulo e reflexões sobre os resultados.

Os dados apresentados nesta seção são provenientes do processo de compilação, síntese e implementação de *hardware* realizado com o conjunto de ferramentas Vivado e Vivado HLS (Sozzo *et al.*, 2022). Eles representam estimativas obtidas durante a fase de síntese e implementação e não os resultados de uma execução em uma FPGA. Os detalhes completos dos testes da execução em FPGA são descritos na Seção 3.4.3, e estão fora do escopo desta seção, que se concentra exclusivamente nos dados relacionados à síntese e à implementação lógica fornecidas pelo ferramental Vivado e Vivado HLS para compilação e síntese.

4.3.1. Resultados ADON-HD preliminar

É pertinente lembrar que comparações relativas à implementação do preliminar da ADON para o algoritmo Random Forest foram realizadas e publicadas em Pordeus *et al.* (2023). Neste trabalho preliminar acima citado, comparou-se uma primeira implementação da ADON-HD com uma implementação *ad hoc* desenvolvida usando o Vivado HLS, que representa o estado da técnica em HLS conforme apontado em *surveys* como Cong *et al.* (2022) e Sozzo *et al.* (2022).

Entretanto, esta primeira implementação da ADON-HD estava limitada a números inteiros, pois ainda não havia suporte para aritmética de ponto fixo nos

componentes do PON-HD. Embora esta limitação não impactasse significativamente os experimentos com algoritmo o algoritmo *Random Forest* devido a forma que são realizados os métodos de votação, esta limitação de números inteiros do PON-HD inviabilizava a implementação do algoritmo *Gradient Boosting* em *hardware*, que requer cálculos com números decimais em seu processo de votação.

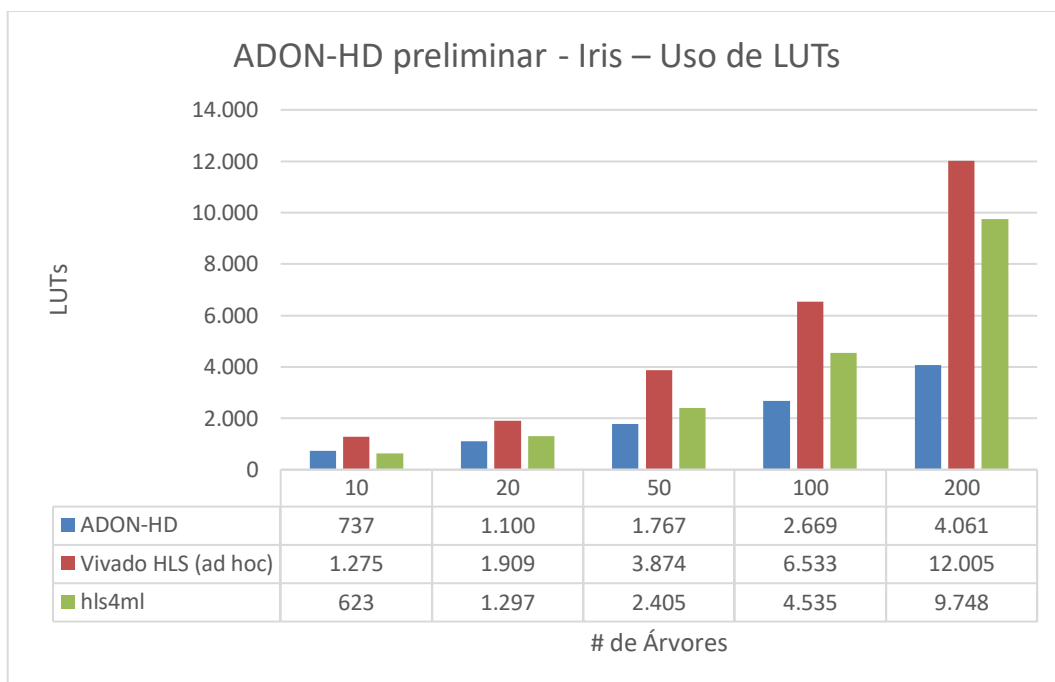
Os resultados iniciais comparando a ADON-HD com a implementação *ad hoc* em Vivado HLS foram promissores à ADON⁹. Como exemplo, a Figura 34, mostra o número de unidades lógicas (LUTs), enquanto a Figura 35 mostra o número de registradores (FFs) para o *dataset* Iris. Nestes experimentos usando o *dataset* Iris com 200 árvores, a ADON-HD utilizava aproximadamente 66,2% menos LUTs e 76,9% menos FFs que a implementação em Vivado HLS (*ad hoc*). Entretanto, apesar dos bons resultados, havia uma preocupação de que a implementação *ad hoc*, mesmo seguindo boas práticas de desenvolvimento, pudesse ser considerada enviesada para favorecer a solução ADON.

Para endereçar esta questão e realizar uma comparação mais justa e imparcial, buscou-se na literatura uma referência para implementação de árvores de decisão em *hardware*. Conforme discutido no final do capítulo 2 (Seção 2.2.3), identificou-se a solução *hls4ml* como estado da arte neste domínio, sendo utilizada e validada em trabalhos recentes como Summers *et al.* (2020), Damiani *et al.* (2022) e Alsharari *et al.* (2024). Nestes experimentos usando o *dataset* Iris com 200 árvores, a ADON-HD utilizava aproximadamente 58,3% menos LUTs e 58,3% menos FFs que a implementação em Vivado HLS (*hls4ml*).

Daqui para frente as comparações são consideradas justas e acima explicadas. Por sua vez, os demais resultados são apresentados no APÊNDICE A e artigo publicado.

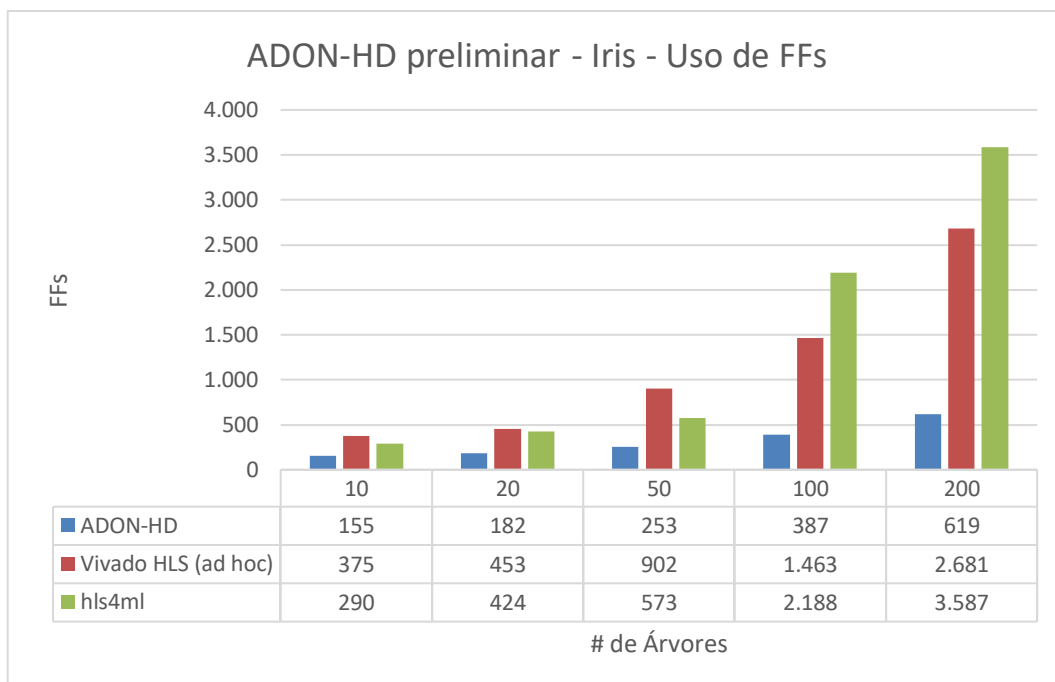
⁹ Inicialmente, foram realizadas comparações com os resultados gerados pela biblioteca *hls4ml* diretamente para o Vivado HLS. No entanto, durante a análise dos resultados iniciais observou-se uma discrepância entre os resultados da ADON-HD, em relação ao Vivado HLS *ad hoc* e Vivado HLS *hls4ml*. Para resolver este problema e garantir uma comparação justa e correta, faltava executar a fase implementação manualmente, pois esta fase não estava sendo executada por padrão.

Figura 34 Uso de LUTs com o algoritmo *Random Forest* para o *dataset Iris* na comparação ADON-HD preliminar vs Vivado HLS (*ad hoc*) vs Vivado HLS (*hls4ml*).



Fonte: Autoria própria

Figura 35 Uso de FFs com o algoritmo *Random Forest* para o *dataset Iris* na comparação ADON-HD preliminar vs Vivado HLS (*ad hoc*) vs Vivado HLS (*hls4ml*).

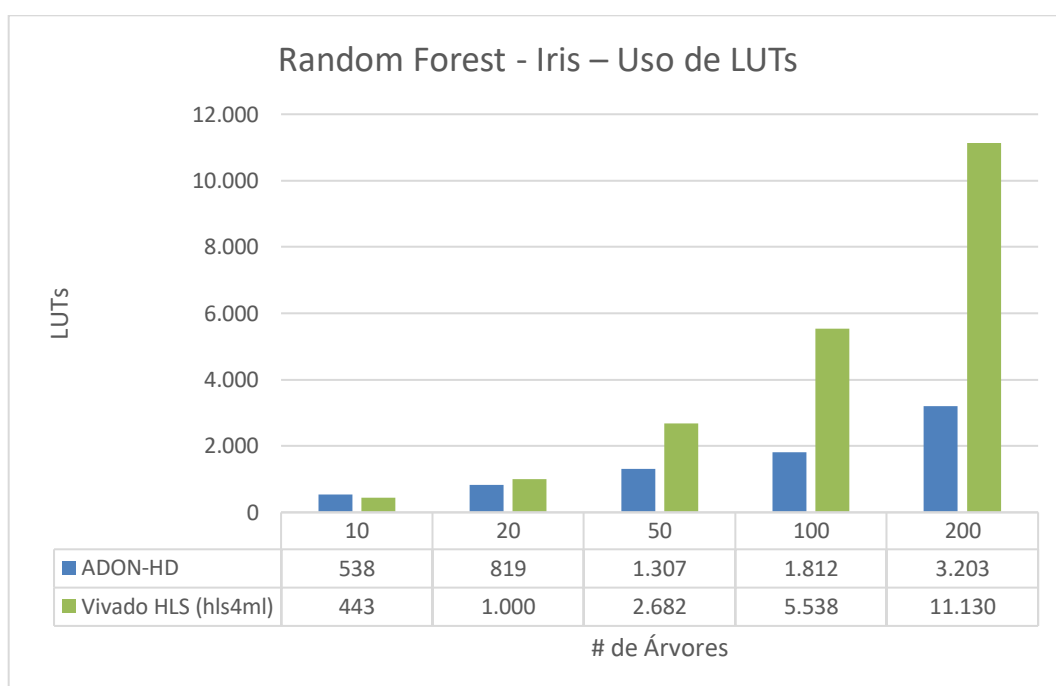


Fonte: Autoria própria

4.3.2. Resultados com *Random Forest*

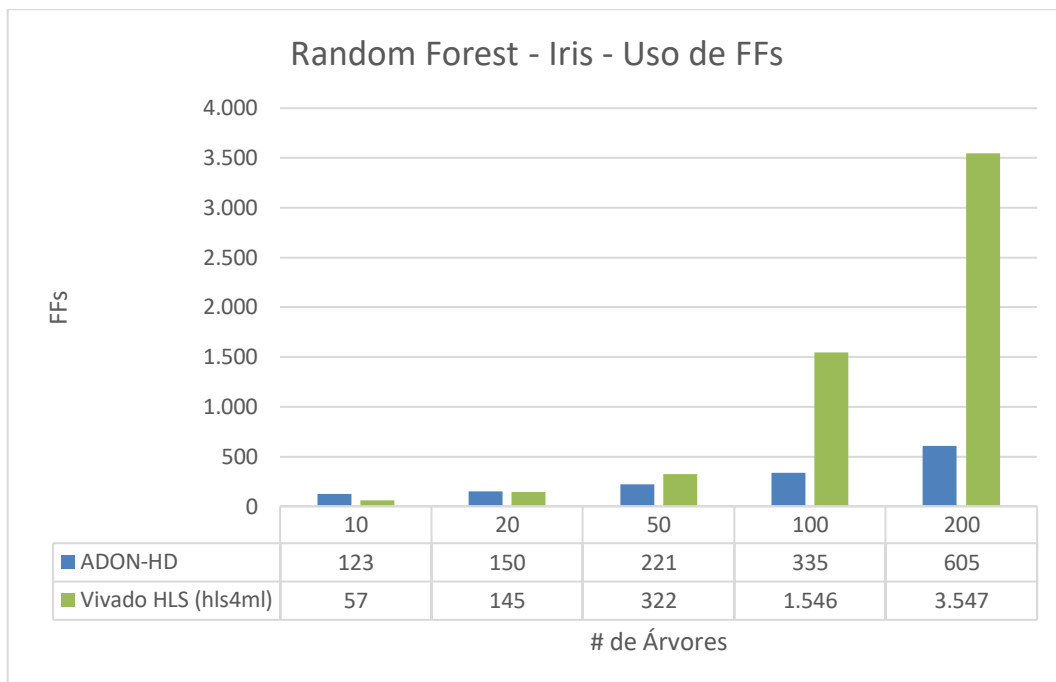
A primeira análise dos resultados diz respeito ao número de elementos lógicos utilizados em cada experimento com o algoritmo *Random Forest*. O conjunto de figuras (Figura 36, Figura 38 e Figura 40) mostra o número de unidades lógicas (LUTs), enquanto o conjunto das figuras (Figura 37, Figura 39 e Figura 41) mostra o número de registradores (FFs).

Figura 36 Uso de LUTs com o algoritmo *Random Forest* para o *dataset Iris*.



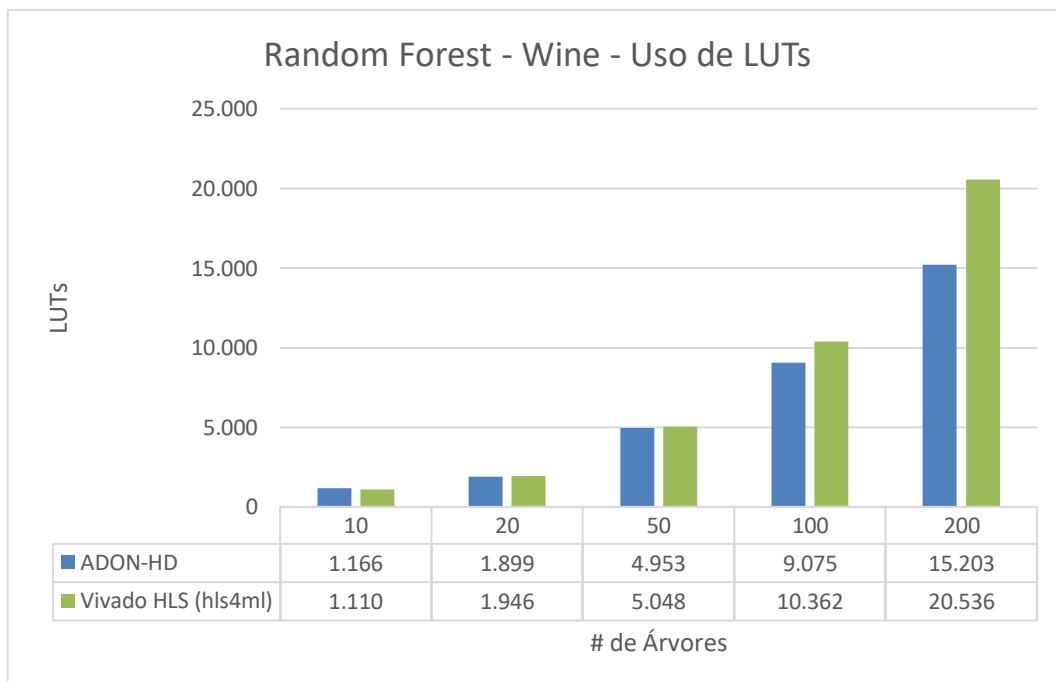
Fonte: Autoria própria

Figura 37 Uso de FFs com o algoritmo *Random Forest* para o dataset *Iris*.



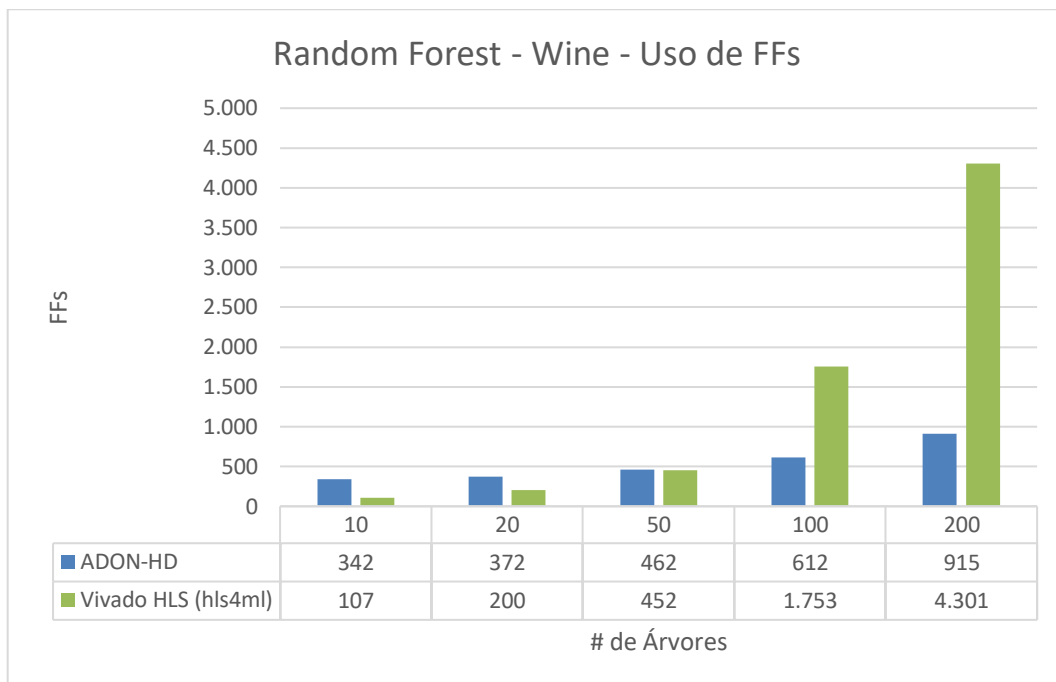
Fonte: Autoria própria

Figura 38 Uso de LUTs com o algoritmo *Random Forest* para o dataset *Wine*.



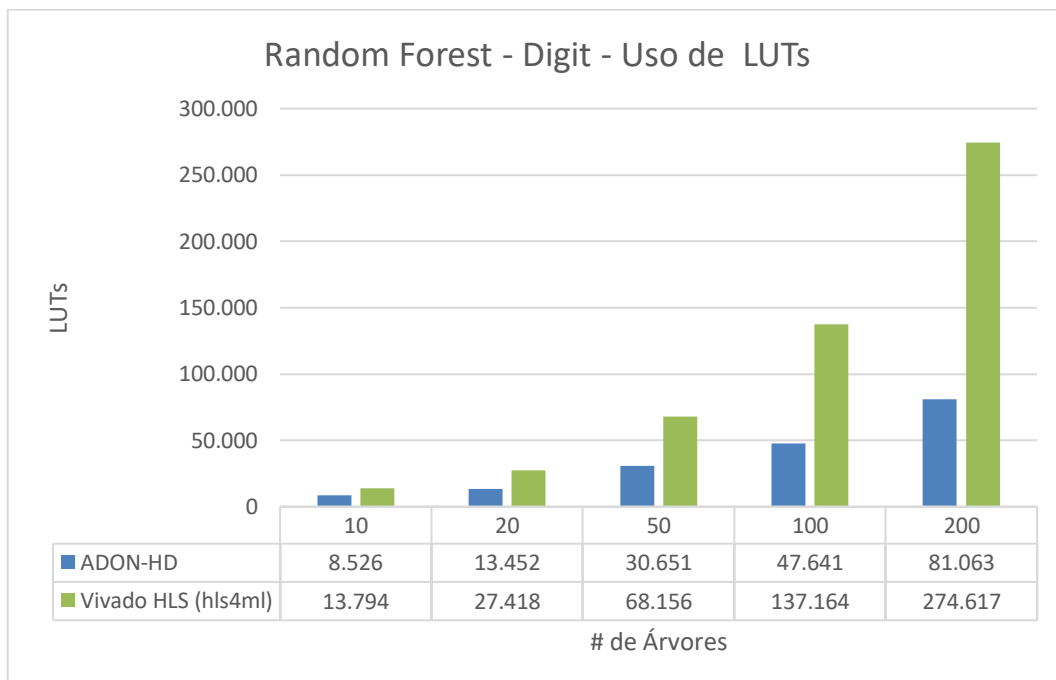
Fonte: Autoria própria

Figura 39 Uso de FFs com o algoritmo *Random Forest* para o *dataset Wine*.



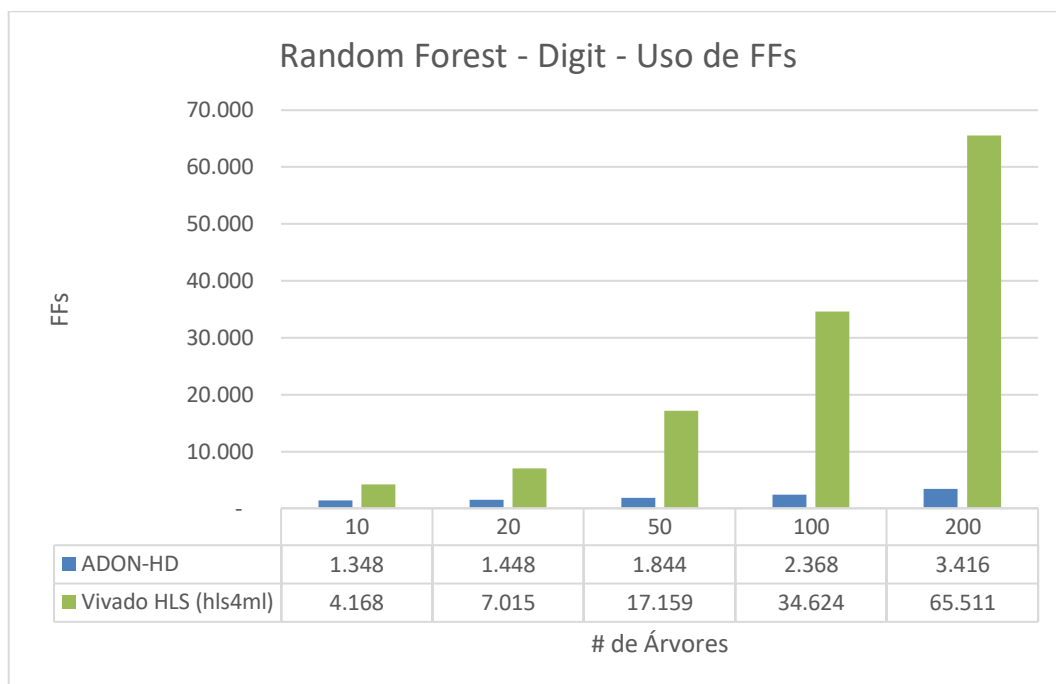
Fonte: Autoria própria

Figura 40 Uso de LUTs com o algoritmo *Random Forest* para o *dataset Digit*.



Fonte: Autoria própria

Figura 41 Uso de FFs com o algoritmo *Random Forest* para o *dataset Digit*.



Fonte: Autoria própria

Ao analisar os experimentos, os circuitos gerados com a biblioteca *hls4ml* apresentaram melhores resultados para os conjuntos com menor quantidade de árvores de decisão, ou seja menor consumo de recursos de LUTs e FFs. Ao aumentar o número de árvores, ou a quantidade de features analisadas, o circuito que utilizou a menor quantidade de recursos foi a abordagem com a arquitetura ADON via PON-HD.

Para o *dataset Iris*, a ADON usa aproximadamente 21,7% mais LUTs do que a abordagem *hls4ml* e aproximadamente 115,8% mais FFs com 10 árvores. Entretanto, à medida que o número de árvores aumenta, a ADON utiliza aproximadamente 71,2% menos LUTs e aproximadamente 82,9% menos FFs do que o *hls4ml* quando este experimento é treinado com 200 árvores.

Para o *dataset Wine*, a abordagem com a arquitetura ADON usa aproximadamente 5,0% mais LUTs do que a abordagem *hls4ml* e aproximadamente 219,6% mais FFs com 10 árvores. Da mesma forma, à medida que o número de árvores aumenta, a ADON passa a utilizar menos recursos lógicos. Quando este algoritmo é treinado com 200 árvores, a ADON usa aproximadamente 26,0% menos elementos lógicos e 78,7% menos FFs do que o *hls4ml*.

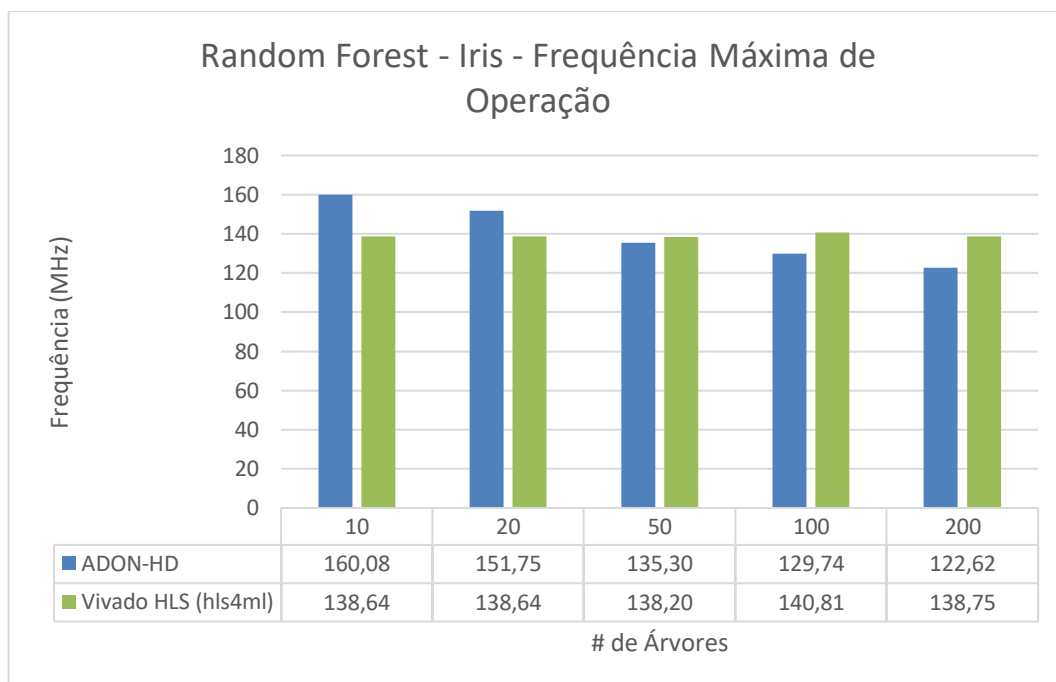
Para o *dataset Digit*, a ADON sempre requisita menos recursos lógicos do que a abordagem *hls4ml*. Por exemplo, a ADON usa aproximadamente 38,2%

menos LUTs e 67,7% menos FFs do que a abordagem *hls4ml* com 10 árvores, e a ADON usa aproximadamente 70,5% menos LUTs e 94,8% menos FFs do que o *hls4ml* com 200 árvores. No entanto, devido a quantidade de elementos LUTs (53.200) disponíveis na FPGA escolhida, os circuitos referentes às implementações do *hls4ml* acima de 50 árvores não são possíveis de serem sintetizados. Para a ADON, existe a mesma limitação, porém, apenas para a configuração com 200 árvores.

A segunda análise compara a frequência máxima (MHz) em que cada circuito pode operar. Os resultados dessa análise são apresentados no conjunto de figuras: Figura 42, Figura 44 e Figura 46. Nesta análise, tanto a ADON quanto a abordagem *hls4ml* apresentaram frequências de operação semelhantes. No entanto, com o aumento na quantidade de árvores de decisão, a frequência máxima de operação da ADON tende a diminuir enquanto a frequência de operação do *hls4ml* permanece constante, o que seria uma característica negativa para o ADON-HD.

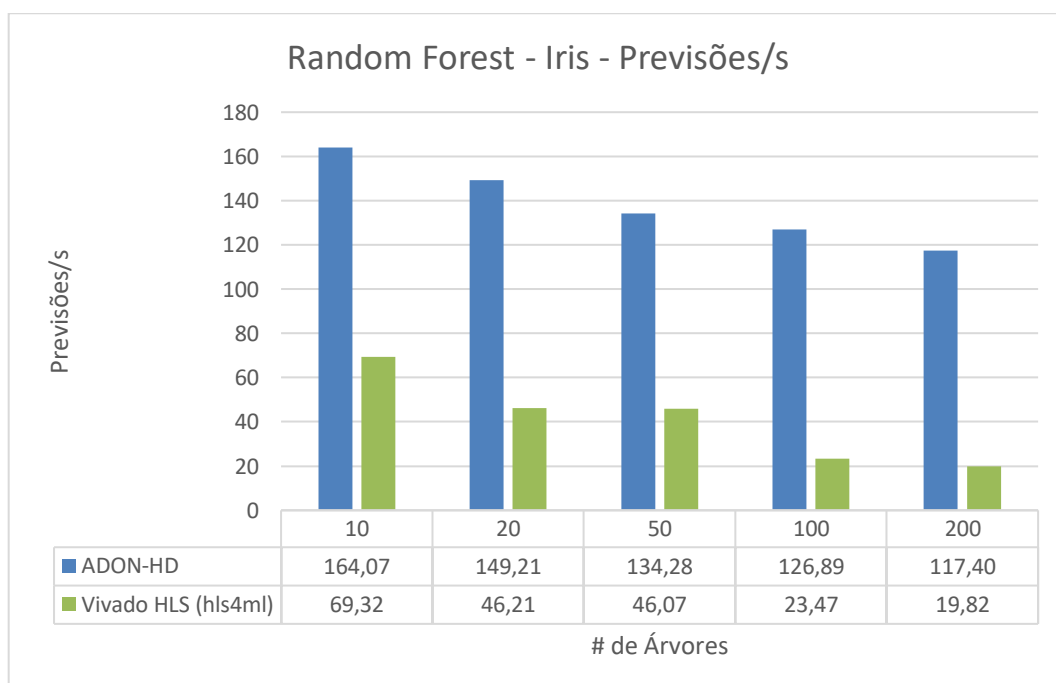
Estes resultados de frequência máxima são usados para a análise subsequente, pois fornecem dados essenciais para o cálculo da taxa de classificação. Esta análise subsequente compara o número máximo de previsões ou classificações por segundo (previsão/s) possível para cada abordagem, representando a taxa de classificação. As figuras: Figura 43, Figura 45 e Figura 47 mostram os gráficos representando a terceira análise, de forma que o cálculo da taxa de classificação é dado pela frequência máxima de operação dividida pelo número de ciclos de *clock* necessários para executar uma classificação.

Figura 42 Frequência Máxima de operação (MHz) com o algoritmo *Random Forest* para o *dataset Iris*.



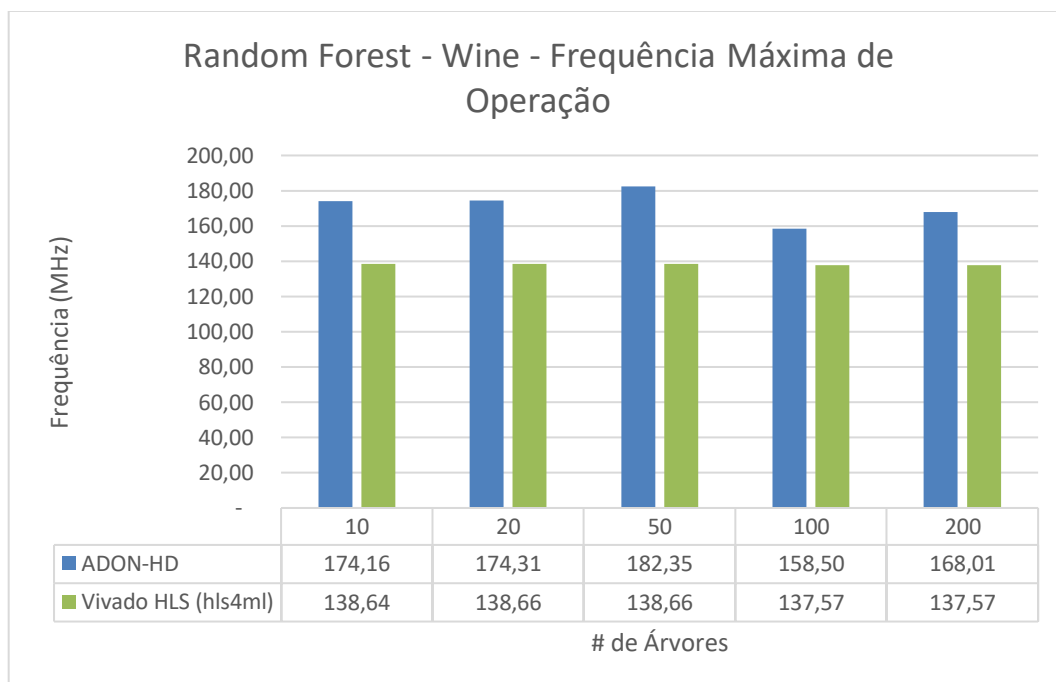
Fonte: Autoria própria

Figura 43 Predições/s com o algoritmo *Random Forest* para o *dataset Iris*.



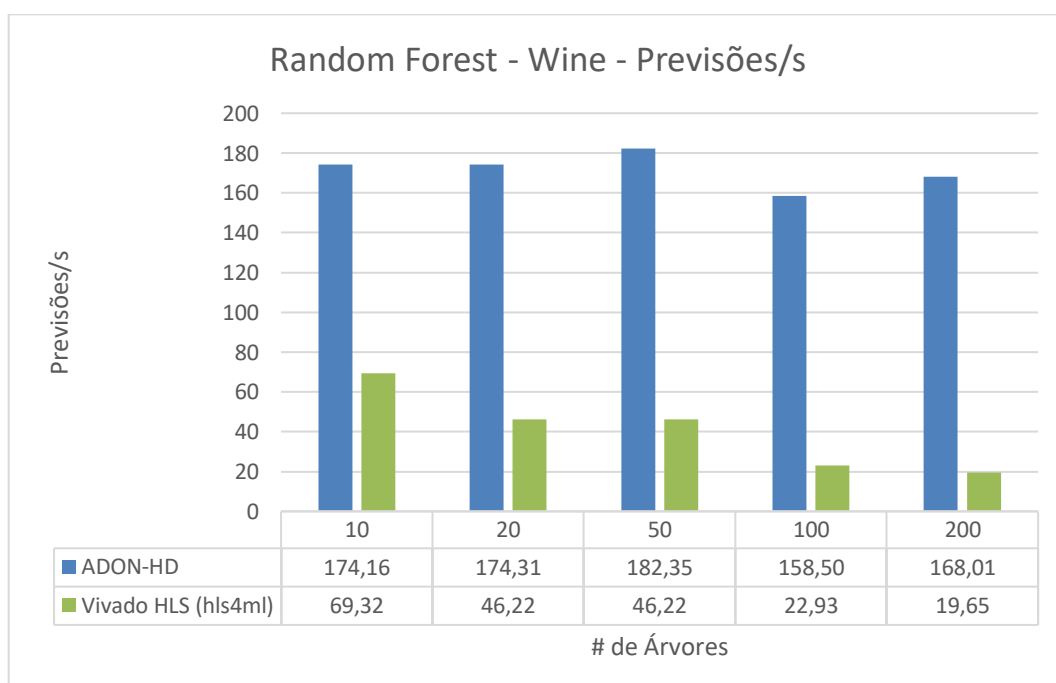
Fonte: Autoria própria

Figura 44 Frequência Máxima de operação (MHz) com o algoritmo *Random Forest* para o *dataset Wine*.



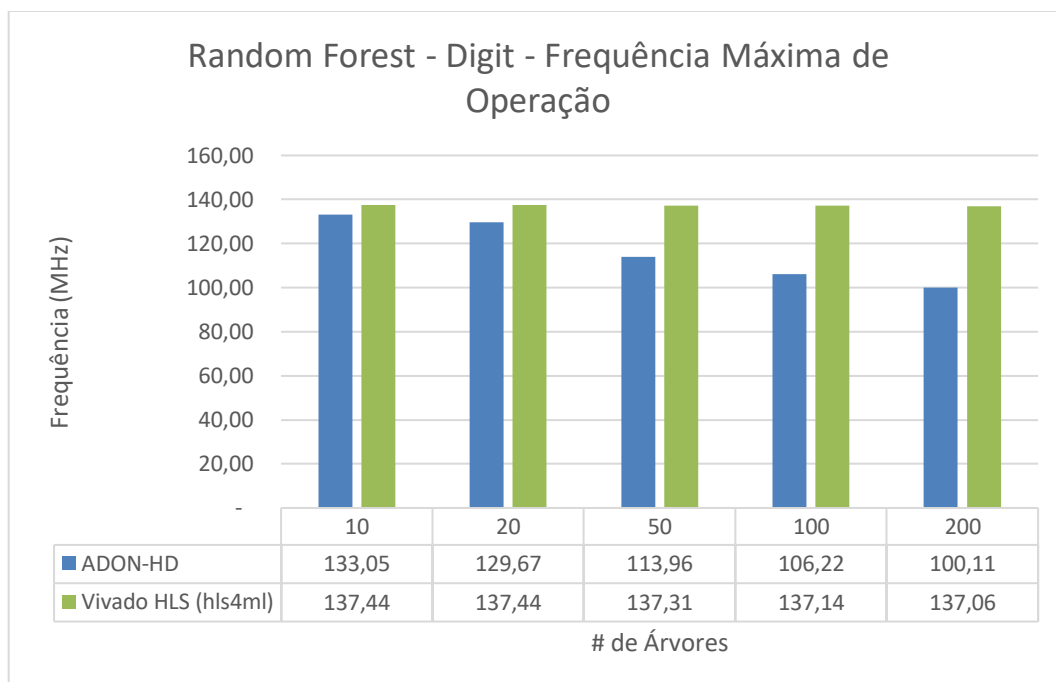
Fonte: Autoria própria

Figura 45 Predições/s com o algoritmo *Random Forest* para o *dataset Wine*.



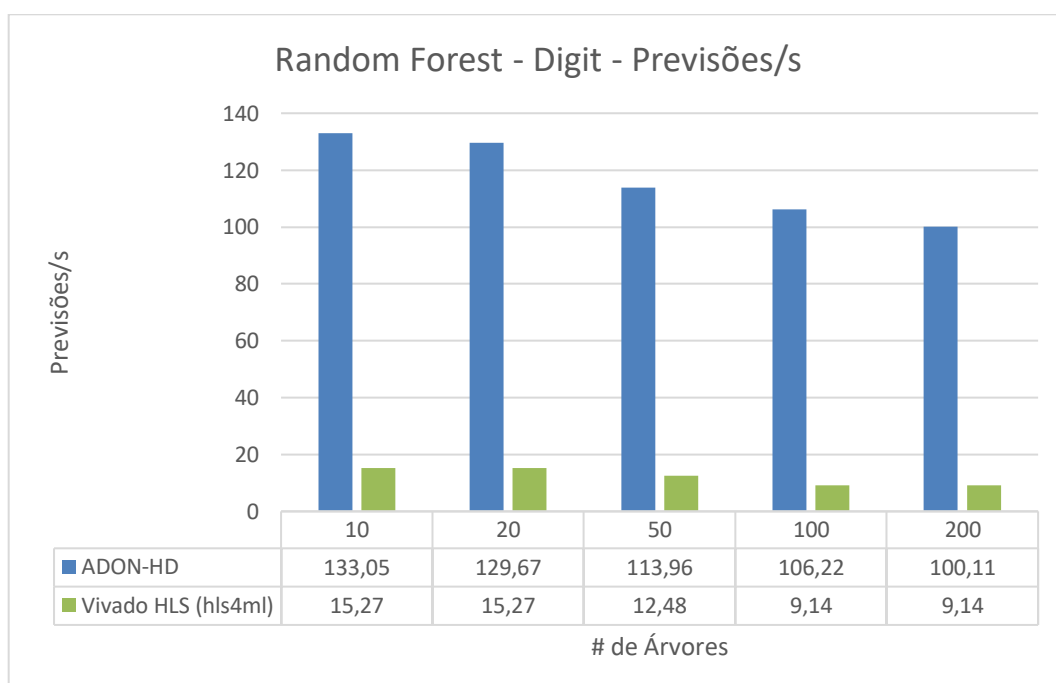
Fonte: Autoria própria

Figura 46 Frequência Máxima de operação (MHz) com o algoritmo *Random Forest* para o *dataset Digit*.



Fonte: Autoria própria

Figura 47 Predições/s com o algoritmo *Random Forest* para o *dataset Digit*.



Fonte: Autoria própria

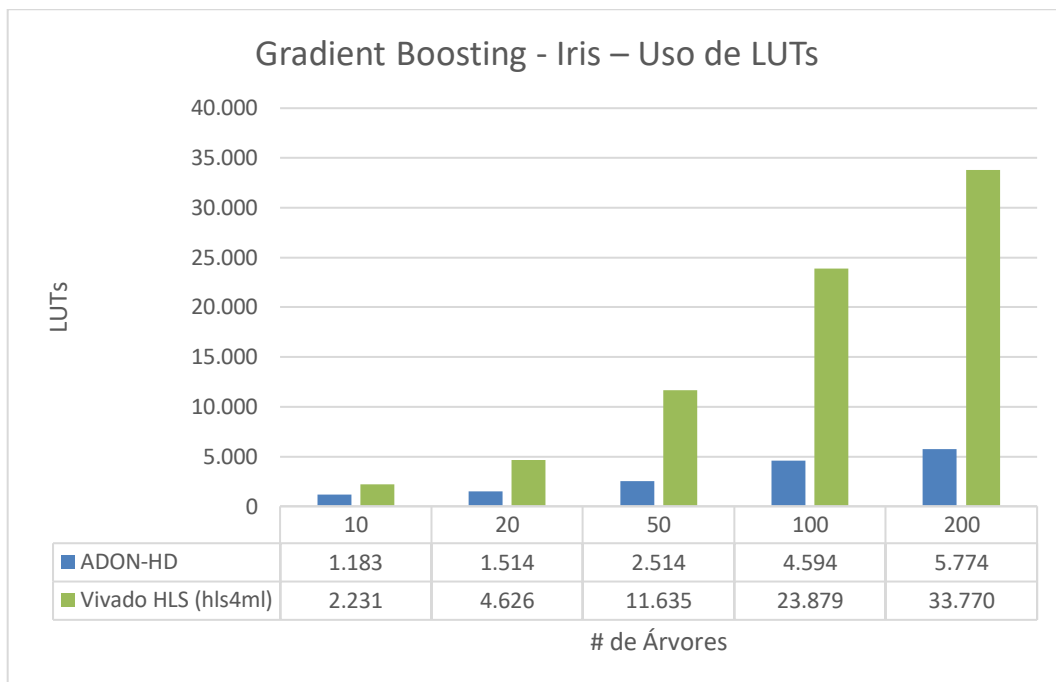
Devido à necessidade de apenas um ciclo de *clock* para a classificação, mesmo com resultados de frequência de operação tendendo a ser mais baixos com

o aumento no número de árvores, o número de previsões/s da ADON foi superior à abordagem *hls4ml* em até 11,62 vezes no caso do *dataset Digit*. Esses resultados ocorrem devido à estrutura dos componentes presentes na arquitetura ADON, pois apenas o componente *Attribute* precisa ser sincronizado pelo *clock*. Em contraste, os outros componentes do PON-HD são essencialmente combinacionais, assim não exigem sincronização através dos ciclos de *clock*. Mais especificamente para o algoritmo *Random Forest*, apenas as características do *dataset* e os atributos de resultado/votação são mapeados para elementos *Attribute*. Portanto, apenas esses elementos precisam ser sincronizados pelo *clock*. Por sua vez, o *hls4ml* precisa de mais registradores ao compilar o mesmo modelo.

4.3.3. Resultados com Gradient Boosting

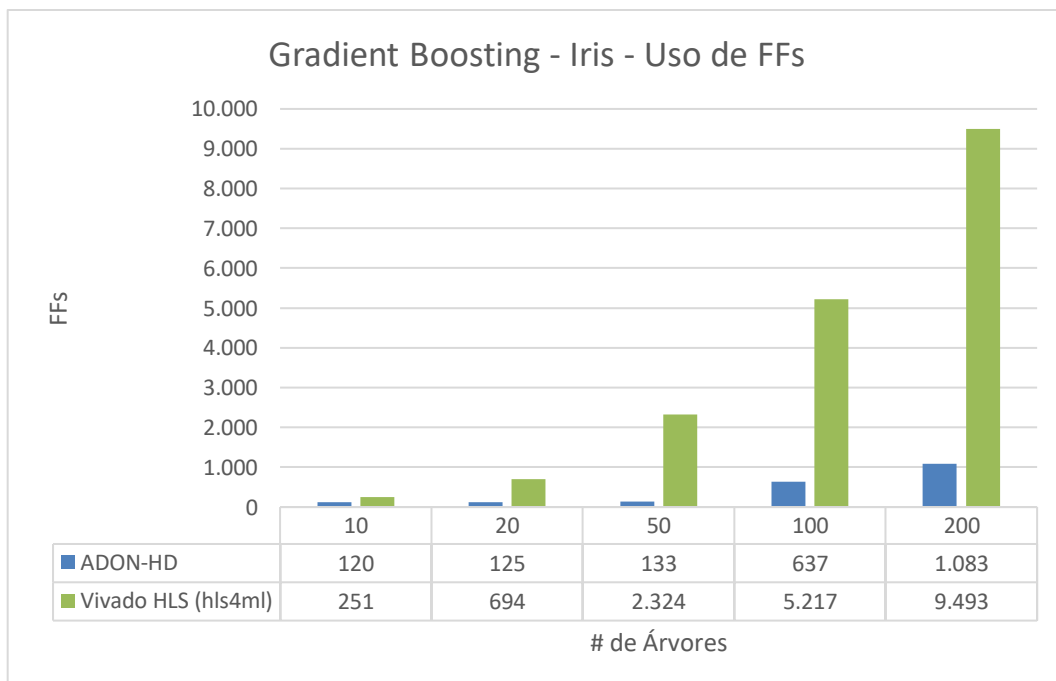
Após a realização dos experimentos e avaliação dos resultados para o algoritmo *Random Forest*, são repetidos os mesmos procedimentos e análises para o algoritmo *Gradient Boosting*. A primeira análise dos resultados diz respeito ao número de elementos lógicos utilizados em cada experimento com o algoritmo *Gradient Boosting*. As figuras: Figura 48, Figura 50 e Figura 52 mostram o número de unidades lógicas (LUTs), enquanto as figuras: Figura 49, Figura 51 e Figura 53 mostram o número de registradores (FFs).

Figura 48 Uso de LUTs com o algoritmo *Gradient Boosting* para o dataset *Iris*.



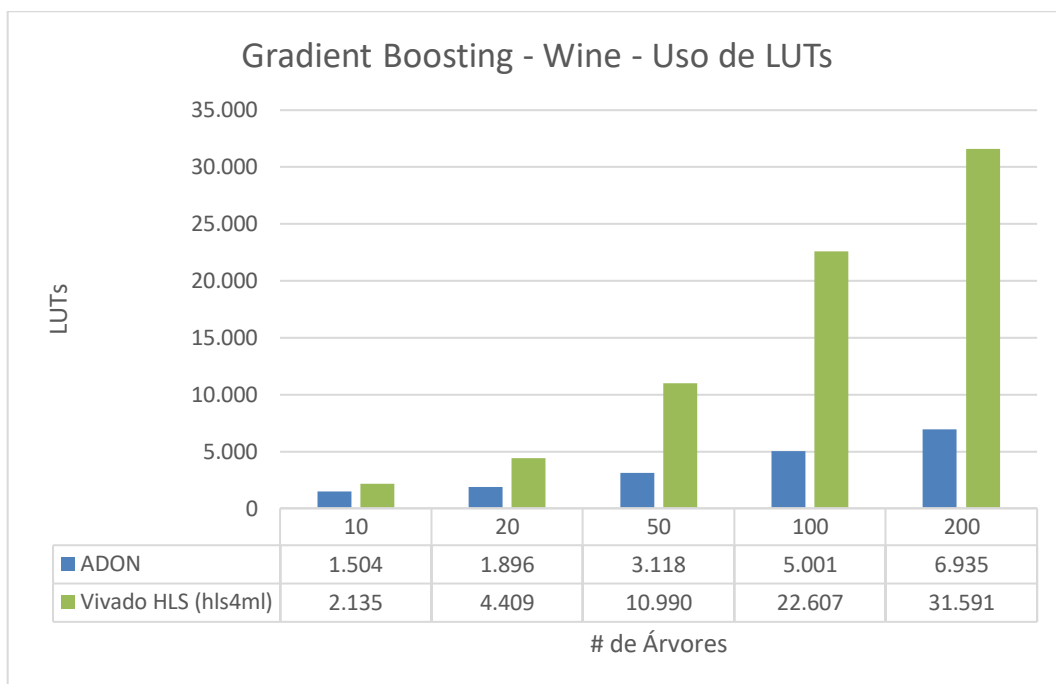
Fonte: Autoria própria

Figura 49 Uso de FFs com o algoritmo *Gradient Boosting* para o dataset *Iris*.



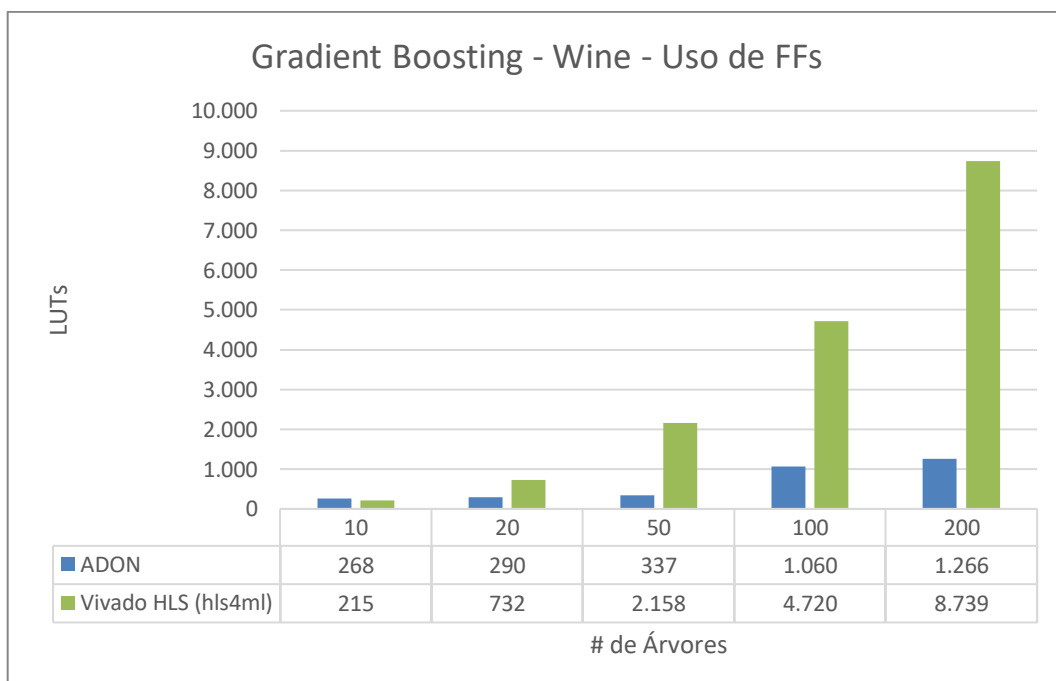
Fonte: Autoria própria

Figura 50 Uso de LUTs com o algoritmo *Gradient Boosting* para o dataset *Wine*.



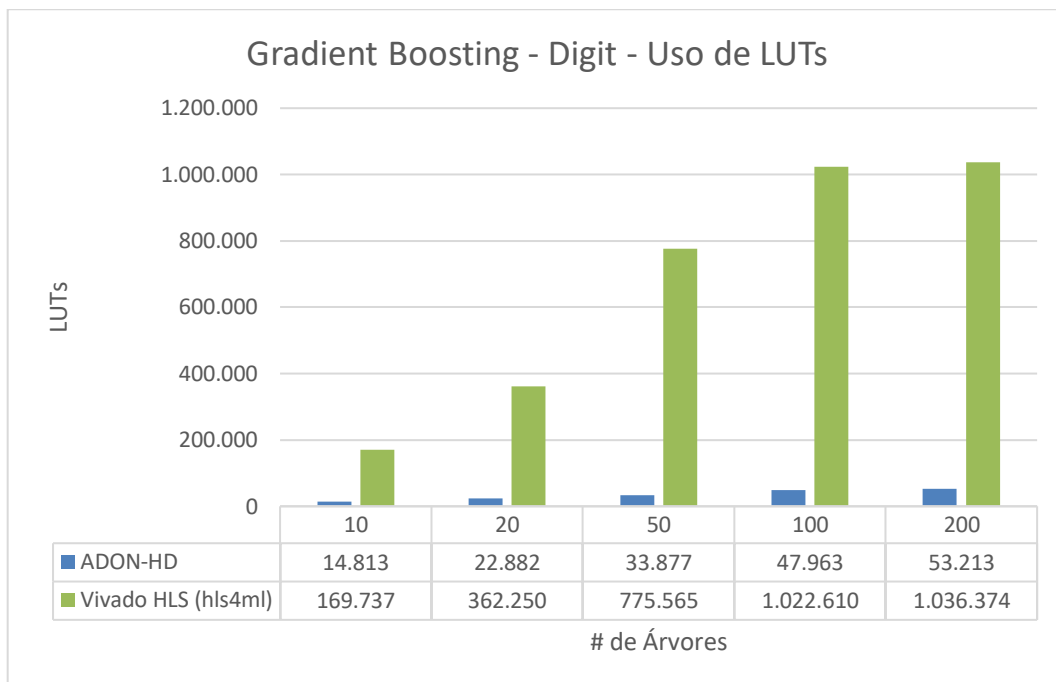
Fonte: Autoria própria

Figura 51 Uso de FFs com o algoritmo *Gradient Boosting* para o dataset *Wine*.



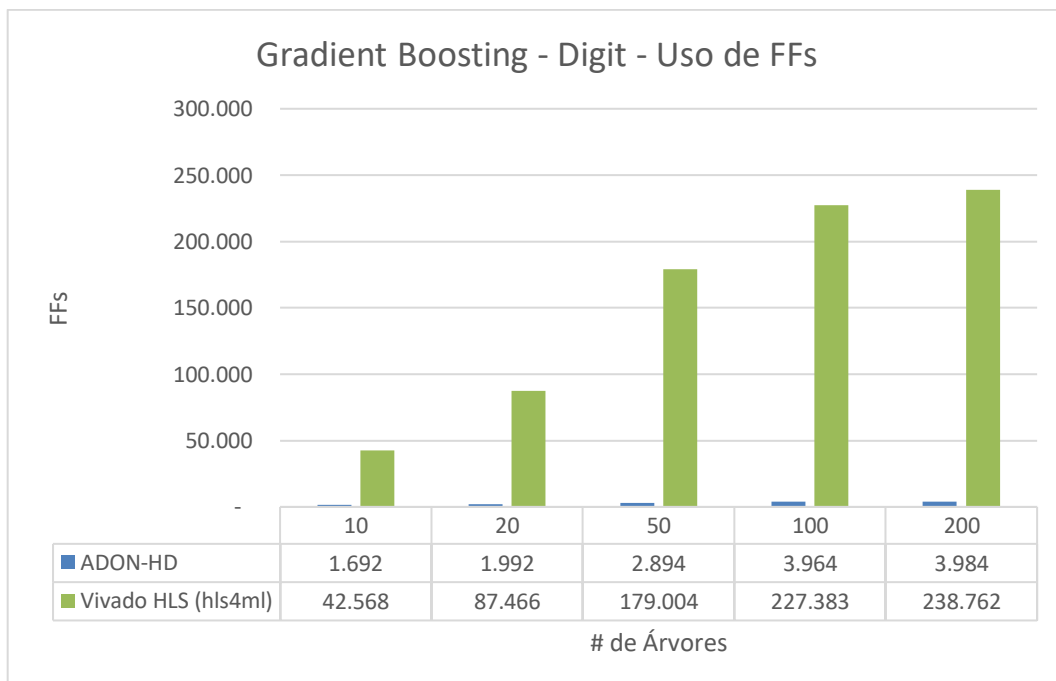
Fonte: Autoria própria

Figura 52 Uso de LUTs com o algoritmo *Gradient Boosting* para o dataset *Digit*.



Fonte: Autoria própria

Figura 53 Uso de FFs com o algoritmo *Gradient Boosting* para o dataset *Digit*.



Fonte: Autoria própria

Nos experimentos com *Gradient Boosting* os circuitos gerados com base na arquitetura ADON via PON-HD utilizaram em média menos elementos lógicos e registradores do que a abordagem *hls4ml*. Assim como no caso do algoritmo *Random Forest*, ao aumentar o número de árvores, ou a quantidade de *features* analisadas, o circuito que utilizou a menor quantidade de recursos foi a solução ADON via PON-HD. Porém o percentual de diferença de uso de recursos entre ADON e *hls4ml* passa a ser maior com o algoritmo *Gradient Boosting*.

Essa vantagem ocorre principalmente graças a característica da ADON em se beneficiar efetivamente dos modelos baseados em *Gradient Boosting* devido à natureza sequencial destes algoritmos, que acarreta a repetição de diversas partes das árvores. Assim sendo, como é observado na Seção 4.2, à medida que o número de árvores nos modelos aumenta, a proporção de *Premises* em relação às avaliações lógicas (*i.e.*, avaliações *se-então*) diminui. Isso indica que a ADON, devido às propriedades e conceitos oriundos do PON, tende a se tornar mais eficiente na eliminação de redundâncias em modelos mais complexos.

Para o *dataset Iris*, a ADON usa aproximadamente 47,0% menos LUTs do que a abordagem *hls4ml* e aproximadamente 52,2% menos FFs com 10 árvores. Além disso, a ADON utiliza aproximadamente 82,9% menos LUTs e aproximadamente 88,6% menos FFs do que o *hls4ml* quando este experimento é treinado com 200 árvores.

Para o *dataset Wine*, a abordagem com a arquitetura ADON usa aproximadamente 29,6% menos LUTs do que a abordagem *hls4ml* e aproximadamente 24,7% mais FFs com 10 árvores. Quando este algoritmo é treinado com 200 árvores, a ADON usa aproximadamente 78,0% menos elementos lógicos e 85,5% menos FFs do que o *hls4ml*.

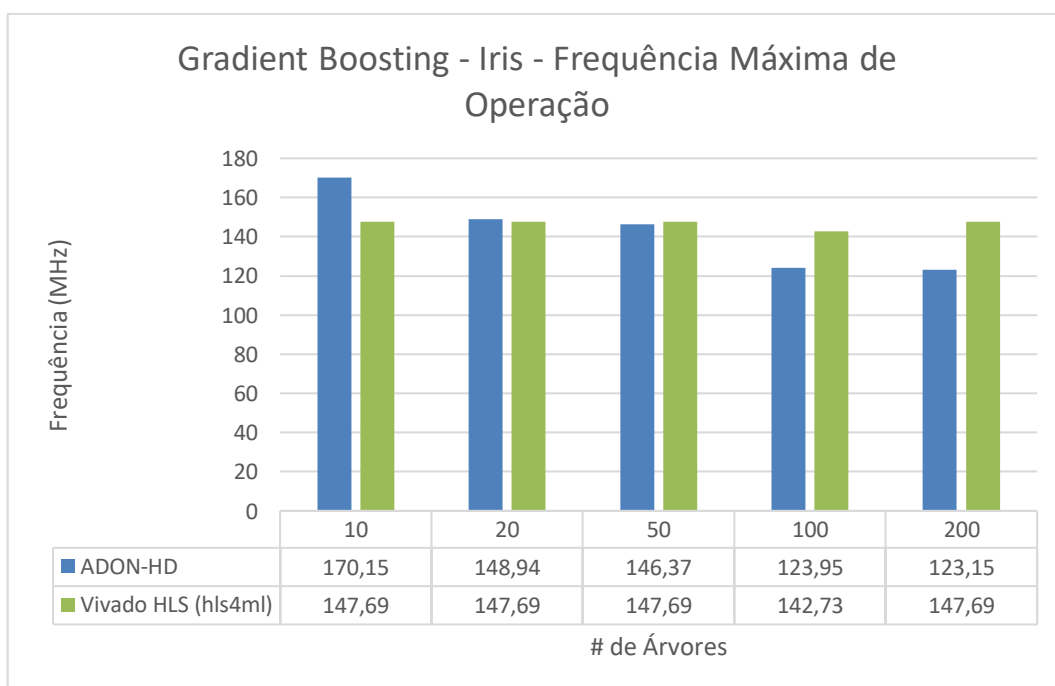
Para o *dataset Digit*, a ADON usa aproximadamente 91,3% menos LUTs e 96,0% menos FFs do que a abordagem *hls4ml* com 10 árvores, e a ADON usa aproximadamente 94,9% menos LUTs e 98,3% menos FFs do que o *hls4ml* com 200 árvores.

Devido à quantidade de elementos LUTs (53.200) disponíveis na FPGA escolhida, os circuitos referentes às implementações do *hls4ml* no dataset *Digit* não são possíveis de serem sintetizados. Enquanto para a implementação com a arquitetura ADON, existe a mesma limitação, porém, apenas para a configuração com 200 árvores houve uma limitação de 13 LUTs. Além das limitações referentes a

LUTs, a implementação *hls4ml* também teve limitações quanto ao número de registradores (FFs), enquanto a implementação com a ADON não apresentou nenhuma limitação. Esta melhora em termos de quantidade de registradores ocorre em função de que apenas os elementos *Attributes* exigem este tipo de componente.

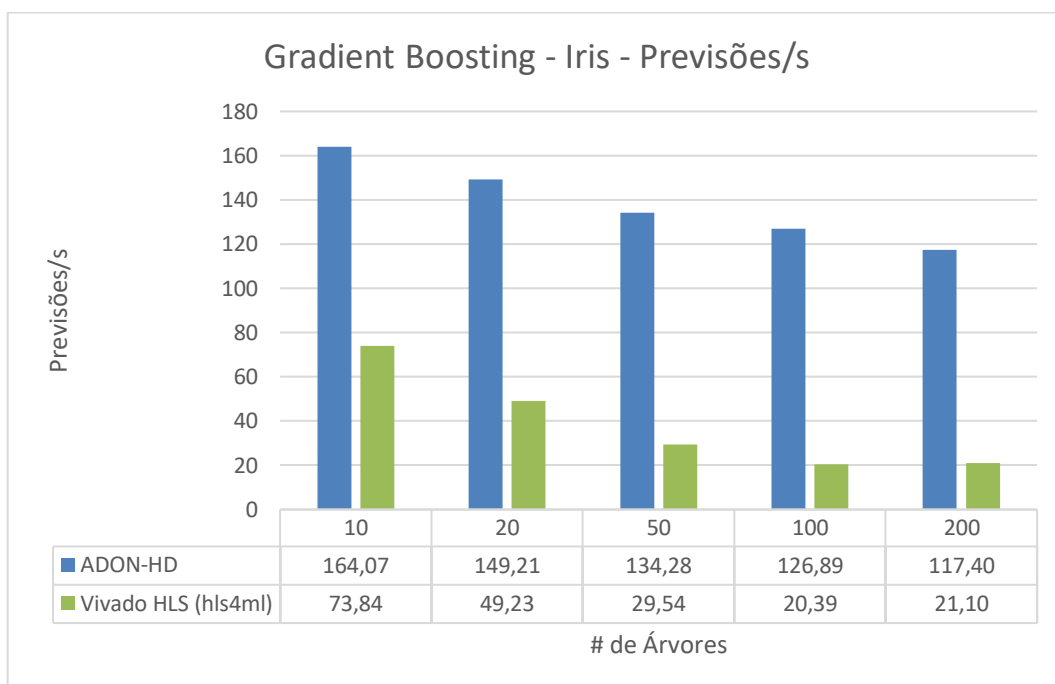
A segunda análise compara a frequência máxima (MHz) em que cada circuito pode operar. Os resultados dessa análise são apresentados na Figura 54, Figura 56 e Figura 58. Nesta análise tanto a ADON, quanto a abordagem *hls4ml* apresentaram frequências de operação semelhantes, com alguma vantagem para a abordagem *hls4ml* que se mantém constante mesmo com o aumento no número de árvores, enquanto a ADON-HD tem perdas graduais com este aumento. Em tempo, estes resultados são usados para calcular a análise subsequente em que se compara o número máximo de previsões ou classificações por segundo (previsão/s) possível para cada abordagem, representando a taxa de classificação. A Figura 55, Figura 57 e Figura 59 mostram os gráficos representando a terceira análise, em que o cálculo da taxa de classificação é dado pela frequência máxima de operação dividida pelo número de ciclos de *clock* necessários para executar uma classificação.

Figura 54 Frequência Máxima de operação (MHz) com o algoritmo *Gradient Boosting* para o *dataset Iris*.



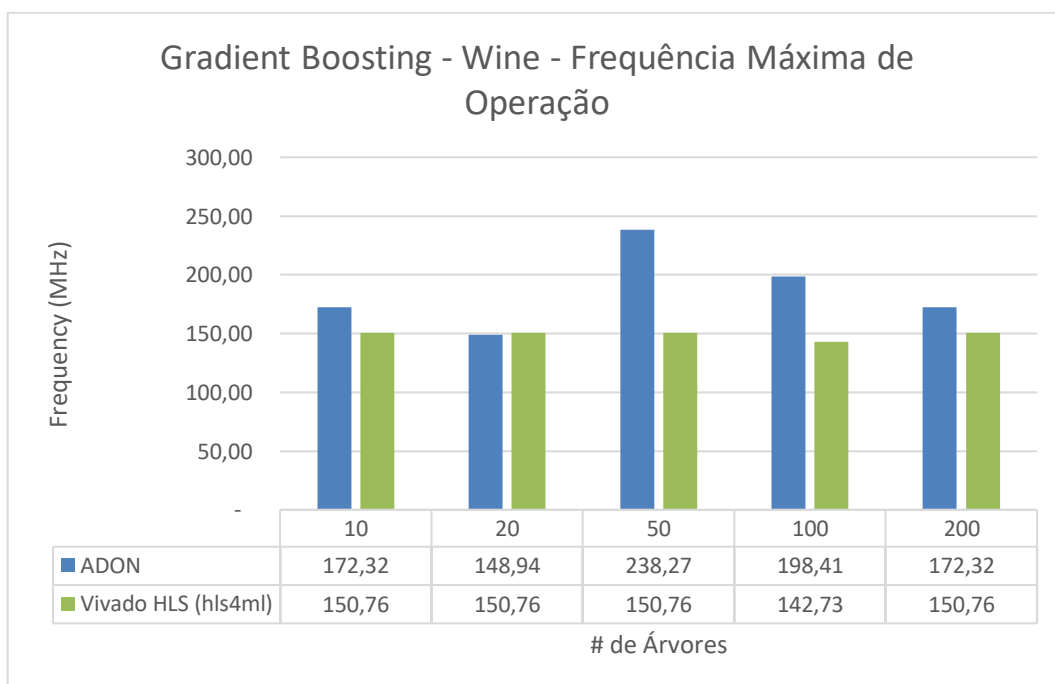
Fonte: Autoria própria

Figura 55 Predições/s com o algoritmo *Gradient Boosting* para o *dataset Iris*.



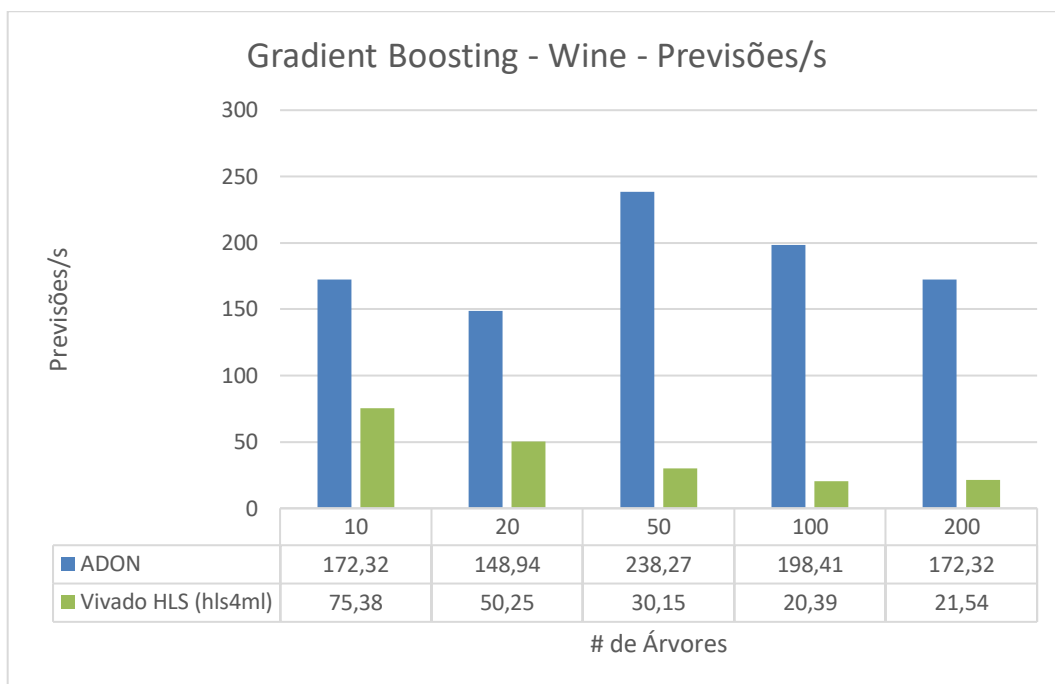
Fonte: Autoria própria

Figura 56 Frequência Máxima de operação (MHz) com o algoritmo *Gradient Boosting* para o *dataset Wine*.



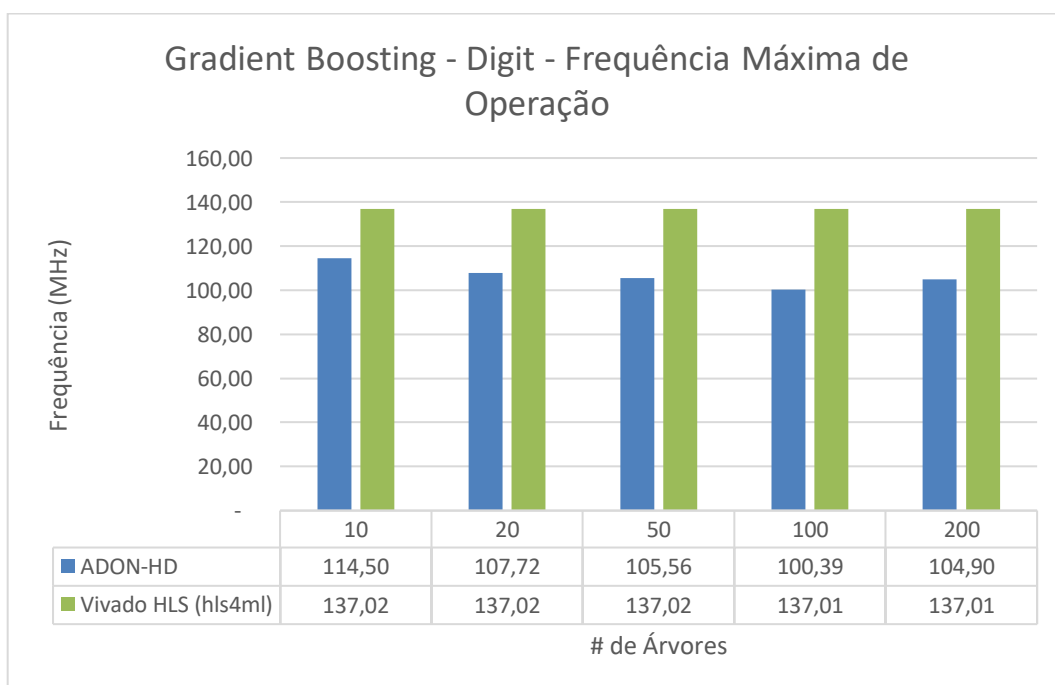
Fonte: Autoria própria

Figura 57 Predições/s com o algoritmo *Gradient Boosting* para o dataset *Wine*.



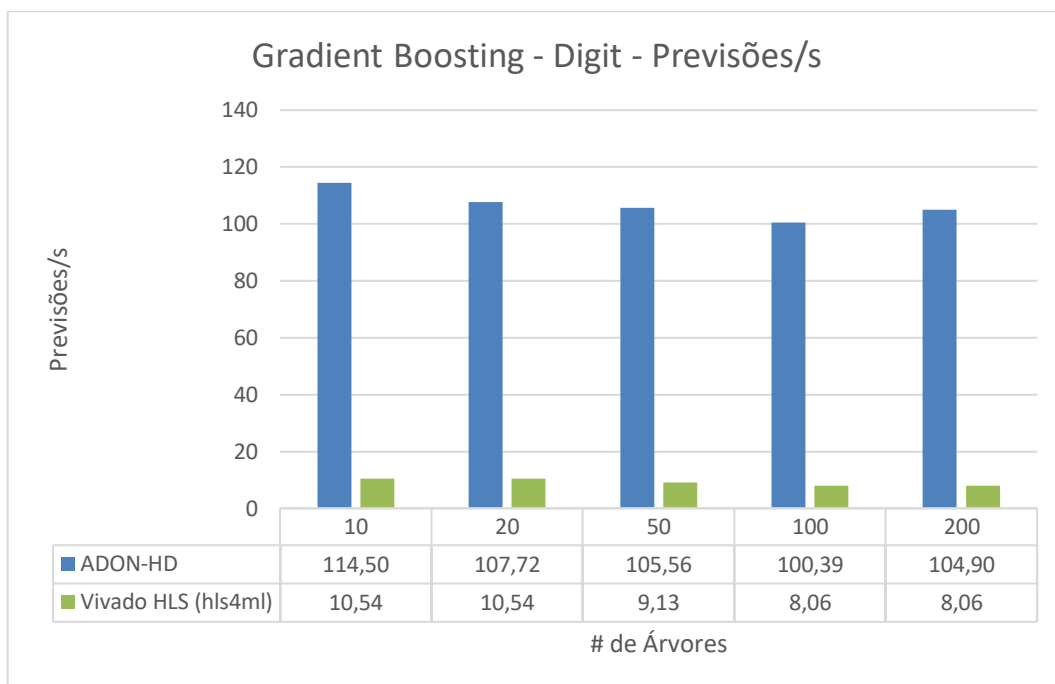
Fonte: Autoria própria

Figura 58 Frequência Máxima de operação (MHz) com o algoritmo *Gradient Boosting* para o dataset *Digit*.



Fonte: Autoria própria

Figura 59 Predições/s com o algoritmo *Gradient Boosting* para o *dataset Digit*.



Fonte: Autoria própria

Os experimentos em relação a frequência máxima de operação e predições por segundo realizados com algoritmo *Gradient Boosting* utilizando as abordagens ADON via PON-HD e *hls4ml* apresentaram resultados similares aos mesmos experimentos realizados com o algoritmo *Random Forest*. Para o *dataset Digit* a ADON apresentou até 13,02 vezes mais previsões/s do que a abordagem *hls4ml*. Esses resultados ocorrem novamente devido à estrutura dos componentes presentes na arquitetura ADON, já que apenas o componente *Attribute* precisa ser sincronizado pelo *clock*.

Ao avaliar a arquitetura ADON por meio da implementação ADON-HD, os resultados apresentados confirmam que a organização de avaliações lógico-causais, presente nos algoritmos de árvores de decisão, torna este conjunto de algoritmos ímpar para se beneficiar dos conceitos e propriedades do PON em conjunto com suas expansões (*i.e. Premise Dual* e *Encadeamento de SubConditions*), pois essas avaliações apresentam redundâncias estruturais e temporais, que resultam em ineficiência em relação ao tempo de execução e ao tamanho do *hardware* sintetizado. Estes mesmos resultados se tornaram mais evidentes ao fazer uso do algoritmo *Gradient Boosting* que adota uma abordagem sequencial para o treinamento e execução das árvores, que acarreta a repetição de diversas partes das árvores para a correção de erros deste modelo. Isso acarreta uma maior

quantidade de redundâncias estruturais e temporais durante a execução para a realização das previsões, principalmente quando há muitas árvores utilizadas.

De forma complementar, a validação em escala da ADON-HD foi conduzida por meio da conformação à ADON, implementação e testes com os algoritmos *Random Forest* e *Gradient Boosting*. Os resultados obtidos evidenciaram que a ADON-HD proporciona vantagens significativas, principalmente em relação à redução de redundâncias, o que possibilita um maior paralelismo granular. Como consequência desses aspectos, observou-se um expressivo aumento da eficiência computacional quando comparado aos métodos tradicionais de execução em FPGA.

4.3.4. Resultados suplementares sobre eficiência energética

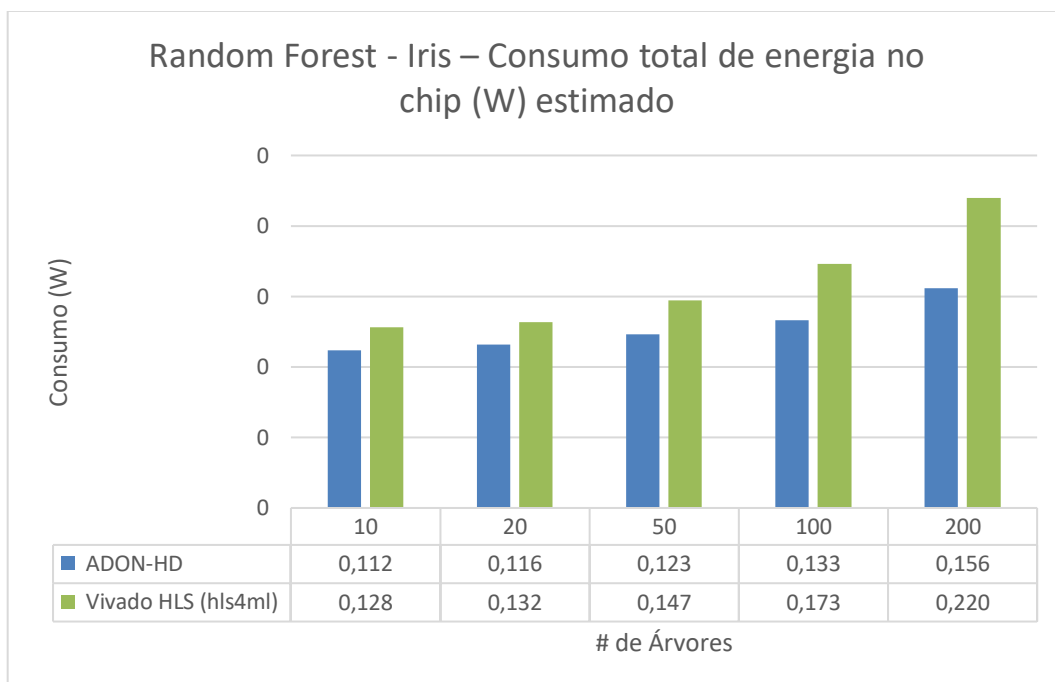
Como experimentos adicionais deste trabalho de pesquisa, buscou-se realizar uma análise da estimativa de consumo de energia por meio dos relatórios gerados pela ferramenta de síntese Vivado. Embora esta avaliação não conste nos objetivos específicos desta pesquisa de doutorado, ela possibilita ter uma visão da tendência de gastos energéticos do circuito gerado para o ADON-HD em relação ao circuito da abordagem *hls4ml*. Por apresentar uma menor quantidade de recursos lógicos e uma melhor eficiência em relação a previsões/s, há a possibilidade de que o ADON-HD também apresente um menor gasto energético.

Neste sentido, no contexto de desenvolvimento de *hardware*, as ferramentas de síntese fornecem relatórios que permitem analisar se o projeto respeita as restrições térmicas e de energia do *hardware* em questão. Os relatórios também podem ser utilizados para fornecer uma análise preliminar a respeito da eficiência energética. Eles apresentam parâmetros referentes a estimativa de consumo de energia, como o consumo total de energia (*Total On-Chip Power*) do projeto em *watts* (W), que é composto pelo consumo dinâmico e pelo consumo estático. O consumo dinâmico é referente as atividades de comutação do circuito e é estimado a partir do cálculo $P = a \cdot C \cdot V^2 \cdot f$, onde a representa a atividade de chaveamento do circuito (*Toggle Rate*), C é a capacitância, V é a tensão de operação e f é a frequência de operação. Já o consumo estático inclui perdas por correntes de fuga nos transistores e os circuitos necessários para o funcionamento do dispositivo. Este parâmetro depende do processo de fabricação, da tensão de operação e da

temperatura, representando o vazamento natural e contínuo de corrente no dispositivo (AMD, 2024).

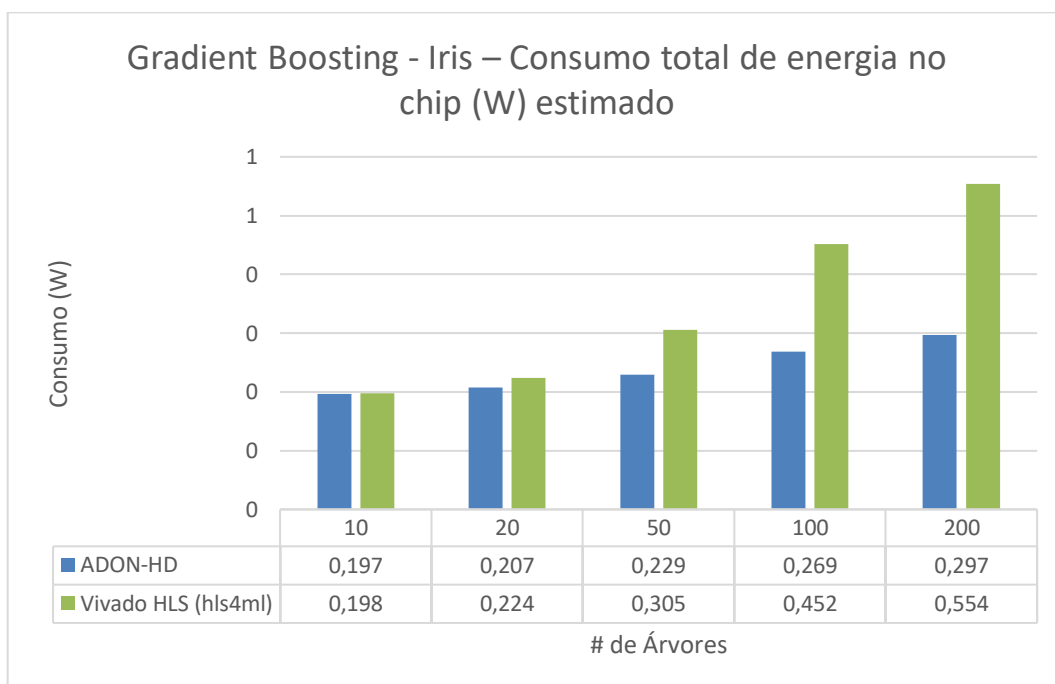
Após a realização da síntese e análise dos resultados referentes as métricas de utilização de recursos lógicos, frequência de operação máxima e número de previsões por segundo para o experimento com *dataset Iris*, realizou-se a geração dos relatórios de consumo energético, que seus dados apresentados nas Figura 60 para o algoritmo *Random Forest* e Figura 61 para o algoritmo *Gradient Boosting*. A partir das figuras apresentadas, observa-se que à medida que a quantidade de árvores aumenta, o ADON-HD apresenta um menor consumo energético em comparação com a abordagem *hls4ml*. No caso do algoritmo *Random Forest*, o ADON-HD apresenta um consumo de 0,156W para 200 árvores, enquanto a *hls4ml* consome 0,220W, representando uma redução de aproximadamente 29,1% no consumo energético. De forma similar, para o *Gradient Boosting*, a diferença é ainda mais significativa, com o ADON-HD consumindo 0,297W contra 0,554W da *hls4ml* para 200 árvores, uma redução de aproximadamente 46,4% no consumo total de energia. Estes resultados demonstram que o ADON-HD também apresenta sinais de bons resultados neste âmbito, o que enseja trabalhos futuros de pesquisa relacionados a eficiência energética do ADON em *hardware*.

Figura 60 Experimentos adicionais referente ao consumo de energia para o algoritmo *Random Forest* com *dataset Iris*.



Fonte: Autoria própria

Figura 61 Experimentos adicionais referente ao consumo de energia para o algoritmo *Gradient Boosting* com *dataset Iris*.



Fonte: Autoria própria

Entretanto, esta análise preliminar apresenta uma estimativa com base em valores padrão da ferramenta de síntese. Embora estes valores possam ser utilizados como referência em uma análise inicial, eles não representam com precisão o comportamento real do circuito em operação. Para uma análise mais precisa do consumo energético, é necessário realizar as estimativas a partir de simulações com base na operação real do circuito. Isto permitiria utilizar taxas de atividade de comutação, temperaturas e tensões nas condições específicas de operação. Além disso, também seriam importantes medições do consumo com o *hardware* real para validar estas estimativas teóricas (XILINX, 2024). Naturalmente, um trabalho de pesquisa também tem a função de ensinar trabalhos subsequentes, sendo que esta análise energética mais acurada estaria nesta alçada.

4.4. Considerações finais

Este capítulo apresentou os casos de estudo realizados para a avaliação da eficácia da arquitetura ADON por meio da implementação específica ADON-HD. Estes casos de estudos realizados permitiram a elaboração de algumas conclusões em relação aos avanços na eficiência do uso de recursos computacionais e no desempenho geral para a composição e execução de árvores de decisão agrupadas em conjuntos correlatos, como o tipo *ensemble* (e.g., *Random Forest* e *Gradient Boosting*), tal como vislumbrado na justificativa deste trabalho de pesquisa.

Mais especificamente, avaliou-se tanto o processo de composição quanto o de execução de árvores propostos pela arquitetura ADON. Em relação ao processo de composição, foram empregados três *datasets* distintos e amplamente conhecidos, tais como *Iris*, *Wine* e *Digit*. Em cada *dataset*, inicialmente foram avaliadas as quantidades de avaliações lógicas (i.e., avaliações 'se-então') geradas pelos modelos *Random Forest* e *Gradient Boosting* e em seguida, também foram avaliadas as quantidades de elementos *Attributes*, *Methods*, *Rules*, *Premises* e *Instigations* mapeados pela ferramenta construída para este processo de conformação.

No contexto da ADON, os modelos de árvore de decisão se destacam devido à grande quantidade de expressões condicionais 'se-então' em cada uma de suas árvores e ao fato dessas expressões poderem ser totalmente desacopladas e executadas em paralelo. Essa característica favorece a paralelização e a distribuição

destes algoritmos, além proporcionar a eliminação das redundâncias estruturais presentes nos modelos de árvores de decisão. Como primeiro resultado, observou-se que a conformação à ADON dos modelos de árvore de decisão treinados permitiu eliminar grande parte das avaliações redundantes durante o processo.

Ao aplicar a eliminação destas redundâncias no contexto de aceleração de *hardware*, era esperado reduzir o uso de recursos em dispositivos FPGAs. Nesse contexto, foi possível analisar o desempenho da arquitetura proposta em comparação com outra abordagem de execução de árvores de decisão em *hardware* presente na literatura, neste caso, utilizando a biblioteca *open-source hls4ml*.

Aplicando o conjunto de *datasets* previamente definidos e variando suas configurações para validar o modelo proposto com base nas métricas definidas e apresentadas na Seção 4.1.2, foi possível realizar comparações da ADON em relação a outra implementação de árvores de decisão como o *Random Forest* e *Gradient Boosting*. Nestes experimentos foram avaliados o número de elementos lógicos, a frequência operacional máxima e a quantidade de ciclos de *clock* necessários para a execução de uma previsão. Ao executar estes experimentos confirmou-se que a eliminação de avaliações redundantes reduziu a quantidade de recursos de *hardware*, tais como LUTs e FF, além de melhorar o desempenho na fase de inferência e diminuindo o tempo de processamento e aumentando a taxa de classificação.

Por sua vez, os resultados obtidos com o modelo *Gradient Boosting* conformado à ADON se destacam significativamente. Este algoritmo tem como uma de suas principais características a sua abordagem sequencial e iterativa na construção de árvores, resultando em muitas redundâncias estruturais e temporais. Isso ocorre devido à sua natureza de corrigir erros progressivamente, frequentemente repetindo partes das árvores anteriores. Desta forma, a conformação à ADON demonstrou ser especialmente eficaz neste cenário, reduzindo significativamente essas redundâncias. Esta eficiência confirma o PON e sua implementação PON-HD como uma solução robusta para a construção de sistemas computacionais de alto desempenho e altamente paralelizáveis. Essa eficiência é especialmente benéfica para implementações em *hardware*, como em dispositivos FPGA, onde a área e o consumo de energia são restritos. Assim, a conformação à ADON contribui para *designs* de *hardware* mais eficientes e escaláveis.

A implementação das árvores seguiu uma estrutura estática em todos os casos avaliados, ou seja, os circuitos são definidos no momento da síntese e carregados permanentemente no *hardware*. Estas arquiteturas são focadas na otimização para um único modelo ou um *dataset* específico de modelos que não serão alvos de mudanças ao longo do tempo. Isso resulta em uma execução mais rápida e previsível, porém, essa abordagem carece de flexibilidade, pois qualquer alteração nos modelos requer uma nova síntese e reprogramação do dispositivo FPGA. Além disso, a quantidade de árvores implementadas depende diretamente da quantidade de recursos disponíveis no dispositivo FPGA utilizado.

Neste cenário de restrição de recursos, o ADON-HD apresentou melhores resultados do que o estado da arte e técnica acessível. Para o experimento com o algoritmo *Gradient Boosting* com *dataset Digit*, a implementação *hls4ml* apresentou limitações a partir de 50 árvores implementadas, enquanto o ADON-HD possibilitou o uso de até 200 árvores na mesma plataforma de execução, o que amenizou as restrições referentes a execução de árvores de decisão estáticas em dispositivos FPGAs com recursos limitados.

5. CONCLUSÃO E TRABALHOS FUTUROS

Neste capítulo são apresentadas as conclusões deste presente trabalho de doutorado. Para tal, são detalhadas as principais contribuições realizadas à luz dos objetivos. Subsequentemente, também são apresentados os possíveis trabalhos futuros relacionados ao tema desta pesquisa. Isto posto, inicia-se essa conclusão relembrando o objetivo principal do trabalho de tese:

Propor e avaliar uma arquitetura orientada a notificações distinta, de acrônimo ADON (Árvores de Decisão Orientada a Notificações), para a composição e execução de árvores de decisão agrupadas em conjuntos correlatos (i.e., tipo ensemble), com destaque para o Random Forest e Gradient Boosting, a partir do mapeamento, conformação e ampliação de conceitos oriundos do PON (Paradigma Orientado a Notificações) em um encadeamento próprio, que permita a sua implementação em diversas materializações, em particular a sua aplicação direta em hardware digital, de acrônimo ADON-HD, na qual deve ser propriamente validada.

Primeiramente, salienta-se que a tese cumpriu o objetivo geral proposto, elaborando a ADON com elemento nuclear, que foi testada na forma da ADON-HD e que é factível em demais materializações baseadas em PON dada a sua natural aderência aos conceitos dele. Neste âmbito, além deste núcleo da ADON, este trabalho estabelece para ela uma solução para o tratamento de árvores de decisão (nomeadamente do tipo *Random Forest* e *Gradient Boosting*) treinadas com a biblioteca *scikit-learn* para serem conformadas às suas entidades notificantes. Tal solução é implementada por meio de um conjunto de módulos (extensíveis) resultando em um ferramental que, em suma, recebe modelos treinados de *Random Forest* ou *Gradient Boosting* e então gera códigos específicos à ADON-HD, mas passíveis de generalização.

À luz deste contexto dado, após aplicar o ADON (ferramental e seu núcleo) em conjunto de experimentos com ADON-HD via *benchmarks* conhecidos e bem aceitos, bem como analisar os resultados obtidos, é pertinente afirmar que a pesquisa apresentada neste trabalho de doutorado traz avanços significativos na eficiência do uso de recursos computacionais e no desempenho de aplicações

baseadas em árvores de decisão em *hardware* digital, especificamente em FPGAs via uma solução nova e distinta. Esses avanços são especialmente relevantes para aplicações que demandam alto desempenho e possuem limitações quanto à quantidade de recursos disponíveis. Justamente, tais resultados e contribuições são discutidas na seção que segue deste capítulo conclusivo de tese.

5.1. Principais contribuições

Conforme dito previamente, esta conclusão é orientada aos objetivos e, portanto, são aqui lembrados os objetivos específicos e então feitas as reflexões conclusivas sobre o que foi alcançado em relação a eles.

Revisar os aspectos técnicos e teóricos a respeito do estado da arte no tocante às arquiteturas para execução de árvores de decisão agrupadas em conjuntos correlatos, com destaque para os algoritmos *Random Forest* e *Gradient Boosting* em *hardware*, para compreender e destacar a lacuna existente.

No decorrer deste trabalho de pesquisa de doutorado, foram apresentados trabalhos de pesquisa que tiveram como foco a revisão de aspectos técnicos e teóricos a respeito de árvores de decisão, com ênfase aos algoritmos do tipo *ensemble Random Forest* e *Gradient Boosting*. Em seguida, realizou-se uma revisão dos principais trabalhos recentes relacionados à implementação em *hardware* de árvores de decisão e seus derivados, com foco nos avanços relevantes para os objetivos desta tese.

A partir dos trabalhos apresentados, observa-se o foco das pesquisas nas aplicações de tecnologias de *hardware* como GPUs e FPGAs para acelerar tanto os processos de treinamento ou aprendizado, quanto os processos de tomada de decisões desses algoritmos. No contexto específico da aceleração do processo de inferência em FPGAs, os trabalhos inicialmente visaram explorar o paralelismo intrínseco das árvores de decisão dentro dos limites de recursos disponíveis pelas plataformas. No entanto, as restrições de recursos disponíveis nesses dispositivos limitam a extensão de paralelismo que pode ser atingido.

Como consequência dessas limitações, os trabalhos subsequentes passaram a focar em técnicas de otimização, como melhorias no acesso à memória, redução do tamanho das árvores e técnicas de quantização, para aumentar a eficiência da execução das árvores de decisão em *hardware*. Isso assim se dá porque essas abordagens são baseadas em paradigmas tradicionais de computação e assim não conseguem explorar plenamente o paralelismo oferecido pelas plataformas de execução. Além disso, elas mantêm redundâncias estruturais e temporais que impactam negativamente tanto o tempo de execução quanto o consumo de recursos nos dispositivos FPGAs. Esta lacuna identificada na literatura motivou o desenvolvimento de uma nova arquitetura baseada em um paradigma que naturalmente favorece o paralelismo e elimina redundâncias, nomeadamente o PON.

Revisar os aspectos técnicos e teóricos fundamentais a respeito do PON e do PON-HD, avaliando suas propriedades e materializações, com o objetivo de guiar as decisões sobre quais de seus conceitos e demais elementos são essenciais para alcançar a ADON e a ADON-HD, bem como quais deles devem ser reinventados, rearticulados, simplificados e expandidos.

A revisão dos aspectos técnicos e teóricos do PON e PON-HD permitiu avaliar as propriedades, conceitos, elementos essenciais e materializações para a concepção da ADON e ADON-HD. Em particular, esta revisão foi fundamental para abordar o processo de conformação dos elementos estruturais e funcionais das árvores de decisão para as entidades definidas pelo metamodelo do PON. No entanto, esta avaliação também revelou a necessidade de expandir o PON com dois novos conceitos para tratar adequadamente as peculiaridades das árvores de decisão.

O primeiro conceito proposto foi a *Premise Dual*, desenvolvida para otimizar avaliações complementares em nós de árvores de decisão. Estas avaliações surgem porque cada nó gera tipicamente duas condições mutuamente exclusivas (e.g., $petalWidth\ 1 \leq 0.8$ ou $petalWidth\ 1 > 0.8$). A *Premise Dual* permite que essas avaliações opostas sejam tratadas como uma única entidade lógica, eliminando a

necessidade de múltiplas avaliações do mesmo atributo e consequentemente reduzindo redundâncias estruturais na árvore.

O segundo conceito foi o *Encadeamento de SubConditions*, que passou a estabelecer a dependência hierárquica entre *SubConditions*. Este conceito é particularmente importante no contexto de árvores de decisão, em que cada caminho da árvore desde o nó raiz (*Root*) até uma folha (*Leaf*) é composto por uma sequência de avaliações interdependentes. Por exemplo, em cada nível da árvore, uma *SubCondition* é formada não apenas por *Premises*, mas também depende de notificação das *SubConditions* dos níveis anteriores, criando um encadeamento de notificações que refina progressivamente o caminho de decisão. Esta abordagem permite tratar cada caminho da árvore orientada a notificações sem redundâncias estruturais e temporais, extrapolando os princípios de base do PON de não redundância de forma inovadora e pertinente ao domínio de árvores de decisão em questão.

Quanto ao PON-HD, por sua vez, a sua revisão indicou a necessidade de adaptações nos componentes para suportar operações com números decimais usando aritmética de ponto fixo, essencial para implementar os métodos de votação dos algoritmos *Random Forest* e *Gradient Boosting* em *hardware*, sendo particularmente importante para o *Gradient Boosting* cujo método de votação necessita realizar cálculos exponenciais e logarítmicos. Estas extensões e adaptações foram fundamentais para que a ADON pudesse aproveitar as propriedades do PON de forma otimizada no contexto específico de árvores de decisão, ao mesmo tempo em que a ADON-HD pudesse implementar eficientemente estas estruturas em *hardware*.

Estabelecer a ADON via um processo de mapeamento entre conceitos de árvores de decisão e conceitos oriundos do PON, os rearticulando, conformando e expandindo, para a execução orientada a notificações em um encadeamento próprio de árvores do tipo *ensemble*, particularmente *Random Forest* e *Gradient Boosting*.

Durante a proposição da arquitetura ADON, como ponto de partida, foram detalhados os requisitos para o seu desenvolvimento, que abrangem os aspectos funcionais e não funcionais. A partir destes requisitos, foi desenvolvido o processo

de conformação das árvores de decisão de acordo com as entidades do metamodelo do PON, traduzindo elementos estruturais como *Features*, *Root*, *Internal Nodes*, *Leaf* e *Voting* para componentes PON como *FBEs*, *Attributes*, *Methods*, *Rules*, *Premises*, *Conditions*, *Actions* e *Instigations*. Durante este processo de conformação, verificou-se que era necessário expandir o PON por meio de dois novos conceitos que visaram a eliminação de redundâncias que ainda persistiam mesmo após a conformação inicial ao PON, sendo os novos conceitos *Premise Dual* e o *Encadeamento de SubConditions*, duas novas contribuições deste trabalho ao PON já explicada nesta presente conclusão.

Eis que o processo de mapeamento estabelecido pelos requisitos, em conjunto com estas expansões do PON, resultou na arquitetura ADON proposta. Esta arquitetura não apenas preserva as propriedades de paralelismo e desacoplamento do PON, mas também as potencializa no contexto específico de árvores de decisão pelos motivos já detalhados. Particularmente, após os experimentos com a ADON em FPGA, percebe-se que ela consegue assim explorar tanto o paralelismo entre diferentes árvores em modelos *ensemble* quanto o paralelismo entre avaliações independentes dentro de cada árvore, ao mesmo tempo em que elimina redundâncias estruturais e temporais de forma mais eficiente que as abordagens tradicionais, representadas pelo estado da arte e da técnica *hls4ml*.

Instanciar a ADON-HD via um processo de mapeamento entre conceitos da ADON e conceitos oriundos do PON-HD, os rearticulando, conformando e expandindo, para a execução orientada a notificações em FPGA via um encadeamento próprio de árvores do tipo *ensemble*, particularmente *Random Forest* e *Gradient Boosting*.

A instanciação do ADON em *hardware* digital, denominada ADON-HD, foi realizada através do mapeamento dos elementos conformados à ADON para os componentes do PON-HD. Este mapeamento permitiu que os elementos da ADON, fossem implementados em *hardware* preservando seu encadeamento distinto, ao mesmo tempo em que aproveitou as capacidades de paralelismo e execução desacoplada oferecidas pelo PON-HD.

Para viabilizar esta implementação, foi necessário expandir os componentes do PON-HD para atender os requisitos específicos dos algoritmos *Random Forest* e *Gradient Boosting*. Neste âmbito, uma adaptação necessária foi a implementação do suporte a operações com números decimais através de aritmética de ponto fixo, uma vez que a versão original do PON-HD era limitada ao uso de números inteiros.

Esta extensão envolveu a criação de novos tipos de dados e a adaptação dos componentes fundamentais do PON-HD, como *Attributes*, *Premises* e *Methods* para trabalhar com aritmética de ponto fixo. Além disso, foram desenvolvidas em VHDL as implementações de funções matemáticas necessárias aos métodos de votação, como funções exponenciais e logarítmicas, essenciais para o algoritmo *Gradient Boosting*.

Estabelecer para a ADON um módulo para tratamento de árvores de decisão treinadas (*Random Forest* e *Gradient Boosting*) a serem conformadas às entidades notificantes, o qual é implementado por meio de uma ferramenta que possa receber modelos treinados e gerar os códigos específicos para a ADON-HD.

No decorrer deste trabalho de pesquisa, foi desenvolvida uma solução especializada em mapear modelos para a arquitetura ADON de árvores de decisão treinados externamente, que se expressa por uma ferramenta expansível que atende aos requisitos estabelecidos de integração e geração de código. A natureza expansível desta arquitetura permite a adição de novos tipos de modelos de árvores de decisão, e o suporte a novas plataformas de execução, possibilitando assim a integração entre os modelos tradicionais de árvores de decisão e diferentes implementações da ADON, especialmente em *hardware* digital (ADON-HD).

A solução desenvolvida recebe como entrada modelos de árvores de decisão treinados, particularmente os gerados pela biblioteca *scikit-learn*. A partir dessa entrada, ela realiza o processo de mapeamento dos modelos para a arquitetura ADON, e em seguida gera códigos específicos otimizados para a implementação específica ADON-HD. Esse processo foi projetado para manter a total integridade das características dos modelos originais, preservando seus resultados de inferência enquanto adapta a execução para aproveitar as vantagens da ADON. Além disso, a ferramenta simplifica a adoção de novos *datasets*,

permitindo a exploração de aplicações práticas com o potencial da execução de árvores de decisão no contexto da arquitetura ADON.

Aplicar conjuntos de *datasets* clássicos, variando suas configurações, para validar o modelo proposto com base nas métricas: números de elementos lógicos, frequência operacional máxima e a quantidade de ciclos de *clock* necessários para a execução de uma previsão, fazendo comparações da ADON-HD em relação a arquitetura do estado da arte em árvores de decisão do tipo *ensemble* (*Random Forest* e *Gradient Boosting*), nomeadamente *hls4ml*.

As contribuições referentes aos objetivos específicos anteriores, relativas à proposta e implementação da ADON, permitiram avançar com as avaliações e conclusões sobre esta arquitetura a partir da execução dos experimentos e avaliação dos resultados obtidos. Foram considerados três casos de estudo, fazendo uso dos *datasets Iris*, *Wine* e *Digit*. Nestes três casos de estudo, inicialmente avaliou-se a efetividade do processo de conformação à ADON, analisando-se a quantidade de elementos do metamodelo do PON mapeados. Como resultado, observou-se que a conformação à ADON dos modelos de árvore de decisão treinados permitiu eliminar grande parte das avaliações redundantes presentes nos modelos de árvore de decisão avaliados.

Em seguida, foram realizados os experimentos comparativos da ADON em relação à implementação *hls4ml*. Nestes experimentos foram avaliados o número de elementos lógicos, a frequência operacional máxima e a quantidade de ciclos de *clock* necessários para a execução de previsões de árvores de decisão com os algoritmos *Random Forest* e *Gradient Boosting*. Com os resultados obtidos, observa-se que a ADON se destaca para ambas as classes de algoritmos avaliados, apresentando redução na quantidade de recursos de *hardware* dos dispositivos FPGAs de até 70,5% menos LUTs e 94,8% menos FFs, e melhora no desempenho na fase de inferência de até 13,02 vezes através da diminuição do tempo de processamento e aumento na taxa de classificação.

Quanto maior o número de árvores de decisão utilizadas no modelo, há um maior número de avaliações lógicas, que demandam por uma maior quantidade de recursos do dispositivo FPGA. No entanto, quando a ADON é utilizada, à medida que o número de árvores aumenta, a quantidade de avaliações redundantes

eliminadas é maior. Isto acarreta um aumento no uso de recursos de *hardware* em uma proporção inferior ao da abordagem *hls4ml*.

No caso do algoritmo *Gradient Boosting* há uma discrepância maior ainda nos resultados entre as abordagens ADON e *hls4ml*. Essa discrepância ocorre principalmente devido à forma como este algoritmo é construído, sendo composto por diversas árvores que têm partes de sua estrutura repetidas para correção dos erros cometidos pelas árvores executadas anteriormente. Isso demonstra que o algoritmo *Gradient Boosting* apresenta vantagens significativas na eficiência do uso de recursos computacionais e no desempenho geral ao se utilizar a ADON para o processo de inferência em *hardware*.

A partir dos resultados obtidos, a ADON é confirmada como uma alternativa promissora para a execução dos algoritmos derivados de árvores de decisão em *hardware*, proporcionando um melhor uso dos recursos disponíveis. Principalmente para o uso em aplicações que demandam alto desempenho, mas que possuem limitações quanto à quantidade de recursos disponíveis. Por fim, os resultados da ADON, cuja essência utiliza e evolui o PON, ajuda a validar a potencialidade deste paradigma para soluções distintas, conforme já argumentava Simão e Stadzisz em (SIMÃO; STADZISZ, 2009) e começavam a demonstrar e confirmar pesquisadores como (PETERS, 2012; LINHARES, 2015; KERSCHBAUMER, 2018; RONSZCKA, 2019).

5.2. Trabalhos futuros

A ADON faz parte de um conjunto de soluções do grupo de pesquisa do PON que foram baseadas nos conceitos oriundos do conjunto de pesquisas realizadas nesta alçada. Mais especificamente, a ADON é a solução que alcançou os melhores resultados em termos de eficacidade, levando a reflexão se o caminho referente as novas pesquisas relativas a orientação a notificações devem ser mais orientadas a generalização do paradigma ou mais orientadas a criação de soluções inovadoras para domínios específicos nas quais as suas propriedades possam ser exacerbadas e expandidas em contextos altamente demandados, justamente como árvores de decisão, redes neurais, indústria 4.0 e assim por diante.

Em todo caso, a partir dos resultados obtidos com a ADON, em conjunto com as análises efetuadas sobre a própria arquitetura, verificou-se que há

possibilidade de novas pesquisas relacionadas à arquitetura ADON em si, tais como expansões para novos modelos e plataformas, além de novos parâmetros de análise. Desta forma, nesta seção, são apresentadas sugestões de possíveis trabalhos futuros que estão diretamente relacionados a esta pesquisa e suas respectivas conclusões.

Expansão dos estudos da ADON com novos *datasets*

Inicialmente, sugere-se expandir os estudos da ADON utilizando novos *datasets*. Embora os *datasets Iris*, *Wine* e *Digit* utilizados nos experimentos sejam amplamente conhecidos e ofereçam um ambiente adequado para o teste e validação da ADON, a realização de estudos com base em *datasets* mais diversos, com diferentes características e maior dimensionalidade, permitiria uma avaliação mais abrangente do desempenho e eficiência da ADON. Além disso, os experimentos com novos *datasets* permitiriam avaliar diferentes cenários em que o pipeline de busca de novos dados (*fetch*) para a realização de previsões possa ter influência nos resultados, como mudanças totais ou parciais nas características avaliadas. Essa abordagem possibilitaria a validação desta arquitetura em diferentes domínios de aplicação, contribuindo para o refinamento e otimizações contínuos da ADON.

Expansão dos modelos de árvores de decisão suportados

Outro trabalho futuro sugerido consiste na expansão dos modelos suportados pela ADON, com foco particular em algoritmos *ensemble* que sejam derivados do algoritmo *Gradient Boosting*, como por exemplo *XGBoost* e *LightGBM*. Estes algoritmos têm ganhado popularidade devido aos seus bons resultados e também são alvos de pesquisas em relação à aceleração de *hardware* (SUMMERS *et al.*, 2020; KANANI; VAIDYA; AGARWAL, 2021; DAMIANI *et al.*, 2022; GAJJAR *et al.*, 2022; WANG *et al.*, 2023; ALSHARARI *et al.*, 2024). Desta forma, a expansão da ADON para suportar estes modelos ampliaria a possibilidade de novos experimentos, além de validar a arquitetura com novos domínios de aplicação que fazem uso destes dois modelos. Outrossim, dado que a ADON permite saber quais

fluxos de notificação ocorrem, isso poderia ensejar pesquisas no tocante a explicabilidades dos modelos estudados e mesmo nesses outros aqui aventados.

Experimentos relacionados a eficiência energética

Os resultados dos experimentos adicionais em relação a eficiência energética confirmam a tendência do ADON-HD em apresentar bons resultados neste âmbito. Entretanto, estes experimentos foram realizados usando valores inferidos pela própria ferramenta de síntese. Embora estes valores possam ser utilizados como referência em uma análise inicial, eles não representam com precisão o comportamento real do circuito em operação. Neste sentido, trabalhos futuros poderiam ser realizados com o objetivo de alcançar estimativas realistas com base na operação real do circuito, além da realização de medições do consumo com o *hardware* real para validar estas estimativas teóricas.

Aplicação de técnicas de quantização

Durante os experimentos realizados nesta pesquisa de doutorado, a ADON apresentou maior eficiência no uso de recursos de *hardware* em relação à abordagem *hls4ml*, considerando que ambas as implementações não empregam técnicas de quantização e mantêm a acurácia original dos modelos. Entretanto, a aplicação de técnicas de quantização é particularmente relevante no contexto de implementações em FPGAs em que os recursos são limitados. Por exemplo, a quantização pode envolver a redução da precisão numérica dos parâmetros dos modelos, levando a uma diminuição significativa no uso de recursos, sem comprometer a acurácia do modelo de forma significativa (WANG et al., 2022; WEI et al., 2024). Este estudo permitiria não apenas avaliar o desempenho da ADON em termos de eficiência, mas também avaliar como a ADON interage com técnicas de quantização.

Comparações com a NeuroPON

O NeuroPON consiste em uma abordagem que aplica os conceitos e princípios PON ao desenvolvimento e execução de Redes Neurais Artificiais (RNAs) em *software* e *hardware* por meio de uma linguagem declarativa em alto nível (SCHÜTZ *et al.*, 2015; SCHÜTZ, 2019). Assim como a ADON, o modelo computacional do NeuroPON visa o aproveitamento das características de paralelismo e não redundância do PON para otimizar a execução de um modelo de *machine learning* em uma plataforma de *hardware* em FPGA, reduzindo redundâncias computacionais e promovendo maior eficiência na execução paralela. Neste sentido, um trabalho futuro sugerido consiste na realização de experimentos e comparação dos resultados entre a abordagem ADON e NeuroPON, em relação à acurácia de ambas as soluções e à quantidade de recursos utilizados em uma plataforma de *hardware*.

Implementação do suporte a ArqPON

A implementação da ADON utilizando a ArqPON é outro trabalho futuro vislumbrado. Esta abordagem é motivada pela necessidade de superar eventuais limitações relacionadas à quantidade de árvores de decisão que podem ser implementadas em um único dispositivo FPGA ou relacionadas à falta de flexibilidade pois, para aplicar um novo modelo, é necessária a reconfiguração do *hardware*. Essencialmente, a ArqPON permitiria executar modelos com um número arbitrário de árvores de decisão, superando a atual restrição imposta pelos recursos finitos de um único dispositivo FPGA. Este benefício é particularmente relevante para modelos *ensemble* complexos que atualmente não poderiam ser totalmente implementados em FPGA.

Adicionalmente, a ArqPON possibilitaria carregar e executar diferentes modelos de árvores de decisão sem necessidade de reconfiguração do *hardware*, diferente da atual implementação que requer uma nova síntese para cada novo modelo. Esta capacidade permitiria que um mesmo *hardware* executasse diferentes modelos ADON conforme necessário. Entretanto, para a aplicação da ArqPON, seriam necessárias extensões do conjunto de instruções para poder suportar valores decimais, essenciais para os algoritmos de votação presentes nos modelos

ensemble como *Random Forest* e *Gradient Boosting*. Ademais, possivelmente a ArqPON ela mesma deveria ser evoluída, para melhor executar aplicações orientadas a notificações, conforme discutido em Pordeus (2020).

Implementação do suporte a LingPON

Por fim, a implementação do suporte ao LingPON pela ADON é um trabalho pertinente ao grupo de pesquisa do PON. Ao gerar código em LingPON a partir da ADON, vislumbra-se expressar os modelos de árvores de decisão de uma maneira mais natural em relação aos princípios do PON. Além disso, a ADON poderia ser executado em qualquer plataforma que a Tecnologia LingPON também suporte, como por exemplo em *software*: *Framework* PON C++ 4.0 e 4.5, e C++ Notificante (OSHIRO, 2021; LEDESMA; SIMÃO, 2022; SKORA; CHIERICI; SIMÃO, 2022). Entretanto, o uso das Tecnologias LingPON em suas versões 2.0 e 3.0 ainda são de natureza prototipal em relação às materializações PON-HD e ArqPON, sendo necessário o desenvolvimento da geração de código-alvo para estas materializações.

6. REFERÊNCIAS

AEBERHARD, Stefan; FORINA, M. **Wine**. UCI Machine Learning Repository, 1992.

ALCOLEA, A.; RESANO, J. **FPGA accelerator for gradient boosting decision trees**. Electronics (Switzerland), v. 10, n. 3, p. 1–15, 1 fev. 2021.

ALPAYDIN, E.; KAYNAK, C. **Optical recognition of handwritten digits**. UCI Machine Learning Repository, 1998.

ALSHARARI, Majed; MAI, Son T.; GARNIER, Romain; REAÑO, Carlos; WOODS, Roger. **An intelligent image processing system for enhancing blood vessel segmentation on low-power SoC**. In: International Conference On Embedded Computer Systems, 2023. *Proceedings*. Cham: Springer, 2023. p. 123–138.

ALSHARARI, Majed; MAI, Son T.; WOODS, Roger; REAÑO, Carlos. **Efficient integer-only-inference of gradient boosting decision trees on low-power devices**. IEEE Transactions on Circuits and Systems I: Regular Papers, IEEE, 2024.

AMD. **Power in AMD Devices. Vivado Design Suite User Guide: Power Analysis and Optimization (UG907)**, versão 2024.2, 18 nov. 2024. Disponível em: <https://docs.amd.com/r/en-US/ug907-vivado-power-analysis-optimization/Power-in-AMD-Devices>. Acesso em: 27 nov. 2024.

ATHAYDE, E. B. **Paradigma Orientado A Notificações Como Alternativa Para Gerenciamento De Energia Em Sistemas Embarcados**. Seminário de Acompanhamento I. PPGCA, UTFPR. Curitiba - PR, Brasil, 2016.

AWAD, M. **FPGA supercomputing platforms: A survey**. FPL 09: 19th International Conference on Field Programmable Logic and Applications, p. 564–568, 2009.

BABU, Abu Ahammed. **Notification Oriented Paradigm as a Green Technology**. 2022. Orientação de outra natureza. (GENIAL - Green Networking and Cloud Computing) - Universidade Tecnológica Federal do Paraná, ERAMUS+ Programme of the European Union.

BANASZEWSKI, R. F.; SIMÃO, J. M.; TACLA, C. A.; STADZISZ, P. C. **Notification Oriented Paradigm (NOP) – A Software Development Approach based on Artificial Intelligence Concepts**. VI Congress of Logic Applied to Technology - LAPTEC 2007. Santos, 2007.

BANASZEWSKI, R. F. **Paradigma Orientado a Notificações : Avanços e Comparações**. Dissertação de Mestrado. CPGEI, UTFPR. Curitiba, Brasil, 2009.

BARBA, J.; SANTOFIMIA, M. J.; DONDO, J.; et al. **FPGA acceleration of semantic tree reasoning algorithms**. Journal of Systems Architecture, v. 61, n. 3–4, p. 185–196, 2015.

BARBARESCHI, M.; BARONE, S.; MAZZOCCA, N. **Advancing synthesis of decision tree-based multiple classifier systems: an approximate computing case study**. Knowledge and Information Systems, v. 63, n. 6, p. 1577–1596, 2021.

BARBARESCHI, M.; DEL PRETE, S.; GARGIULO, F.; MAZZEO, A.; SANSONE, C. **Decision tree-based multiple classifier systems: An FPGA perspective**. Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), v. 9132, p. 194–205, 2015.

BARRETO, W. R. M.; VENDRAMIN, A. C. B. K.; SIMÃO, J. M. **Notification Oriented Paradigm for Distributed Systems**. In: Computer on the Beach 2018, Florianopolis, 2018.

BATISTA, M. V. **Paradigma Orientado a Notificações - Pon: Criação de uma Aplicação Comercial Baseada em Regras e Notificações com Modelagem**

por Rede de Petri. Framework NOP/PPON C++ 1.0. Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2013.

BAUMGARTNER, S.; HUEMER, M.; LUNGLMAYR, M. **Efficient Majority Voting in Digital Hardware.** IEEE Transactions on Circuits and Systems II: Express Briefs, v. 69, n. 4, p. 2266–2270, 1 abr. 2022.

BELMONTE, D.; SIMÃO, J. M.; STADZISZ, P. C. **Proposta de um Método para Distribuição da Carga de Trabalho Usando o Paradigma Orientado a Notificações (PON).** Revista SODEBRAS, 7(84), p. 10-17, 2012.

BELMONTE, D. **Método para Distribuição da Carga de Trabalho dos Softwares PON em Multicore.** Trabalho de Qualificação de Doutorado, CPGEI, UTFPR. Curitiba, Brasil, 2012.

BELMONTE, D.; LINHARES, R. R.; STADZISZ, P. C.; SIMAO, J. M.. **A new Method for Dynamic Balancing of Workload and Scalability in Multicore Systems.** IEEE Latin America Transactions, ISSN: 1548-0992. 2016.

BENTÉJAC, C.; CSÖRGŐ, A.; MARTÍNEZ-MUÑOZ, G. **A comparative analysis of gradient boosting algorithms.** Artificial Intelligence Review, v. 54, n. 3, p. 1937–1967, 24 mar. 2021.

BERTOLINI, M.; MEZZOGORI, D.; NERONI, M.; ZAMMORI, F. **Machine Learning for industrial applications: A comprehensive literature review.** Expert Systems with Applications, 1. ago. 2021.

BREIMAN, L. **Random Forests.** Machine learning, v. 45, p. 5–32, 2001.

BUSCHJÄGER, S.; MORIK, K. **Decision Tree and Random Forest Implementations for Fast Filtering of Sensor Data.** IEEE Transactions on Circuits and Systems I: Regular Papers, v. 65, n. 1, p. 209–222, 2018.

CANIS, A.; CHOI, J.; FORT, B.; et al. **From software to accelerators with LegUp high-level synthesis**. 2013 International Conference on Compilers, Architecture and Synthesis for Embedded Systems, CASES 2013, 2013.

CAPOGROSSO, L. et al. **A Machine Learning-Oriented Survey on Tiny Machine Learning**. IEEE Access, v. 12, p. 23406–23426, 2024.

COMPTON, K.; HAUCK, S. **Reconfigurable computing: a survey of systems and software**. ACM Computing Surveys, v. 34, n. 2, p. 171–210, 2002.

COSTA, V. G.; PEDREIRA, C. E. **Recent advances in decision trees: an updated survey**. Artificial Intelligence Review, v. 56, n. 5, p. 4765–4800, 10 maio 2023.

CONG, J.; FANG, Z.; LO, M.; et al. **Understanding Performance Differences of FPGAs and GPUs**. Proceedings - 26th IEEE International Symposium on Field-Programmable Custom Computing Machines, FCCM 2018, 2018.

CRIMINISI, A.; SHOTTON, J.; KONUKOGLU, E. **Decision forests: A unified framework for classification, regression, density estimation, manifold learning and semi-supervised learning**. Foundations and Trends in Computer Graphics and Vision, v. 7, n. 2–3, p. 81–227, 2011.

DAMIANI, A.; SOZZO, E. DEL; SANTAMBROGIO, M. D. **Large Forests and Where to “Partially” Fit Them**. 2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC). Anais...IEEE, 17 jan. 2022. Disponível em: <https://ieeexplore.ieee.org/document/9712534/>

DINH, T. P. et al. **A flexible and efficient FPGA-based random forest architecture for IoT applications**. Internet of Things, v. 22, p. 100813, 1 jul. 2023.

FERREIRA, C. A. **Linguagem e compilador para o paradigma orientado a notificações (PON): Avanços e comparações**. Dissertação de Mestrado, PPGCA, UTFPR. Curitiba, Brasil, 2015.

FIGUEIREDO, L. H. P.. **PON para Sistema Distribuídos**. Dissertação de Mestrado, PPGCA, UTFPR. Curitiba, Brasil, 2022.

FIGUEIREDO, L. H. P.; SIMÃO, J. M.; VENDRAMIN, A. C. B. K. **Paradigma Orientado a Notificações para Aplicações de Internet das Coisas em Cidades Inteligentes**. Anais Estendidos do XVIII Simpósio Brasileiro de Sistemas de Informação (SBSI Estendido 2022). Anais Sociedade Brasileira de Computação (SBC), 16 maio 2022.

FISHER, R. A. **THE USE OF MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS**. Annals of Eugenics, v. 7, n. 2, p. 179–188, 1936.

GAJJAR, A. et al. **FAXID: FPGA-Accelerated XGBoost Inference for Data Centers using HLS**. 2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), IEEE, 15 maio, 2022.

GONSKI, J.; GUPTA, A.; JIA, H.; KIM, H.; ROTA, L.; RUCKMAN, L.; DRAGONE, A.; HERBST, R. **Embedded FPGA developments in 130 nm and 28 nm CMOS for machine learning in particle detector readout**. Journal of Instrumentation, v. 19, n. 08, p. P08023, ago. 2024.

GONZÁLEZ, S.; GARCÍA, S.; DEL SER, J.; ROKACH, L.; HERRERA, F. **A practical tutorial on bagging and boosting based ensembles for machine learning: Algorithms, software tools, performance study, practical perspectives and opportunities**. Information Fusion, v. 64, p. 205–237, 2020.

HAYKIN, S.. **Neural Networks and Learning Machines**. 3. ed. Upper Saddle River: Pearson Education, 2009.

HE, M.; THOTTETHODI, M.; VIJAYKUMAR, T. N. **Booster: An Accelerator for Gradient Boosting Decision Trees Training and Inference**. Proceedings - 2022 IEEE 36th International Parallel and Distributed Processing Symposium, IPDPS 2022. **Anais...**Institute of Electrical and Electronics Engineers Inc., 2022.

HENZEN, A. F. **PORTABILIDADE DO FRAMEWORK PON DE C++ STANDARD PARA C# E JAVA**. Relatório da disciplina de Tópicos Especiais Em Ec: Paradigma Orientado A Notificações. CPGEI-PPGCA/UTFPR, Curitiba - PR, Brasil, 2015.

HUANG, L.; LI, D.-L.; WANG, K.-P.; GAO, T.; TAVARES, A. **A Survey on Performance Optimization of High-Level Synthesis Tools**. Journal of Computer Science and Technology, v. 35, n. 3, p. 697–720, 2020

INGGS, G.; FLEMING, S.; THOMAS, D.; LUK, W. **Is high level synthesis ready for business? A computational finance case study**. Proceedings of the 2014 International Conference on Field-Programmable Technology, FPT 2014, p. 12–19, 2015.

JAHANI, H.; JAIN, R.; IVANOV, D. **Data science and big data analytics: a systematic review of methodologies used in the supply chain and logistics research**. Annals of Operations Research, 2023.

KANANI, A.; VAIDYA, S.; AGARWAL, H. **LightFPGA: Scalable and Automated FPGA Acceleration of LightGBM for Machine Learning Applications**. 2021 25th International Symposium on VLSI Design and Test, VDAT 2021. **Anais...**Institute of Electrical and Electronics Engineers Inc., 16 set. 2021.

KARN, Rupesh Raj; KNECHTEL, Johann; SINANOGLU, Ozgur. **Code-based cryptography for confidential inference on FPGAs: an end-to-end methodology**. In: International Symposium On Quality Electronic Design (ISQED), 25., 2024.

KAUR, R.; GABRIJELČIČ, D.; KLOBUČAR, T. **Artificial intelligence for cybersecurity: Literature review and future research directions**. Information Fusion, v. 97, 1 set. 2023.

KERSCHBAUMER, R.; SIMÃO, J. M.; LINHARES, R. R.; STADZISZ, P. C.; LIMA, C. R. E. **Paradigma Orientado a Notificações para a Síntese de Lógica Reconfigurável**. LA-CCI/CBIC, Curitiba, Paraná, Brasil, 2015.

KERSCHBAUMER R.; LINHARES, R. R.; SIMÃO, J. M.; STADZISZ, P. C.; Lima C. R. E. **Notification Oriented Paradigm to Implement Digital Hardware**. Journal of Circuits Systems and Computers, 2018a.

KERSCHBAUMER R.; R. R.; SIMÃO; FABRO J. A; LINHARES , J. M.; STADZISZ, P. C.; Lima C. R. E. **A Tool for Digital Circuits Synthesis Based on Notification Oriented Paradigm**. IEEE Latin America Transactions, 2018b

KERSCHBAUMER, R. **Proposição do paradigma orientado a notificações no desenvolvimento de circuitos lógico-digitais reconfiguráveis**. Tese de Doutorado, CPGEI, UTFPR. Brasil, 2018.

KOSSOSKI, C., **Proposta de um método de teste para processos de desenvolvimento de software usando o Paradigma Orientado a Notificações**. Dissertação de Mestrado, CPGEI, UTFPR, 2015.

KUMAR, Y. et al. **Artificial intelligence in disease diagnosis: a systematic literature review, synthesizing framework and future research agenda**. Journal of Ambient Intelligence and Humanized Computing, v. 14, n. 7, p. 8459–8486, 1 jul. 2023.

LAHTI, S.; SJOVALL, P.; VANNE, J.; HAMALAINEN, T. D. **Are We There Yet? A Study on the State of High-Level Synthesis**. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, v. 38, n. 5, p. 898–911, 2019

LEDESMA, M. K.; SIMÃO, J. M.. **Avanços em Paradigma Orientado a Notificações - Framework PON C++ 4.5 e seu gerador de código em Tecnologia LingPON 3.0**. In: XII Seminário de Extensão e Inovação & XXVII Seminário de Iniciação Científica e Tecnológica da UTFPR, 2022, Santa Helena - PR. XII SEI & XXVII SICITE 2022, 2022.

LINHARES, R. R.; RENAUX, D. P. B.; STADZISZ, P. C.; SIMÃO, J. M. **Evaluation of the Notification Oriented Paradigm applied to Real-Time Systems**. Brazilian Symposium on Computing Systems Engineering (SBESC), Manaus-AM, Brazil, 2014.

LINHARES, R. R., **Contribuição para o desenvolvimento de uma arquitetura de computação própria ao paradigma orientado a notificações**. Tese de Doutorado, CPGEI, UTFPR. Brasil, 2015.

LINHARES, R. R.; SIMAO, J. M.; STADZISZ, P. C. **NOCA - A Notification-Oriented Computer Architecture**. Latin America Transactions, IEEE (Revista IEEE America Latina), v. 13, n. 5, p. 1593–1604, 2015.

LINHARES, R. R.; PORDEUS, L. F.; SIMÃO, J. M.; STADZISZ, P. C. **NOCA — A Notification-Oriented Computer Architecture: Prototype and Simulator**. IEEE Access, v. 8, p. 37287–37304, 2020.

LIN, Xiang; BLANTON, R. D. Shawn; THOMAS, Donald E.. **Random forest architectures on FPGA for multiple applications**. In: Great Lakes Symposium on VLSI, 2017, Banff, Alberta, Canada. *Proceedings*. New York, NY: Association for Computing Machinery, 2017. p. 415–418.

LIU, Y.; LI, Y.; QI, Z.; GUAN, H. **A scala based framework for developing acceleration systems with FPGAs**. Journal of Systems Architecture, v. 98, p. 231–242, 2019.

LUCCA, J.; BANASZEWSKI, R. F.; TACLA, C. A.; STADZISZ, P. C.; SIMÃO, J. M.. **Ambiente de Controle Holônico Sobre o Simulador ANALYTICE II e**

Comparações de Políticas de Controle de Manufatura. In: Simpósio Brasileiro de Automação Inteligente - SBAI - 2009 (a ser realizado), 2009, Brasília. ANAIS DO IX SIMPÓSIO BRASILEIRO DE AUTOMAÇÃO, 2009.

MAMANN, L. V. S.; Simao, J. M.; DELGADO, M. R. B. S.; PIGATTO, D. F.. **Paradigma Orientado a Notificações Aplicado à Programação de Microcontroladores**. In: XI SBESC - Brazilian Symposium on Computing Systems Engineering, 2021, Florianópolis. XI SBESC - Brazilian Symposium on Computing Systems Engineering, 2021.

MARTINI, G. H. K. **NOP language on multi-core architecture computers. Definição Framework NOP AKKA**. Aluno Externo CPGEI/UTFPR. Disciplina sobre Paradigma Orientado a Notificações (PON), CPGEI-PPGCA/UTFPR (Profs. J. M. Simão & R. R. Linhares), Curitiba - PR, Brasil, 2018.

MARTINI, G. H. K.; SIMÃO J. M.; FABRO, J. A.; RONSZCKA, ADRIANO F. **Multithreading capability evaluation of the Notification Oriented Programming Language for the x86 Architecture**. In: 11th International Conference on Information Society and Technology - ICIST, 2021, Kopaonik. Book: ICIST 2021 Proceedings - Proceedings of the 11th International Conference on Information Society and Technology. Belgrade, Serbia: ISOS Conference Proceedings Series, 2021. v. 1. p. 44-49

MEEUS, W; VAN B, K; GOEDEMEÉ, T; *et al.* **An overview of today's high-level synthesis tools**. Design Automation for Embedded Systems, v. 16, n. 3, p. 31-51, 2012.

MELO, L. C. V.; SIMÃO, J. M.; FABRO, J. A. **Adaptation of the Notification Oriented Paradigm (NOP) for the Development of Fuzzy Systems**. Mathware & Soft Computing, v. 22, n. 1, p. 40–64, 2015.

MELO, L. C., **Adaptação Do Paradigma Orientado A Notificações Para Desenvolvimento De Sistemas Fuzzy**. Dissertação de Mestrado, Programa de Pós-Graduação em Computação Aplicada (PPGCA), UTFPR. Curitiba, Brasil, 2016.

MENDONÇA, I. T. M., SIMÃO, J. M., WIECHETECK, L. V. B., STADZISZ, P. C. **Método para Desenvolvimento de Sistemas Orientados a Regras utilizando o Paradigma Orientado a Notificações**. LA-CCI/CBIC, 2015.

MENDONÇA, I. T. M. **Metodologia de projeto de software orientado a notificações**. 2020. Tese (Doutorado em Engenharia Elétrica e Informática Industrial) - Universidade Tecnológica Federal do Paraná, Curitiba, 2020.

MENDONÇA, I. T. M.; SIMÃO, J. M.; STADZISZ, P. C. **Notification-oriented software design methodology**. IEEE Latin America Transactions, v. 18, n. 06, p. 1144–1153, 2020.

MIENYE, I. D.; JERE, N. **A Survey of Decision Trees: Concepts, Algorithms, and Applications**. IEEE Access, v. 12, p. 86716–86727, 2024.

MIENYE, I. D.; SUN, Y. **A Survey of Ensemble Learning: Concepts, Algorithms, Applications, and Prospects**. IEEE Access, Institute of Electrical and Electronics Engineers Inc., 2022.

MOHAMMADI, M. et al. **Deep learning for IoT big data and streaming analytics: A survey**. IEEE Communications Surveys and Tutorials, Institute of Electrical and Electronics Engineers Inc., 1 out. 2018.

MOLINA, R. et al. **Efficient traversal of decision tree ensembles with FPGAs**. Journal of Parallel and Distributed Computing, v. 155, p. 38–49, 1 set. 2021.

NAKAHARA, H. et al. **An acceleration of a random forest classification using Altera SDK for OpenCL**. 2016 International Conference on Field-Programmable Technology (FPT). Anais...IEEE, dez. 2016. Disponível em: <http://ieeexplore.ieee.org/document/7929555/>

NANE, R.; SIMA, V. M.; PILATO, C.; et al. **A Survey and Evaluation of FPGA High-Level Synthesis Tools**. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, v. 35, n. 10, p. 1591–1604, 2016.

NARAYANAN, R. et al. **An FPGA Implementation of Decision Tree Classification**. 2007 Design, Automation & Test in Europe Conference & Exhibition, abr. 2007.

NAZARETH, N.; RAMANA REDDY, Y. V. **Financial applications of machine learning: A literature review**. Expert Systems with Applications. Elsevier Ltd, 1 jun. 2023.

NECHI, A.; GROTH, L.; MULHEM, S.; et al. **FPGA-based Deep Learning Inference Accelerators: Where Are We Standing?**. ACM Transactions on Reconfigurable Technology and Systems, v. 16, n. 4, 2023.

NEGRINI, F. **Tecnologia NOPL Erlang-Elixir: paradigma orientado a notificações via uma abordagem orientada a microatores assíncronos**. Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2019.

NEGRINI, F.; PORDEUS, L. F.; LINHARES, R. R.; SIMÃO, J. M. **Linguagem do Paradigma Orientado a Notificações: Comparativos via Simulador de Tráfego**. Revista SODEBRAS, v. 15, p. 102-107, 2020.

NEVES, F. S. **Framework PON C++ 4.0: Contribuição para a Concepção no Paradigma Orientado a Notificações por meio de Programa Genérico**. Dissertação de Mestrado – Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI), UTFPR, 2021.

NOVAES, P. J. D. **Método e linguagem para modelagem gráfica de requisitos de software e sistemas**, Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2019.

OLIVEIRA, R. N. et al. **Notification Oriented Paradigm Applied to Ambient Assisted Living Tool**. IEEE Latin America Transactions, v. 16, n. 2, p. 647–653, fev. 2018.

OLIVEIRA, R. N.; OTHERS. **Assistência à autonomia domiciliar empregando paradigma orientado a notificações**, Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2019.

OWAIDA, M.; ZHANG, H.; ZHANG, C.; ALONSO, G. **Scalable Inference of Decision Tree Ensembles: Flexible Design for CPU-FPGA Platforms.**, 2014.

OWAIDA, M.; ALONSO, G. **Application partitioning on FPGA clusters: Inference over decision tree ensembles**. Proceedings - 2018 International Conference on Field-Programmable Logic and Applications, FPL 2018. Anais...Institute of Electrical and Electronics Engineers Inc., 9 nov. 2018.

OSHIRO, L. K. **Contribuição em paradigma orientado a notificações: evolução da tecnologia LingPON 2.0 via aprimoramento da linguagem e compilador para código notificante modular em C++**. 2021. Dissertação de Mestrado, CPGEI, UTFPR, Curitiba, 2021.

PETERS, E. **Coprocessador para Aceleração de Aplicações Desenvolvidas Utilizando Paradigma Orientado a Notificações**. Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2012.

PETERS, E.; JASINSKI, R. P.; PEDRONI, V. A.; SIMAO, J. M. **A new hardware coprocessor for accelerating Notification-Oriented applications**. International Conference on Field Programmable Technology (FPT), South Korea, 2012.

POORHERAVI, P. A.; GAUDET, V. **FPGA-Based Architectures for Random Forest Acceleration**. Midwest Symposium on Circuits and Systems. Anais...Institute of Electrical and Electronics Engineers Inc., 2022.

PORDEUS, L. F.; KERSCHBAUMER, R.; LINHARES, R. R.; WITT, F. A.; STADZISZ, P. C.; LIMA, C. R. E.; SIMÃO, J. M.. **NOTIFICATION ORIENTED PARADIGM TO DIGITAL HARDWARE**. Revista SODEBRAS, v. 11, p. 116-122, 2016.

PORDEUS, L. F. **Simulação de uma arquitetura de computação própria ao paradigma orientado a notificações**. Dissertação de Mestrado, Universidade Tecnológica Federal do Paraná - UTFPR, Curso de Pós-Graduação em Engenharia Elétrica e Informática Industrial - CPGEI, Curitiba, 2017.

PORDEUS, L. F. **ArqTotalPON - Contribuição para Arquitetura de Computação própria e efetiva ao Paradigma Orientado a Notificações**. Qualificação de Doutorado, Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI), UTFPR, 2020.

PORDEUS, L. F.; LINHARES, R. R.; STADZISZ, P. C.; SIMÃO, J. M. **NOP-DH – Evaluation Over Bitonic Sort Algorithm**. Microprocessors and Microsystems, v. 85, p. 104314, 2021.

PORDEUS, L. F.; LAZZARETTI, A. E.; LINHARES, R. R.; SIMÃO, J. M. **Notification Oriented Paradigm to Digital Hardware — A benchmark evaluation with Random Forest algorithm**. Microprocessors and Microsystems, v. 103, p. 104951, 2023.

REUTHER, A.; MICHALEAS, P.; JONES, M.; et al. **Survey and Benchmarking of Machine Learning Accelerators**. 2019 IEEE High Performance Extreme Computing Conference (HPEC), set. 2019.

ROKACH, L.. **Decision forest: Twenty years of research**. Information Fusion, v. 27, p. 111–125, 2016. Elsevier B.V.

RONSZCKA, A. F. **Contribuição Para a Concepção de Aplicações no Paradigma Orientado a Notificações (PON) Sob o Viés de Padrões**. 2012. Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2012.

RONSZCKA, A. F.; BANASZEWSKI, R. F.; LINHARES, R. R.; TACLA, C. A.; STADZISZ, P. C.; SIMAO, J. M. **Notification-Oriented and Rete Network Inference: A Comparative Study**. Systems, Man, and Cybernetics (SMC), 2015 IEEE International Conference on. p.807–814, 2015.

RONSZCKA, A. F.; FERREIRA, C. A.; STADZISZ, P. C.; FABRO J. A, SIMÃO, J. M. **Notification Oriented Programming Language and Compiler**. VII SBESC - Brazilian Symposium on Computing Systems Engineering - Curitiba - Paraná – Brasil, 2017.

RONSZCKA, A. F. **Método para a Criação de Linguagens de Programação e Compiladores para o Paradigma Orientado a Notificações em Plataformas Distintas**. Tese de Doutorado, CPGEI, UTFPR. Brasil, 2019.

ROY, P. VAN. **Programming Paradigms for Dummies: What Every Programmer Should Know**. New Computational Paradigms for Computer Music, p. 9–47, 2009.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Hoboken: Pearson, 2021.

SAIDI, A. et al. **FPGA-based implementation of classification techniques: A survey**. Integration Elsevier B.V., 1 nov. 2021.

SANTOS, L. A. **Linguagem e Compilador para o Paradigma Orientado a Notificações: avanços para facilitar a codificação e sua validação em uma aplicação de controle de futebol de robôs**. Dissertação de Mestrado, CPGEI, UTFPR. Brasil, 2017.

SANTOS, L. A.; SIMÃO, J. M.; FABRO J. A. **Linguagem e Compilador para o Paradigma Orientado a Notificações Avanços para a Redução de Complexidade de Código**. VII SBESC - Brazilian Symposium on Computing Systems Engineering - Curitiba - Paraná – Brasil, 2017.

SARKER, I. H. **Machine Learning: Algorithms, Real-World Applications and Research Directions**. SN Computer Science, 1. maio 2021.

SAYED, R.; AZMI, H.; NASSAR, A. M.; SHAWKEY, H. **Design automation and implementation of machine learning classifier chips**. IEEE Access, v. 8, p. 192155–192164, 2020.

SCHÜTZ, F.; FABRO, J. A.; LIMA, C. R. E.; STADZISZ, P. C.; SIMÃO, J. M. **Training of an Artificial Neural Network with Backpropagation Algorithm Using Notification Oriented Paradigm**. LA-CCI/CBIC, Outubro, 2015.

SCHÜTZ, F.; Fabro J. A.; RONSZCKA, A. F.; STADZISZ, P. C.; SIMÃO, J. M. **Proposal of a declarative and parallelizable artificial neural network using the notification-oriented paradigm**. Neural Computing and Applications, p. 1-12, 2018.

SCHÜTZ, F. **NeuroPON: uma abordagem para o desenvolvimento de redes neurais artificiais utilizando o paradigma orientado a notificações**. Tese de Doutorado, CPGEI, UTFPR. Brasil, 2019.

SHAH, M. et al. **Accelerating Random Forest Classification on GPU and FPGA**. ACM International Conference Proceeding Series. Anais...Association for Computing Machinery, 29 ago. 2022.

SIMÃO, J. M. **Proposta de uma Arquitetura de Controle para Sistemas Flexíveis de Manufatura Baseada em Regras e Agentes**. 2001. Dissertação de Mestrado, Universidade Tecnológica Federal do Paraná - UTFPR, Curso de Pós-Graduação em Engenharia Elétrica e Informática Industrial - CPGEI, Curitiba, 2001.

SIMÃO, J. M.; STADZISZ, P. C. **An Agent-Oriented Inference Engine applied for Supervisory Control of Automated Manufacturing Systems**. Frontiers in Artificial Intelligence and Applications (Advances in Logic, Art. Int. and Robotics -

LAPTEC 2002 Edited by Abe J. M., Silva Filho J. I.), IOS Press Books, Amsterdam - The Netherlands, Vol. 85, pp. 234-241. ISBN: 1 58603 292 5, 2002.

SIMÃO, J. M.; FABRO, J. A.; STADZISZ, P. C. **An Agent-Oriented Fuzzy Inference Engine**. In: 6. Simpósio Brasileiro de Automação Inteligente, 2003, Bauru-SP. Anais do VI SBAI, 2003.

SIMÃO, J. M. **A Contribution To The Development Of A HMS Simulation Tool And Proposition Of A Meta-Model For Holonic Control**. Tese de doutorado. CPGEI, CEFET-PR. Curitiba, Brasil, 2005.

SIMÃO, J. M. ; STADZISZ, P. C. **Paradigma Orientado a Notificações (PON) - Uma Técnica de Composição e Execução de Software Orientada a Notificações**. Pedido de patente: Privilégio de Inovação. Número do registro: PI08055181. Data de depósito: 26 nov. 2008. Data de recusa: 31 ago. 2023 (pedido tido como de tema pertinente mas abstrato). Instituição de registro: INPI - Instituto Nacional de Propriedade Industrial. Instituição financiadora: Universidade Tecnológica Federal do Paraná. Brasil, 2008.

SIMÃO, J. M.; STADZISZ, P. C. **Inference Based on Notifications: A Holonic Metamodel Applied to Control Issues**. IEEE Transactions on Systems, Man and Cybernetics. Part A, Systems and Humans, v. 39, p. 238-250, 2009.

SIMÃO, J. M.; TACLA, C. A.; STADZISZ, P. C.; BANASZEWSKI, R. F. **Notification Oriented Paradigm (NOP) and Imperative Paradigm: A Comparative Study**. Journal of Software Engineering and Applications, 5(6), p. 402-416, 2012a.

SIMÃO, J. M.; LINHARES, R. R. ; WITT, F. A. ; LIMA, C. R. E. ; STADZISZ, P. C. **Método de Materialização do Paradigma Orientado a Notificações (em Hardware Digital*)**. Pedido de patente: Privilégio de Inovação. Número do registro: BR102012026429. Data de depósito: 16 out. 2012. Instituição de registro: INPI - Instituto Nacional da Propriedade Industrial. Instituição financiadora: Universidade Tecnológica Federal do Paraná (UTFPR). Brasil, 2012b.

SIMÃO, J. M.; STADZISZ, P. C.; TACLA, C. A.; Linhares, R. R.; Belmonte, D. L.; Banaszewski, R. F. **Comparações entre duas materializações do Paradigma Orientado a Notificações (PON): Framework PON Prototipal versus Framework PON Primário**. IV Congreso Internacional de Computación y Telecomunicaciones. Lima, Peru, 2012c.

SIMAO, J. M.; RENAUX, D. P. B.; LINHARES, R. R.; STADZISZ, P. C. **Evaluation of the Notification Oriented Paradigm Applied to Sentient Computing**. 2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing, p. 253–260, 2014.

SIMÃO, J. M.; PANETTO, H.; LIAO, Y.; STADZISZ, P. C. **A notification-oriented approach for systems requirements engineering**. In: IPSE (Org.); 23rd IPSE International Conference on Transdisciplinary Engineering, Transdisciplinary Engineering: Crossing Boundaries. v. 4, p.229–238. Curitiba, Brazil, 2016.

SKORA, L. E. B.; CHIERICI, G. B.; SIMÃO, J. M.. **Avanços nos alvos de compilação da linguagem do paradigma orientado a notificações**. In: XII Seminário de Extensão e Inovação & XXVII Seminário de Iniciação Científica e Tecnológica da UTFPR, 2022, Santa Helena - PR. XII SEI & XXVII SICITE 2022, 2022.

SOUZA, C. A.; WESTPHALL, C. B.; MACHADO, R. B.; et al. **Intrusion detection and prevention in fog based IoT environments: A systematic literature review**. Computer Networks, v. 214, p. 109154, 2022.

SOZZO, E. DEL; CONFICCONI, D.; ZENI, A.; et al. **Pushing the Level of Abstraction of Digital System Design: A Survey on How to Program FPGAs**. ACM Computing Surveys, v. 55, n. 5, 2022.

STRUHARIK, R.; VUKOBRATOVIĆ, B. **A system for hardware aided decision tree ensemble evolution**. Journal of Parallel and Distributed Computing, v. 112, p. 67–83, 2018.

SUMMERS, S.; GUGLIELMO, G. DI; DUARTE, J.; et al. **Fast inference of Boosted Decision Trees in FPGAs for particle physics**. Journal of Instrumentation, v. 15, n. 05, p. P05026–P05026, 2020.

UNWIN, A.; KLEINMAN, K. **The iris data set: In search of the source of virginica**. Significance, v. 18, n. 6, p. 26–29, 2021.

TALAU, M. **PONIP: Uso do Paradigma Orientado a Notificações em Redes IP**. Relatório da disciplina de Tópicos Especiais Em Ec: Paradigma Orientado A Notificações. CPGEI-PPGCA/UTFPR, Curitiba - PR, Brasil, 2016.

VALENÇA, G. Z. **Contribuição para Materialização do Paradigma Orientado a Notificações (PON) Via Framework e Wizard**. Dissertação de Mestrado, Programa de Pós-Graduação em Computação Aplicada (PPGCA), UTFPR. Curitiba, Brasil, 2012.

VAN ESSEN, B. et al. **Accelerating a random forest classifier: Multi-core, GP-GPU, or FPGA?** Proceedings of the 2012 IEEE 20th International Symposium on Field-Programmable Custom Computing Machines, FCCM 2012, 2012.

WANG, H.; LI, J.; HE, K. **Hierarchical Ensemble Reduction and Learning for Resource-constrained Computing**. ACM Transactions on Design Automation of Electronic Systems, v. 25, n. 1, p. 1–21, 2020

WANG, H.; WU, Z.; WANG, X.; BIAN, L.; JIN, H. **HardGBM: A Framework for Accurate and Hardware-Efficient Gradient Boosting Machines**. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, v. 42, n. 7, p. 2122–2135, 2023.

WANG, M.; FU, W.; HE, X.; HAO, S.; WU, X. **A Survey on Large-Scale Machine Learning**. IEEE Transactions on Knowledge and Data Engineering, v. 34, n. 6, p. 2574–2594, 2022.

WEI, L.; MA, Z.; YANG, C.; YAO, Q. **Advances in the Neural Network Quantization: A Comprehensive Review**. Applied Sciences (Switzerland), 1. set. 2024

WIECHETECK, L. V. B. **Método para projeto de software usando o paradigma orientado a notificações – PON**. Dissertação de Mestrado, CPGEI, UTFPR. Curitiba, Brasil, 2011.

WINDH, S; MA, X; HALSTEAD, R J; *et al.* **High-Level Language Tools for Reconfigurable Computing**. Proceedings of the IEEE, v. 103, n. 3, p. 390-408, 2015.

WITT, F. A.; SIMAO, J. M.; LINHARES, R. R.; STADZISZ, P. C.; LIMA, C. R. E. **Comparação entre o Paradigma Orientado a Objetos (POO) e o Paradigma Orientado a Notificações (PON) em um Controle Discreto em Lógica Reconfigurável**. Em: XVI SICITE - Seminário de Iniciação Científica e Tecnológica da UTFPR, 2011, Ponta Grossa - PR. Anais do XVI SICITE, 2011.

WOLNY, Sabine; MAZAK, Alexandra; CARPELLA, Christine; GEIST, Verena; WIMMER, Manuel. **Thirteen years of SysML: a systematic mapping study**. Software and Systems Modeling, v. 19, p. 111–169, 2020. Springer.

XAVIER, R. D. **Paradigmas de desenvolvimento de software: comparação entre abordagens orientada a eventos e orientada a notificações**. Dissertação de Mestrado, PPGCA, UTFPR. Curitiba, Brasil, 2014.

ZAMAN, K. S.; REAZ, M. B. I.; MD ALI, S. H.; BAKAR, A. A. A.; CHOWDHURY, M. E. H. **Custom Hardware Architectures for Deep Learning on Portable Devices: A Review**. IEEE Transactions on Neural Networks and Learning Systems, v. 33, n. 11, p. 6068–6088, 2022.

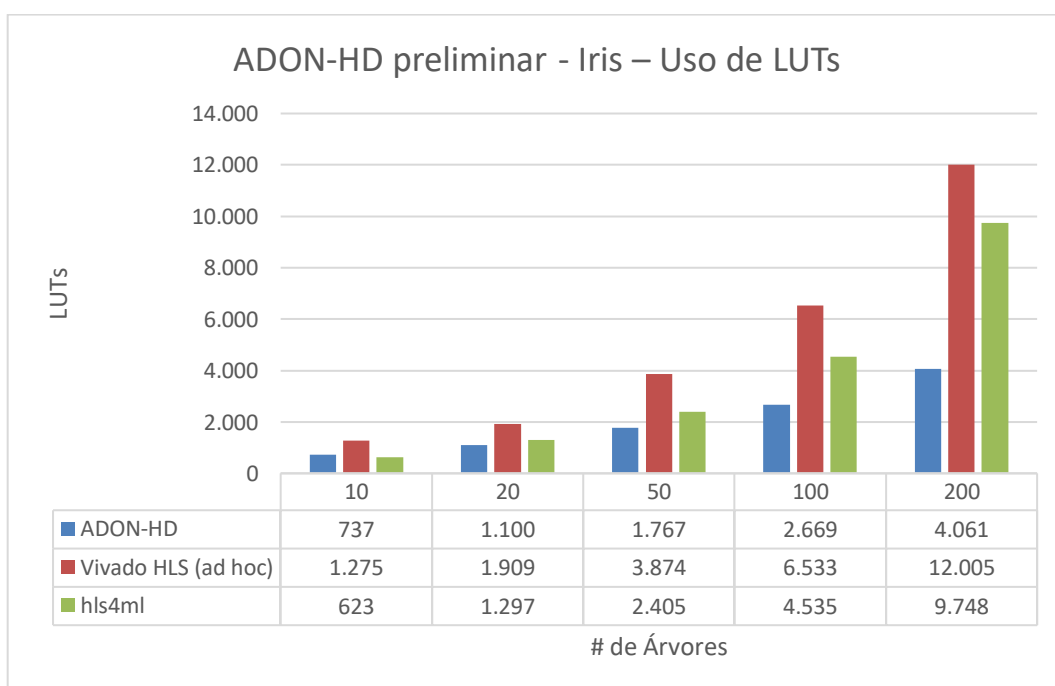
ZHU, M. et al. **An Efficient FPGA-based Accelerator for Deep Forest**. Proceedings - IEEE International Symposium on Circuits and Systems. Anais...Institute of Electrical and Electronics Engineers Inc., 2022.

APÊNDICE A

Este apêndice traz os resultados relativos à implementação do preliminar da ADON-HD para o algoritmo *Random Forest* que publicados em Pordeus *et al.* (2023). Neste trabalho preliminar, comparou-se a primeira implementação da ADON-HD em relação uma implementação em Vivado HLS (*ad hoc*) e a implementação Vivado HLS (*hls4ml*). Entretanto, esta primeira implementação da ADON-HD estava limitada a números inteiros, pois ainda não havia suporte para aritmética de ponto fixo nos componentes do PON-HD. Para os resultados apresentados, foram utilizados os três *datasets* de referência utilizados neste mesmo trabalho de pesquisa: *Iris*, *Wine* e *Digit*. Dito isto, são apresentadas as figuras referentes aos resultados obtidos em relação as métricas:

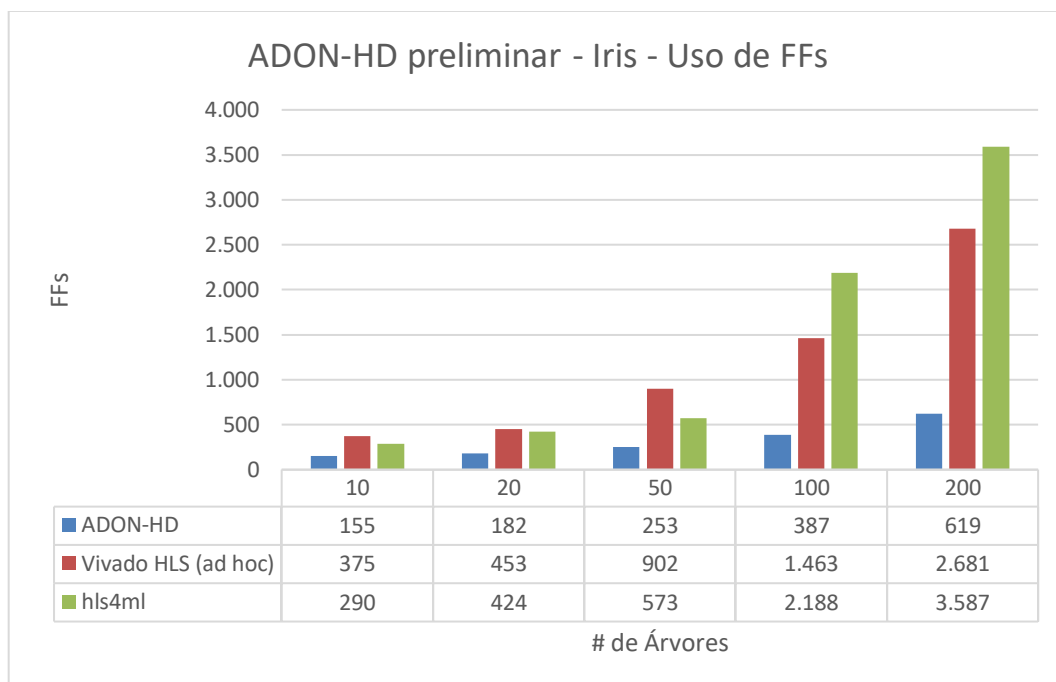
- **Utilização de recursos lógicos:** Número de LUTs e FFs utilizados.
- **Frequência de operação máxima:** Máxima frequência em que o circuito pode operar de forma estável.
- **Número de previsões por segundo:** Número de previsões ou classificações que o circuito pode realizar por segundo.

Figura 62 Uso de LUTs para *dataset Iris* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



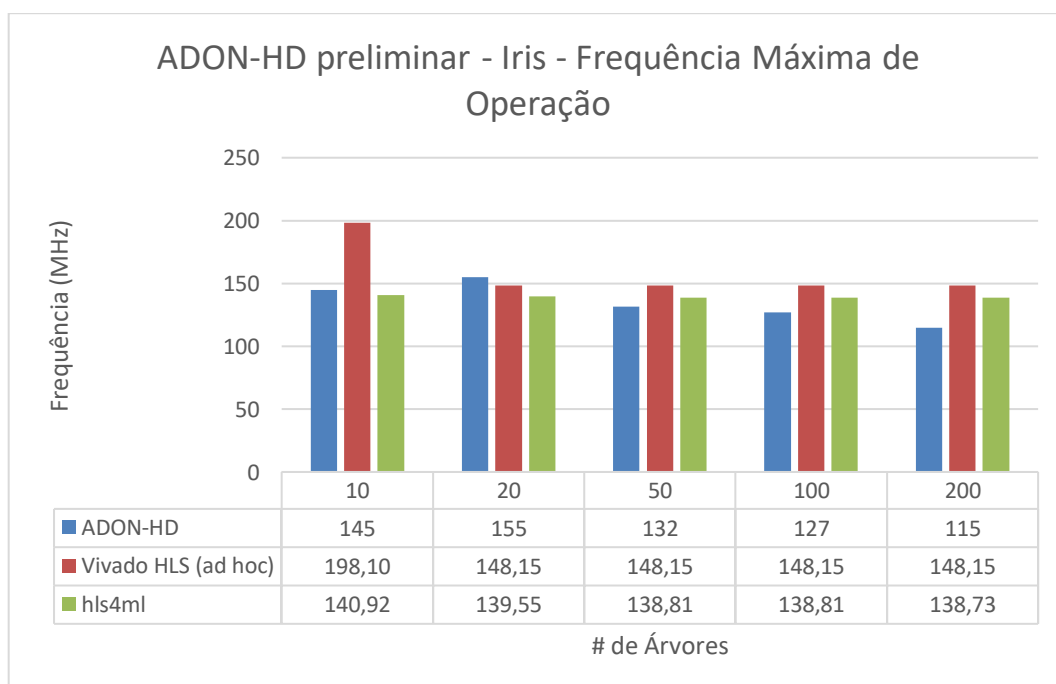
Fonte: Autoria própria

Figura 63 Uso de FFs para *dataset Iris* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



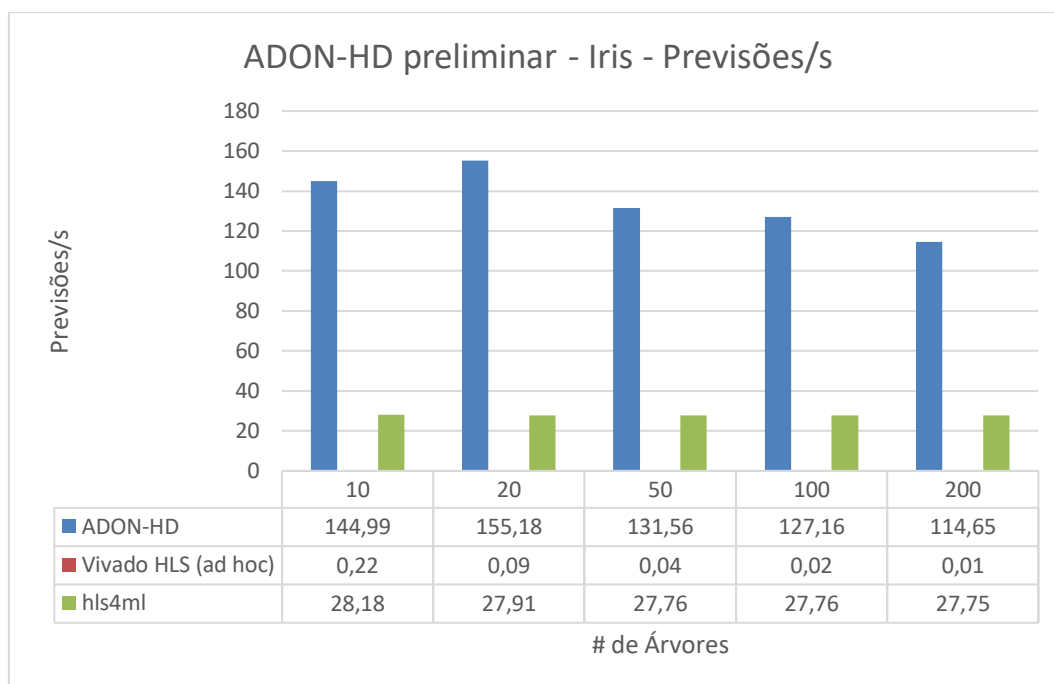
Fonte: Autoria própria

Figura 64 Frequência Máxima de Operação para *dataset Iris* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



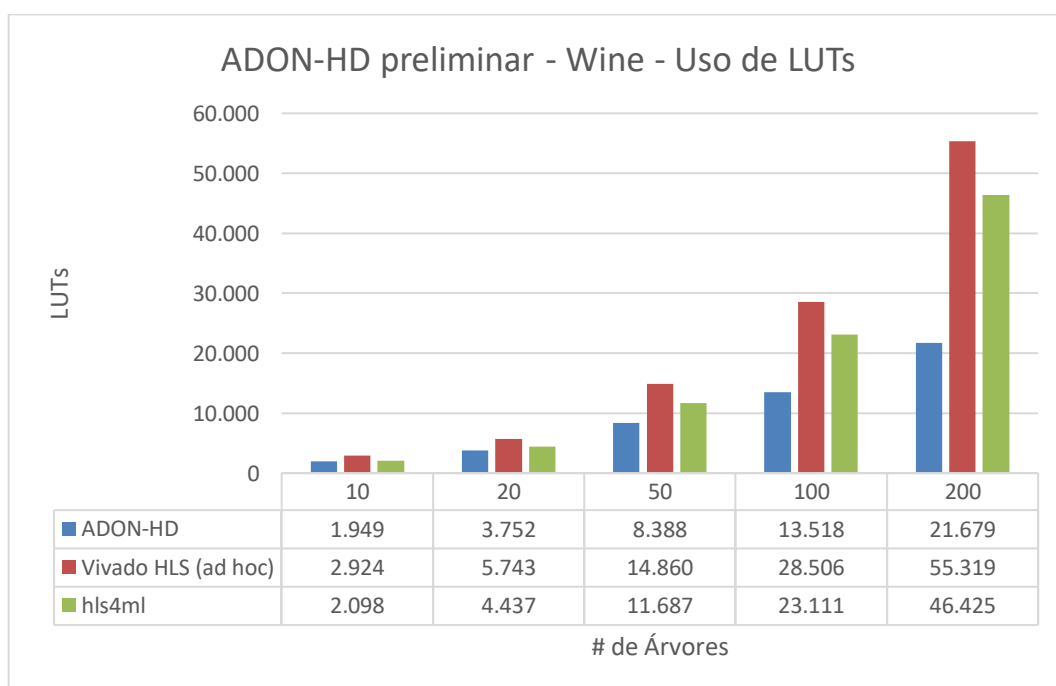
Fonte: Autoria própria

Figura 65 Previsões/s para *dataset Iris* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



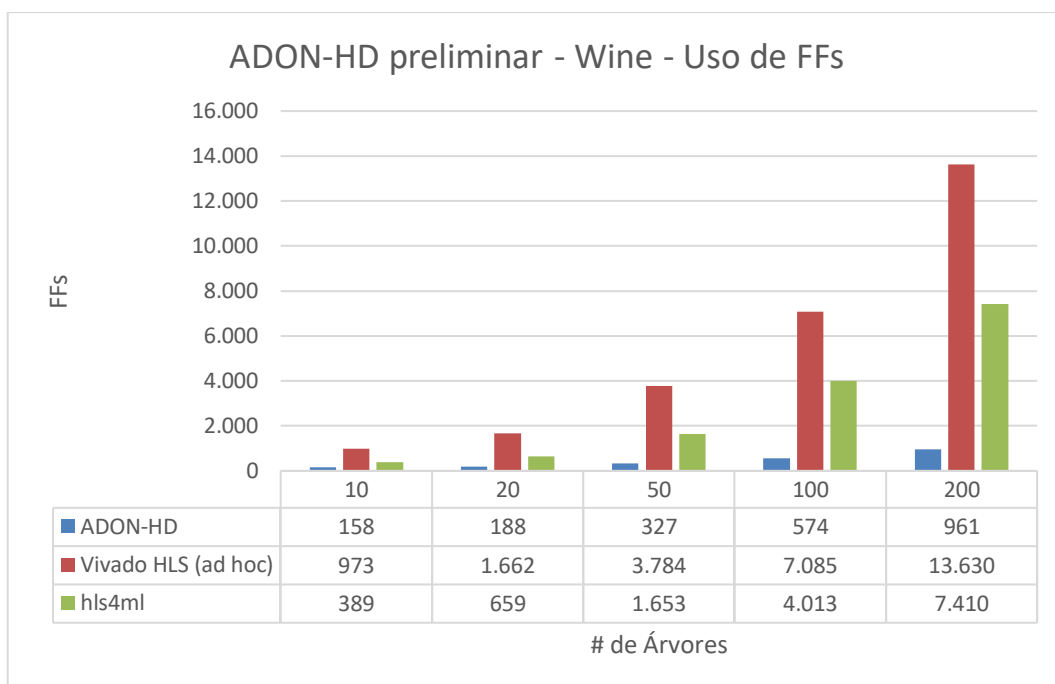
Fonte: Autoria própria

Figura 66 Uso de LUTs para *dataset Wine* com ADON-HD preliminar vs Vivado LHS (*ad hoc*) e Vivado HLS (*hls4ml*).



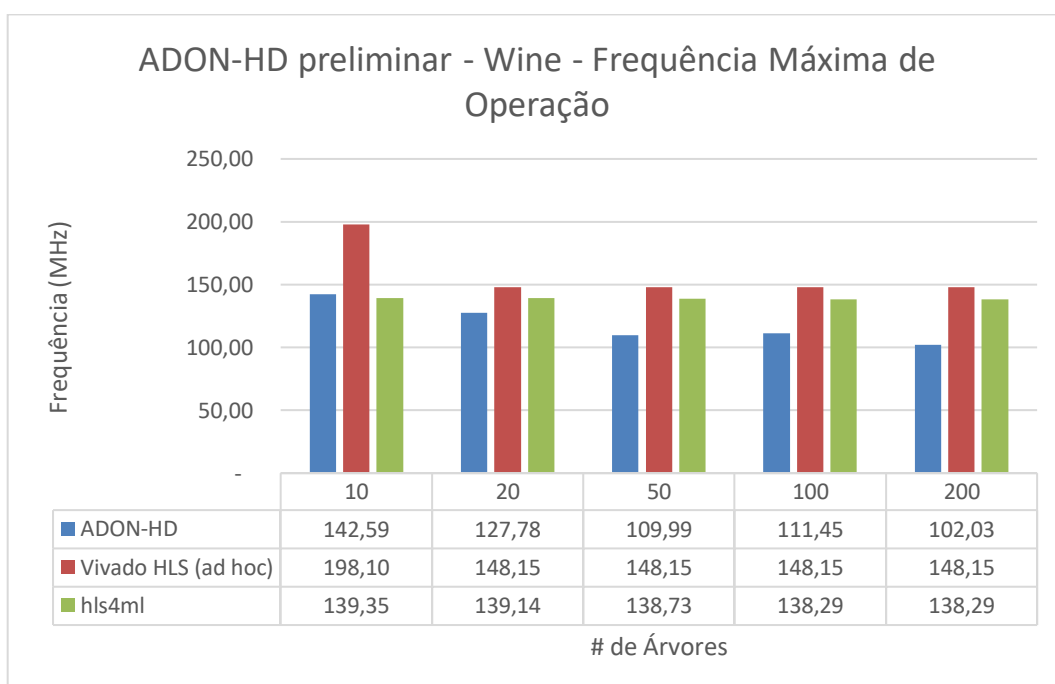
Fonte: Autoria própria

Figura 67 Uso de FFs para *dataset Wine* com ADON-HD preliminar vs Vivado LHS (*ad hoc*) e Vivado HLS (*hls4ml*).



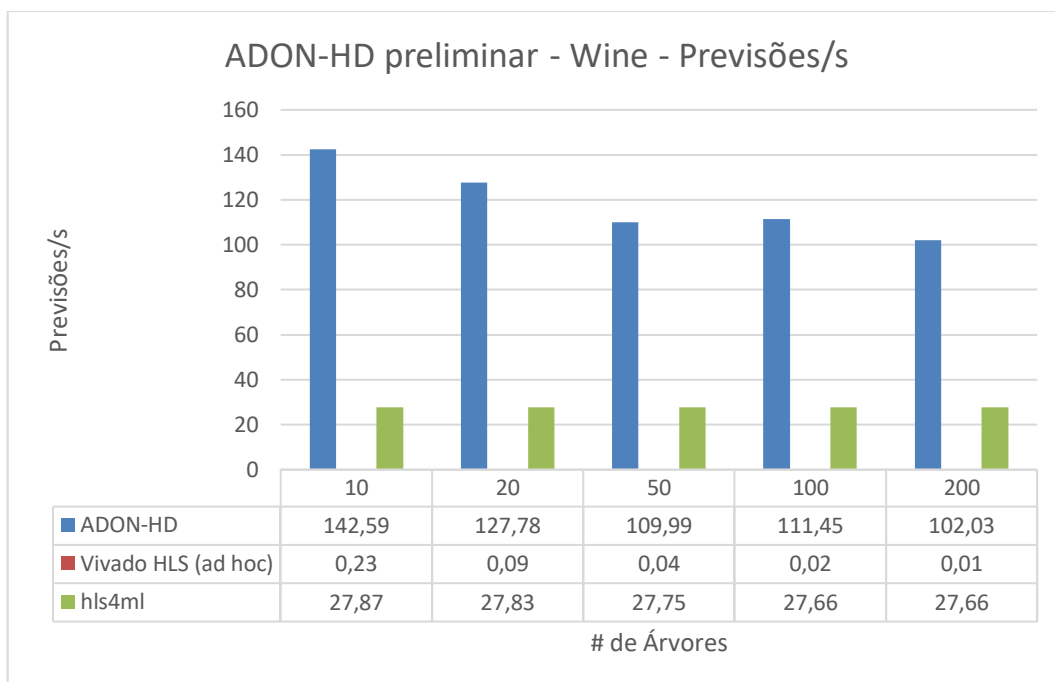
Fonte: Autoria própria

Figura 68 Frequência Máxima de Operação para *dataset Wine* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



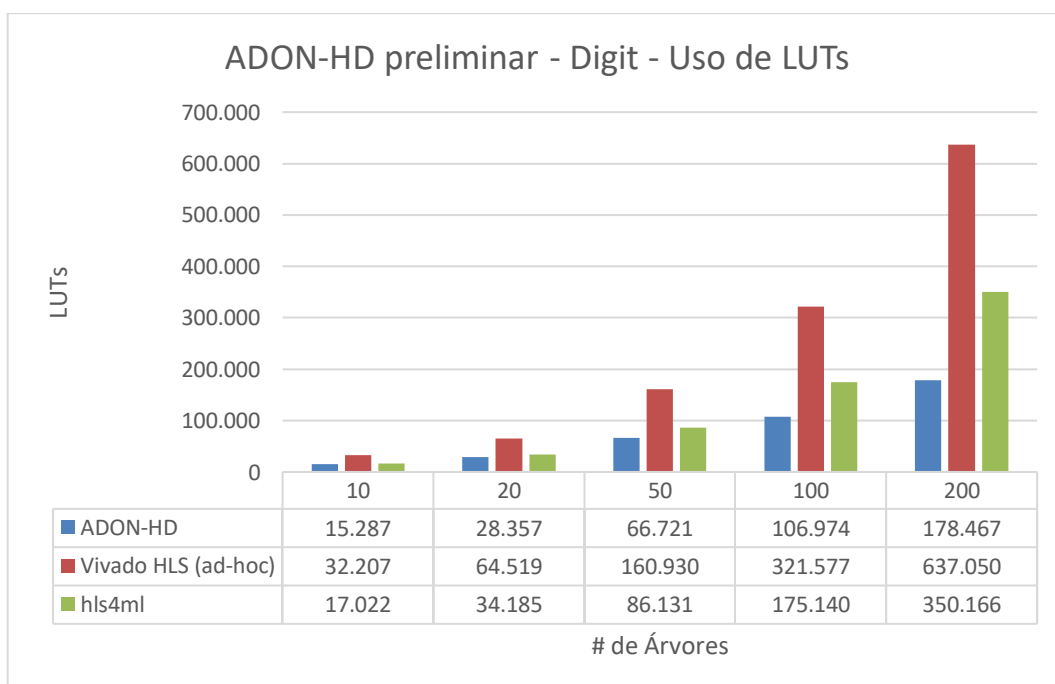
Fonte: Autoria própria

Figura 69 Previsões/s para *dataset Wine* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



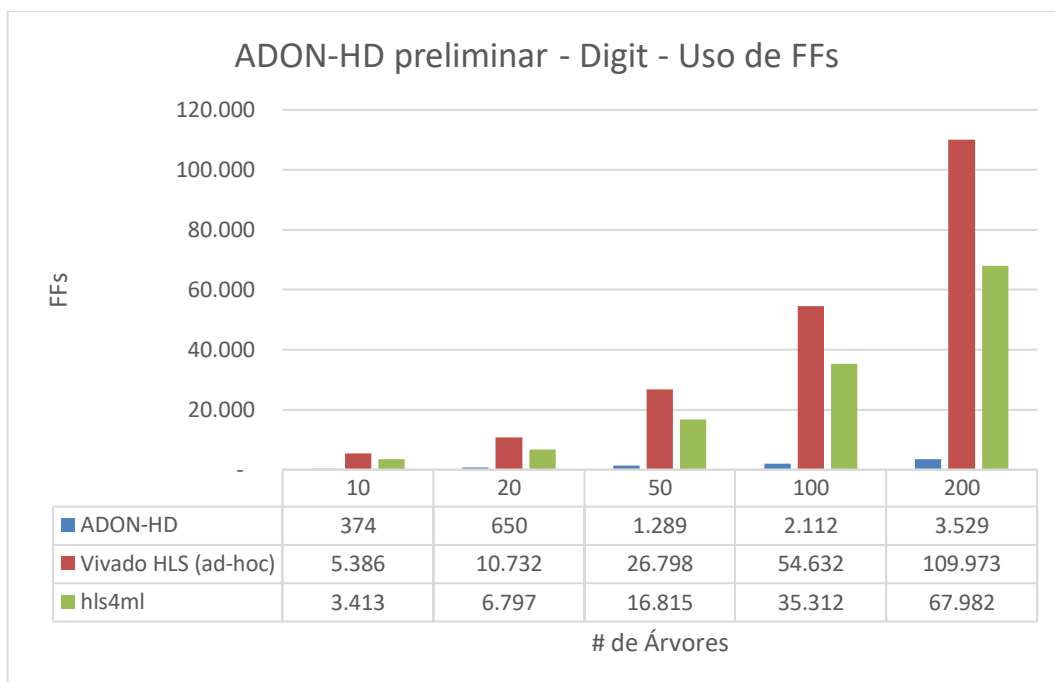
Fonte: Autoria própria

Figura 70 Uso de LUTs para *dataset Digit* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



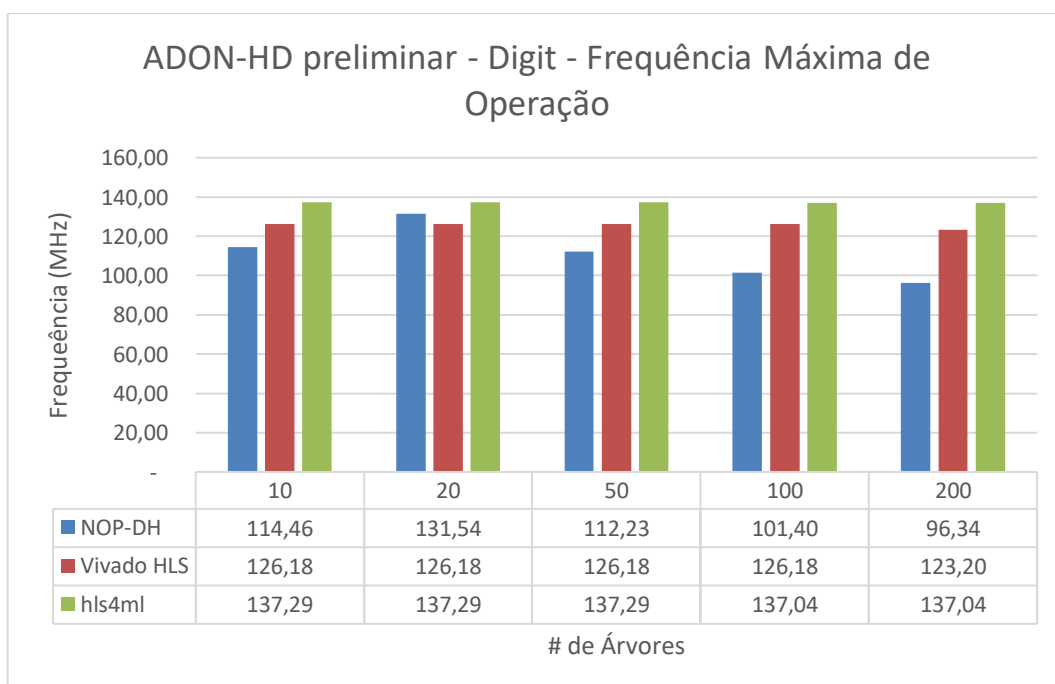
Fonte: Autoria própria

Figura 71 Uso de FFs para *dataset Digit* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



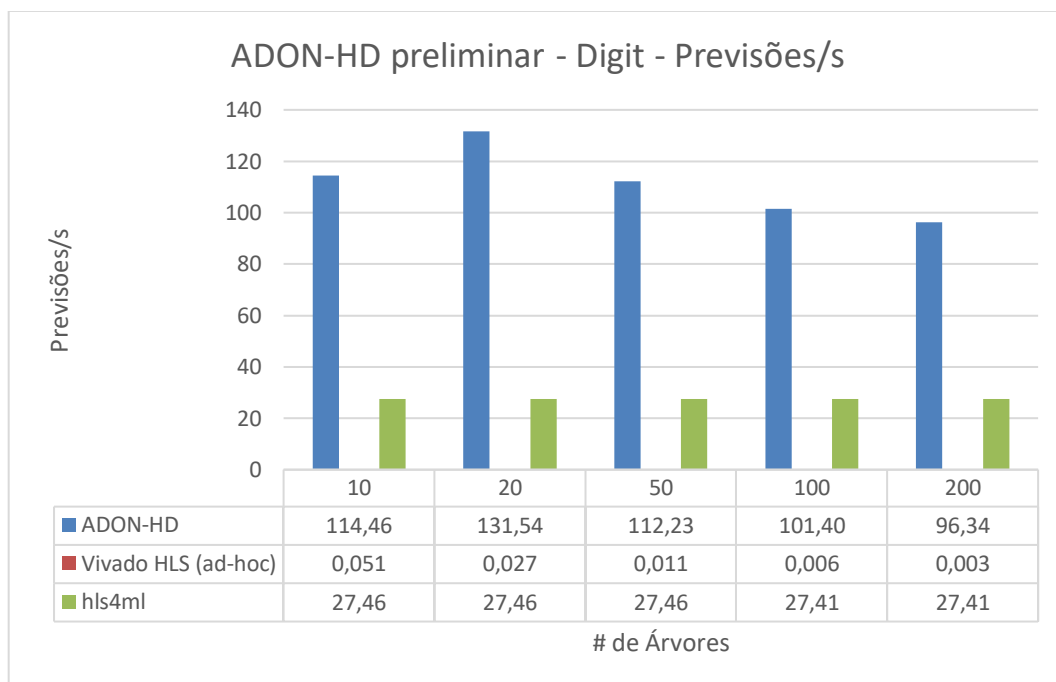
Fonte: Autoria própria

Figura 72 Frequência Máxima de Operação para *dataset Digit* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



Fonte: Autoria própria

Figura 73 Previsões/s para *dataset Digit* com ADON-HD preliminar vs Vivado HLS (*ad hoc*) e Vivado HLS (*hls4ml*).



Fonte: Autoria própria